IF3170 Inteligensi Artifisial

LAPORAN TUGAS BESAR 1

Optimasi Pemuatan Paket pada Peti Kemas Truk Tiga Dimensi Menggunakan Algoritma Local Search

Disusun oleh:

Tim **GGFerkenEZ**

Nathan E C Marpaung	— 13524062
Kurt Mikhael Purba	— 13524065
Aurelia Jennifer Gunawan	— 13524089

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

Daftar Isi

1 Deskripsi Persoalan **1**

2 Dasar Teori **3**
2.1 Algoritma *Hill Climbing* . . . 3
2.1.1 *Steepest Ascent Hill Climbing* . . . 3
2.1.2 *Hill Climbing with Sideways Move* . . . 4
2.1.3 *Random Restart Hill Climbing* . . . 5
2.1.4 *Stochastic Hill Climbing* . . . 7
2.2 Algoritma *Simulated Annealing* . . . 8
2.3 Algoritma *Genetic Algorithm* . . . 10

3 Implementasi **12**
3.1 Arsitektur dan Struktur Program . . . 12
3.2 Representasi State, Fungsi Objektif, dan Constraints . . . 14
3.2.1 Representasi State . . . 14
3.2.2 Fungsi Objektif . . . 15
3.2.3 Constraints . . . 15
3.3 Representasi Neighbor dan State Awal . . . 19
3.3.1 Pembangkitan State Awal . . . 19
3.3.2 Operasi Tetangga . . . 20
3.4 Implementasi Kelas-kelas Pendukung . . . 24
3.4.1 truck.py . . . 24
3.4.2 package.py . . . 25
3.4.3 problem.py . . . 26
3.4.4 placement.py . . . 27
3.4.5 state.py . . . 27
3.4.6 reader.py . . . 29
3.4.7 common.py . . . 30
3.5 Implementasi Algoritma Hill-climbing . . . 31
3.5.1 hill_climb.py . . . 31
3.6 Implementasi Algoritma Simulated Annealing . . . 40
3.6.1 simulated_annealing.py . . . 40
3.7 Implementasi Algoritma Genetic Algorithm . . . 46
3.7.1 genetic_algorithm.py . . . 46
3.8 Implementasi Visualizer . . . 59

3.8.1 visualize.py . . . 59
3.8.2 visualization_charts.py . . . 69

4 Hasil Eksperimen dan Analisis **76**
4.1 Eksperimen Algoritma *Hill-climbing* . . . 76
4.1.1 *Steepest Ascent Hill-climbing* . . . 76
4.1.2 *Hill-climbing with Sideways Move* . . . 86
4.1.3 *Random Restart Hill-climbing* . . . 97
4.1.4 *Stochastic Hill-climbing* . . . 108
4.2 Eksperimen Algoritma *Simulated Annealing* . . . 118
4.3 Eksperimen Algoritma *Genetic Algorithm* . . . 135
4.3.1 Populasi Tetap, Variasi Banyak Iterasi . . . 136
4.3.2 Iterasi Tetap, Variasi Jumlah Populasi . . . 189
4.4 Analisis Hasil Eksperimen . . . 243
4.4.1 Ringkasan Perbandingan Nilai Akhir dan Durasi . . . 243
4.4.2 Kedekatan terhadap Nilai Terbaik yang Ditemukan . . . 245
4.4.3 Konsistensi Hasil antar-*Run* . . . 246
4.4.4 Perbandingan Durasi dan Biaya Pencarian . . . 247
4.4.5 Perilaku Khusus Algoritma *Hill-climbing* dan *Simulated Annealing* . . . 247
4.4.6 Pengaruh Parameter pada *Genetic Algorithm* . . . 248
4.4.7 Kesimpulan Analisis . . . 249

5 Kesimpulan dan Saran **251**
5.1 Kesimpulan . . . 251
5.2 Saran . . . 253

Referensi **255**

Lampiran **256**
Pembagian Tugas . . . 256
Pernyataan . . . 257

Bab 1

Deskripsi Persoalan

Pemuatan barang ke dalam kendaraan angkut merupakan salah satu permasalahan umum dalam industri logistik yang melibatkan berbagai variabel, seperti dimensi paket, berat, nilai barang, kerapuhan, keterbatasan ruang, dan kapasitas kendaraan. Penentuan paket yang dimuat dan penempatannya dalam peti kemas truk memerlukan pengaturan yang efisien agar tidak terjadi pelanggaran, seperti paket yang melayang atau menembus dinding peti kemas, barang pecah-belah yang tertindih paket lain, atau total berat muatan yang melebihi kapasitas truk. Apabila dilakukan secara manual, proses ini sangat tidak efektif karena ruang solusi yang meningkat secara eksponensial seiring bertambahnya jumlah paket, dengan kombinasi pilihan paket, posisi koordinat, dan enam kemungkinan orientasi untuk setiap paket.

Tugas besar 1 mata kuliah IF3170 Inteligensi Artifisial bertujuan agar peserta kuliah memahami dan mampu mengimplementasikan serta mengevaluasi berbagai algoritma *local search* untuk menyelesaikan permasalahan pengisian paket pada truk peti kemas Mejiro Express. Persoalan ini merupakan variasi dari *3D bin packing* dan *knapsack problem*, dengan tujuan memaksimalkan total nilai paket yang berada dalam peti kemas. Tidak seluruh paket harus dimuat, sehingga paket yang tidak terpilih dapat ditempatkan di luar truk. Dalam konteks inteligensi artifisial, proses ini dapat dimodelkan sebagai sebuah *problem-solving agent* yang bertindak secara rasional dengan mempertimbangkan keterbatasan ruang dan kapasitas berat. Agen diharapkan dapat memilik aksi terbaik berdasarkan *state* saat ini untuk mencapai kondisi optimal.

Untuk mencapai tujuan tersebut, digunakan pendekatan *local search*. *Local search* cocok untuk masalah yang tidak membutuhkan pencarian jalur menuju solusi dan hanya berorientasi pada pencarian *final state* terbaik. Pencarian dimulai dari *initial state* yang dibangkitkan secara acak, kemudian dieksplorasi terhadap *state* tetangga untuk memperoleh nilai *objective function* yang lebih baik. Perpindahan antar-*state* dibatasi pada tiga jenis langkah, yaitu menukar dua paket, memindahkan sebuah paket ke koordinat yang berbeda, dan memutar paket 90 derajat terhadap sumbu x, y, atau z. Setiap *state* harus memenuhi batasan yang ditetapkan, yaitu paket tidak boleh melayang atau menembus dinding peti kemas, barang pecah-belah tidak boleh menjadi penyangga paket lain, dan total berat muatan tidak boleh melebihi kapasitas truk.

Dalam tugas besar 1, dilakukan implementasi beberapa varian algoritma *local search*, yaitu *Hill-climbing, Simulated Annealing*, dan *Genetic Algorithm*. Setiap algoritma memiliki karakteristik dan strategi pencarian yang berbeda. *Hill-climbing* berfokus pada perbaikan bertahap menuju solusi terbaik, *Simulated-Annealing* mengombinasikan *Hill-climbing* dengan probabilitas berpindah ke solusi yang lebih buruk untuk menghindari *local optimum*, serta *Genetic Algorithm* yang mempertahankan sekumpulan solusi dan memperbaruinya melalui proses seleksi, *crossover*, dan mutasi. Hasil dari setiap algoritma kemudian dianalisis berdasarlan nilai *objective function* akhir, durasi pencarian, dan konsistensi hasil dari beberapa kali percobaan.

Bab 2

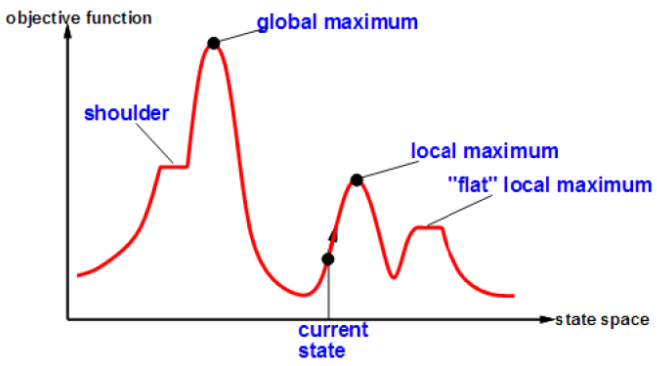
Dasar Teori

Local search merupakan pendekatan dalam kecerdasan buatan yang mencari solusi dengan berpindah dari satu *state* ke *state* tetangganya. *Classical search*, seperti BFS dan DFS, mementingkan jalur menuju solusi. Sedangkan *local search* hanya mempertahankan satu *state* aktif sehingga cocok untuk masalah yang hasil akhirnya lebih penting daripada urutan langkahnya. Setiap *state* merepresentasikan konfigurasi solusi yang lengkap. Perpindahan dilakukan dengan menghasilkan *successor* atau *neighbor* dari *state* saat ini, lalu kualitasnya dinilai menggunakan fungsi objektif. Algoritma akan memaksimalkan fungsi objektif atau meminimalkan fungsi biaya. Tujuan utama dari *local search* adalah untuk menemukan *global optimum*. Namun, pencarian dapat terhenti atau terjebak pada *local optimum*, yaitu *state* yang lebih baik daripada semua tetangganya tetapi bukan yang terbaik secara keseluruhan. Untuk mengatasi permasalahan ini, sebagian algoritma menambahkan mekanisme seperti langkah acak, penerimaan solusi yang sementara lebih buruk, atau pengembangan banyak kandidat sekaligus.

2.1 Algoritma *Hill Climbing*

2.1.1 *Steepest Ascent Hill Climbing*

Steepest ascent hill-climbing adalah varian dasar algoritma pencarian *hill-climbing*, yaitu algoritma pencarian lokal yang bergerak secara terus-menerus ke arah nilai yang lebih baik, baik berupa kenaikan nilai objektif maupun penurunan nilai biaya (*cost*).



Gambar 2.1: Grafik Fungsi Objektif terhadap State Space pada Steepest Ascent HC

Algoritma diawali dari sebuah *initial state* yang dibangkitkan secara acak. Pada setiap iterasi, algoritma membangkitkan seluruh *successor* dari *state* saat ini, lalu memilih *neighbor* yang merupakan *highest-valued successor*, yaitu *successor* dengan nilai tertinggi. Jika terdapat lebih dari satu *successor* terbaik, pemilihan dilakukan secara acak di antara *successor* tersebut. Berikut adalah pseudocode algoritma *steepest ascent hill-climbing*:

Algorithm 1: Hill-Climbing (Steepest Ascent)

Input: *problem*
Output: state yang merupakan local maximum

```
current ← MAKE-NODE(problem.INITIAL-STATE)
while true do
    neighbor ← a highest-valued successor of current
    if neighbor.VALUE ≤ current.VALUE then
        return current.STATE
    current ← neighbor
```

Alur kerja algoritma *steepest ascent HC* terdiri dari tiga bagian. Pertama, inisialisasi dengan *state* acak. Kedua, *loop* yang terus berpindah ke *neighbor* terbaik selama nilainya lebih tinggi daripada *state* saat ini. Ketiga, terminasi ketika tidak ada *neighbor* yang memiliki nilai lebih tinggi dari *state* saat ini. Kondisi ini mencakup puncak maupun area datar, karena kondisi berhenti menggunakan operator $\leq$.

2.1.2 *Hill Climbing with Sideways Move*

Hill-climbing with sideways move adalah salah satu varian dari *steepest ascent hill-climbing* yang dirancang untuk mengatasi kegagalan algoritma dasar pada kondisi *shoulder*. Pada *steepest ascent HC*, pencarian berhenti ketika tidak ada *neighbor* yang bernilai lebih tinggi dari *state* saat ini. Kondisi berhenti ini juga terpicu pada area datar, padahal pada area tersebut, seperti *shoulder*, masih terdapat kemungkinan untuk mencapai nilai yang lebih baik jika pencarian dilanjutkan.

Algoritma HC dengan *sideways move* ini mengizinkan algoritma berpindah ke *neighbor* yang nilainya sama dengan *state* saat ini, yang disebut *sideways move*. Perubahan ini diwujudkan dengan mengganti kondisi terminasi dari $neighbor.Value \leq current.Value$ menjadi $neighbor.Value < current.Value$. Dengan perubahan tersebut, algoritma hanya berhenti apabila seluruh *neighbor* memiliki nilai yang benar-benar lebih rendah, sehingga perpindahan ke *neighbor* bernilai sama tetap diperbolehkan.

Perpindahan sideways tanpa batas dapat menyebabkan *infinite loop*, yaitu ketika algoritma terus berpindah antar-*state* bernilai sama pada *flat local maximum* yang bukan *shoulder*. Maka, jumlah *sideways move* yang dilakukan secara berurutan perlu dibatasi dengan suatu nilai batas. Apabila batas tersebut tercapai, algoritma dihentikan dan mengembalikan *state* saat ini. Nilai limit menjadi parameter yang memengaruhi keseimbangan antara peluang keberhasilan dan waktu komputasi. Berikut adalah pseudocode algoritma *hill-climbing with sideways move*:

Algorithm 2: Hill-Climbing dengan Sideways Move

Input: *problem*, *maxSideways*

Output: state yang merupakan local maximum

```
current ← MAKE-NODE(problem.INITIAL-STATE)
sideways ← 0
while true do
    neighbor ← a highest-valued successor of current
    if neighbor.VALUE < current.VALUE then
        return current.STATE
    if neighbor.VALUE = current.VALUE then
        sideways ← sideways + 1
        if sideways > maxSideways then
            return current.STATE
    else
        sideways ← 0
    current ← neighbor
```

2.1.3 *Random Restart Hill Climbing*

Random restart hill-climbing adalah salah satu varian *hill-climbing* yang mengatasi masalah terjebak pada optimum lokal dengan cara mengulang pencarian dari *state* awal yang berbeda. Algoritma ini menjalankan serangkaian *hill-climbing*, dan setiap pencarian dimulai dari *initial state* yang dibangkitkan secara acak. Proses diulang hingga ditemukan *state* yang memenuhi tujuan, yaitu *state* dengan nilai *global maximum*. Jika pencarian dimulai dari area yang mengarah ke *local maximum*, algoritma akan berhenti di sana dan tidak dapat keluar. Dengan memulai ulang dari *state* acak yang berbeda, setiap percobaan memiliki peluang tersendiri untuk berada pada area yang mengarah ke *global maximum*. Semakin banyak percobaan dilakukan, semakin besar peluang setidaknya satu di antaranya berhasil. Setiap percobaan merupakan

hill-climbing biasa dan bersifat independen terhadap percobaan sebelumnya.

Kinerja algoritma ini dapat dianalisis dengan probabilitas. Misalkan p adalah probabilitas sebuah percobaan hill-climbing berhasil mencapai solusi, s adalah rata-rata jumlah langkah pada percobaan yang berhasil, dan f adalah rata-rata jumlah langkah pada percobaan yang gagal. Maka jumlah *restart* yang diharapkan hingga ditemukan solusi adalah

$$E[\text{percobaan}] = \frac{1}{p} \tag{2.1}$$

dan jumlah langkah yang diharapkan adalah

$$E[\text{langkah}] = s + \frac{f(1-p)}{p} \tag{2.2}$$

Persamaan kedua diperoleh dari dua komponen. Rata-rata terdapat $\frac{1-p}{p}$ percobaan gagal sebelum percobaan pertama yang berhasil, dan tiap percobaan gagal membutuhkan f langkah. Percobaan terakhir yang berhasil menambah s langkah. Dari persamaan tersebut terlihat bahwa nilai p yang kecil membuat jumlah restart membesar, tetapi total langkah tetap dapat rendah selama setiap percobaan berlangsung singkat. Hill-climbing yang cepat berhenti sehingga cocok digunakan secara berulang.

Algoritma ini juga bersifat *complete* secara probabilistik. Selama $p > 0$, solusi pada akhirnya akan ditemukan dengan probabilitas mendekati 1 jika jumlah restart tidak dibatasi. Namun, waktu komputasi tidak dapat dipastikan karena bergantung pada keberuntungan pembangkitan state awal. Berikut adalah pseudocode algoritma *random restart hill climbing*

Algorithm 3: Random Restart Hill-Climbing

Input: *problem*

Output: state dengan nilai global maximum

```
while true do
    current ← MAKE-NODE(RANDOM-STATE(problem))
    while true do
        neighbor ← a highest-valued successor of current
        if neighbor.VALUE ≤ current.VALUE then
            break
        current ← neighbor
    if current.VALUE = problem.GLOBAL-MAXIMUM then
        return current.STATE
```

2.1.4 *Stochastic Hill Climbing*

Stochastic hill-climbing adalah salah satu varian *hill-climbing* yang tidak mengevaluasi seluruh *successor* dari *state* saat ini. Pada *steepest ascent HC*, algoritma membangkitkan seluruh *successor* lalu memilih yang bernilai tertinggi. Pada varian ini, algoritma hanya membangkitkan satu *successor* secara acak, kemudian memutuskan apakah perpindahan dilakukan berdasarkan perbandingan nilainya dengan *state* saat ini. Algoritma akan berpindah ke *neighbor* acak tersebut hanya jika nilainya lebih baik daripada *state* saat ini. Jika *neighbor* yang terpilih lebih buruk atau sama, *neighbor* tersebut diabaikan dan algoritma mencoba membangkitkan *successor* acak yang lain.

Algoritma tidak dapat menentukan bahwa ia telah mencapai puncak hanya dari satu kali percobaan yang gagal, karena tidak ada jaminan bahwa *successor* acak yang dipilih lebih baik. Oleh karena itu, kondisi terminasi adalah tercapainya jumlah iterasi maksimum (n_{max}). Algoritma berhenti setelah n_{max} iterasi dan mengembalikan *state* terakhir yang dicapai. Dibandingkan dengan *steepest ascent*, *stochastic hill-climbing* memerlukan biaya per iterasi yang lebih rendah karena hanya satu *successor* yang dievaluasi, tetapi secara keseluruhan membutuhkan lebih banyak langkah untuk mencapai hasil yang sebanding sehingga berjalan lebih lambat. Algoritma ini tetap dapat terjebak pada *local maximum*, karena hanya menerima perpindahan yang memperbaiki nilai. Berikut adalah pseudocode algoritma *stochastic hill-climbing*:

Algorithm 4: Stochastic Hill-Climbing

Input: *problem*, n_{max}

Output: state hasil pencarian

```
current ← MAKE-NODE(problem.INITIAL-STATE)
for i ← 1 to n_max do
    neighbor ← a random successor of current
    if neighbor.VALUE > current.VALUE then
        current ← neighbor
return current.STATE
```

2.2 Algoritma *Simulated Annealing*

Simulated annealing adalah algoritma *local search* berupa versi *stochastic hill-climbing* yang mengizinkan sebagian langkah buruk agar dapat keluar dari *local maximum*. Frekuensi langkah buruk ini dikurangi secara bertahap selama pencarian berlangsung. Nama algoritma ini berasal dari proses *annealing* pada metalurgi, yaitu pemanasan logam lalu pendinginan perlahan. Analoginya, parameter T berperan sebagai "temperatur" yang bernilai tinggi di awal dan menurun mengikuti jadwal pendinginan, yaitu fungsi yang memetakan waktu t ke nilai T.

Pada setiap iterasi, algoritma memilih satu successor acak dari *state* saat ini, lalu menghitung selisih nilainya, yaitu $\Delta E = next.\text{VALUE} - current.\text{VALUE}$. Jika $\Delta E > 0$, *neighbor* lebih baik dan algoritma selalu berpindah. Jika $\Delta E < 0$, *neighbor* lebih buruk, tetapi algoritma tetap dapat berpindah dengan probabilitas $e^{\Delta E/T}$. Probabilitas ini makin kecil ketika langkah semakin buruk dan ketika T semakin rendah. Akibatnya, pada T tinggi algoritma berperilaku seperti *random walk*, sedangkan pada T mendekati 0 algoritma berperilaku seperti *stochastic hill-climbing*. Algoritma berhenti ketika $T = 0$ dan mengembalikan *state* saat ini.

$$\Delta E = next.\text{VALUE} - current.\text{VALUE} \tag{2.3}$$

Keputusan perpindahan bergantung pada tanda ΔE. Jika $\Delta E > 0$, *neighbor* lebih baik dan algoritma selalu berpindah. Jika $\Delta E < 0$, *neighbor* lebih buruk, tetapi algoritma tetap dapat berpindah dengan probabilitas

$$P = e^{\Delta E/T} \tag{2.4}$$

Karena $\Delta E < 0$, nilai P selalu berada di antara 0 dan 1. Probabilitas ini menurun ketika langkah semakin buruk (nilai $|\Delta E|$ makin besar) dan ketika temperatur semakin rendah. Pada temperatur tinggi, P mendekati 1 sehingga hampir semua langkah buruk diterima. Pada temperatur mendekati 0, P mendekati 0 sehingga langkah buruk hampir tidak pernah diterima.

Untuk menentukan apakah langkah buruk diterima, nilai P dibandingkan dengan suatu nilai pembanding. Terdapat dua pendekatan, yaitu membandingkan dengan nilai statis (misalnya langkah diterima jika $P > 0{,}5$), atau membandingkan dengan bilangan acak pada rentang $[0, 1]$ (langkah diterima jika $P > \text{random}(0, 1)$). Pendekatan kedua lebih sesuai dengan sifat stokastik algoritma karena langkah buruk yang kecil tetap memiliki peluang diterima secara proporsional dengan nilai P.

Algoritma berhenti ketika $T = 0$ dan mengembalikan *state* saat ini. Secara teoritis, apabila T diturunkan cukup lambat, *simulated annealing* akan menemukan global optimum dengan probabilitas mendekati 1. Kelemahannya, penurunan temperatur yang lambat membuat waktu pencarian lebih lama, sehingga pemilihan jadwal pendinginan menjadi parameter penting. Berikut adalah pseudocode algoritma *simulated annealing*:

Algorithm 5: Simulated Annealing

Input: *problem*, *schedule*
Output: state solusi

```
current ← MAKE-NODE(problem.INITIAL-STATE)
for t ← 1 to ∞ do
    T ← schedule(t)
    if T = 0 then
        return current.STATE
    next ← a randomly selected successor of current
    ΔE ← next.VALUE − current.VALUE
    if ΔE > 0 then
        current ← next
    else
        current ← next only with probability e^(ΔE/T)
```

2.3 Algoritma *Genetic Algorithm*

Genetic algorithm (GA) adalah algoritma *local search* yang mempertahankan sekumpulan *state* (populasi). Genetic algorithm membangkitkan *successor* melalui penggabungan dua *state* induk (*parent*). Setiap *state*, yang disebut individu, direpresentasikan sebagai *string* pada alfabet berhingga, umumnya *string* bernilai 0 dan 1. Setiap karakter pada *string* merepresentasikan satu variabel permasalahan. Kualitas setiap individu dinilai oleh fungsi evaluasi yang disebut fungsi kebugaran (*fitness function*), dengan nilai yang lebih tinggi menunjukkan *state* yang lebih baik. Fungsi *fitness* harus dirancang sesuai permasalahan, dan nilai maksimumnya menjadi global maximum.

Algoritma diawali dengan populasi awal berisi k individu yang dibangkitkan secara acak. Generasi berikutnya dibentuk melalui tiga operasi berikut.

1. **Selection**

   Dua individu induk dipilih secara acak dengan probabilitas proporsional terhadap nilai *fitness*. Probabilitas terpilihnya individu i adalah

   $$P(i) = \frac{F(i)}{\sum_{j=1}^{k} F(j)} \tag{2.5}$$

   Individu dengan *fitness* lebih tinggi lebih berpeluang terpilih, tetapi individu dengan *fitness* rendah tetap memiliki peluang. Implementasinya dapat berupa *roulette wheel*, yaitu membangkitkan bilangan acak pada rentang $[0, 100]$ lalu mencocokkannya dengan interval kumulatif probabilitas setiap individu.

2. **Crossover**

   Sebuah titik potong (*crossover point*) dipilih secara acak pada string. Anak dibentuk dengan menggabungkan bagian awal *string* induk pertama dan bagian akhir *string* induk kedua. Jika dilakukan pada kedua arah, dihasilkan dua anak.

3. **Mutation**

   Dengan probabilitas kecil, satu karakter pada *string* anak diubah secara acak pada titik mutasi (*mutation point*). Mutasi menjaga keragaman populasi dan memungkinkan munculnya karakteristik yang tidak dimiliki kedua induk.

Proses selection, crossover, dan mutation diulang hingga terbentuk populasi baru

berukuran sama dengan populasi sebelumnya, kemudian populasi lama digantikan. Iterasi berlanjut hingga ada individu yang cukup fit atau waktu yang ditentukan habis. Algoritma mengembalikan individu terbaik pada populasi menurut fungsi *fitness*. Berikut adalah pseudocode algoritma *genetic algorithm*:

Algorithm 6: Genetic Algorithm

Input: *population*, FITNESS-FN

Output: individu terbaik

```
repeat
    new_population ← ∅
    for i ← 1 to SIZE(population) do
        x ← RANDOM-SELECTION(population, FITNESS-FN)
        y ← RANDOM-SELECTION(population, FITNESS-FN)
        child ← REPRODUCE(x, y)
        if probabilitas acak kecil terpenuhi then
            child ← MUTATE(child)
        tambahkan child ke new_population
    population ← new_population
until ada individu yang cukup fit atau waktu habis
return individu terbaik pada population menurut FITNESS-FN
```

Bab 3

Implementasi

3.1 Arsitektur dan Struktur Program

Program pencarian konfigurasi pemuatan paket ke dalam peti kemas truk ini dibangun menggunakan bahasa pemrograman Python dengan pendekatan berorientasi objek. Entitas masalah, representasi solusi (*placement* dan *state*), dan ketiga algoritma *local search* dipisahkan ke dalam modul masing-masing. Pemisahan ini memungkinkan seluruh algoritma memakai fungsi pembangkit *state* awal, pembangkit tetangga, validasi *constraint*, dan *objective function* yang sama, sehingga perbandingan antar algoritma dilakukan pada kondisi yang setara. Perhitungan inti hanya memakai pustaka standar Python (json, random, math, time, dan argparse); pustaka pandas dan plotly hanya dipakai oleh modul visualisasi.

Tabel 3.1: Modul program

Modul	**Deskripsi**
main.py	Program utamayang membaca berkas masukan, memilih algoritma dan parameternya, menjalankan setiap algoritma beberapa kali dengan *seed* berbeda, lalu menyimpan hasil tiap *run* sebagai berkas JSON di output/<nama tc>/.
reader.py	Membaca berkas JSON masukan dan membentuk objek Problem yang berisi Truck dan daftar Package.
truck.py, package.py, problem.py	Merepresentasikan entitas masalah: dimensi dan kapasitas berat truk, atribut paket (dimensi, *value*, berat, status *fragile*, ETA), serta kumpulan keduanya. package.py juga menyimpan daftar enam orientasi paket.
placement.py, state.py	Placement menyimpan posisi dan orientasi satu paket, sedangkan State menyimpan Placement seluruh paket.

Modul	Deskripsi
constraints.py	Memvalidasi sebuah *state* terhadap seluruh *constraint* (dinding, *overlap*, kapasitas berat, paket melayang, dan paket *fragile*).
objective.py	Menghitung *objective function*, yaitu total *value* paket yang berada di dalam truk.
initialization.py	Membangkitkan *state* awal acak yang valid untuk *Hill-climbing* dan *Simulated Annealing*.
neighbors.py	Membangkitkan tetangga valid dari sebuah *state* dengan operator *move*, *rotate*, dan *swap*, baik secara menyeluruh maupun secara acak.
common.py	Menyusun format hasil *run* yang seragam untuk semua algoritma.
hill_climb.py	Implementasi *Hill-climbing* dengan empat varian: *steepest ascent*, *sideways move*, *random restart*, dan *stochastic*.
simulated_annealing.py	Implementasi *Simulated Annealing* dengan jadwal pendinginan geometris dan pencatatan kejadian *stuck*.
genetic_algorithm.py	Implementasi *Genetic Algorithm*, yaitu pencarian berbasis populasi dengan *roulette wheel selection*, *uniform crossover*, dan *mutation*.
visualize.py	Membuat satu halaman HTML interaktif per berkas hasil, berisi tampilan 3D *state*, peta sel per lapisan, tabel paket, metrik, dan grafik riwayat.
visualization_charts.py	Membuat grafik hasil eksperimen dengan plotly, yaitu algorithm_view untuk satu *run* dan experiment_view untuk kumpulan *run*.

Alur eksekusi program adalah sebagai berikut:

1. main.py memuat masalah dengan load_problem,
2. Untuk setiap algoritma dan setiap *run*, program membuat generator acak ran-

dom.Random(seed) dengan *seed* bernilai seed awal + nomor run - 1, sehingga setiap eksperimen dapat diulang.

3. Pada algoritma *Hill-climbing* dan *Simulated Annealing*, *state* awal dibangkitkan lebih dahulu dengan generate_initial_state, sedangkan pada algoritma *Genetic Algorithm*, program membangkitkan populasi awalnya sendiri.

4. Hasil setiap *run* (nilai awal dan akhir, jumlah iterasi, durasi, alasan berhenti, dan riwayat nilai per iterasi) disimpan dalam satu berkas JSON.

3.2 Representasi State, Fungsi Objektif, dan Constraints

3.2.1 Representasi State

Peti kemas truk dimodelkan sebagai ruang tiga dimensi diskret berukuran $W \times L \times H$ satuan, dengan sumbu x sepanjang lebar (w), sumbu y sepanjang panjang (l), dan sumbu z sepanjang tinggi (h). Setiap paket berbentuk balok dengan dimensi bilangan bulat. Posisi sebuah paket adalah koordinat (x, y, z) dari sudut balok dengan nilai koordinat terkecil, sehingga paket yang diletakkan pada posisi tersebut dengan dimensi efektif (d_x, d_y, d_z) menempati rentang $[x, x+d_x) \times [y, y+d_y) \times [z, z+d_z)$.

Satu solusi atau *state* direpresentasikan oleh objek State yang memuat sebuah *dictionary* placements dengan bentuk

placements[id_paket] = Placement(position, orientation)

untuk *seluruh* paket pada masalah, baik yang dimuat maupun tidak. Atribut position berupa list [x, y, z] bila paket dimuat ke dalam truk, dan bernilai None bila paket tidak dimuat. Atribut orientation berupa permutasi dari sumbu ("x", "y", "z") yang menyatakan sumbu truk yang dipetakan berturut-turut oleh dimensi w, l, dan h milik paket.

Terdapat enam permutasi yang mungkin berdasarkan konstanta ORIENTATIONS pada package.py, misalnya orientasi ("x", "z", "y") berarti panjang paket sejajar sumbu z dan tingginya sejajar sumbu y. Dimensi efektif paket pada sumbu truk diperoleh dari Package.effective_dims. Dengan representasi ini, memuat atau mengeluarkan paket cukup dilakukan dengan mengubah position antara koordinat dan None.

Selain itu, State menyediakan metode copy untuk menyalin *state*, key untuk mengubah *state* menjadi *tuple* terurut yang dapat di-*hash* (untuk menghindari tetangga duplikat), dan to_dict untuk serialisasi ke JSON.

3.2.2 Fungsi Objektif

Tujuan masalah adalah memaksimalkan total *value* paket yang berhasil dimuat ke dalam truk. Misalkan v_i adalah *value* paket i dan pos_i adalah posisinya pada *state* S, maka *objective function* yang dimaksimalkan adalah

$$f(S) = \sum_{i\,:\,\text{pos}_i \neq None} v_i. \tag{3.1}$$

Fungsi ini diimplementasikan oleh evaluate pada objective.py (Kode 3.1). Karena seluruh *state* yang dikunjungi algoritma dijaga selalu memenuhi *constraint* (lihat Section 3.2.3 dan Section 3.3), *objective function* tidak memerlukan suku penalti pelanggaran. Pada *Genetic Algorithm*, nilai $f(S)$ digunakan langsung sebagai nilai *fitness* karena $f(S) \geq 0$ dan semakin besar semakin baik.

```
def evaluate(problem, state):
    values = {p.id: p.value for p in problem.packages}
    return sum(
        values[pid] for pid, p in state.placements.items() if p.position is
 not None
    )
```

Kode 3.1: Fungsi objektif pada objective.py

3.2.3 Constraints

Sebuah *state* dinyatakan valid apabila memenuhi lima *constraint* pada Table 3.2. Seluruh pengecekan dilakukan oleh validate_state pada constraints.py yang mengembalikan pasangan (valid, violations), dengan violations berisi daftar pesan pelanggaran. Setiap paket yang dimuat lebih dahulu diubah menjadi kotak (x, y, z, d_x, d_y, d_z) oleh collect_boxes; paket yang menembus dinding dicatat sebagai pelanggaran dan tidak diikutsertakan pada pengecekan berikutnya.

Tabel 3.2: *Constraint* dan fungsi pengecekannya

Fungsi	**Deskripsi**
collect_boxes(problem, state, pkgs, violations)	**Tidak menembus dinding.** Paket harus berada sepenuhnya di dalam truk: $x, y, z \geq 0$ serta $x + d_x \leq W$, $y + d_y \leq L$, dan $z + d_z \leq H$.
check_overlap(inside, violations)	**Tidak saling tumpang tindih.** Dua paket *overlap* bila rentang keduanya beririsan pada sumbu x, y, dan z sekaligus (box_overlap). Rentang dianggap setengah terbuka sehingga paket yang hanya bersentuhan pada sisinya tidak dihitung *overlap*.
check_capacity(problem, inside, pkgs, violations)	**Kapasitas berat.** Total berat seluruh paket yang dimuat tidak boleh melebihi max_capacity truk.
check_floating(inside, pkgs, violations)	**Tidak melayang.** Paket dengan $z = 0$ berada di lantai. Paket dengan $z > 0$ harus ditopang paling sedikit satu paket *non-fragile* yang sisi atasnya tepat berada pada ketinggian z dan proyeksi alasnya beririsan dengan proyeksi alas paket tersebut (footprint_overlap).
check_fragile(inside, pkgs, violations)	**Paket *fragile* tidak boleh tertindih.** Untuk setiap paket *fragile*, tidak boleh ada paket lain yang proyeksi alasnya beririsan dan posisi dasarnya berada di atas atau tepat pada sisi atas paket *fragile* tersebut. Bersama check_floating, hal ini membuat paket *fragile* tidak dapat menjadi penyangga.
validate_state(problem, state)	Menjalankan semua pengecekan di atas dan menyatakan *state* valid hanya jika violations kosong.

```
def overlap_1d(a0, a1, b0, b1):
    return max(a0, b0) < min(a1, b1)


```

```
def footprint_overlap(box_a, box_b):
    xa, ya, _, dxa, dya, _ = box_a
    xb, yb, _, dxb, dyb, _ = box_b
    return overlap_1d(xa, xa + dxa, xb, xb + dxb) and overlap_1d(ya, ya + dya, yb, yb + dyb)


def box_overlap(box_a, box_b):
    _, _, za, _, _, dza = box_a
    _, _, zb, _, _, dzb = box_b
    return footprint_overlap(box_a, box_b) and overlap_1d(za, za + dza, zb, zb + dzb)


def collect_boxes(problem, state, pkgs, violations):
    truck = problem.truck
    inside = {}

    for pid, p in state.placements.items():
        if p.position is None:
            continue
        x, y, z = p.position
        dx, dy, dz = pkgs[pid].effective_dims(p.orientation)
        if not (x >= 0 and y >= 0 and z >= 0 and x + dx <= truck.w and y + dy <= truck.l and z + dz <= truck.h):
            violations.append(f"{pid}: menembus dinding")
            continue
        inside[pid] = (x, y, z, dx, dy, dz)

    return inside


def check_overlap(inside, violations):
    ids = list(inside)
    for a in range(len(ids)):
        for b in range(a + 1, len(ids)):
            i, j = ids[a], ids[b]
            if box_overlap(inside[i], inside[j]):
                violations.append(f"{i}/{j}: overlap")

```

```
42
43  def check_capacity(problem, inside, pkgs, violations):
44      totalW = sum(pkgs[pid].weight for pid in inside)
45      if totalW > problem.truck.max_capacity:
46          violations.append(f"overload {totalW} > {problem.truck.max_capacity
    }")
47
48
49  def check_floating(inside, pkgs, violations):
50      for pid, box in inside.items():
51          z = box[2]
52          if z == 0:
53              continue
54          supported = any(
55              not pkgs[q].is_fragile
56              and other[2] + other[5] == z
57              and footprint_overlap(box, other)
58              for q, other in inside.items()
59              if q != pid
60          )
61          if not supported:
62              violations.append(f"{pid}: melayang")
63
64
65  def check_fragile(inside, pkgs, violations):
66      for pid, box in inside.items():
67          if not pkgs[pid].is_fragile:
68              continue
69          top = box[2] + box[5]
70          for q, other in inside.items():
71              if q == pid:
72                  continue
73              if footprint_overlap(box, other) and other[2] >= top:
74                  violations.append(f"{pid}: fragile tertindih {q}")
75                  break
76
77
78  def validate_state(problem, state):
79      pkgs = problem.package_map
80      violations = []
```

```
81
82      inside = collect_boxes(problem, state, pkgs, violations)
83      check_overlap(inside, violations)
84      check_capacity(problem, inside, pkgs, violations)
85      check_floating(inside, pkgs, violations)
86      check_fragile(inside, pkgs, violations)
87
88      return (len(violations) == 0, violations)
```

Kode 3.2: Validasi *constraint* pada constraints.py

3.3 Representasi Neighbor dan State Awal

3.3.1 Pembangkitan State Awal

Pada *Hill-climbing* dan *Simulated Annealing*, *state* awal dibangkitkan oleh generate_initial_state pada initialization.py (Kode 3.3). Fungsi ini dimulai dari *state* kosong, yaitu semua paket berposisi None. Urutan paket diacak, dan untuk setiap paket urutan keenam orientasinya juga diacak. Pada setiap orientasi, seluruh posisi bulat yang membuat paket muat di dalam dimensi truk (candidate_positions) diacak, lalu dicoba satu per satu; posisi pertama yang membuat seluruh *state* tetap valid menurut validate_state dipakai. Jika tidak ada kombinasi orientasi dan posisi yang valid, paket dibiarkan di luar truk. Dengan demikian, *state* awal selalu valid, tetapi tidak dioptimalkan sama sekali dan dapat berbeda pada setiap *seed*.

```
1  from src.constraints import validate_state
2  from src.neighbors import candidate_positions
3  from src.package import ORIENTATIONS
4  from src.placement import Placement
5  from src.state import State
6
7
8  def generate_initial_state(problem, rng):
9      pkgs = problem.package_map
10     state = State(
11         {pid: Placement(None, ["x", "y", "z"]) for pid in pkgs}
12     )
13     order = list(pkgs)
14     rng.shuffle(order)
```

```
    for pid in order:
        oris = list(ORIENTATIONS)
        rng.shuffle(oris)
        placed = False
        for ori in oris:
            positions = list(candidate_positions(problem, pkgs[pid], ori))
            rng.shuffle(positions)
            for pos in positions:
                cand = state.copy()
                cand.placements[pid] = Placement(pos, list(ori))
                valid, _ = validate_state(problem, cand)
                if valid:
                    state = cand
                    placed = True
                    break
            if placed:
                break
    return state
```

Kode 3.3: Pembangkitan *state* awal pada initialization.py

Pada *Genetic Algorithm*, *state* awal berupa populasi yang tiap individunya adalah *state* lengkap. Individu dibangkitkan oleh metode GeneticAlgorithm._generate_initial_state yang menyisipkan paket secara acak satu per satu dengan pengecekan *constraint* inkremental. Individu terbaik pada populasi awal dicatat sebagai initial_state pada hasil *run*.

3.3.2 Operasi Tetangga

Tetangga dari sebuah *state* adalah *state* baru yang diperoleh dengan tepat satu langkah perubahan terhadap *state* tersebut dan memenuhi seluruh *constraint*. Terdapat tiga operator pada neighbors.py:

1. **Move**: memindahkan satu paket ke koordinat (x, y, z) lain dengan orientasi tetap. Jika paket yang dipilih sebelumnya berada di luar truk, operator ini sekaligus memuatnya ke dalam truk.

2. **Rotate**: merotasi satu paket sebesar 90° terhadap sumbu x, y, atau z dengan posisi tetap. Rotasi terhadap suatu sumbu menukar dua sumbu lainnya pada orientation (Placement.rotate).

3. **Swap**: menukar posisi dua paket, baik keduanya di dalam truk maupun salah satunya di luar truk. Orientasi masing-masing paket tidak berubah. Pasangan yang keduanya berada di luar truk dilewati karena tidak mengubah *state*.

Paket hanya dapat dikeluarkan dari truk melalui operator *swap* dengan paket yang berada di luar truk. Semua tetangga yang sama dengan *state* saat ini atau dengan tetangga yang sudah dihasilkan sebelumnya dibuang menggunakan himpunan seen yang berisi State.key(), dan tetangga yang melanggar *constraint* dibuang oleh is_new_valid.

Terdapat dua cara pembangkitan tetangga, yaitu:

- Fungsi iter_valid_neighbors menghasilkan *seluruh* tetangga valid secara berurutan (operator *move*, kemudian *rotate*, kemudian *swap*) dan digunakan oleh varian *Hill-climbing* yang memerlukan seluruh tetangga, yaitu *steepest ascent*, *sideways move*, dan *random restart*.

- Fungsi sample_random_neighbor menghasilkan *satu* tetangga valid secara acak dan digunakan oleh *stochastic Hill-climbing* serta *Simulated Annealing*: urutan ketiga operator diacak, lalu untuk setiap operator dicoba hingga max_attempts (bawaan 50) kandidat acak, dan kandidat valid pertama dikembalikan. Fungsi ini mengembalikan None apabila tidak ditemukan tetangga valid.

Pada *Genetic Algorithm*, konsep tetangga digantikan oleh operator *crossover* dan *mutation*. Operator *mutation* memakai tiga jenis perubahan yang sama dengan operator tetangga di atas.

```
from src.constraints import validate_state


def candidate_positions(problem, package, orientation):
    truck = problem.truck
    dx, dy, dz = package.effective_dims(orientation)
    W, L, H = truck.w, truck.l, truck.h
    if dx > W or dy > L or dz > H:
        return
    for x in range(W - dx + 1):
        for y in range(L - dy + 1):
            for z in range(H - dz + 1):
                yield [x, y, z]

```

```

def is_new_valid(problem, cand, seen):
    key = cand.key()
    if key in seen:
        return False
    seen.add(key)
    valid, _ = validate_state(problem, cand)
    return valid
```

Kode 3.4: Fungsi bantu pembangkit tetangga pada neighbors.py

```
def iter_valid_neighbors(problem, state):
    pkgs = problem.package_map
    ids = list(state.placements)
    seen = {state.key()}

    # Move: pindahkan satu paket ke koordinat lain, orientasi tetap
    for pid in ids:
        cur = state.placements[pid]
        for pos in candidate_positions(problem, pkgs[pid], cur.orientation)
:
            cand = state.copy()
            cand.placements[pid].position = pos
            if is_new_valid(problem, cand, seen):
                yield cand

    # Rotate: rotasi satu paket 90° terhadap sumbu x, y, atau z, posisi
tetap
    for pid in ids:
        for axis in ("x", "y", "z"):
            cand = state.copy()
            cand.placements[pid].rotate(axis)
            if is_new_valid(problem, cand, seen):
                yield cand

    # Swap: tukar posisi dua paket, di dalam maupun di luar truk
    for a in range(len(ids)):
        for b in range(a + 1, len(ids)):
            i, j = ids[a], ids[b]
            pi, pj = state.placements[i].position, state.placements[j].
```

```
position
            if pi is None and pj is None:
                continue
            cand = state.copy()
            cand.placements[i].position = list(pj) if pj is not None else
None
            cand.placements[j].position = list(pi) if pi is not None else
None
            if is_new_valid(problem, cand, seen):
                yield cand
```

Kode 3.5: Pembangkitan seluruh tetangga valid (iter_valid_neighbors)

```
# Ambil satu tetangga acak yang valid dari state
def sample_random_neighbor(problem, state, rng, max_attempts=50):
    pkgs = problem.package_map
    ids = list(state.placements)
    # tanpa paket tidak ada tetangga yang bisa dibangkitkan
    if not ids:
        return None
    seen = {state.key()}
    operators = [0, 1, 2]
    rng.shuffle(operators)

    for op in operators:
        for _ in range(max_attempts):
            if op == 0:
                # Move
                pid = rng.choice(ids)
                cur = state.placements[pid]
                positions = list(candidate_positions(problem, pkgs[pid],
cur.orientation))
                if not positions:
                    continue
                pos = rng.choice(positions)
                cand = state.copy()
                cand.placements[pid].position = pos
                if is_new_valid(problem, cand, seen):
                    return cand

```

```
            elif op == 1:
                # Rotate
                pid = rng.choice(ids)
                axis = rng.choice(("x", "y", "z"))
                cand = state.copy()
                cand.placements[pid].rotate(axis)
                if is_new_valid(problem, cand, seen):
                    return cand

            else:
                # Swap
                if len(ids) < 2:
                    continue
                i, j = rng.sample(ids, 2)
                pi = state.placements[i].position
                pj = state.placements[j].position
                if pi is None and pj is None:
                    continue
                cand = state.copy()
                cand.placements[i].position = list(pj) if pj is not None
else None
                cand.placements[j].position = list(pi) if pi is not None
else None
                if is_new_valid(problem, cand, seen):
                    return cand

    return None
```

Kode 3.6: Pengambilan satu tetangga acak valid (sample_random_neighbor)

3.4 Implementasi Kelas-kelas Pendukung

3.4.1 truck.py

Tabel 3.3: Kelas/fungsi pada truck.py

Kelas/Fungsi	Deskripsi
class Truck	Merepresentasikan peti kemas truk.

Kelas/Fungsi	Deskripsi
__init__(self, w, l, h, max_capacity)	Menyimpan dimensi truk (lebar w, panjang l, tinggi h) dan kapasitas berat maksimum max_capacity.

```python
class Truck:
    def __init__(self, w, l, h, max_capacity):
        self.w = w
        self.l = l
        self.h = h
        self.max_capacity = max_capacity
```

Kode 3.7: Kelas Truck

3.4.2 package.py

Tabel 3.4: Kelas/fungsi pada package.py

Kelas/Fungsi	Deskripsi
ORIENTATIONS	Daftar enam permutasi sumbu ("x", "y", "z") yang merepresentasikan seluruh orientasi balok.
class Package	Merepresentasikan satu paket beserta atributnya.
__init__(self, id, w, l, h, value, weight, is_fragile, eta)	Menyimpan identitas, dimensi, *value*, berat, status *fragile*, dan ETA paket. Atribut eta dibaca dari masukan tetapi tidak digunakan pada *objective function* maupun *constraint* implementasi ini.
effective_dims(self, orientation)	Mengembalikan dimensi paket (dx, dy, dz) pada sumbu x, y, z truk untuk sebuah orientasi, dengan memetakan dimensi w, l, h paket ke sumbu yang ditunjuk orientasi.

```python
ORIENTATIONS = [
    ("x", "y", "z"),
    ("x", "z", "y"),
```

```
    ("y", "x", "z"),
    ("z", "x", "y"),
    ("y", "z", "x"),
    ("z", "y", "x"),
]


class Package:
    def __init__(self, id, w, l, h, value, weight, is_fragile, eta):
        self.id = id
        self.w = w
        self.l = l
        self.h = h
        self.value = value
        self.weight = weight
        self.is_fragile = is_fragile
        self.eta = eta

    def effective_dims(self, orientation):
        w_axis, l_axis, h_axis = orientation
        size = {w_axis: self.w, l_axis: self.l, h_axis: self.h}
        return (size["x"], size["y"], size["z"])
```

Kode 3.8: Kelas Package

3.4.3 problem.py

Tabel 3.5: Kelas/fungsi pada problem.py

Kelas/Fungsi	Deskripsi
class Problem	Menyatukan truk dan seluruh paket menjadi satu instans masalah.
__init__(self, truck, packages)	Menyimpan objek Truck, daftar Package, dan package_map yang memetakan id paket ke objek paket untuk akses cepat.

```
class Problem:
    def __init__(self, truck, packages):
```

```
        self.truck = truck
        self.packages = packages
        self.package_map = {p.id: p for p in packages}
```

Kode 3.9: Kelas Problem

3.4.4 placement.py

Tabel 3.6: Kelas/fungsi pada placement.py

Kelas/Fungsi	Deskripsi
class Placement	Merepresentasikan penempatan satu paket.
__init__(self, position, orientation)	Menyimpan posisi [x, y, z] (atau None jika paket tidak dimuat) dan orientasi paket.
rotate(self, axis)	Merotasi paket 90° terhadap sumbu axis dengan menukar dua sumbu lainnya pada orientation.

```
class Placement:
    def __init__(self, position, orientation):
        self.position = position
        self.orientation = orientation

    def rotate(self, axis):
        # rotasi 90° terhadap suatu sumbu = dua sumbu lainnya bertukar
        a, b = {"x": ("y", "z"), "y": ("x", "z"), "z": ("x", "y")}[axis]
        self.orientation = [
            b if s == a else a if s == b else s
            for s in self.orientation
        ]
```

Kode 3.10: Kelas Placement

3.4.5 state.py

Tabel 3.7: Kelas/fungsi pada state.py

Kelas/Fungsi	**Deskripsi**
class State	Merepresentasikan satu solusi, yaitu penempatan seluruh paket.
__init__(self, placements)	Menyimpan *dictionary* id paket ke Placement.
copy(self)	Membuat salinan mendalam *state*, termasuk list posisi dan orientasi, agar perubahan pada tetangga tidak memengaruhi *state* asal.
key(self)	Mengubah *state* menjadi *tuple* terurut berdasarkan id paket yang dapat di-*hash*, sehingga dapat disimpan pada himpunan untuk mendeteksi *state* yang sama.
to_dict(self)	Mengubah *state* menjadi *dictionary* yang dapat diserialisasi ke JSON.

```python
from src.placement import Placement


class State:
    def __init__(self, placements):
        self.placements = placements

    def copy(self):
        return State({
            pid: Placement(
                list(p.position) if p.position is not None else None,
                list(p.orientation),
            )
            for pid, p in self.placements.items()
        })

    # menyimpan dict sebuah state ke dalam bentuk tuple
    def key(self):
        return tuple(
            sorted(
                (
```

```
                    pid,
                    tuple(p.position) if p.position is not None else None,
                    tuple(p.orientation),
                )
                for pid, p in self.placements.items()
            )
        )

    def to_dict(self):
        return {
            "placements": {
                pid: {"position": p.position, "orientation": p.orientation}
                for pid, p in self.placements.items()
            }
        }
```

Kode 3.11: Kelas State

3.4.6 reader.py

Tabel 3.8: Fungsi pada reader.py

Kelas/Fungsi	Deskripsi
load_problem(path)	Membuka berkas JSON pada path, membentuk objek Truck dari data truck (dimensi dan max_capacity), membentuk satu objek Package untuk setiap entri packages, lalu mengembalikan objek Problem.

```
import json

from src.package import Package
from src.problem import Problem
from src.truck import Truck


def load_problem(path):
    with open(path) as f:
```

```
        data = json.load(f)
    td = data["truck"]["dimensions"]
    truck = Truck(td["w"], td["l"], td["h"], data["truck"]["max_capacity"])
    packages = [
        Package(p["id"], p["dimensions"]["w"], p["dimensions"]["l"], p["
dimensions"]["h"],
                p["value"], p["weight"], p["is_fragile"], p["eta"])
        for p in data["packages"]
    ]
    return Problem(truck, packages)
```

Kode 3.12: Fungsi load_problem

3.4.7 common.py

Tabel 3.9: Fungsi pada common.py

Kelas/Fungsi	**Deskripsi**
ser(s)	Mengubah objek State menjadi *dictionary* melalui to_dict; nilai yang sudah berupa *dictionary* dikembalikan apa adanya.
build_result(...)	Menyusun *dictionary* hasil *run* dengan format seragam untuk semua algoritma: nama algoritma, parameter, *state* dan nilai awal, *state* dan nilai akhir, jumlah iterasi, jumlah perpindahan yang diterima, jumlah evaluasi *objective function*, durasi, alasan berhenti, riwayat per iterasi, dan informasi tambahan khusus algoritma (extra).

```
def ser(s):
    return s.to_dict() if hasattr(s, "to_dict") else s


def build_result(
    algorithm,
    parameters,
    initial_state,
```

```
    initial_value,
    final_state,
    final_value,
    iterations,
    accepted_moves,
    objective_evaluations,
    duration_seconds,
    stop_reason,
    history,
    extra,
):
    return {
        "algorithm": algorithm,
        "parameters": parameters,
        "initial_state": ser(initial_state),
        "final_state": ser(final_state),
        "initial_value": initial_value,
        "final_value": final_value,
        "iterations": iterations,
        "accepted_moves": accepted_moves,
        "objective_evaluations": objective_evaluations,
        "duration_seconds": duration_seconds,
        "stop_reason": stop_reason,
        "history": history,
        "extra": extra,
    }
```

Kode 3.13: Fungsi pada common.py

3.5 Implementasi Algoritma Hill-climbing

3.5.1 hill_climb.py

Seluruh varian *Hill-climbing* diimplementasikan sebagai metode pada kelas HillClimb dan memaksimalkan *objective function*. Ke-empat varian menerima *state* awal sebagai argumen dan mengembalikan hasil dalam format build_result.

Tabel 3.10: Kelas/fungsi pada hill_climb.py

Kelas/Fungsi	**Deskripsi**
class HillClimb	Pembungkus seluruh varian *Hill-climbing* untuk sebuah Problem.
__init__(self, problem, rng)	Menyimpan instans masalah dan generator acak rng yang digunakan untuk memilih di antara tetangga yang nilainya sama dan untuk pembangkitan tetangga acak.
steepest_ascent(self, initial_state)	Pada setiap iterasi, membangkitkan seluruh tetangga valid, lalu berpindah ke tetangga dengan nilai tertinggi bila nilainya lebih besar dari *state* saat ini. Jika beberapa tetangga bernilai tertinggi sama, salah satunya dipilih secara acak. Berhenti jika tidak ada tetangga yang lebih baik.
sideways(self, initial_state, max_sideways=10)	Serupa dengan *steepest ascent*, tetapi jika tidak ada tetangga yang lebih baik dan terdapat tetangga bernilai sama, pencarian tetap berpindah ke salah satunya (*sideways move*) selama jumlah *sideways move* belum mencapai max_sideways.
random_restart(self, initial_state, max_restart=3)	Menjalankan *steepest ascent* dari *state* awal yang diberikan, kemudian dari max_restart *state* awal acak baru, lalu mengambil hasil dengan nilai akhir tertinggi. Riwayat seluruh pencarian digabung pada satu sumbu iterasi global.
stochastic(self, initial_state, max_iterations=10_000)	Pada setiap iterasi mengambil satu tetangga acak valid dan berpindah hanya jika nilainya lebih baik. Berhenti setelah max_iterations iterasi atau jika tidak ditemukan tetangga valid.

Kelas HillClimb dan varian *steepest ascent* ditunjukkan pada Kode 3.14. Pada setiap iterasi, iter_valid_neighbors dievaluasi satu per satu; variabel best_value diinisialisasi dengan nilai *state* saat ini sehingga best_list hanya memuat tetangga yang nilainya lebih besar atau sama dengan *state* saat ini. Pencarian berhenti de-

ngan alasan no_improving_neighbor bila tidak ada tetangga yang lebih baik, atau no_valid_neighbors bila tidak ada satu pun tetangga yang nilainya sama maupun lebih baik.

```python
import time

from src.algorithms.common import build_result
from src.neighbors import iter_valid_neighbors, sample_random_neighbor
from src.objective import evaluate


class HillClimb:
    def __init__(self, problem, rng):
        self.problem = problem
        self.rng = rng

    def steepest_ascent(self, initial_state):
        # referensi: materi kuliah AI BeyondClassicalSearch bagian Stochastic Hill-climbing
        # setiap state menhidupi semua successor dan cari succesor yang menghasilkan state value terbaik. Jika tidak ada successor yang lebih baik, berhenti.
        t0 = time.perf_counter()
        start = initial_state.copy()
        current = initial_state.copy()
        current_value = evaluate(self.problem, current)
        evals = 1
        history = [{"iteration": 0, "current_value": current_value, "best_value": current_value}]
        accepted = 0
        iteration = 0

        while True:
            # cari semua tetangga yang valid, pilih yang terbaik
            best_value, best_list = current_value, []
            for cand in iter_valid_neighbors(self.problem, current):
                v = evaluate(self.problem, cand)
                evals += 1
                if v > best_value:
                    best_value, best_list = v, [cand]
```

```
                elif v == best_value:
                    best_list.append(cand)
            # jika ada tetangga yang lebih baik, pindah ke salah satunya. Jika tidak ada, berhenti.
            if best_value > current_value:
                current = self.rng.choice(best_list) if len(best_list) > 1 else best_list[0]
                current_value = best_value
                accepted += 1
                iteration += 1
                history.append(
                    {"iteration": iteration, "current_value": current_value, "best_value": current_value}
                )
            else:
                iteration += 1
                history.append(
                    {"iteration": iteration, "current_value": current_value, "best_value": current_value}
                )
                stop = "no_valid_neighbors" if not best_list else "no_improving_neighbor"
                break

        return build_result(
            "steepest_ascent", {}, start, history[0]["current_value"],
            current, current_value, iteration, accepted, evals,
            time.perf_counter() - t0, stop, history, {},
        )
```

Kode 3.14: Kelas HillClimb dan *steepest ascent*

Varian *sideways move* pada Kode 3.15 menambahkan penghitung sideways. Perpindahan ke tetangga bernilai sama hanya dilakukan bila tidak ada tetangga yang lebih baik dan penghitung belum mencapai max_sideways. Penghitung ini dihitung kumulatif sepanjang pencarian dan tidak di-*reset* ketika ditemukan perbaikan. Pencarian berhenti dengan alasan sideways_limit_reached ketika batas tercapai.

```
    def sideways(self, initial_state, max_sideways=10):
        # referensi: materi kuliah AI BeyondClassicalSearch bagian
```

```
Stochastic Hill-climbing
        # sama seperti steepest ascent, tapi jika ada tetangga yang sama baiknya dengan
        t0 = time.perf_counter()
        start = initial_state.copy()
        current = initial_state.copy()
        current_value = evaluate(self.problem, current)
        evals = 1
        history = [{"iteration": 0, "current_value": current_value, "best_value": current_value}]
        accepted = sideways = iteration = 0

        while True:
            best_value, best_list = current_value, []
            for cand in iter_valid_neighbors(self.problem, current):
                v = evaluate(self.problem, cand)
                evals += 1
                if v > best_value:
                    best_value, best_list = v, [cand]
                elif v == best_value:
                    best_list.append(cand)
            # jika ada tetangga yang lebih baik, pindah ke salah satunya. Jika tidak ada, tapi ada tetangga yang sama baiknya dan belum melewati batas sideways, pindah ke salah satunya. Jika tidak ada tetangga yang lebih baik atau sama baiknya, berhenti.
            if best_value > current_value:
                move, sideways_move = True, False
            # jika ada tetangga yang sama baiknya, pindah ke salah satunya, iterasi sideways bertambah 1.
            elif best_list and sideways < max_sideways:
                move, sideways_move = True, True
            else:
                move = False
            iteration += 1
            if not move:
                history.append(
                    {"iteration": iteration, "current_value": current_value, "best_value": current_value}
                )
                if not best_list:
```

```
                    stop = "no_valid_neighbors"
                elif sideways >= max_sideways:
                    stop = "sideways_limit_reached"
                else:
                    stop = "no_improving_neighbor"
                break
            current = self.rng.choice(best_list) if len(best_list) > 1 else
 best_list[0]
            current_value = best_value
            accepted += 1
            sideways += sideways_move
            history.append(
                {"iteration": iteration, "current_value": current_value, "
best_value": current_value,
                 "sideways": sideways_move}
            )
            if sideways_move and sideways >= max_sideways:
                stop = "sideways_limit_reached"
                break

        return build_result(
            "sideways", {"maximum_sideways_move": max_sideways},
            start, history[0]["current_value"], current, current_value,
            iteration, accepted, evals, time.perf_counter() - t0, stop,
history,
            {"maximum_sideways_move": max_sideways, "sideways_moves":
sideways},
        )
```

Kode 3.15: Varian *sideways move*

Varian *random restart* pada Kode 3.16 menjalankan *steepest ascent* sebanyak max_restart +1 kali: satu kali dari *state* awal yang diberikan dan sisanya dari *state* awal baru yang dibangkitkan generate_initial_state. Jumlah iterasi, perpindahan yang diterima, dan evaluasi dijumlahkan dari seluruh pencarian, sedangkan *state* akhir diambil dari pencarian dengan nilai akhir tertinggi.

```
    def random_restart(self, initial_state, max_restart=3):
        from src.initialization import generate_initial_state
        # referensi: materi kuliah AI BeyondClassicalSearch bagian
```

```
    Stochastic Hill-climbing
        # jalankan steepest ascent beberapa kali dari initial state yang berbeda, ambil hasil terbaik

        t0 = time.perf_counter()
        starts = [initial_state.copy()] + [
            generate_initial_state(self.problem, self.rng) for _ in range(max_restart)
        ]
        runs = [self.__class__(self.problem, self.rng).steepest_ascent(s) for s in starts]
        best_idx = max(range(len(runs)), key=lambda i: runs[i]["final_value"])
        best = runs[best_idx]
        searches = [
            {
                "search_id": i,
                "initial_state": starts[i].to_dict(),
                "final_state": r["final_state"],
                "initial_value": r["initial_value"],
                "final_value": r["final_value"],
                "iterations": r["iterations"],
                "accepted_moves": r["accepted_moves"],
                "stop_reason": r["stop_reason"],
                "history": r["history"],
            }
            for i, r in enumerate(runs)
        ]

        # gabungin riwayat semua pencarian dalam satu sumbu iterasi global
        history, offset, global_best = [], 0, None
        for i, r in enumerate(runs):
            for h in r["history"]:
                if i > 0 and h["iteration"] == 0:
                    offset += 1
                global_best = h["current_value"] if global_best is None else max(global_best, h["current_value"])
                history.append({
                    "iteration": offset + h["iteration"],
                    "search_id": i,
```

```
                    "local_iteration": h["iteration"],
                    "current_value": h["current_value"],
                    "best_value": global_best,
                })
            offset += r["history"][-1]["iteration"]
        total_iterations = sum(r["iterations"] for r in runs)

        return build_result(
            "random_restart", {"maximum_restart": max_restart},
            starts[0].copy(), runs[0]["initial_value"],
            best["final_state"], best["final_value"],
            total_iterations, sum(r["accepted_moves"] for r in runs),
            sum(r["objective_evaluations"] for r in runs),
            time.perf_counter() - t0, best["stop_reason"], history,
            {
                "maximum_restart": max_restart,
                "num_searches": len(runs),
                "restart_count": len(runs) - 1,
                "best_search": best_idx,
                "total_iterations": total_iterations,
                "iterations_per_search": [r["iterations"] for r in runs],
                "restarts": searches,
            },
        )
```

Kode 3.16: Varian *random restart*

Varian *stochastic* pada Kode 3.17 tidak mengevaluasi seluruh tetangga, melainkan hanya satu tetangga acak per iterasi yang diterima bila nilainya lebih tinggi dari *state* saat ini (delta > 0). *State* terbaik dicatat terpisah pada variabel best.

```
    def stochastic(self, initial_state, max_iterations=10_000):
        # ambil satu  neighbour acak, pindah hanya jika lebih baik dari state saat ini
        # referensi: materi kuliah AI BeyondClassicalSearch bagian Stochastic Hill-climbing
        t0 = time.perf_counter()
        start = initial_state.copy()
        current = initial_state.copy()
        current_value = evaluate(self.problem, current)
```

```
        evals = 1
        best = current.copy()
        best_value = current_value
        accepted = iteration = 0
        history = [
            {"iteration": 0, "current_value": current_value, "best_value": best_value, "accepted": False}
        ]
        stop = "max_iterations"

        while iteration < max_iterations:
            neighbour = sample_random_neighbor(self.problem, current, self.rng)
            if neighbour is None:
                stop = "no_valid_neighbors"
                break

            neighbour_value = evaluate(self.problem, neighbour)
            evals += 1
            delta = neighbour_value - current_value

            is_accepted = delta > 0
            if is_accepted:
                current = neighbour
                current_value = neighbour_value
                accepted += 1
                if current_value > best_value:
                    best = current.copy()
                    best_value = current_value

            iteration += 1
            history.append(
                {
                    "iteration": iteration,
                    "current_value": current_value,
                    "best_value": best_value,
                    "accepted": is_accepted,
                }
            )

```

```
46          return build_result(
47              "stochastic", {"max_iterations": max_iterations},
48              start, history[0]["current_value"], best, best_value,
49              iteration, accepted, evals, time.perf_counter() - t0, stop,
history,
50              {"max_iterations": max_iterations, "rejected_moves": iteration
- accepted},
51          )
```

Kode 3.17: Varian *stochastic*

3.6 Implementasi Algoritma Simulated Annealing

3.6.1 simulated_annealing.py

Tabel 3.11: Kelas/fungsi pada simulated_annealing.py

Kelas/Fungsi	**Deskripsi**
class SimulatedAnnealing	Implementasi *Simulated Annealing* untuk memaksimalkan *objective function*.
__init__(self, problem, rng)	Menyimpan instans masalah dan generator acak.
run(self, initial_state, initial_temperature=100.0, minimum_temperature=0.1, cooling_rate=0.995, max_iterations=10_000, stuck_patience=100)	Menjalankan pencarian dari *state* awal. Pada setiap iterasi diambil satu tetangga acak valid; tetangga yang lebih baik selalu diterima, sedangkan tetangga yang sama atau lebih buruk diterima dengan probabilitas $e^{\Delta E/T}$. Suhu T diturunkan secara geometris dengan faktor cooling_rate. Selain itu, dicatat jumlah kejadian *stuck* dan jumlah perpindahan buruk yang diterima maupun ditolak.

Pada metode run, *state* saat ini (current) dan *state* terbaik yang pernah dikunjungi (best) disimpan terpisah, karena penerimaan perpindahan yang lebih buruk dapat menurunkan nilai *state* saat ini. Hasil akhir yang dikembalikan adalah best. Kode 3.18 menunjukkan parameter, inisialisasi variabel pencacah, dan format riwayat per iterasi.

```python
import math
import time

from src.algorithms.common import build_result
from src.neighbors import sample_random_neighbor
from src.objective import evaluate


class SimulatedAnnealing:

    def __init__(self, problem, rng):
        self.problem = problem
        self.rng = rng

    def run(
        self,
        initial_state,
        initial_temperature=100.0,
        minimum_temperature=0.1,
        cooling_rate=0.995,
        max_iterations=10_000,
        stuck_patience=100, # default
    ):
        t0 = time.perf_counter()

        start = initial_state.copy()
        current = initial_state.copy()
        current_value = evaluate(self.problem, current)
        evals = 1
        best = current.copy()
        best_value = current_value
        temperature = initial_temperature
        iteration = 0
        accepted = 0
        bad_accepted = 0
        bad_rejected = 0
        # definisi stuck: state dianggap terjebak selama periode tertentu jika selama stuck_patience iterasi berturut-turut tidak ada satu pun tetangga acak yang lebih baik dari state saat ini. Satu episode stuck
```

```
dihitung sekali dan dianggap lolos ketika akhirnya ditemukan tetangga yang
lebih baik.
        stuck_count = 0
        stuck_iterations = 0
        no_improve_streak = 0
        stuck_events = []

        history = [
            {
                "iteration": 0,
                "temperature": temperature,
                "current_value": current_value,
                "best_value": best_value,
                "delta_e": None,
                "exp_delta_e_over_t": None,
                "acceptance_probability": None,
                "stuck": False,
            }
        ]

        stop_reason = "max_iterations"
```

Kode 3.18: Parameter dan inisialisasi *Simulated Annealing*

Perulangan utama pada Kode 3.19 berjalan selama jumlah iterasi belum mencapai max_iterations dan suhu masih lebih besar atau sama dengan minimum_temperature. Pada setiap iterasi dihitung $\Delta E = f(\text{tetangga}) - f(\text{saat ini})$. Jika $\Delta E > 0$, tetangga diterima dengan probabilitas 1. Jika tidak, tetangga diterima dengan probabilitas

$$P = e^{\Delta E/T} \tag{3.2}$$

dengan membandingkan P terhadap bilangan acak seragam pada $[0, 1)$ (aturan penerimaan Metropolis). Karena $\Delta E = 0$ menghasilkan $P = 1$, tetangga bernilai sama selalu diterima. Pembatas min(delta / temperature, 700.0) pada pemanggilan math.exp dipakai untuk mencegah *overflow* numerik. Setelah setiap iterasi, suhu diperbarui dengan $T \leftarrow \alpha T$, dengan α adalah cooling_rate.

Selain itu, perulangan melacak kejadian *stuck*. Pencarian dianggap *stuck* apabila terdapat stuck_patience iterasi berturut-turut tanpa tetangga yang lebih baik dari *state* saat ini. Setiap episode *stuck* dihitung satu kali pada stuck_count, dicatat

pada stuck_events beserta iterasi deteksi dan iterasi lolosnya, dan dianggap lolos ketika akhirnya ditemukan tetangga yang lebih baik.

```
        # reference to materi kuliah AI BeyondClassicalSearch, tp jika ada yg diluar itu maka akan ditulis secara explicit
        while iteration < max_iterations and temperature >= minimum_temperature:
            neighbour = sample_random_neighbor(self.problem, current, self.rng)

            if neighbour is None:
                stop_reason = "no_valid_neighbors"
                break

            neighbour_value = evaluate(self.problem, neighbour)
            evals += 1
            delta = neighbour_value - current_value
            exp_value = math.exp(min(delta / temperature, 700.0))

            if delta > 0:
                # gerakan lebih baik selalu diterima (reference to materi kuliah AI BeyondClassicalSearch)
                acceptance_prob = 1.0
                current = neighbour
                current_value = neighbour_value
                accepted += 1

                if current_value > best_value:
                    best = current.copy()
                    best_value = current_value
            else:
                # gerakan sama atau lebih buruk bisa diterima dengan probabilitas (reference to materi kuliah AI BeyondClassicalSearch)
                acceptance_prob = exp_value

                # aturan penerimaan acak (Metropolis acceptance rule utk SA)
                # reference: S. Kirkpatrick, C. D. Gelatt, Jr., dan M. P. Vecchi, "Optimization by Simulated Annealing", Science, 1983. https://doi.org/10.1126/science.220.4598.671
```

```
                if self.rng.random() < acceptance_prob:
                    current = neighbour
                    current_value = neighbour_value
                    accepted += 1
                    bad_accepted += 1
                else:
                    bad_rejected += 1

            iteration += 1
            if delta > 0:
                if no_improve_streak >= stuck_patience:
                    stuck_events[-1]["escaped_at"] = iteration
                no_improve_streak = 0
            else:
                no_improve_streak += 1
                if no_improve_streak == stuck_patience:
                    stuck_count += 1
                    stuck_events.append({
                        "detected_at": iteration,
                        "value": current_value,
                        "escaped_at": None,
                    })
            is_stuck = no_improve_streak >= stuck_patience
            stuck_iterations += is_stuck

            history.append(
                {
                    "iteration": iteration,
                    "temperature": round(temperature, 6),
                    "current_value": current_value,
                    "best_value": best_value,
                    "delta_e": delta,
                    "exp_delta_e_over_t": None if exp_value is None else
round(exp_value, 6),
                    "acceptance_probability": round(acceptance_prob, 6),
                    "stuck": is_stuck,
                }
            )
            temperature *= cooling_rate
```

Kode 3.19: Perulangan utama *Simulated Annealing*

Pencarian berhenti karena salah satu dari tiga alasan, yaitu max_iterations (batas iterasi tercapai), temperature_minimum_reached (suhu turun di bawah suhu minimum), atau no_valid_neighbors (tidak ditemukan tetangga valid). Dengan parameter bawaan ($T_0 = 100$, $T_{\min} = 0.1$, $\alpha = 0.995$), suhu turun di bawah suhu minimum setelah 1.379 iterasi, sehingga pencarian umumnya berhenti karena suhu, bukan karena batas iterasi 10.000. Kode 3.20 menunjukkan penyusunan hasil akhir.

```
        if stop_reason == "max_iterations" and temperature < minimum_temperature:
            stop_reason = "temperature_minimum_reached"

        params = {
            "initial_temperature": initial_temperature,
            "minimum_temperature": minimum_temperature,
            "cooling_rate": cooling_rate,
            "max_iterations": max_iterations,
            "stuck_patience": stuck_patience,
        }
        extra = {
            **params,
            "stuck_definition": (
                f"{stuck_patience} proposal berturut-turut tanpa tetangga yang lebih baik"
            ),
            "stuck_count": stuck_count,
            "stuck_iterations": stuck_iterations,
            "stuck_events": stuck_events,
            "bad_accepted": bad_accepted,
            "bad_rejected": bad_rejected,
        }

        return build_result(
            "simulated_annealing",
            params,
            start,
            history[0]["current_value"],
```

```
            best,
            best_value,
            iteration,
            accepted,
            evals,
            time.perf_counter() - t0,
            stop_reason,
            history,
            extra,
        )
```

Kode 3.20: Penentuan alasan berhenti dan penyusunan hasil *Simulated Annealing*

3.7 Implementasi Algoritma Genetic Algorithm

3.7.1 genetic_algorithm.py

Genetic Algorithm yang diimplementasikan menggunakan State sebagai individu (kromosom), dengan Placement setiap paket sebagai gen. Populasi dievolusikan selama sejumlah iterasi (generasi) tertentu melalui seleksi orang tua, *crossover*, dan *mutation*. Berbeda dengan algoritma lain, individu populasi awal dan hasil *crossover* dibangun dengan pengecekan *constraint* inkremental pada metode _can_insert, sedangkan hasil *mutation* divalidasi dengan validate_state. Dengan demikian, setiap individu pada populasi selalu valid.

Tabel 3.12: Kelas/fungsi pada genetic_algorithm.py

Kelas/Fungsi	**Deskripsi**
class GeneticAlgorithm	Implementasi *Genetic Algorithm* yang menyediakan siklus seleksi, *crossover*, dan *mutation*. Konstanta kelas: CROSSOVER_PROBABILITY = 0,80, MUTATION_PROBABILITY = 0,30, INITIAL_PLACEMENT_ATTEMPTS = 20, dan MUTATION_ATTEMPTS = 10.
__init__(self, problem, rng)	Menyimpan instans masalah, generator acak, dan package_map.

Dilanjutkan ke halaman berikutnya

Tabel 3.12 – lanjutan

Kelas/Fungsi	**Deskripsi**
random_inside_position(self, package, orientation, rng, heights)	Mengembalikan posisi acak yang membuat paket berada di dalam dimensi truk, atau None bila paket tidak muat pada orientasi tersebut. Jika heights diberikan, koordinat z dipilih hanya dari daftar ketinggian tersebut.
select_parents(self, population, fitness_scores)	Memilih dua orang tua dengan *roulette wheel selection* dengan pengembalian, dengan peluang sebanding *fitness*. Jika seluruh *fitness* bernilai nol, pemilihan dilakukan seragam.
_empty_state(self)	Membuat *state* kosong, yaitu semua paket tidak dimuat.
_can_insert(self, pid, position, orientation, inside)	Mengecek apakah sebuah paket dapat disisipkan pada posisi dan orientasi tertentu terhadap paket yang sudah berada di dalam truk (inside), mencakup batas truk, *overlap*, paket *fragile*, dan penyangga.
_try_initial_placement(self, state, pid, orientation, inside, loaded_weight, rng)	Mencoba menempatkan satu paket pada *state* awal. Pengecekan berat dilakukan lebih dahulu, kemudian hingga INITIAL_PLACEMENT_ATTEMPTS posisi acak dicoba pada ketinggian kandidat. Mengembalikan True bila berhasil.
_generate_initial_state(self, rng)	Membangkitkan satu individu acak dengan menyisipkan paket dalam urutan acak.
crossover(self, parent_a, parent_b)	Menghasilkan satu anak dari dua orang tua melalui pewarisan gen per paket tanpa mencari posisi pengganti, lalu membuang paket yang melanggar *constraint*.
mutate(self, state)	Menerapkan satu *mutation* (*swap*, *move*, atau *rotate*) yang menghasilkan *state* valid; jika gagal, mengembalikan salinan *state* semula.

Dilanjutkan ke halaman berikutnya

Tabel 3.12 – lanjutan

Kelas/Fungsi	**Deskripsi**
_statistics(iteration, scores, best_value)	Menghitung statistik populasi per generasi (nilai maksimum, rata-rata, dan nilai terbaik sejauh ini).
genetic_algorithm(self, population_size=50, iterations=100)	Menjalankan seluruh proses evolusi dan mengembalikan hasil dalam format build_result.

Kode 3.21 menunjukkan konstanta dan konstruktor, serta Kode 3.22 menunjukkan fungsi pembangkit posisi acak di dalam truk dan seleksi orang tua. Seleksi menggunakan random.choices dengan bobot berupa nilai *fitness* yang sama dengan nilai *objective function*. Seleksi dilakukan dengan pengembalian sehingga kedua orang tua dapat merupakan individu yang sama, dan akan muncul ValueError bila terdapat *fitness* negatif.

```python
import random
import time

from src.algorithms.common import build_result
from src.constraints import box_overlap, validate_state
from src.objective import evaluate
from src.package import ORIENTATIONS
from src.placement import Placement
from src.state import State


class GeneticAlgorithm:
    CROSSOVER_PROBABILITY = 0.80
    MUTATION_PROBABILITY = 0.30
    INITIAL_PLACEMENT_ATTEMPTS = 20
    MUTATION_ATTEMPTS = 10

    def __init__(self, problem, rng):
        self.problem = problem
        self.rng = rng
        self.packages = problem.package_map
```

Kode 3.21: Konstanta dan konstruktor GeneticAlgorithm

```
    def random_inside_position(self, package, orientation, rng=None, heights=None):
        rng = self.rng if rng is None else rng
        dx, dy, dz = package.effective_dims(orientation)
        limits = (self.problem.truck.w - dx, self.problem.truck.l - dy,self.problem.truck.h - dz)

        if min(limits) < 0:
            return None

        if heights is None:
            z = rng.randint(0, limits[2])
        else:
            allowed = [height for height in heights if 0 <= height <= limits[2]]
            if not allowed:
                return None
            z = rng.choice(allowed)

        return [rng.randint(0, limits[0]), rng.randint(0, limits[1]), z]

    def select_parents(self, population, fitness_scores):
        if any(score < 0 for score in fitness_scores):
            raise ValueError("Fitness scores must be nonnegative.")

        weights = fitness_scores if sum(fitness_scores) else None
        return tuple(self.rng.choices(population, weights=weights, k=2))

    def _empty_state(self):
        return State({
            pid: Placement(None, ["x", "y", "z"])
            for pid in self.packages
        })
```

Kode 3.22: Pembangkit posisi acak, seleksi orang tua, dan *state* kosong

Metode _can_insert pada Kode 3.23 merupakan pengecekan *constraint* secara inkremental terhadap paket yang sudah berada di dalam truk. Metode ini menolak posisi yang keluar dari truk, bertumpang tindih dengan paket lain, berada di atas

paket *fragile*, atau (bila paket yang disisipkan *fragile*) berada di bawah paket lain, serta mensyaratkan paket berada di lantai atau tepat di atas paket *non-fragile*.

```
    def _can_insert(self, pid, position, orientation, inside):
        package = self.packages[pid]
        truck = self.problem.truck
        x, y, z = position
        dx, dy, dz = package.effective_dims(orientation)

        if not (0 <= x <= truck.w - dx and 0 <= y <= truck.l - dy and 0 <= z <= truck.h - dz):
            return False

        supported = z == 0
        for other_id, (ox, oy, oz, odx, ody, odz) in inside.items():
            if not (x < ox + odx and ox < x + dx and y < oy + ody and oy < y + dy):
                continue

            if z < oz + odz and oz < z + dz:
                return False
            if self.packages[other_id].is_fragile and z >= oz + odz:
                return False
            if package.is_fragile and oz >= z + dz:
                return False
            if oz + odz == z and not self.packages[other_id].is_fragile:
                supported = True

        return supported
```

Kode 3.23: Pengecekan penyisipan paket secara inkremental

Individu populasi awal dibangkitkan oleh _generate_initial_state pada Kode 3.24. Urutan paket diacak, orientasi unik (yang menghasilkan dimensi efektif berbeda) diacak, dan setiap paket dicoba ditempatkan oleh _try_initial_placement dengan salah satu orientasi hingga berhasil. _try_initial_placement lebih dahulu menolak paket yang membuat total berat melebihi kapasitas, kemudian menyusun daftar ketinggian kandidat, yaitu lantai dan sisi atas paket *non-fragile* yang sudah ada, dan mencoba posisi acak pada ketinggian tersebut hingga INITIAL_PLACEMENT_ATTEMPTS kali. Paket yang gagal ditempatkan dibiarkan berada di luar truk.

```
    def _try_initial_placement(self, state, pid, orientation, inside, loaded_weight, rng):
        package = self.packages[pid]
        if loaded_weight + package.weight > self.problem.truck.max_capacity:
            return False

        heights = sorted({0} | {box[2] + box[5]
                for other_id, box in inside.items()
                if not self.packages[other_id].is_fragile
            }
        )
        for _ in range(self.INITIAL_PLACEMENT_ATTEMPTS):
            position = self.random_inside_position(package, orientation, rng=rng, heights=heights)

            if position is None:
                return False

            if self._can_insert(pid, position, orientation, inside):
                state.placements[pid] = Placement(position, list(orientation))
                inside[pid] = (*position, *package.effective_dims(orientation))
                return True

        return False

    def _generate_initial_state(self, rng):
        state = self._empty_state()
        inside, loaded_weight = {}, 0
        ids = list(self.packages)
        rng.shuffle(ids)

        for pid in ids:
            orientations = list({
                self.packages[pid].effective_dims(orientation): orientation
                for orientation in ORIENTATIONS
            }.values())
```

```
            rng.shuffle(orientations)
            for orientation in orientations:
                if self._try_initial_placement(state, pid, orientation,
inside, loaded_weight, rng):
                    loaded_weight += self.packages[pid].weight
                    break

        return state
```

Kode 3.24: Penempatan paket dan pembangkitan individu awal

Operator *crossover* pada Kode 3.25 bekerja per paket (*uniform crossover*): untuk setiap paket, gen (posisi dan orientasi) diwarisi dari salah satu orang tua yang dipilih acak. Koordinat dan orientasi warisan dipertahankan; operator tidak mencari posisi pengganti. Pembentukan anak dilakukan dalam empat tahap. Pertama, gen berposisi None atau yang keluar dari batas truk dibuang dari kandidat. Kedua, bila beberapa kandidat saling bertumpang tindih, kandidat dengan *value* lebih besar dipertahankan lebih dahulu, dengan luas alas sebagai pemecah nilai seri. Ketiga, kandidat yang tersisa diperiksa dari ketinggian z terkecil, sehingga paket yang kehilangan penyangga juga ikut dikeluarkan. Keempat, setiap paket dicek terhadap kapasitas berat dan _can_insert. Paket yang tidak lolos tetap berposisi None pada anak, sehingga anak selalu valid tetapi sebagian paket dapat tidak termuat.

```
    def crossover(self, parent_a, parent_b):
        inherited = {
            pid: self.rng.choice((parent_a, parent_b)).placements[pid]
            for pid in self.packages
        }
        child = State({
            pid: Placement(None, list(gene.orientation))
            for pid, gene in inherited.items()
        })
        # Pertahankan koordinat dan orientasi warisan; tidak mencari posisi
 pengganti.
        truck = self.problem.truck
        candidates = {}
        for pid, gene in inherited.items():
            if gene.position is None:
                continue
            x, y, z = gene.position
```

```
            dx, dy, dz = self.packages[pid].effective_dims(gene.orientation
)
            if not (0 <= x <= truck.w - dx and 0 <= y <= truck.l - dy and 0
 <= z <= truck.h - dz):
                continue
            if self.packages[pid].weight > truck.max_capacity:
                continue
            candidates[pid] = (x, y, z, dx, dy, dz)

        # Jika bertabrakan, dahulukan nilai paket; luas alas memecahkan
nilai seri.
        priority = sorted(candidates, key=lambda pid: (
            -self.packages[pid].value,
            -candidates[pid][3] * candidates[pid][4],
            pid,
        ))
        selected = {}
        for pid in priority:
            if not any(box_overlap(candidates[pid], box) for box in
selected.values()):
                selected[pid] = candidates[pid]

        # Periksa dari bawah agar kehilangan penyangga juga mengeluarkan
paket di atasnya.
        # Paket yang melanggar constraint tetap memiliki position=None pada
 child.
        ids = sorted(selected, key=lambda pid: (
            selected[pid][2], -self.packages[pid].value, pid,
        ))
        inside, loaded_weight = {}, 0
        for pid in ids:
            gene = inherited[pid]
            if loaded_weight + self.packages[pid].weight > truck.
max_capacity:
                continue
            if not self._can_insert(pid, gene.position, gene.orientation,
inside):
                continue
            child.placements[pid] = Placement(list(gene.position), list(
gene.orientation))
```

```
48              inside[pid] = selected[pid]
49              loaded_weight += self.packages[pid].weight
50
51          return child
```

Kode 3.25: Operator *crossover*

Operator *mutation* pada Kode 3.26 memilih secara seragam salah satu dari tiga jenis perubahan: *swap* (menukar posisi dua paket, orientasi tetap), *move* (memindahkan satu paket ke posisi acak di dalam truk, dengan peluang 20% paket dikeluarkan dari truk), atau *rotate* (merotasi satu paket pada sumbu acak, hanya dihitung bila dimensi efektif berubah). Kandidat harus berbeda dari *state* semula dan lolos validate_state. Hingga MUTATION_ATTEMPTS percobaan dilakukan, dan bila tidak ada yang valid, salinan *state* semula dikembalikan.

```
1     def mutate(self, state):
2         ids = list(state.placements)
3         if not ids:
4             return state.copy()
5
6         operation = self.rng.choice(("swap", "move", "rotate"))
7         original_key = state.key()
8
9         for _ in range(self.MUTATION_ATTEMPTS):
10            candidate = state.copy()
11
12            if operation == "swap":
13                if len(ids) < 2:
14                    break
15                first_id, second_id = self.rng.sample(ids, 2)
16                first = candidate.placements[first_id]
17                second = candidate.placements[second_id]
18                first.position, second.position = second.position, first.
position
19
20            elif operation == "move":
21                pid = self.rng.choice(ids)
22                placement = candidate.placements[pid]
23                if self.rng.random() < 0.20:
24                    placement.position = None
```

```
                else:
                    position = self.random_inside_position(
                        self.packages[pid], placement.orientation
                    )
                    if position is None:
                        continue
                    placement.position = position

            else:
                pid = self.rng.choice(ids)
                placement = candidate.placements[pid]
                before = self.packages[pid].effective_dims(placement.
orientation)
                axis = self.rng.choice(("x", "y", "z"))
                placement.rotate(axis)
                if self.packages[pid].effective_dims(placement.orientation)
 == before:
                    continue

            if candidate.key() == original_key:
                continue

            valid, _ = validate_state(self.problem, candidate)
            if valid:
                return candidate

        return state.copy()
```

Kode 3.26: Operator *mutation*

Proses evolusi pada Kode 3.27 berjalan sebagai berikut. Populasi awal dibangkitkan dengan generator acak terpisah yang diturunkan dari rng, kemudian dievaluasi, dan individu terbaiknya dicatat sebagai initial_state. Pada setiap generasi, populasi baru dibentuk dengan mengulangi langkah berikut hingga ukurannya sama dengan population_size: dua orang tua dipilih dengan *roulette wheel*; dengan probabilitas 0,80 anak dibentuk melalui *crossover*, selain itu anak berupa salinan salah satu orang tua; kemudian dengan probabilitas 0,30 anak dimutasi. Populasi baru menggantikan seluruh populasi lama (tanpa *elitism*), tetapi individu terbaik sepanjang pencarian disimpan terpisah pada best_state. Pencarian berhenti setelah iterations generasi

dengan alasan iteration_limit_reached, dan jumlah evaluasi *objective function* adalah population_size × (iterations +1). Pada main.py, eksperimen dijalankan pada dua rangkaian: populasi sebagai kontrol dengan tiga variasi jumlah iterasi (bawaan 50, 100, dan 200), dan iterasi sebagai kontrol dengan tiga variasi ukuran populasi (bawaan 20, 50, dan 100). Konfigurasi yang muncul pada kedua rangkaian hanya dijalankan sekali.

```
    @staticmethod
    def _statistics(iteration, scores, best_value):
        return {
            "iteration": iteration,
            "population_max": max(scores),
            "population_mean": sum(scores) / len(scores),
            "best_so_far": best_value,
        }

    def genetic_algorithm(self, population_size=50, iterations=100):
        if population_size < 2:
            raise ValueError("Population size must be at least 2.")
        if iterations < 1:
            raise ValueError("Iterations must be positive.")

        started = time.perf_counter()
        initialization_rng = random.Random(self.rng.getrandbits(128))
        population = [
            self._generate_initial_state(initialization_rng)
            for _ in range(population_size)
        ]
        initial_population = population
        scores = [evaluate(self.problem, state) for state in population]
        best_index = max(range(population_size), key=scores.__getitem__)
        best_state = population[best_index].copy()
        best_value = scores[best_index]
        initial_state, initial_value = best_state.copy(), best_value
        history = [self._statistics(0, scores, best_value)]
        best_generation = crossovers = mutations = accepted_mutations = 0

        for generation in range(1, iterations + 1):
            next_population = []

```

```
            while len(next_population) < population_size:
                parent_a, parent_b = self.select_parents(population, scores
)

                if self.rng.random() < self.CROSSOVER_PROBABILITY:
                    child = self.crossover(parent_a, parent_b)
                    crossovers += 1
                else:
                    child = self.rng.choice((parent_a, parent_b)).copy()

                if self.rng.random() < self.MUTATION_PROBABILITY:
                    mutations += 1
                    before = child.key()
                    child = self.mutate(child)
                    accepted_mutations += child.key() != before

                next_population.append(child)

            population = next_population
            scores = [evaluate(self.problem, state) for state in population
]
            best_index = max(range(population_size), key=scores.__getitem__
)

            if scores[best_index] > best_value:
                best_state = population[best_index].copy()
                best_value = scores[best_index]
                best_generation = generation

            history.append(self._statistics(generation, scores, best_value)
)
```

Kode 3.27: Statistik populasi dan proses evolusi *Genetic Algorithm*

Hasil akhir pada Kode 3.28 berisi individu terbaik sepanjang pencarian sebagai *final state*, jumlah perpindahan yang diterima (jumlah *mutation* yang benar-benar mengubah *state*), serta informasi tambahan berupa populasi awal, *fitness* populasi akhir, generasi tempat individu terbaik ditemukan, jumlah *crossover* dan *mutation*, dan pengaturan yang digunakan.

```
        duration = time.perf_counter() - started

        return build_result(
            algorithm="genetic_algorithm",
            parameters={"population_size": population_size, "iterations": iterations},
            initial_state=initial_state,
            initial_value=initial_value,
            final_state=best_state,
            final_value=best_value,
            iterations=iterations,
            accepted_moves=accepted_mutations,
            objective_evaluations=population_size * (iterations + 1),
            duration_seconds=duration,
            stop_reason="iteration_limit_reached",
            history=history,
            extra={
                "initial_state_selection": "best_individual_in_initial_population",
                "initial_population": [state.to_dict() for state in initial_population],
                "final_population_fitness": scores,
                "best_generation": best_generation,
                "crossovers": crossovers,
                "mutation_calls": mutations,
                "accepted_mutations": accepted_mutations,
                "settings": {
                    "selection": "roulette_wheel_with_replacement",
                    "crossover_probability": self.CROSSOVER_PROBABILITY,
                    "mutation_probability": self.MUTATION_PROBABILITY,
                    "elitism": False,
                    "crossover": "uniform_inheritance_without_relocation",
                    "overlap_priority": "package_value_then_footprint_area",
                    "invalid_placement": "outside",
                    "initial_placement_attempts": self.INITIAL_PLACEMENT_ATTEMPTS,
                    "mutation_attempts": self.MUTATION_ATTEMPTS,
                },
            },
```

```
36          )
```

Kode 3.28: Penyusunan hasil *Genetic Algorithm*

3.8 Implementasi Visualizer

Visualizer mengubah berkas JSON hasil pencarian menjadi halaman HTML interaktif, sehingga *state* awal, *state* akhir, dan riwayat pencarian dapat diperiksa tanpa membaca JSON secara manual. Visualizer terdiri atas dua modul, yaitu visualize.py yang menyusun tampilan *state* dan halaman HTML, serta visualization_charts.py yang menyusun grafik hasil eksperimen. Pustaka pandas dan plotly dipakai pada kedua modul. Setiap berkas JSON hasil menghasilkan satu berkas HTML dengan nama yang sama, ditambah satu berkas plotly.min.js bersama sehingga halaman dapat dibuka tanpa koneksi internet.

3.8.1 visualize.py

Tabel 3.13: Fungsi pada visualize.py

Fungsi	**Deskripsi**
package_colors(problem)	Memberikan satu warna tetap untuk setiap ID paket dari palet plotly.
read_state(problem, data)	Membaca *state* dari JSON, memeriksa kelengkapan ID paket, format posisi dan orientasi, lalu memvalidasinya dengan validate_state. Memunculkan ValueError bila *state* tidak valid.
box_vertices(position, dimensions), box_mesh(position, dimensions, color, info)	Menghitung delapan titik sudut balok dan membentuk objek Mesh3d berwarna untuk satu paket, dengan teks *hover*.
truck_frame(dimensions)	Membentuk rangka garis peti kemas dari dua belas rusuk balok.

Dilanjutkan ke halaman berikutnya

Tabel 3.13 – lanjutan

Fungsi	**Deskripsi**
state_view(problem, data, title)	Membuat tampilan 3D berisi dua *scene*, yaitu paket di dalam peti dan paket di luar peti. Fungsi ini juga mengembalikan tabel paket dan peta sel peti.
grid_view(problem, grid, title)	Membuat peta sel per lapisan z (*heatmap*) dengan *slider* lapisan.
figure_html, table_html, write_page	Mengubah grafik dan tabel menjadi HTML, menambahkan tombol Unduh PNG (2x), dan menulis halaman akhir.
write_result(problem, result, path)	Menyusun seluruh isi halaman satu berkas hasil.
main()	Membaca argumen *command line* dan memproses seluruh berkas hasil.

Fungsi read_state pada Kode 3.29 memeriksa bahwa himpunan ID paket pada hasil sama dengan masukan, setiap posisi berupa tiga bilangan bulat atau None, dan setiap orientasi merupakan permutasi x, y, z. Setelah itu *state* divalidasi dengan validate_state yang sama dengan yang dipakai algoritma, sehingga visualizer tidak pernah menampilkan konfigurasi yang melanggar *constraint*.

```
def read_state(problem, data):
    placements = data["placements"]
    if set(placements) != set(problem.package_map):
        raise ValueError("State harus memetakan seluruh ID paket pada input.")

    for pid, item in placements.items():
        position = item["position"]
        if position is not None and (
            len(position) != 3 or any(type(c) is not int for c in position)
        ):
            raise ValueError(f"{pid}: posisi harus berupa tiga bilangan bulat.")
        orientation = item["orientation"]
        if len(orientation) != 3 or set(orientation) != {"x", "y", "z"}:
```

```
            raise ValueError(f"{pid}: orientasi harus berupa permutasi x, y, z.")

    state = State({
        pid: Placement(
            list(item["position"]) if item["position"] is not None else None,
            list(item["orientation"]),
        )
        for pid, item in placements.items()
    })
    valid, violations = validate_state(problem, state)
    if not valid:
        raise ValueError("State tidak valid: " + "; ".join(violations))
    return state
```

Kode 3.29: Pembacaan dan validasi *state*

Fungsi state_view pada Kode 3.30 membangun tampilan 3D. Paket yang berada di dalam peti digambar pada posisi *state* di *scene* pertama, sedangkan paket di luar peti (posisi None) digambar berderet pada *scene* kedua dengan posisi ilustrasi saja. Judul grafik memuat nilai *objective*, jumlah paket masuk dan luar, persentase volume terisi, dan berat terhadap kapasitas. Fungsi ini juga memetakan setiap sel satuan peti ke ID paket penghuninya yang digunakan oleh grid_view.

```
def state_view(problem, data, title):
    state = read_state(problem, data)
    truck = problem.truck
    dimensions = (truck.w, truck.l, truck.h)
    if any(type(size) is not int or size <= 0 for size in dimensions):
        raise ValueError("Ukuran peti harus berupa bilangan bulat positif.")

    colors = package_colors(problem)
    fig = make_subplots(
        rows=1, cols=2,
        specs=[[{"type": "scene"}, {"type": "scene"}]],
        subplot_titles=("Di dalam peti", "Di luar peti (posisi ilustrasi)"),
    )
```

```
    fig.add_trace(truck_frame(dimensions), row=1, col=1)

    columns = max(1, ceil(sqrt(len(problem.packages))))
    spacing = 1 + max(
        (max(p.w, p.l, p.h) for p in problem.packages), default=1
    )
    rows = []
    loaded_weight = inside_count = 0

    for index, (pid, package) in enumerate(problem.package_map.items()):
        placement = state.placements[pid]
        size = package.effective_dims(placement.orientation)
        inside = placement.position is not None
        position = placement.position if inside else (
            truck.w + 2 + (index % columns) * spacing,
            (index // columns) * spacing,
            0,
        )
        record = {
            "Paket": pid,
            "Status": "Dalam" if inside else "Luar",
            "Posisi state": str(placement.position),
            "Orientasi": str(placement.orientation),
            "Ukuran efektif": str(size),
            "Nilai": package.value,
            "Berat": package.weight,
            "Fragile": package.is_fragile,
            "ETA": package.eta,
        }
        rows.append(record)
        info = "<br>".join(f"{key}: {escape(str(value))}" for key, value in
record.items())
        fig.add_trace(
            box_mesh(position, size, colors[pid], info),
            row=1, col=1 if inside else 2,
        )
        if inside:
            inside_count += 1
            loaded_weight += package.weight

```

```
    occupied = {}
    for pid, placement in state.placements.items():
        if placement.position is None:
            continue
        size = problem.package_map[pid].effective_dims(placement.orientation)
        cells = product(*[
            range(start, start + length)
            for start, length in zip(placement.position, size)
        ])
        occupied.update((cell, pid) for cell in cells)
    grid = {
        cell: occupied.get(cell)
        for cell in product(range(truck.w), range(truck.l), range(truck.h))
    }
    utilization = len(occupied) / len(grid)
    outside_rows = max(1, ceil(len(problem.packages) / columns))
    fig.update_scenes(
        aspectmode="data", xaxis_title="x", yaxis_title="y", zaxis_title="z",
        camera=dict(eye=dict(x=1.7, y=1.7, z=1.3)),
    )
    fig.update_layout(
        height=550,
        title=(
            f"{escape(title)} | Nilai={evaluate(problem, state)} | Valid<br>"
            f"Masuk={inside_count}, luar={len(rows) - inside_count} | "
            f"Volume={utilization:.1%} | Berat={loaded_weight}/{truck.max_capacity}"
        ),
        scene=dict(
            xaxis=dict(range=[0, truck.w]),
            yaxis=dict(range=[0, truck.l]),
            zaxis=dict(range=[0, truck.h]),
        ),
        scene2=dict(
            xaxis=dict(range=[truck.w + 1, truck.w + 2 + columns * spacing]),
            yaxis=dict(range=[-1, outside_rows * spacing]),
```

```
            zaxis=dict(range=[0, spacing]),
        ),
    )
    return fig, pd.DataFrame(rows, columns=PACKAGE_COLUMNS), grid
```

Kode 3.30: Tampilan 3D *state*

Peta sel pada Kode 3.31 menampilkan satu lapisan z sebagai *heatmap* dua dimensi, dengan sel kosong berwarna abu-abu. *Slider* memungkinkan pengguna berpindah antar-lapisan untuk memeriksa susunan tumpukan paket.

```
def grid_view(problem, grid, title):
    truck = problem.truck
    colors = package_colors(problem)
    ids = [None, *problem.package_map]
    codes = {pid: index for index, pid in enumerate(ids)}
    palette = ["#e5e7eb", *[colors[pid] for pid in ids[1:]]]
    scale = [
        (boundary, color)
        for index, color in enumerate(palette)
        for boundary in (index / len(palette), (index + 1) / len(palette))
    ]
    fig = go.Figure()
    for level in range(truck.h):
        occupants = [
            [grid[(x, y, level)] for x in range(truck.w)]
            for y in range(truck.l)
        ]
        fig.add_trace(go.Heatmap(
            x=list(range(truck.w)), y=list(range(truck.l)),
            z=[[codes[pid] for pid in row] for row in occupants],
            customdata=[
                ["Kosong" if pid is None else escape(str(pid)) for pid in
row]
                for row in occupants
            ],
            colorscale=scale, zmin=-0.5, zmax=len(ids) - 0.5,
            showscale=False, visible=(level == 0), xgap=1, ygap=1,
            hovertemplate=(
                "x=%{x}, y=%{y}, z=" + str(level)
```

```
                + "<br>Isi=%{customdata}<extra></extra>"
            ),
        ))
    fig.update_layout(
        title=escape(title), height=500,
        xaxis_title="x (indeks sel)", yaxis_title="y (indeks sel)",
        sliders=[dict(
            currentvalue=dict(prefix="Lapisan z = "),
            steps=[dict(
                label=str(level), method="update",
                args=[{"visible": [i == level for i in range(truck.h)]}],
            ) for level in range(truck.h)],
        )],
    )
    fig.update_xaxes(range=[-0.5, truck.w - 0.5], constrain="domain", dtick
=1)
    fig.update_yaxes(
        range=[-0.5, truck.l - 0.5], constrain="domain", dtick=1,
        scaleanchor="x", scaleratio=1,
    )
    return fig
```

Kode 3.31: Peta sel per lapisan

Fungsi write_result pada Kode 3.32 menyusun halaman satu *run*. Untuk *initial state* dan *final state*, fungsi ini menampilkan grafik 3D, peta sel, dan tabel seluruh paket, setelah memeriksa bahwa nilai *objective* yang dihitung ulang sama dengan initial_value dan final_value pada JSON. Selanjutnya ditampilkan tabel metrik (algoritma, *seed*, nilai awal dan akhir, durasi, jumlah iterasi atau generasi, alasan berhenti, serta metrik khusus algoritma seperti jumlah *sideways*, jumlah *restart*, atau frekuensi *stuck*), tabel parameter, catatan penjelasan, dan grafik riwayat pencarian dari algorithm_view.

```
def write_result(problem, result, path):
    algorithm = result["algorithm"]
    name = ALGORITHM_NAMES.get(algorithm, algorithm)
    title = f"{path.stem} | {name}"
    parts = [
        "<p>Posisi paket di luar peti hanya ilustrasi; posisi state tetap
None. "
```

```
        "Sel (x, y, z) mewakili [x, x+1) &times; [y, y+1) &times; [z, z+1). "
        "Warna abu-abu pada peta sel berarti kosong. Arahkan kursor untuk melihat ID paket.</p>"
        "<p>Atur sudut pandang 3D, zoom, atau lapisan z, lalu klik Unduh PNG (2x) "
        "pada grafik yang ingin disimpan. Gambar mengikuti tampilan grafik saat diunduh.</p>"
    ]
    for phase, label in [("initial_state", "Initial state"), ("final_state", "Final state")]:
        state = read_state(problem, result[phase])
        value_field = "initial_value" if phase == "initial_state" else "final_value"
        if evaluate(problem, state) != result[value_field]:
            raise ValueError(f"{path.stem}: {value_field} tidak cocok dengan nilai paket dalam state.")
        figure, packages, grid = state_view(problem, result[phase], label)
        parts.extend([
            f"<h2>{label}</h2>", figure_html(figure, f"{path.stem}_{phase}_3d"),
            figure_html(
                grid_view(problem, grid, f"{label}: seluruh sel peti"),
                f"{path.stem}_{phase}_grid",
            ),
            "<details><summary>Semua paket</summary>" + table_html(packages) + "</details>",
        ])
    parts.append("<h2>Hasil eksperimen</h2>")
    extra = result.get("extra", {})
    metrics = {
        "Algoritma": name,
        "Run": result.get("run_id", "-"),
        "Seed": result.get("seed", "-"),
        "Nilai objective awal": result["initial_value"],
        "Nilai objective akhir": result["final_value"],
        "Durasi pencarian (detik)": result["duration_seconds"],
        "Generasi dijalankan" if algorithm == "genetic_algorithm" else "Iterasi dijalankan":
            extra.get("total_iterations", result["iterations"]),
```

```
        "Alasan berhenti": result["stop_reason"],
    }
    if algorithm == "sideways":
        metrics["Sideways diterima"] = extra["sideways_moves"]
    elif algorithm == "random_restart":
        metrics["Jumlah restart"] = extra.get("restart_count", extra["
num_searches"] - 1)
        metrics["Pencarian terbaik"] = extra["best_search"]
    elif algorithm == "simulated_annealing":
        metrics["Frekuensi stuck (episode)"] = extra.get("stuck_count", "
Tidak dicatat")
        if "stuck_iterations" in extra:
            metrics["Iterasi dalam kondisi stuck"] = extra["
stuck_iterations"]
    parts.append(table_html(pd.DataFrame(metrics.items(), columns=["Metrik"
, "Nilai"])))

    parameter_labels = {
        "population_size": "Jumlah populasi",
        "iterations": "Batas generasi GA",
        "maximum_sideways_move": "Maksimum sideways move",
        "maximum_restart": "Maksimum restart",
        "max_iterations": "Batas iterasi",
        "initial_temperature": "Suhu awal",
        "minimum_temperature": "Suhu minimum",
        "cooling_rate": "Faktor pendinginan",
        "stuck_patience": "Batas proposal tanpa perbaikan untuk deteksi
stuck",
    }
    parameters = result.get("parameters", {})
    if parameters:
        parts.append(table_html(pd.DataFrame([
            (parameter_labels.get(key, key), value)
            for key, value in parameters.items()
        ], columns=["Parameter", "Nilai"])))
    notes = {
        "sideways": "Jumlah sideways memakai perpindahan yang diterima;
titik datar terakhir dapat berarti berhenti.",
        "random_restart": "Initial state berasal dari Pencarian 0. Final
state adalah hasil terbaik dari seluruh pencarian. Grafik memakai indeks
```

```
riwayat kumulatif, termasuk titik awal setiap restart.",
70          "simulated_annealing": "E adalah nilai objective state saat ini,
bukan energi yang diminimalkan. Delta E adalah nilai proposal dikurangi
nilai state sebelum iterasi. Final state adalah state terbaik yang
ditemukan. Probabilitas proposal tidak menunjukkan keputusan penerimaannya.
",
71          "genetic_algorithm": "Initial state adalah individu terbaik pada
populasi awal. Final state adalah individu terbaik sepanjang pencarian.",
72      }
73      if algorithm in notes:
74          parts.append(f"<p>{notes[algorithm]}</p>")
75      if algorithm == "simulated_annealing" and "stuck_definition" in extra:
76          parts.append(f"<p>Definisi stuck: {escape(extra['stuck_definition
'])}.</p>")
77      parts.append(figure_html(algorithm_view(result), f"{path.stem}
_experiment"))
78      if algorithm == "random_restart":
79          parts.append(table_html(pd.DataFrame([
80              {
81                  "Pencarian (0 = awal)": search["search_id"],
82                  "Iterasi": search["iterations"],
83                  "Nilai awal": search["initial_value"],
84                  "Nilai akhir": search["final_value"],
85                  "Alasan berhenti": search["stop_reason"],
86              }
87              for search in extra["restarts"]
88          ])))
89      write_page(path, title, "".join(parts))
```

Kode 3.32: Penyusunan halaman hasil satu *run*

Program dijalankan dengan perintah python -m src.visualize –input <berkas masukan> yang menerima opsi –results (berkas atau direktori JSON hasil, bawaan output) dan –output-dir (bawaan output/visualizations). Setiap grafik dilengkapi tombol Unduh PNG (2x) untuk menyimpan tampilan grafik saat itu sebagai gambar.

```
1   def main():
2       parser = argparse.ArgumentParser(description="Buat satu HTML per JSON
hasil pencarian, dengan nama file yang sama.")
3       parser.add_argument("--input", required=True, type=Path, help="JSON
```

```
permasalahan yang digunakan saat pencarian")
    parser.add_argument("--results", type=Path, default=Path("output"),
help="File JSON hasil atau direktori hasil")
    parser.add_argument("--output-dir", type=Path, default=Path("output/
visualizations"))
    args = parser.parse_args()
    files = sorted(args.results.glob("*.json")) if args.results.is_dir()
else [args.results]
    if not files:
        parser.error("Tidak ada JSON hasil pencarian pada direktori
tersebut.")

    try:
        problem = load_problem(args.input)
        args.output_dir.mkdir(parents=True, exist_ok=True)
        (args.output_dir / "plotly.min.js").write_text(get_plotlyjs(),
encoding="utf-8")
        for source in files:
            result = json.loads(source.read_text(encoding="utf-8-sig"))
            destination = args.output_dir / source.with_suffix(".html").
name
            write_result(problem, result, destination)
            print(f"Dibuat: {destination.resolve()}")
    except (OSError, ValueError, KeyError, TypeError) as error:
        parser.error(str(error))
    print(f"Selesai: {len(files)} HTML. Buka file HTML hasil run yang ingin
 dilihat.")


if __name__ == "__main__":
    main()
```

Kode 3.33: Fungsi utama visualizer

3.8.2 visualization_charts.py

Modul ini memiliki dua fungsi. algorithm_view membuat grafik riwayat satu *run* sesuai jenis algoritma, dan experiment_view merangkum kumpulan *run* untuk perbandingan antar-algoritma. Pemetaan nama algoritma ke nama tampilan disimpan pada ALGORITHM_NAMES.

Tabel 3.14: Grafik riwayat per algoritma pada algorithm_view

Algoritma	**Grafik**
Steepest ascent, *sideways*, *stochastic*	Satu garis nilai *state* terhadap iterasi, dengan anotasi alasan berhenti.
Random restart	Satu garis per pencarian dan satu garis putus-putus untuk nilai terbaik sepanjang pencarian, dengan iterasi kumulatif.
Simulated Annealing	Beberapa panel bersusun: nilai E (saat ini dan terbaik), ΔE, suhu (skala logaritmik), $e^{\Delta E/T}$, dan probabilitas penerimaan.
Genetic Algorithm	Tiga garis terhadap generasi: maksimum populasi, rata-rata populasi, dan terbaik sepanjang pencarian.

```
def algorithm_view(result):
    algorithm = result["algorithm"]
    history = pd.DataFrame(result["history"])
    extra = result.get("extra", {})
    title = ALGORITHM_NAMES.get(algorithm, algorithm)

    if algorithm in {"steepest_ascent", "sideways", "stochastic"}:
        if algorithm == "sideways":
            title += f" | Sideways diterima: {extra['sideways_moves']}"

        fig = px.line(
            history,
            x="iteration",
            y="current_value",
            markers=True,
            color_discrete_sequence=CHART_COLORS,
            title=title,
            labels={"iteration": "Iterasi", "current_value": "Nilai state"
},
        )
        last = history.iloc[-1]
        fig.add_annotation(
            x=last["iteration"],
```

```
            y=last["current_value"],
            text=f"Berhenti: {result['stop_reason']}",
            showarrow=True,
            ax=0,
            ay=-40,
        )

    elif algorithm == "random_restart":
        fig = go.Figure()
        for search_id, part in history.groupby("search_id"):
            label = f"Pencarian {search_id} ({extra['iterations_per_search'][search_id]} it)"
            if search_id == extra["best_search"]:
                label += " (terbaik)"
            fig.add_scatter(x=part["iteration"], y=part["current_value"], mode="lines+markers", name=label)
        fig.add_scatter(x=history["iteration"], y=history["best_value"], mode="lines",
                        name="Terbaik sepanjang pencarian", line={"dash": "dash"})
        fig.update_layout(
            title=(f"{title} | Restart: {extra['restart_count']} dari maks {extra['maximum_restart']}"
                   f" | Total iterasi: {extra['total_iterations']}"),
            xaxis_title="Iterasi (kumulatif seluruh pencarian)",
            yaxis_title="Nilai state",
        )

    elif algorithm == "simulated_annealing":
        # E is the objective value of the current state, including rejected
        # proposals where E stays unchanged. Delta E describes each proposal.
        panels = [("E: nilai objective state setiap iterasi", [
            ("current_value", "E (state saat ini)"),
            ("best_value", "E terbaik"),
        ], "E")]
        if "delta_e" in history:
            panels.append(("Delta E proposal terhadap state sebelum iterasi", [
                ("delta_e", "Delta E proposal"),
```

```
            ], "Delta E"))
        panels.append(("Suhu", [("temperature", "Suhu")], "T"))
        if "exp_delta_e_over_t" in history:
            panels.append(("e^(Delta E/T) setiap proposal", [
                ("exp_delta_e_over_t", "e^(Delta E/T)"),
            ], "e^(Delta E/T)"))
        panels.append(("Probabilitas penerimaan min(1, e^(Delta E/T))", [
            ("acceptance_probability", "Probabilitas penerimaan"),
        ], "Probabilitas"))
        fig = make_subplots(
            rows=len(panels),
            cols=1,
            shared_xaxes=True,
            subplot_titles=[panel[0] for panel in panels],
        )
        for row, (_, fields, axis_title) in enumerate(panels, start=1):
            for field, label in fields:
                fig.add_trace(
                    go.Scatter(
                        x=history["iteration"], y=history[field],
                        mode="lines+markers", marker_size=3, name=label,
                        connectgaps=False,
                    ), row=row, col=1,
                )
            fig.update_yaxes(title_text=axis_title, row=row, col=1)
            if axis_title == "T":
                fig.update_yaxes(type="log", row=row, col=1)
            elif axis_title == "Probabilitas":
                fig.update_yaxes(range=[0, 1.05], row=row, col=1)
        fig.update_xaxes(title_text="Iterasi", row=len(panels), col=1)
        fig.update_layout(
            title=f"{title} | Episode stuck tercatat: {extra.get('
stuck_count', 0)}",
            height=250 * len(panels),
        )

    elif algorithm == "genetic_algorithm":
        fig = go.Figure()
        for field, label in [
            ("population_max", "Maksimum populasi"),
```

```
            ("population_mean", "Rata-rata populasi"),
            ("best_so_far", "Terbaik sepanjang pencarian"),
        ]:
            fig.add_scatter(x=history["iteration"], y=history[field], mode="lines", name=label)
        fig.update_layout(title=title, xaxis_title="Generasi", yaxis_title="Fitness")

    else:
        raise ValueError(f"Algoritma belum dikenali: {algorithm}")

    fig.update_layout(colorway=CHART_COLORS, template="plotly_white")
    return fig
```

Kode 3.34: Grafik riwayat per algoritma

Fungsi experiment_view pada Kode 3.35 mengumpulkan metrik setiap *run* ke dalam tabel pandas (nilai awal dan akhir, selisih, durasi, jumlah iterasi, dan alasan berhenti), lalu mengelompokkannya berdasarkan algoritma dan parameter untuk menghitung jumlah *run*, rata-rata, simpangan baku, nilai minimum dan maksimum, serta rata-rata durasi. Fungsi mengembalikan data mentah, tabel ringkasan, *box plot* sebaran nilai akhir antar-*run*, dan diagram sebar durasi terhadap nilai akhir.

```
def experiment_view(results):
    records = []
    for result in results:
        extra = result.get("extra", {})
        records.append({
            "algorithm": result["algorithm"],
            "parameters": json.dumps(result["parameters"], sort_keys=True, ensure_ascii=False),
            "seed": result.get("seed"),
            "run_id": result.get("run_id"),
            "initial_value": result["initial_value"],
            "final_value": result["final_value"],
            "gain": result["final_value"] - result["initial_value"],
            "seconds": result["duration_seconds"],
            "iterations": extra.get("total_iterations", result["iterations"]),
            "stop_reason": result["stop_reason"],
```

```
        })

    if not records:
        raise ValueError("Belum ada hasil eksperimen.")

    data = pd.DataFrame(records)
    summary = (
        data.groupby(["algorithm", "parameters"], dropna=False)
        .agg(
            runs=("final_value", "size"),
            mean_value=("final_value", "mean"),
            std_value=("final_value", "std"),
            min_value=("final_value", "min"),
            max_value=("final_value", "max"),
            mean_seconds=("seconds", "mean"),
        )
        .reset_index()
    )
    labels = {
        "algorithm": "Algoritma",
        "parameters": "Parameter",
        "final_value": "Nilai akhir",
        "initial_value": "Nilai awal",
        "seconds": "Durasi (detik)",
        "iterations": "Iterasi/generasi (total untuk random restart)",
        "seed": "Seed",
        "run_id": "Run",
        "stop_reason": "Alasan berhenti",
    }
    scores = px.box(
        data,
        x="algorithm",
        y="final_value",
        color="parameters",
        points="all",
        hover_data=["seed", "run_id", "initial_value"],
        title="Distribusi nilai akhir antar-run",
        labels=labels,
    )
    timing = px.scatter(
```

```
        data,
        x="seconds",
        y="final_value",
        color="algorithm",
        hover_data=["parameters", "seed", "run_id", "iterations", "
stop_reason"],
        title="Durasi pencarian terhadap nilai akhir",
        labels=labels,
    )
    return data, summary, scores, timing
```

Kode 3.35: Ringkasan eksperimen

Bab 4

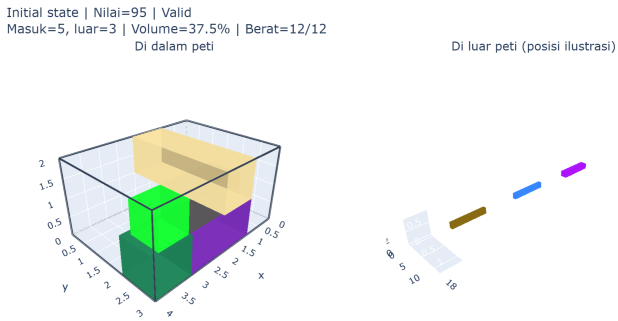
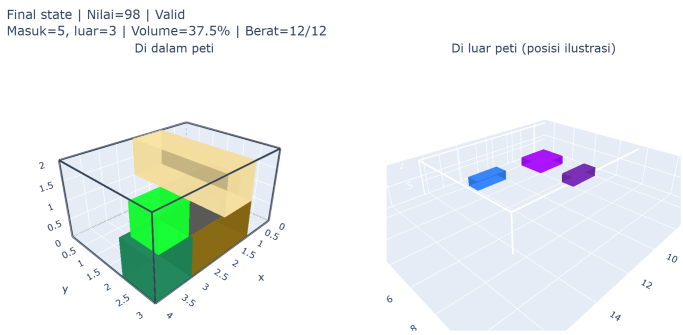
Hasil Eksperimen dan Analisis

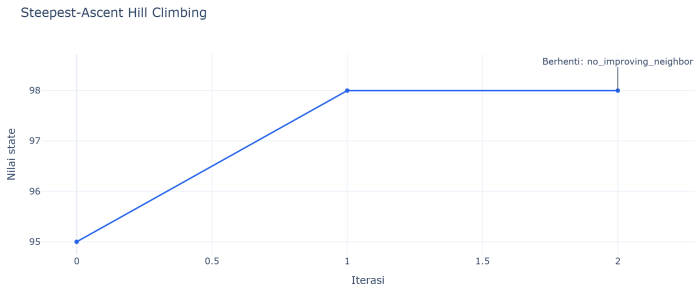
Bab ini menyajikan hasil pengujian setiap algoritma pada *test case* `tc8`, `tc9`, dan `tc10`, beserta analisisnya. Seluruh data diambil dari berkas JSON hasil *run* pada folder output, sedangkan gambar dihasilkan oleh *visualizer* program.

4.1 Eksperimen Algoritma *Hill-climbing*

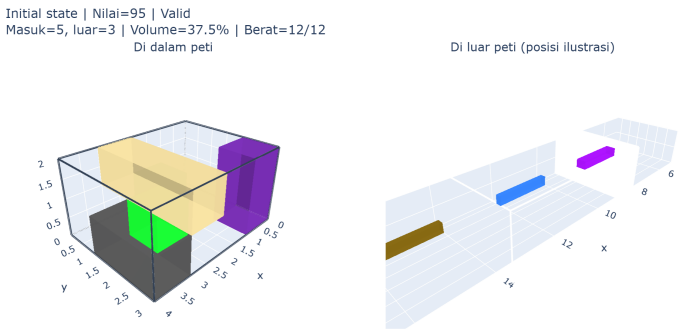
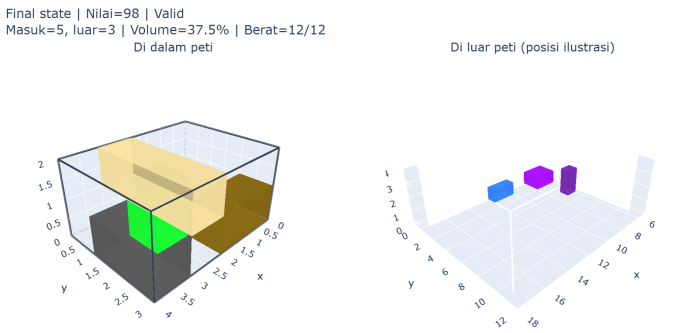
4.1.1 *Steepest Ascent Hill-climbing*

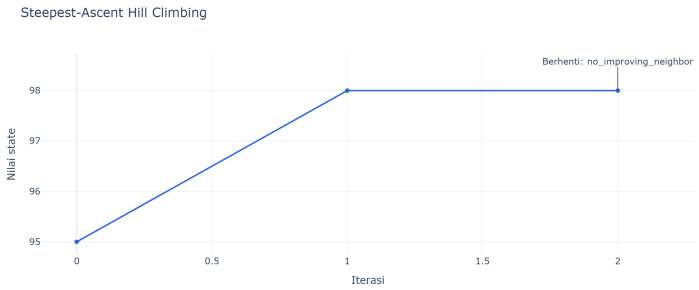
Test case tc8 — Percobaan 1

Informasi yang dicatat	**Data hasil pengujian**
State awal	
State akhir	
Nilai *objective function* akhir	98

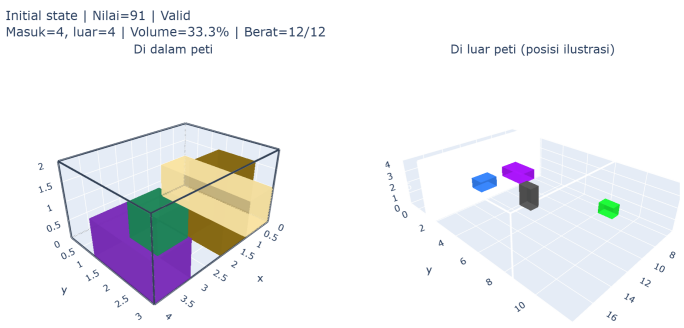
Informasi yang dicatat	Data hasil pengujian
Grafik *objective function* terhadap iterasi	
Durasi pencarian	0.017746 detik
Banyak iterasi hingga berhenti	2

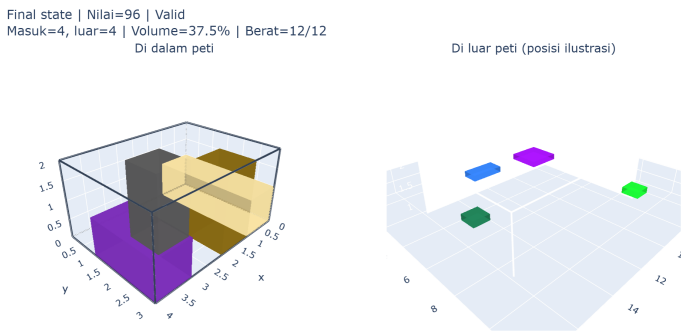
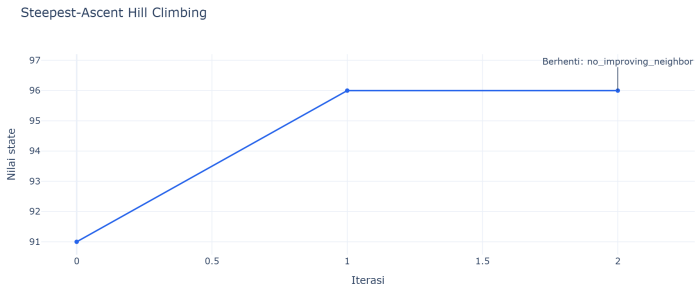
Test case tc8 — Percobaan 2

Informasi yang dicatat	Data hasil pengujian
State awal	
State akhir	

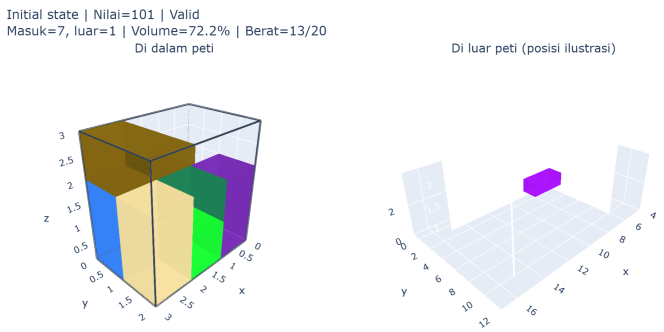
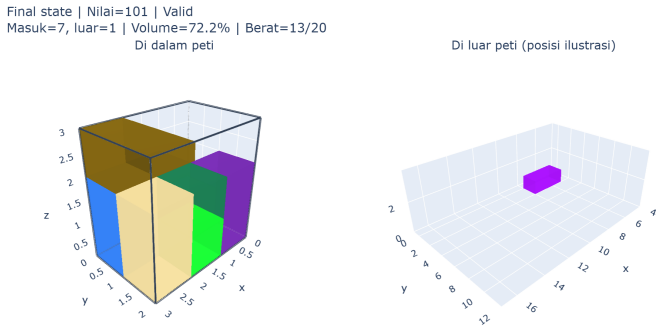
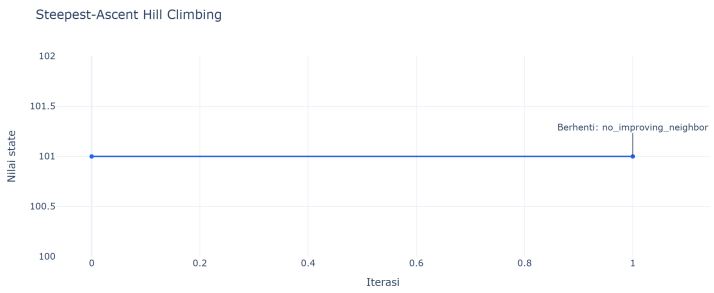
Informasi yang dicatat	**Data hasil pengujian**
Nilai *objective function* akhir	98
Grafik *objective function* terhadap iterasi	
Durasi pencarian	0.015894 detik
Banyak iterasi hingga berhenti	2

Test case tc8 — Percobaan 3

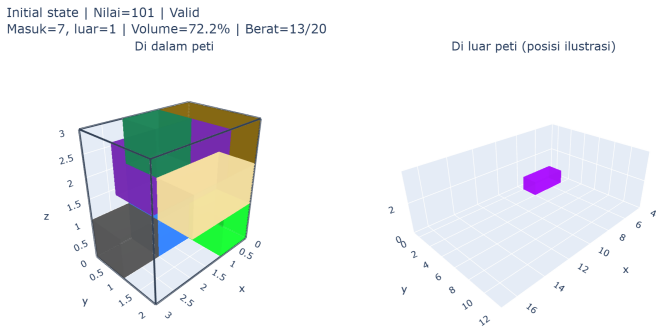
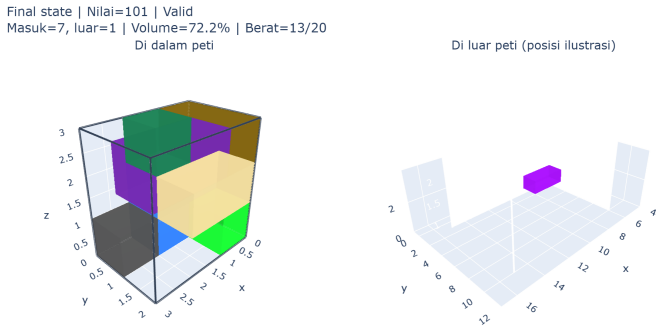
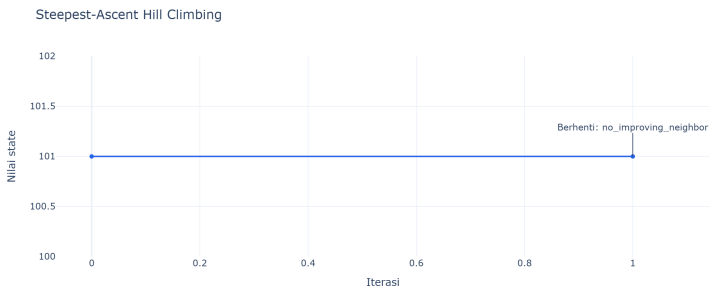
Informasi yang dicatat	**Data hasil pengujian**
State awal	

Informasi yang dicatat	**Data hasil pengujian**
State akhir	
Nilai *objective function* akhir	96
Grafik *objective function* terhadap iterasi	
Durasi pencarian	0.014675 detik
Banyak iterasi hingga berhenti	2

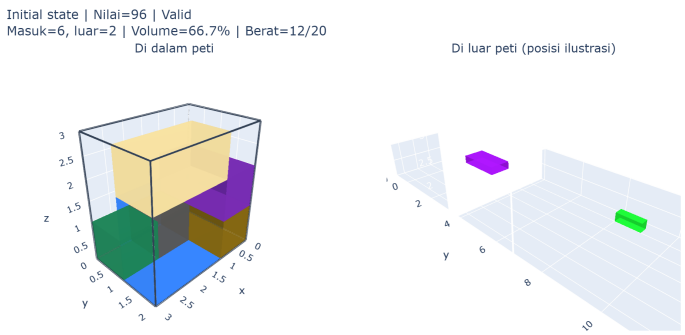
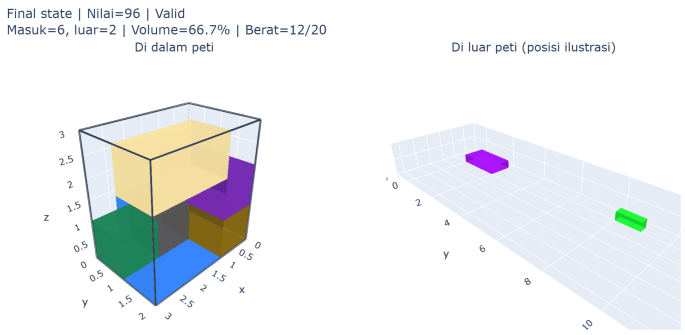
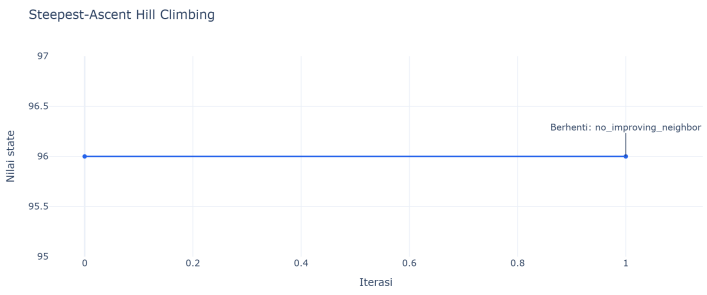
Test case tc9 — Percobaan 1

Informasi yang dicatat	**Data hasil pengujian**
State awal	
State akhir	
Nilai *objective function* akhir	101
Grafik *objective function* terhadap iterasi	
Durasi pencarian	0.010037 detik
Banyak iterasi hingga berhenti	1

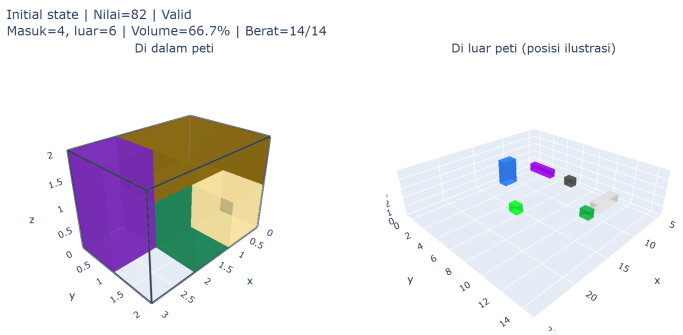
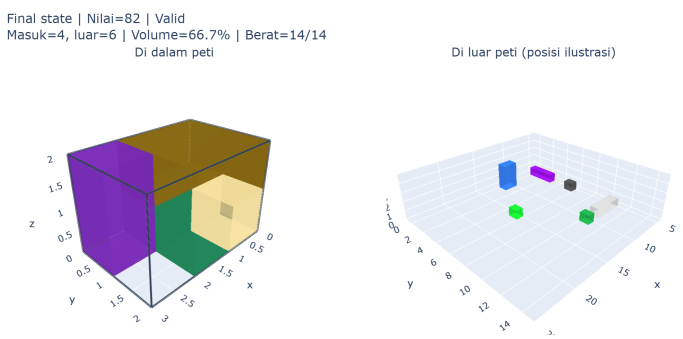
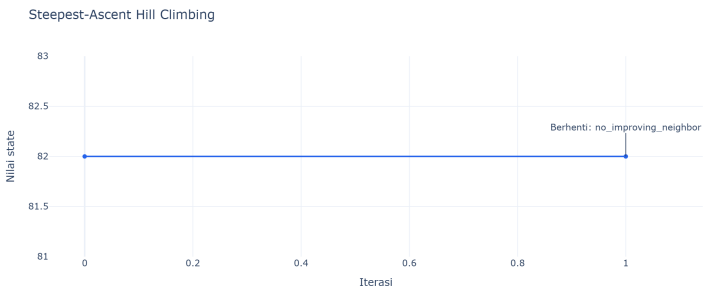
Test case tc9 — Percobaan 2

Informasi yang dicatat	**Data hasil pengujian**
State awal	
State akhir	
Nilai *objective function* akhir	101
Grafik *objective function* terhadap iterasi	
Durasi pencarian	0.010593 detik
Banyak iterasi hingga berhenti	1

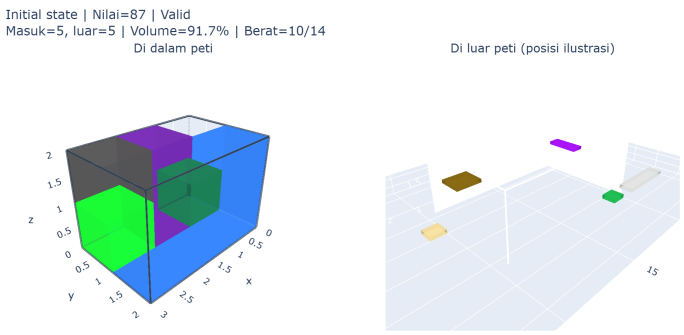
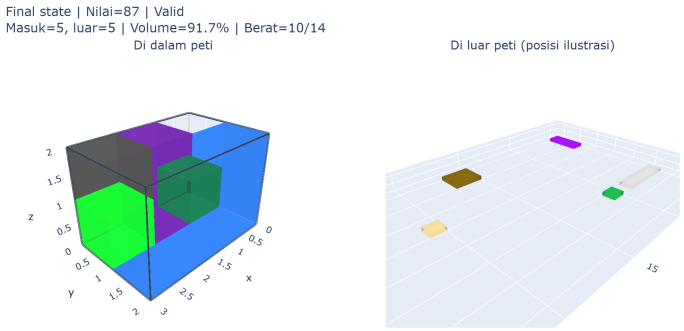
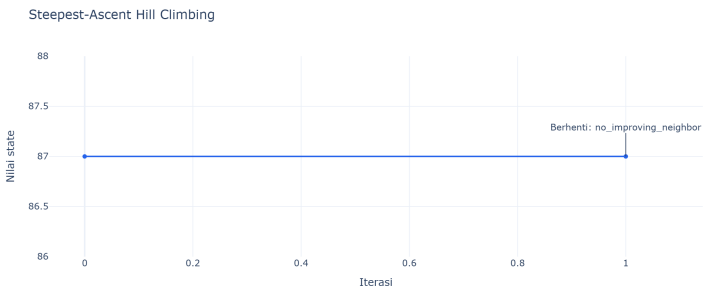
Test case tc9 — Percobaan 3

Informasi yang dicatat	**Data hasil pengujian**
State awal	
State akhir	
Nilai *objective function* akhir	96
Grafik *objective function* terhadap iterasi	
Durasi pencarian	0.007649 detik
Banyak iterasi hingga berhenti	1

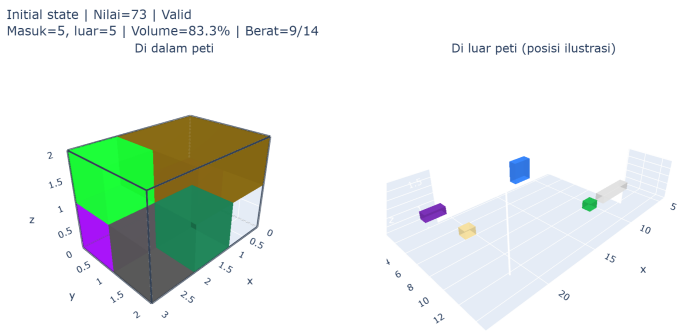
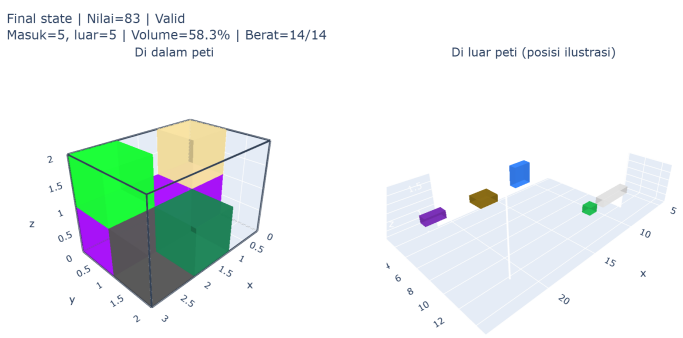
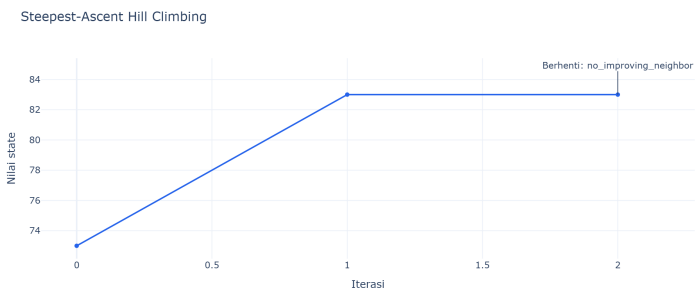
Test case tc10 — Percobaan 1

Informasi yang dicatat	**Data hasil pengujian**
State awal	
State akhir	
Nilai *objective function* akhir	82
Grafik *objective function* terhadap iterasi	
Durasi pencarian	0.00552 detik
Banyak iterasi hingga berhenti	1

Test case tc10 — Percobaan 2

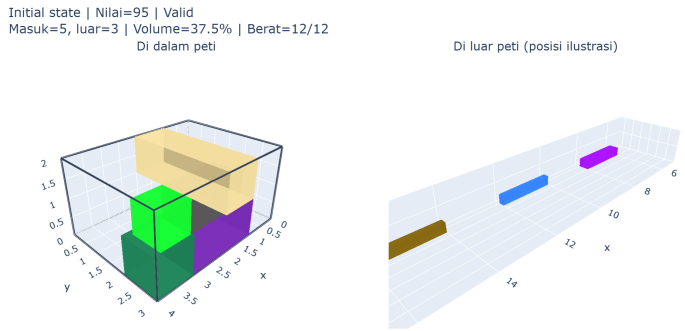
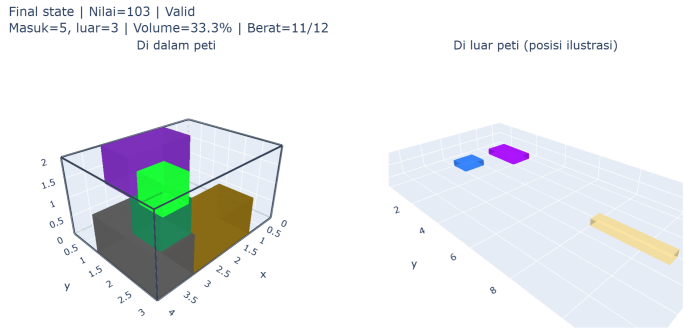
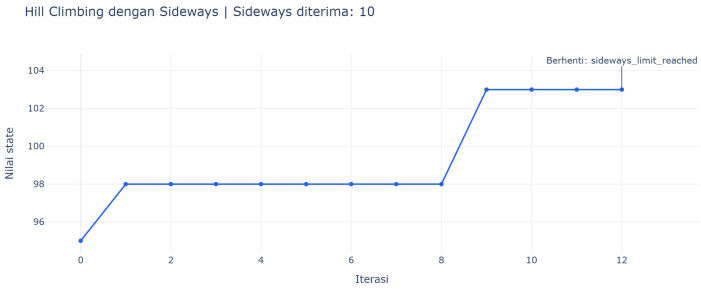
Informasi yang dicatat	**Data hasil pengujian**
State awal	
State akhir	
Nilai *objective function* akhir	87
Grafik *objective function* terhadap iterasi	
Durasi pencarian	0.006447 detik
Banyak iterasi hingga berhenti	1

Test case tc10 — Percobaan 3

Informasi yang dicatat	**Data hasil pengujian**
State awal	
State akhir	
Nilai *objective function* akhir	83
Grafik *objective function* terhadap iterasi	
Durasi pencarian	0.012795 detik
Banyak iterasi hingga berhenti	2

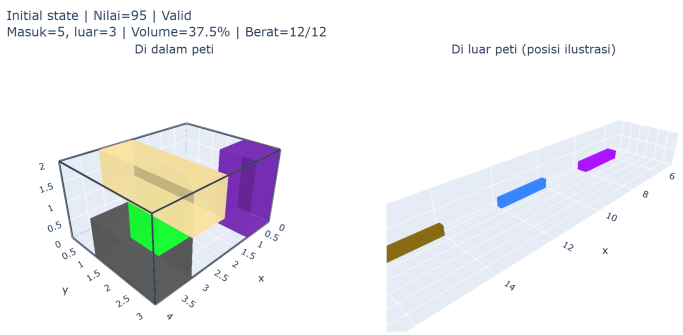
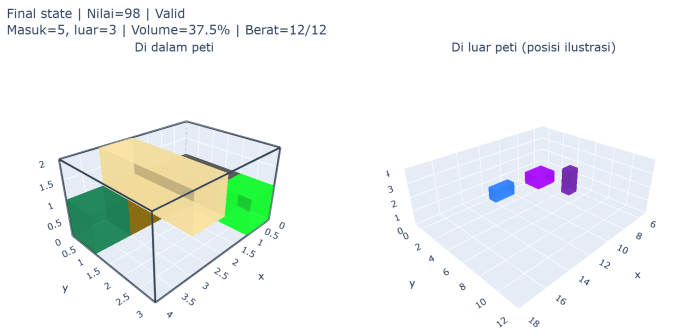
4.1.2 *Hill-climbing with Sideways Move*

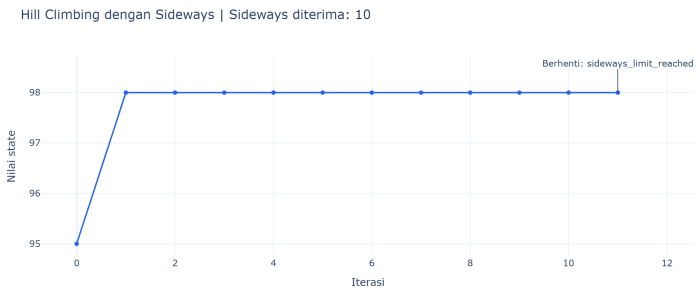
Test case tc8 — Percobaan 1

Informasi yang dicatat	**Data hasil pengujian**
Maximum sideways move	10
State awal	
State akhir	
Nilai *objective function* akhir	103
Grafik *objective function* terhadap iterasi	
Durasi pencarian	0.101828 detik

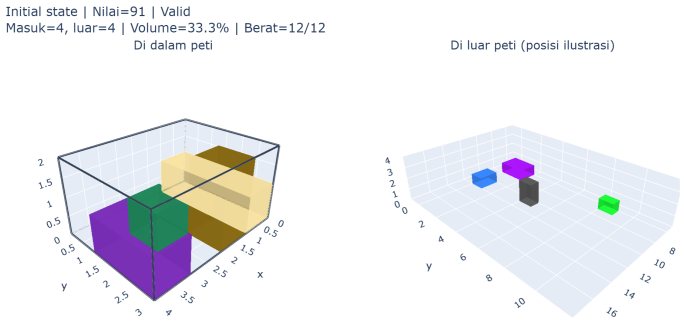
Informasi yang dicatat	**Data hasil pengujian**
Banyak iterasi hingga berhenti	12
Jumlah *sideways move* yang dilakukan	10

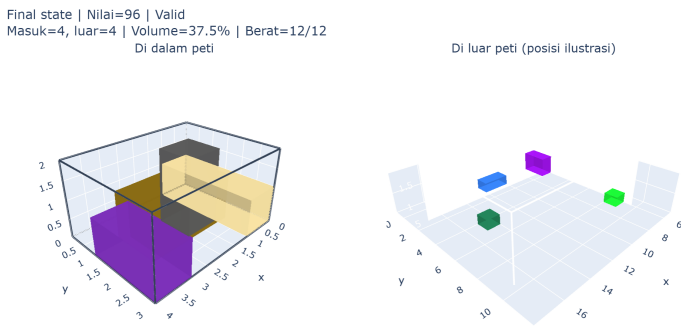
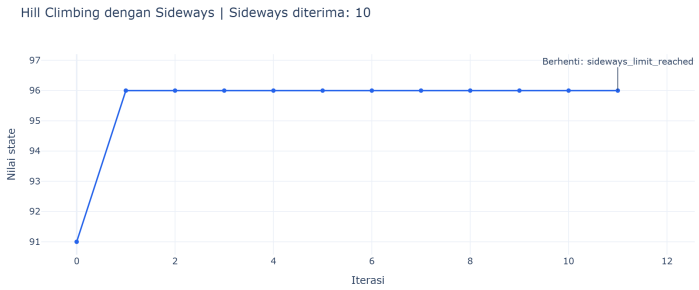
Test case tc8 — Percobaan 2

Informasi yang dicatat	**Data hasil pengujian**
Maximum sideways move	10
State awal	
State akhir	
Nilai *objective function* akhir	98

Informasi yang dicatat	**Data hasil pengujian**
Grafik *objective function* terhadap iterasi	
Durasi pencarian	0.094503 detik
Banyak iterasi hingga berhenti	11
Jumlah *sideways move* yang dilakukan	10

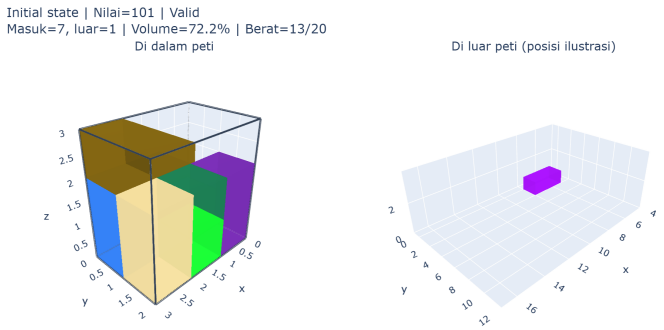
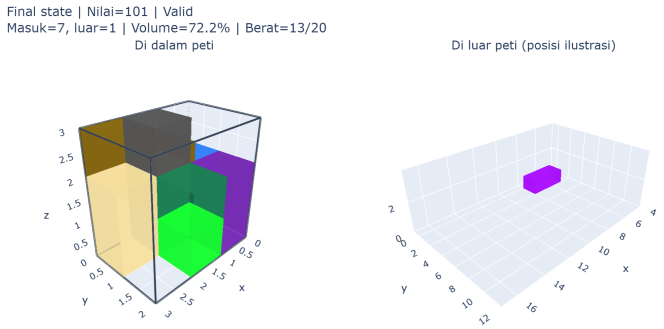
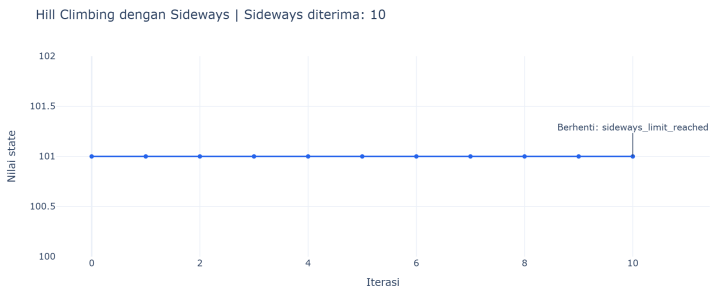
Test case tc8 — Percobaan 3

Informasi yang dicatat	**Data hasil pengujian**
Maximum sideways move	10
State awal	

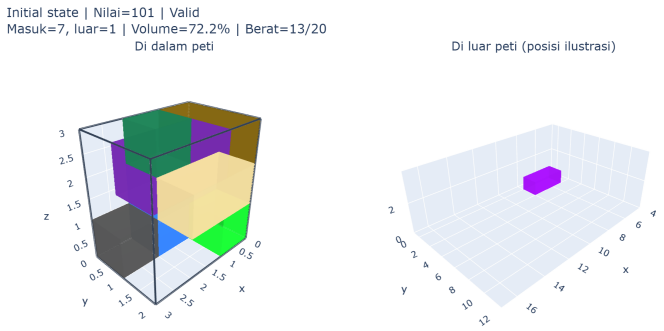
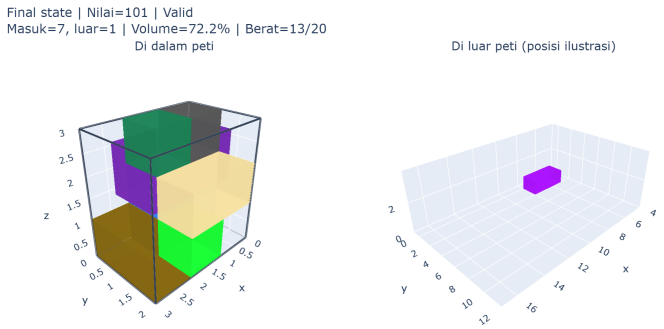
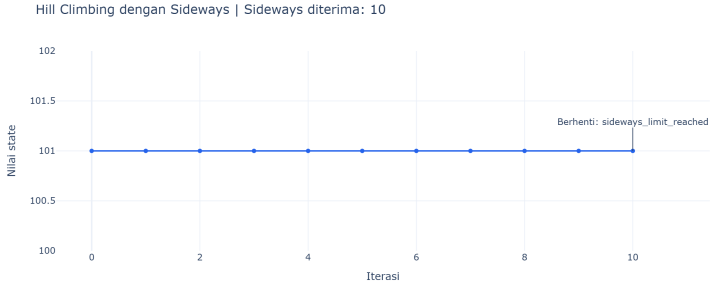
Informasi yang dicatat	**Data hasil pengujian**
State akhir	
Nilai *objective function* akhir	96
Grafik *objective function* terhadap iterasi	
Durasi pencarian	0.070576 detik
Banyak iterasi hingga berhenti	11
Jumlah *sideways move* yang dilakukan	10

Test case tc9 — Percobaan 1

Informasi yang dicatat	**Data hasil pengujian**
Maximum sideways move	10

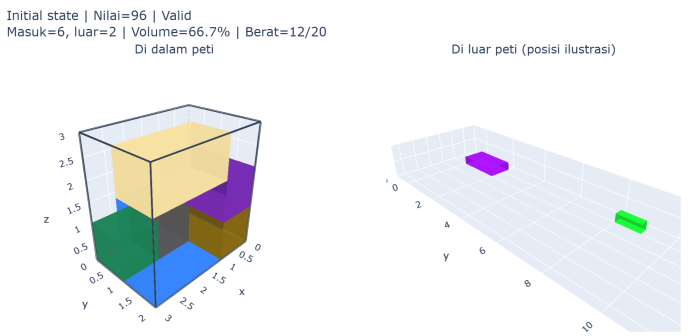
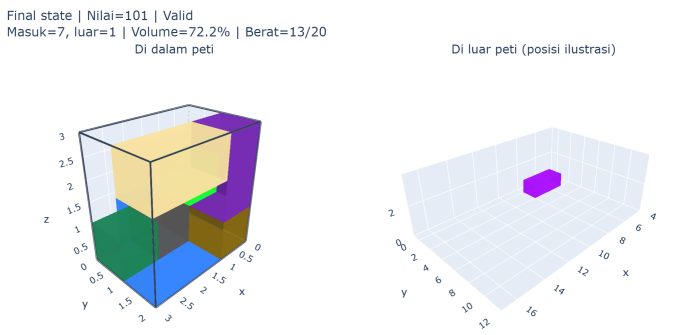
Informasi yang dicatat	**Data hasil pengujian**
State awal	
State akhir	
Nilai *objective function* akhir	101
Grafik *objective function* terhadap iterasi	
Durasi pencarian	0.130282 detik
Banyak iterasi hingga berhenti	10
Jumlah *sideways move* yang dilakukan	10

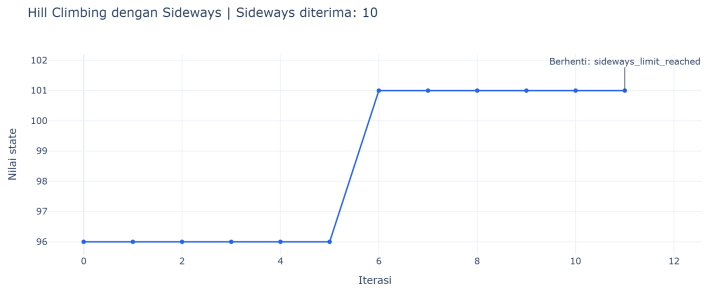
Test case tc9 — Percobaan 2

Informasi yang dicatat	Data hasil pengujian
Maximum sideways move	10
State awal	
State akhir	
Nilai *objective function* akhir	101
Grafik *objective function* terhadap iterasi	
Durasi pencarian	0.101959 detik

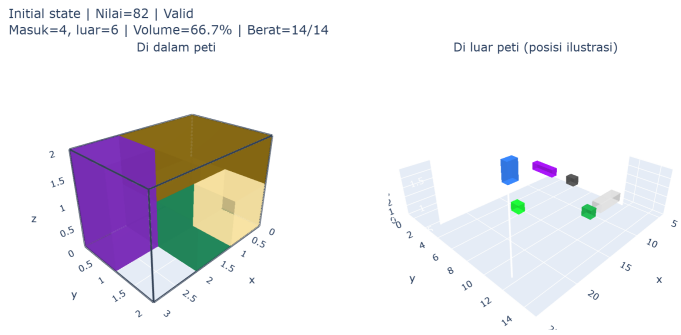
Informasi yang dicatat	**Data hasil pengujian**
Banyak iterasi hingga berhenti	10
Jumlah *sideways move* yang dilakukan	10

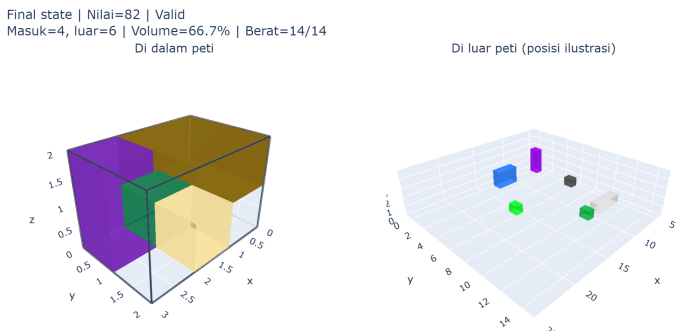
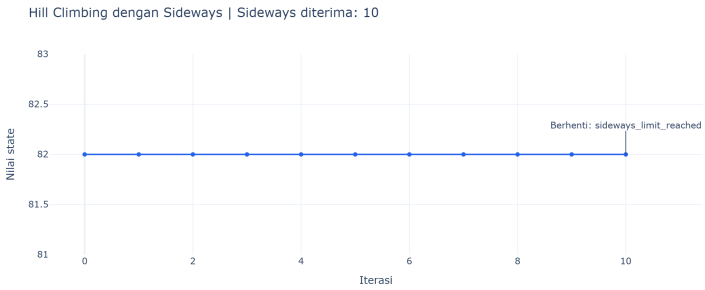
Test case tc9 — Percobaan 3

Informasi yang dicatat	**Data hasil pengujian**
Maximum sideways move	10
State awal	
State akhir	
Nilai *objective function* akhir	101

Informasi yang dicatat	**Data hasil pengujian**
Grafik *objective function* terhadap iterasi	
Durasi pencarian	0.099453 detik
Banyak iterasi hingga berhenti	11
Jumlah *sideways move* yang dilakukan	10

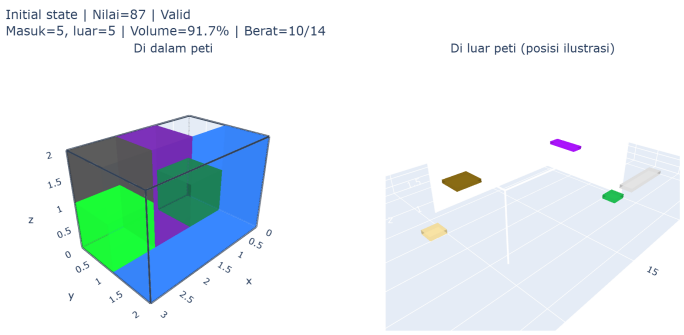
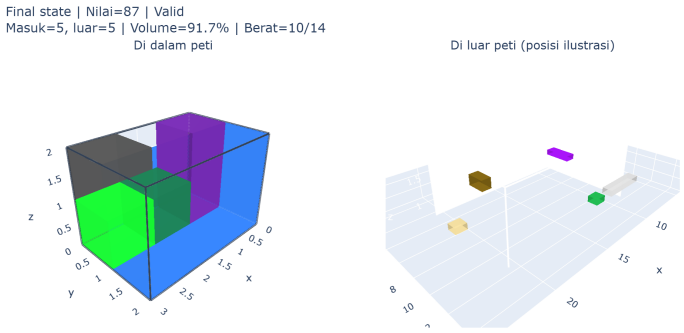
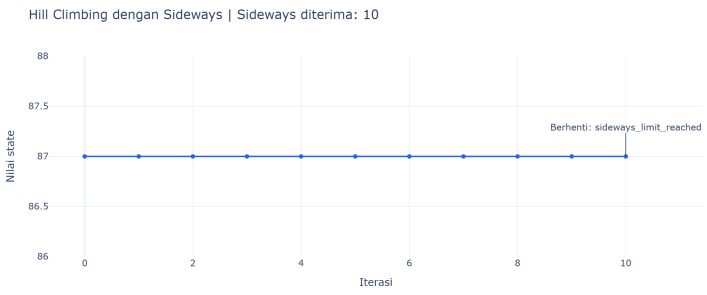
Test case tc10 — Percobaan 1

Informasi yang dicatat	**Data hasil pengujian**
Maximum sideways move	10
State awal	

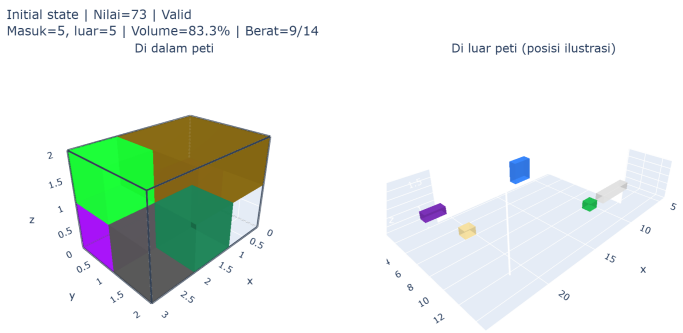
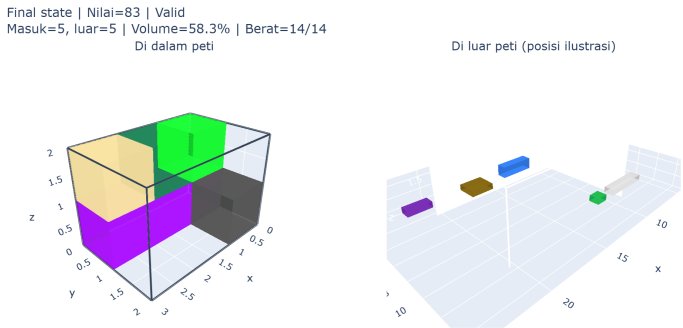
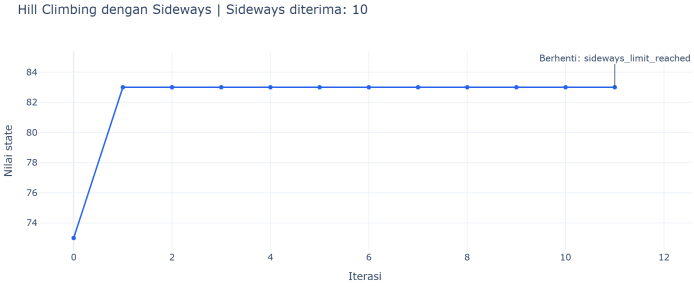
Informasi yang dicatat	**Data hasil pengujian**
State akhir	
Nilai *objective function* akhir	82
Grafik *objective function* terhadap iterasi	
Durasi pencarian	0.055903 detik
Banyak iterasi hingga berhenti	10
Jumlah *sideways move* yang dilakukan	10

Test case tc10 — Percobaan 2

Informasi yang dicatat	**Data hasil pengujian**
Maximum sideways move	10

Informasi yang dicatat	**Data hasil pengujian**
State awal	
State akhir	
Nilai *objective function* akhir	87
Grafik *objective function* terhadap iterasi	
Durasi pencarian	0.065628 detik
Banyak iterasi hingga berhenti	10
Jumlah *sideways move* yang dilakukan	10

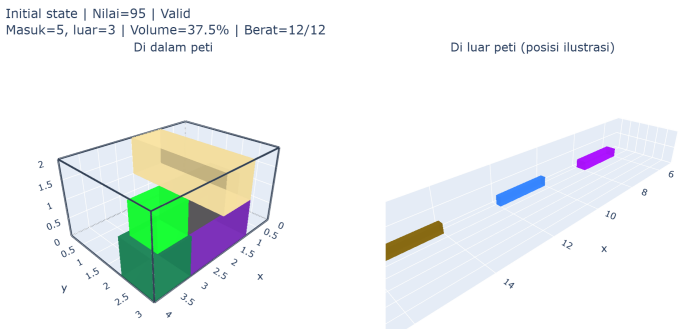
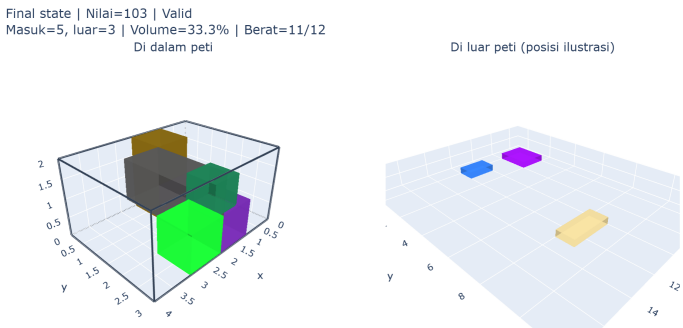
Test case tc10 — Percobaan 3

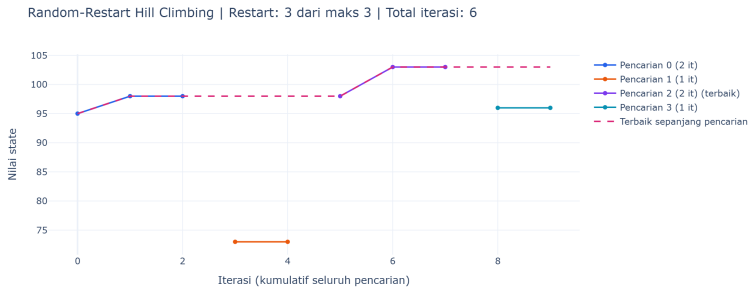
Informasi yang dicatat	Data hasil pengujian
Maximum sideways move	10
State awal	
State akhir	
Nilai *objective function* akhir	83
Grafik *objective function* terhadap iterasi	
Durasi pencarian	0.075649 detik

Informasi yang dicatat	Data hasil pengujian
Banyak iterasi hingga berhenti	11
Jumlah *sideways move* yang dilakukan	10

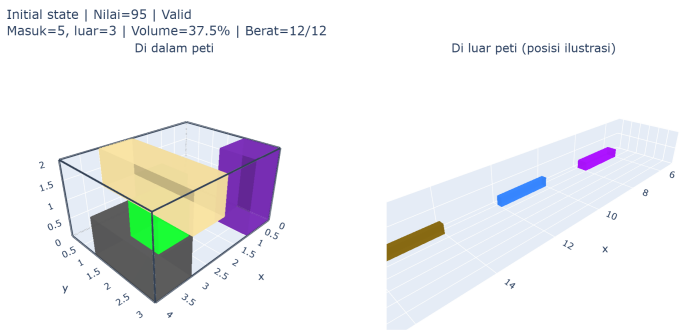
4.1.3 *Random Restart Hill-climbing*

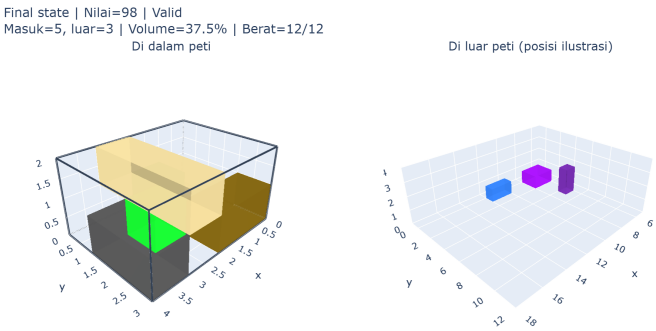
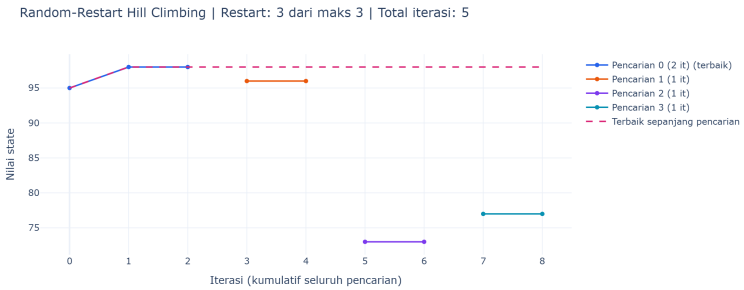
Test case tc8 — Percobaan 1

Informasi yang dicatat	Data hasil pengujian
Maximum restart	3
State awal	
State akhir terbaik	
Nilai *objective function* akhir	103

Informasi yang dicatat	**Data hasil pengujian**
Grafik *objective function* terhadap iterasi	
Durasi pencarian	0.090698 detik
Banyak *restart* yang dilakukan	3
Banyak iterasi per *restart*	2, 1, 2, 1 (rata-rata: 1,5)

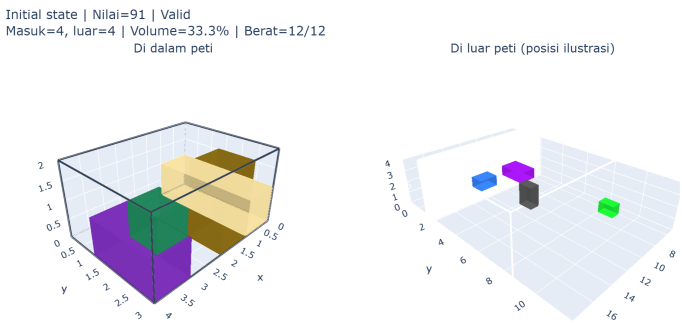
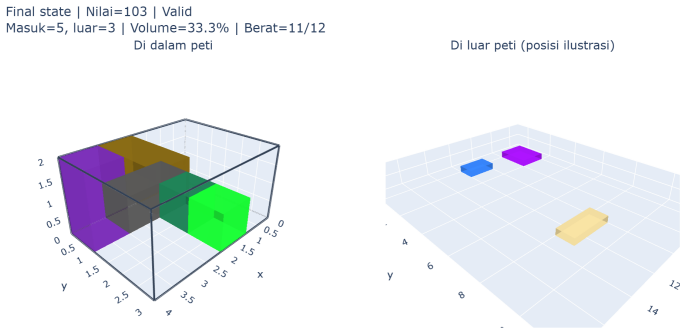
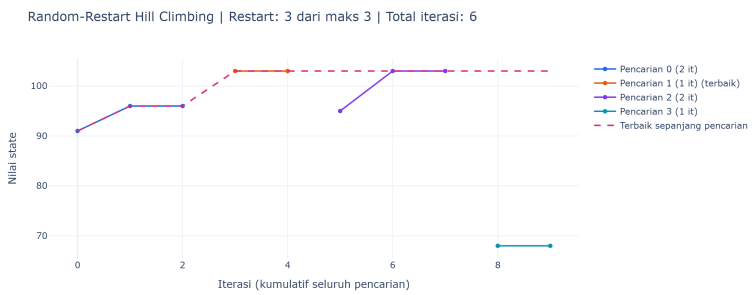
Test case tc8 — Percobaan 2

Informasi yang dicatat	**Data hasil pengujian**
Maximum restart	3
State awal	

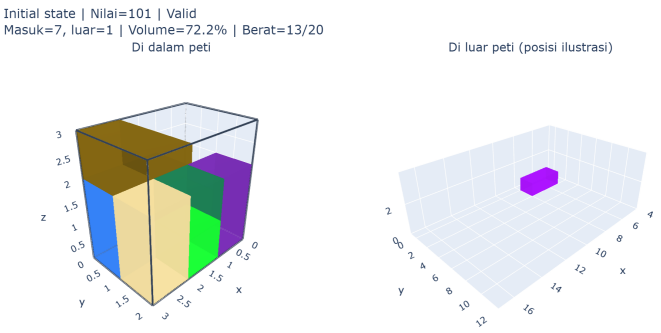
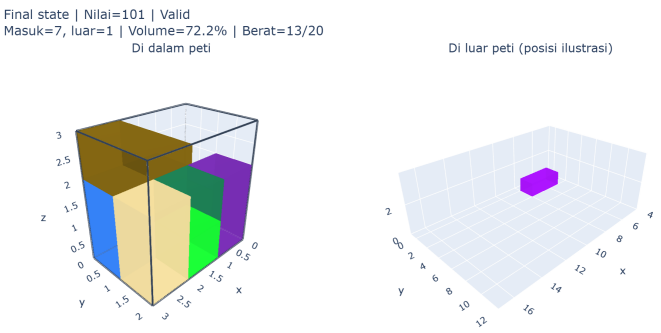
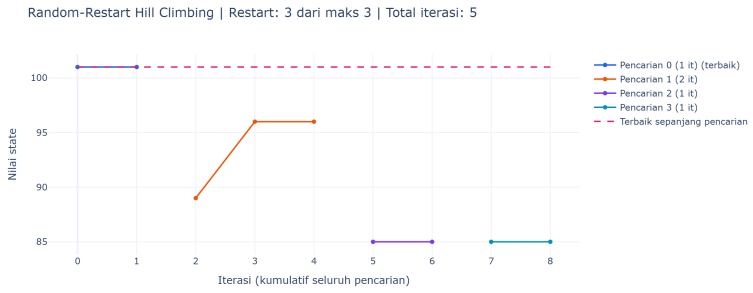
Informasi yang dicatat	**Data hasil pengujian**
State akhir terbaik	
Nilai *objective function* akhir	98
Grafik *objective function* terhadap iterasi	
Durasi pencarian	0.079565 detik
Banyak *restart* yang dilakukan	3
Banyak iterasi per *restart*	2, 1, 1, 1 (rata-rata: 1,25)

Test case tc8 — Percobaan 3

Informasi yang dicatat	**Data hasil pengujian**
Maximum restart	3

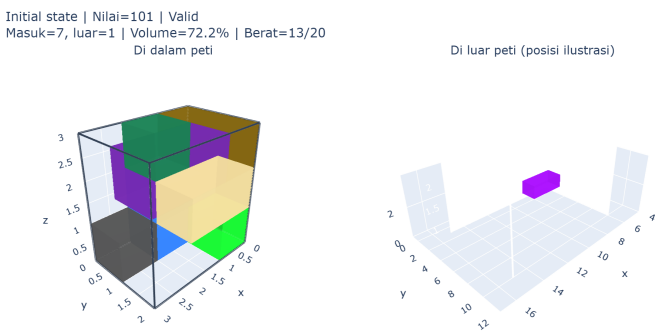
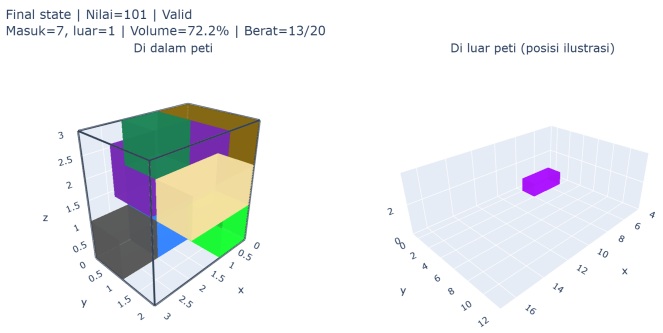
Informasi yang dicatat	**Data hasil pengujian**
State awal	
State akhir terbaik	
Nilai *objective function* akhir	103
Grafik *objective function* terhadap iterasi	
Durasi pencarian	0.074867 detik
Banyak *restart* yang dilakukan	3
Banyak iterasi per *restart*	2, 1, 2, 1 (rata-rata: 1,5)

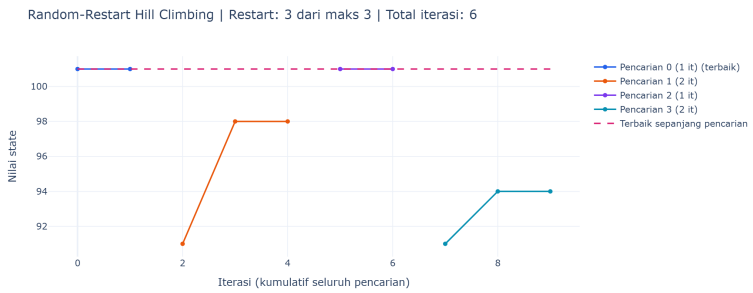
Test case tc9 — Percobaan 1

Informasi yang dicatat	Data hasil pengujian
Maximum restart	3
State awal	
State akhir terbaik	
Nilai *objective function* akhir	101
Grafik *objective function* terhadap iterasi	
Durasi pencarian	0.057637 detik
Banyak *restart* yang dilakukan	3

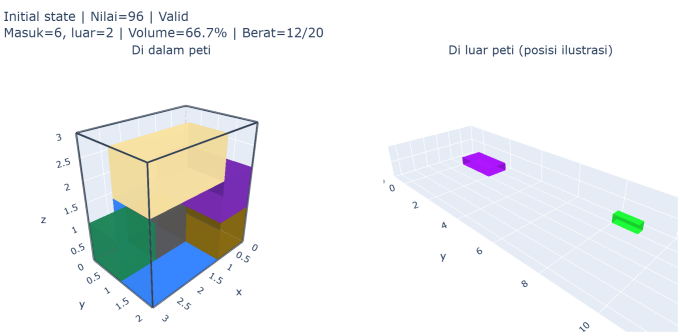
Informasi yang dicatat	**Data hasil pengujian**
Banyak iterasi per *restart*	1, 2, 1, 1 (rata-rata: 1,25)

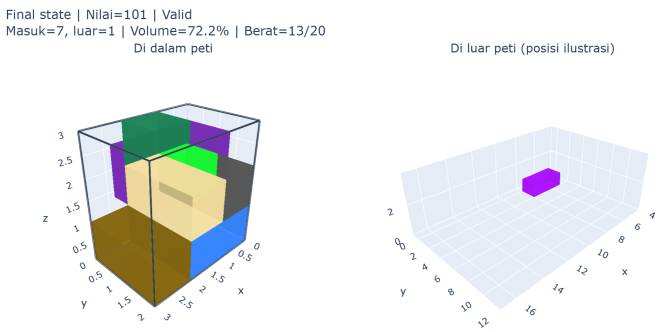
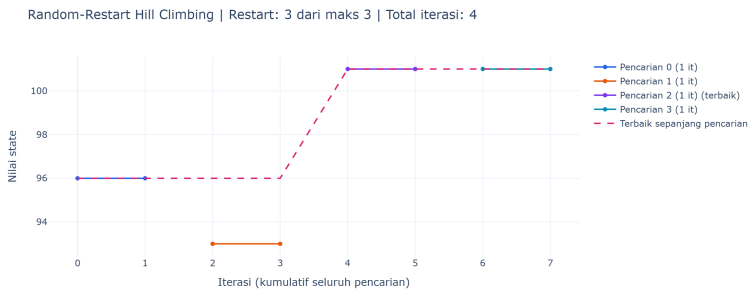
Test case tc9 — Percobaan 2

Informasi yang dicatat	**Data hasil pengujian**
Maximum restart	3
State awal	
State akhir terbaik	
Nilai *objective function* akhir	101

Informasi yang dicatat	**Data hasil pengujian**
Grafik *objective function* terhadap iterasi	
Durasi pencarian	0.074722 detik
Banyak *restart* yang dilakukan	3
Banyak iterasi per *restart*	1, 2, 1, 2 (rata-rata: 1,5)

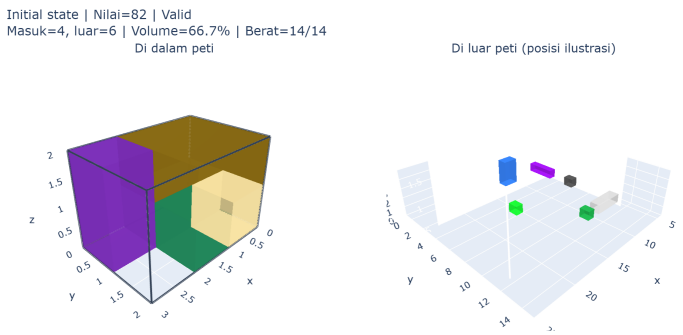
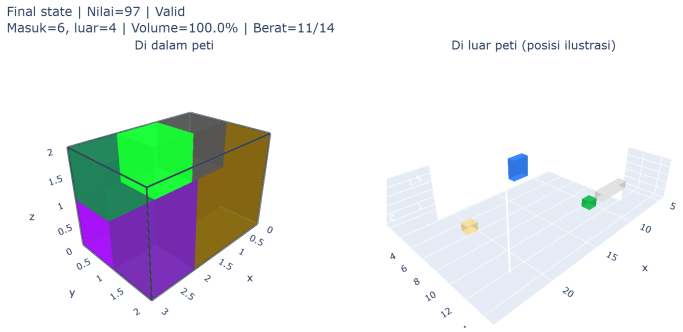
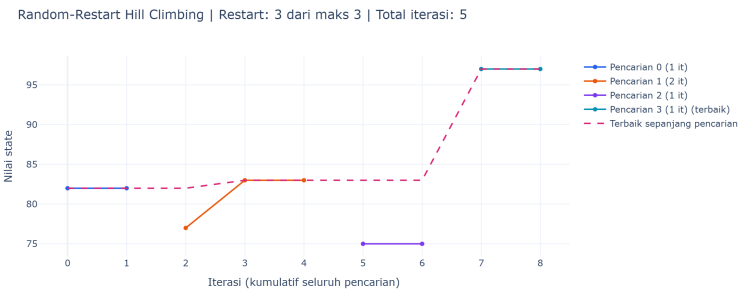
Test case tc9 — Percobaan 3

Informasi yang dicatat	**Data hasil pengujian**
Maximum restart	3
State awal	

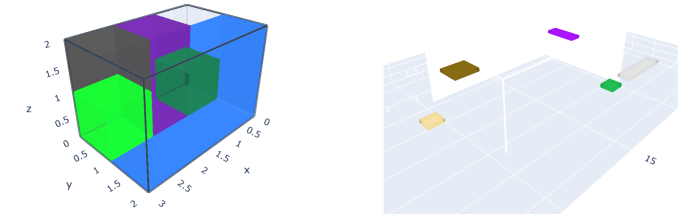
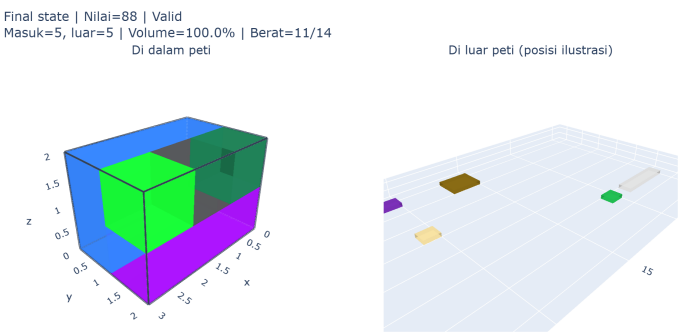
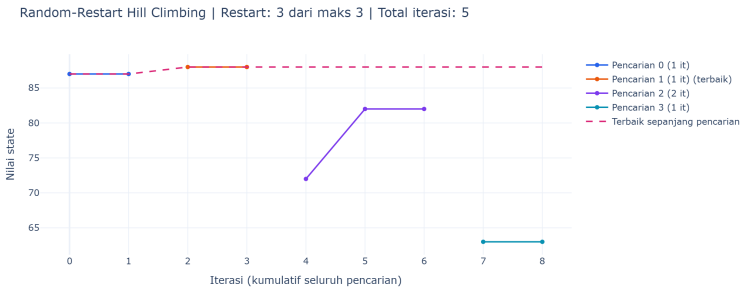
Informasi yang dicatat	**Data hasil pengujian**
State akhir terbaik	
Nilai *objective function* akhir	101
Grafik *objective function* terhadap iterasi	
Durasi pencarian	0.055818 detik
Banyak *restart* yang dilakukan	3
Banyak iterasi per *restart*	1, 1, 1, 1 (rata-rata: 1)

Test case tc10 — Percobaan 1

Informasi yang dicatat	**Data hasil pengujian**
Maximum restart	3

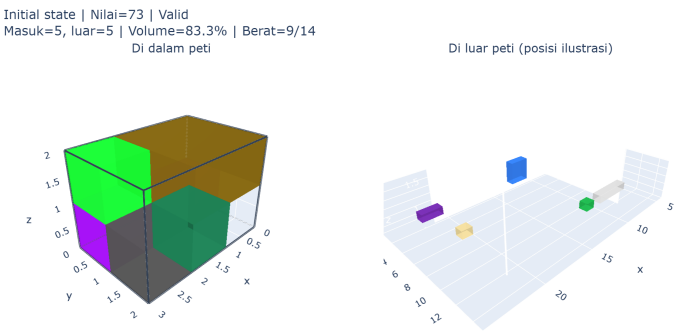
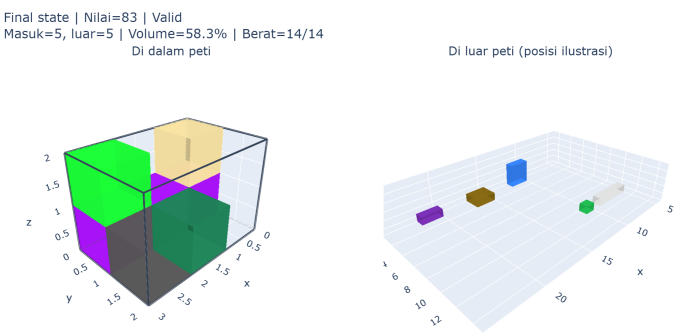
Informasi yang dicatat	**Data hasil pengujian**
State awal	
State akhir terbaik	
Nilai *objective function* akhir	97
Grafik *objective function* terhadap iterasi	
Durasi pencarian	0.053462 detik
Banyak *restart* yang dilakukan	3
Banyak iterasi per *restart*	1, 2, 1, 1 (rata-rata: 1,25)

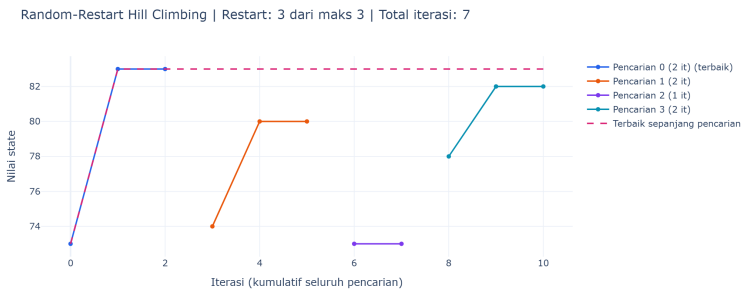
Test case tc10 — Percobaan 2

Informasi yang dicatat	Data hasil pengujian
Maximum restart	3
State awal	
State akhir terbaik	
Nilai *objective function* akhir	88
Grafik *objective function* terhadap iterasi	
Durasi pencarian	0.055554 detik
Banyak *restart* yang dilakukan	3

Informasi yang dicatat	**Data hasil pengujian**
Banyak iterasi per *restart*	1, 1, 2, 1 (rata-rata: 1,25)

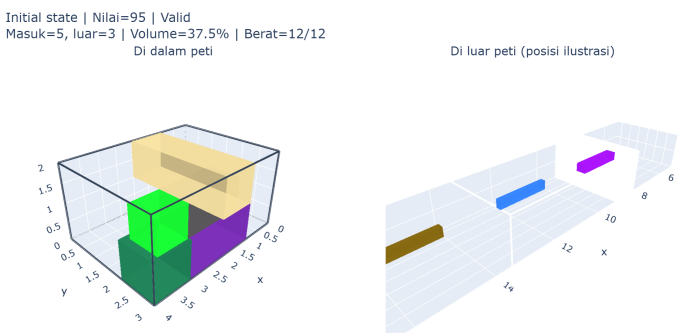
Test case tc10 — Percobaan 3

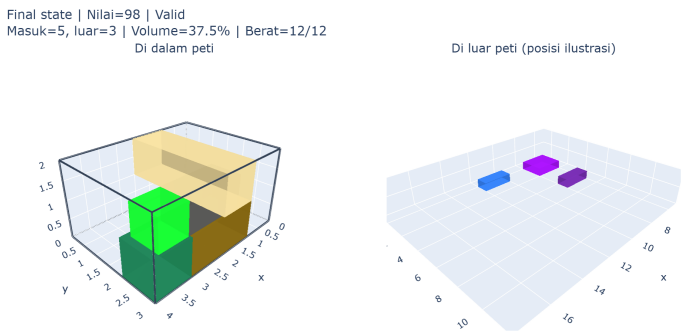
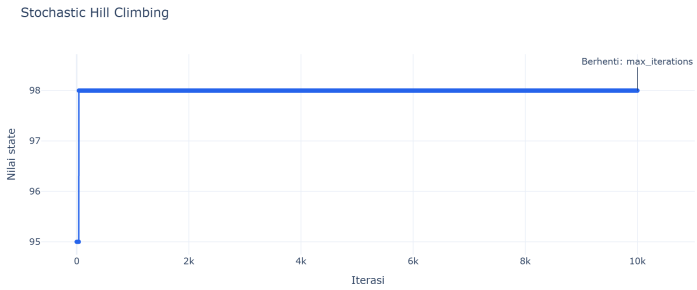
Informasi yang dicatat	**Data hasil pengujian**
Maximum restart	3
State awal	
State akhir terbaik	
Nilai *objective function* akhir	83

Informasi yang dicatat	Data hasil pengujian
Grafik *objective function* terhadap iterasi	
Durasi pencarian	0.075717 detik
Banyak *restart* yang dilakukan	3
Banyak iterasi per *restart*	2, 2, 1, 2 (rata-rata: 1,75)

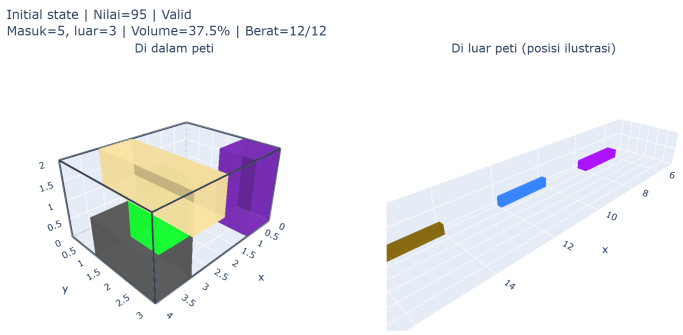
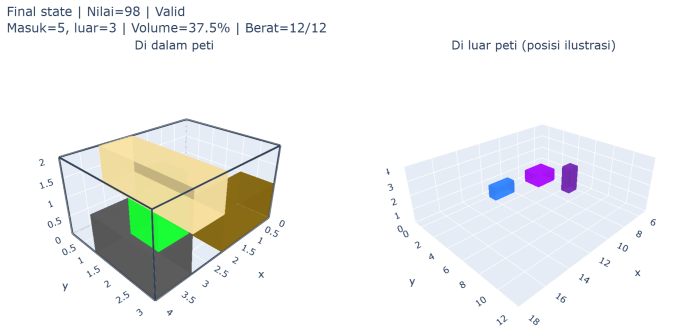
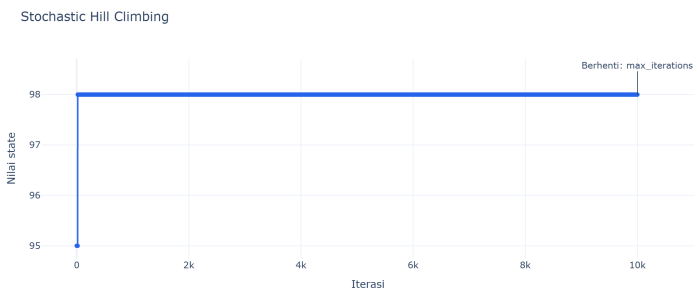
4.1.4 *Stochastic Hill-climbing*

Test case tc8 — Percobaan 1

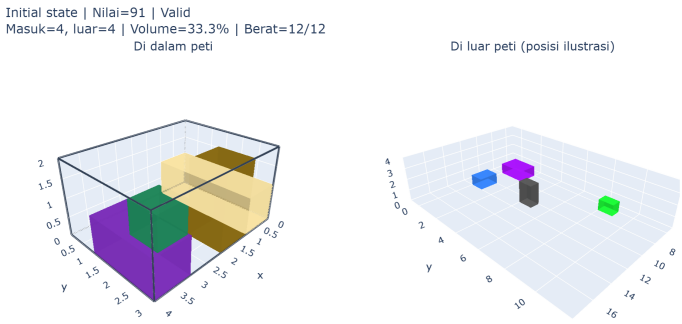
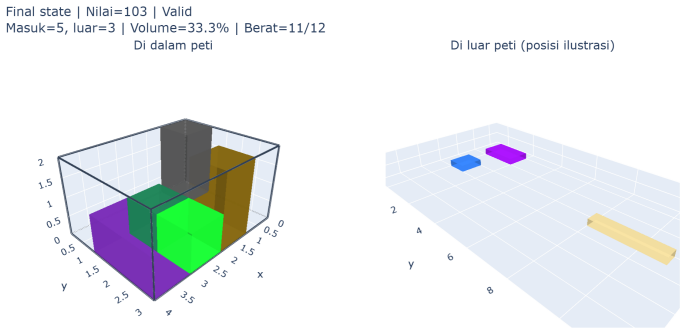
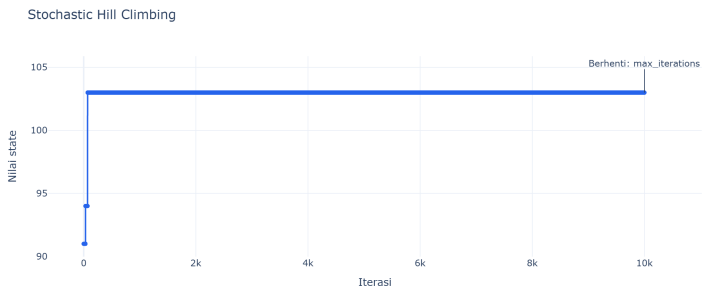
Informasi yang dicatat	Data hasil pengujian
State awal	

Informasi yang dicatat	**Data hasil pengujian**
State akhir	
Nilai *objective function* akhir	98
Grafik *objective function* terhadap iterasi	
Durasi pencarian	2.546581 detik
Banyak iterasi hingga berhenti	10.000

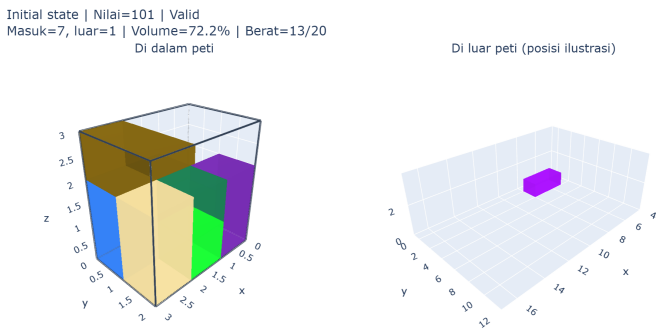
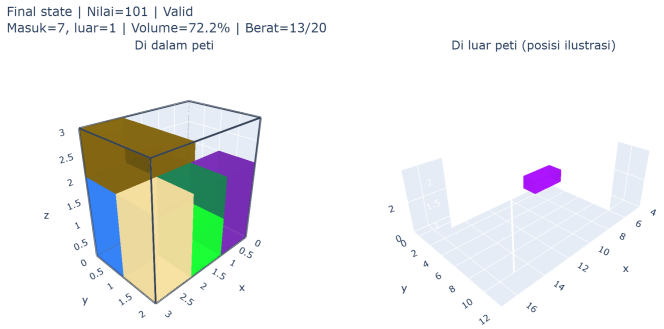
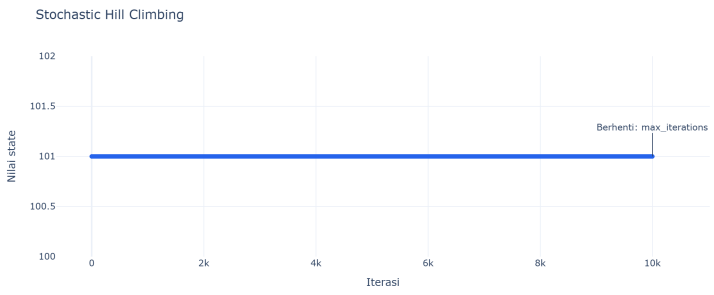
Test case tc8 — Percobaan 2

Informasi yang dicatat	**Data hasil pengujian**
State awal	
State akhir	
Nilai *objective function* akhir	98
Grafik *objective function* terhadap iterasi	
Durasi pencarian	2.677560 detik
Banyak iterasi hingga berhenti	10.000

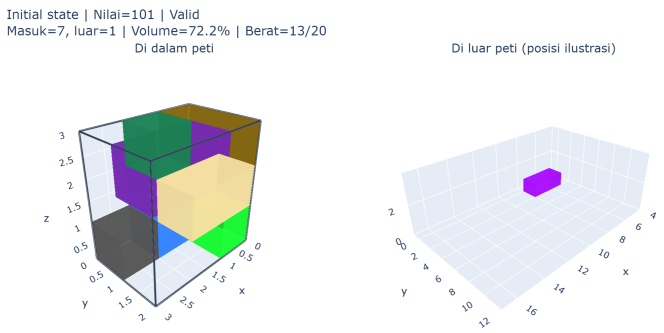
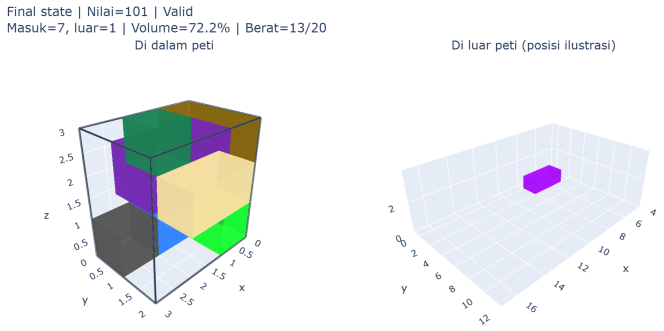
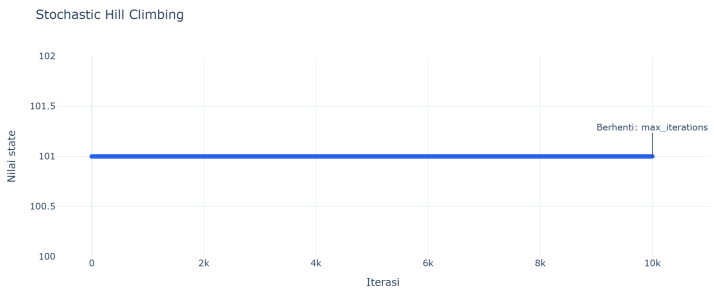
Test case tc8 — Percobaan 3

Informasi yang dicatat	**Data hasil pengujian**
State awal	
State akhir	
Nilai *objective function* akhir	103
Grafik *objective function* terhadap iterasi	
Durasi pencarian	1.668485 detik
Banyak iterasi hingga berhenti	10.000

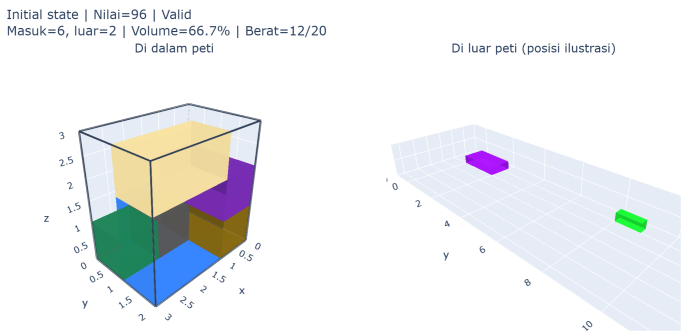
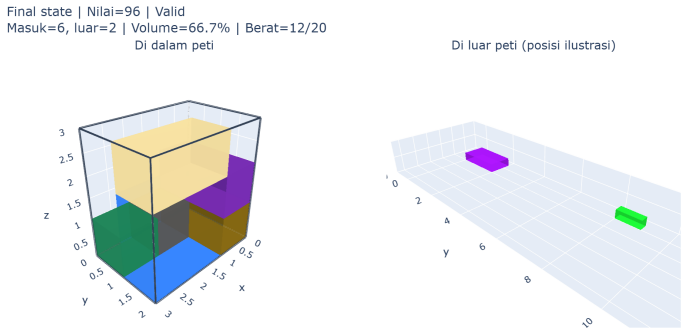
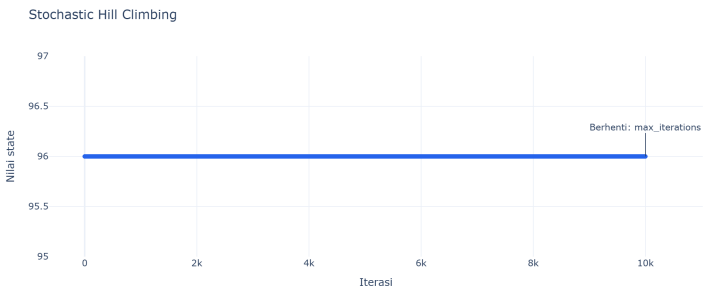
Test case tc9 — Percobaan 1

Informasi yang dicatat	**Data hasil pengujian**
State awal	
State akhir	
Nilai *objective function* akhir	101
Grafik *objective function* terhadap iterasi	
Durasi pencarian	11.263783 detik
Banyak iterasi hingga berhenti	10.000

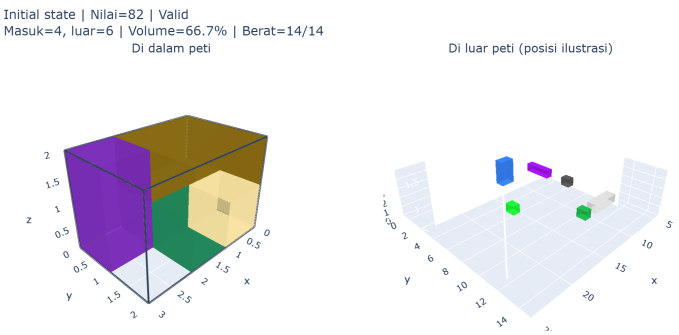
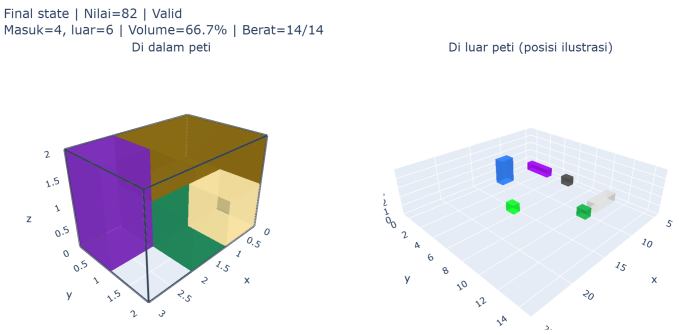
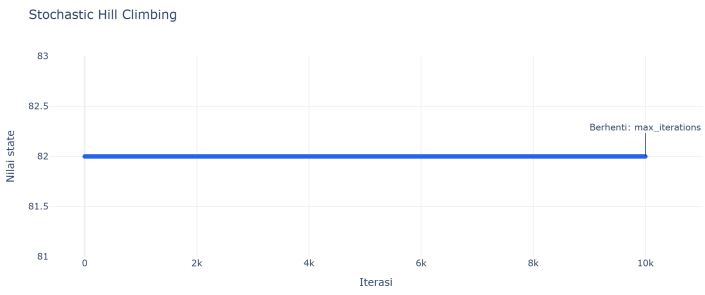
Test case tc9 — Percobaan 2

Informasi yang dicatat	**Data hasil pengujian**
State awal	
State akhir	
Nilai *objective function* akhir	101
Grafik *objective function* terhadap iterasi	
Durasi pencarian	14.746734 detik
Banyak iterasi hingga berhenti	10.000

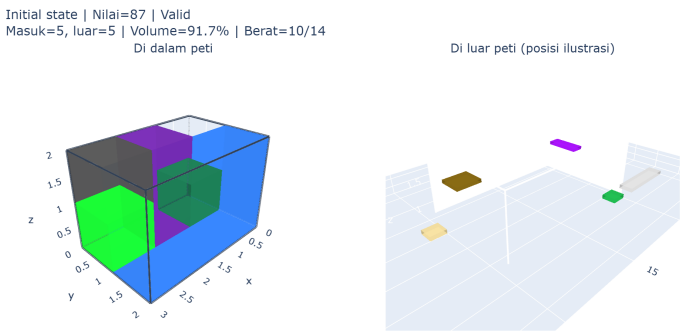
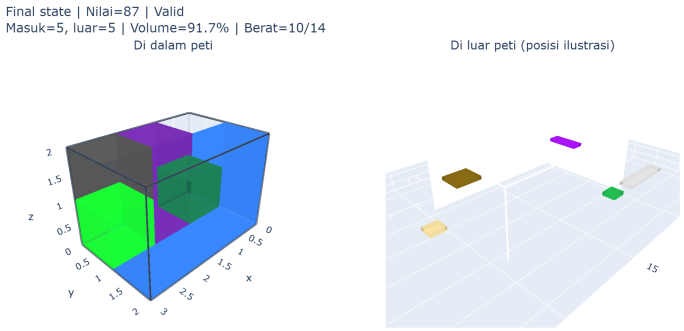
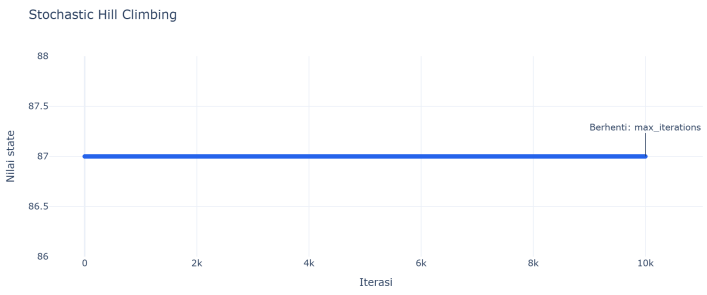
Test case tc9 — Percobaan 3

Informasi yang dicatat	**Data hasil pengujian**
State awal	
State akhir	
Nilai *objective function* akhir	96
Grafik *objective function* terhadap iterasi	
Durasi pencarian	11.694191 detik
Banyak iterasi hingga berhenti	10.000

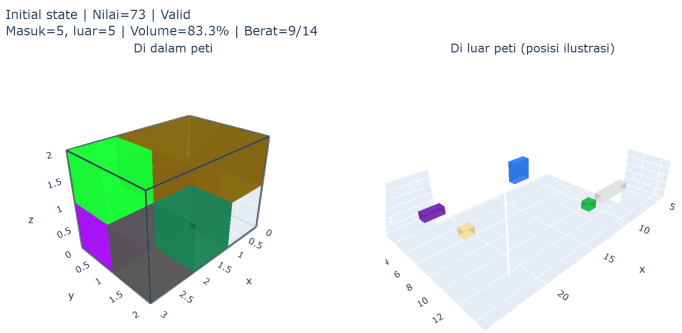
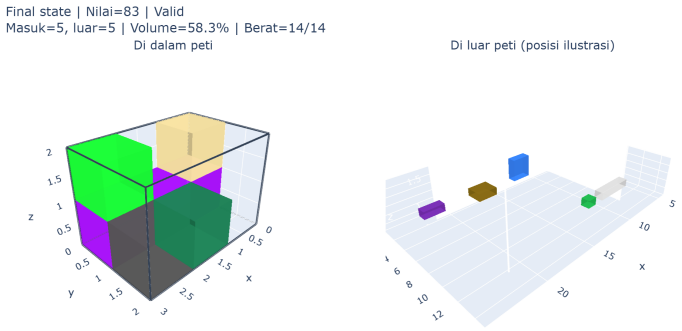
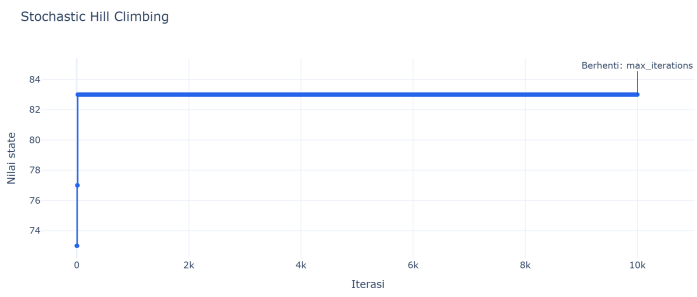
Test case tc10 — Percobaan 1

Informasi yang dicatat	**Data hasil pengujian**
State awal	
State akhir	
Nilai *objective function* akhir	82
Grafik *objective function* terhadap iterasi	
Durasi pencarian	3.434454 detik
Banyak iterasi hingga berhenti	10.000

Test case tc10 — Percobaan 2

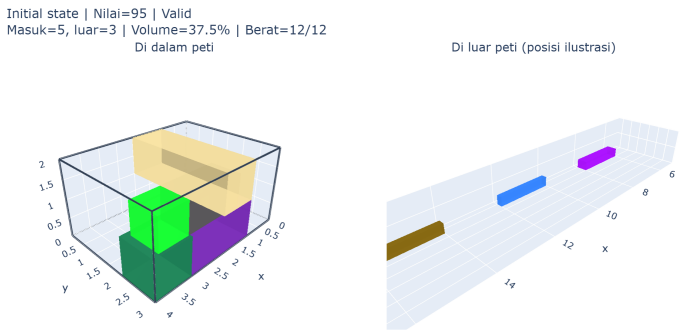
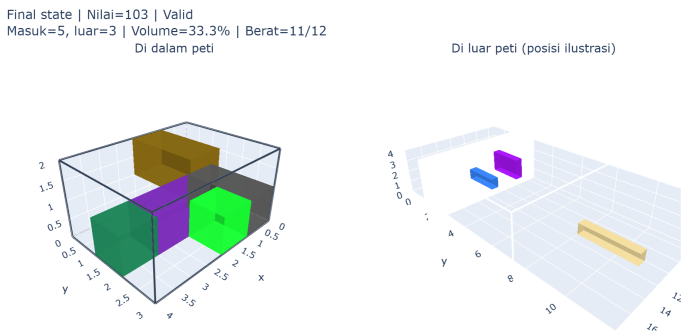
Informasi yang dicatat	**Data hasil pengujian**
State awal	
State akhir	
Nilai *objective function* akhir	87
Grafik *objective function* terhadap iterasi	
Durasi pencarian	9.042528 detik
Banyak iterasi hingga berhenti	10.000

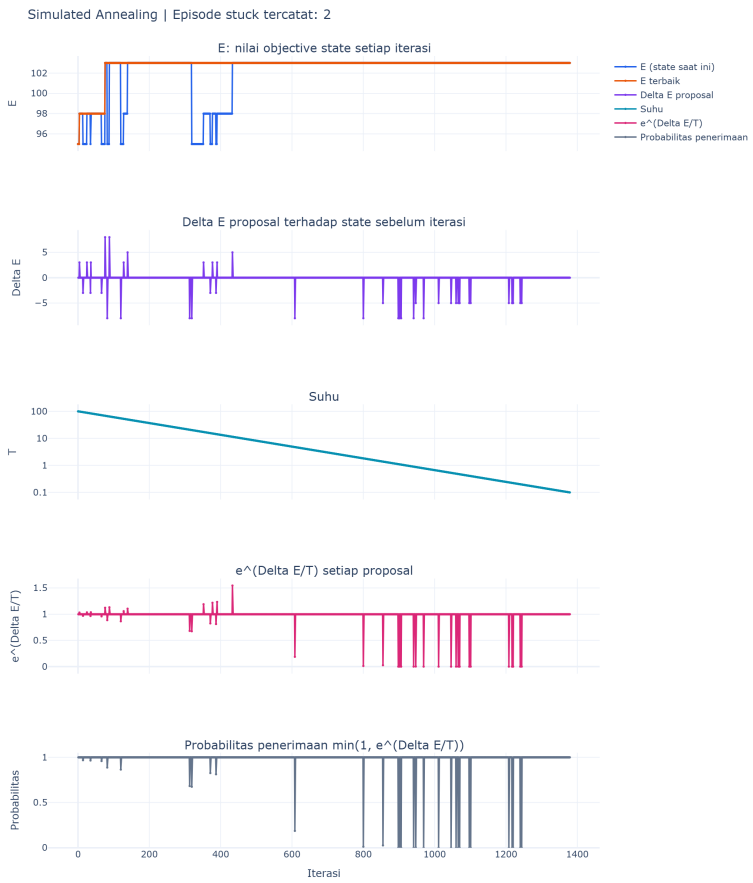
Test case tc10 — Percobaan 3

Informasi yang dicatat	**Data hasil pengujian**
State awal	
State akhir	
Nilai *objective function* akhir	83
Grafik *objective function* terhadap iterasi	
Durasi pencarian	3.207028 detik
Banyak iterasi hingga berhenti	10.000

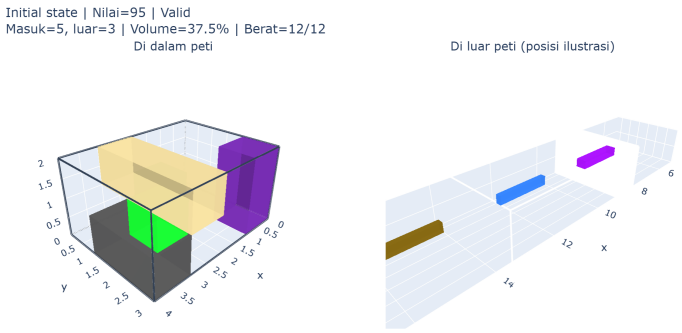
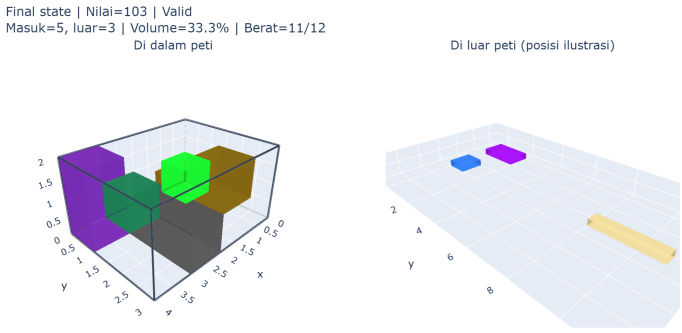
4.2 Eksperimen Algoritma *Simulated Annealing*

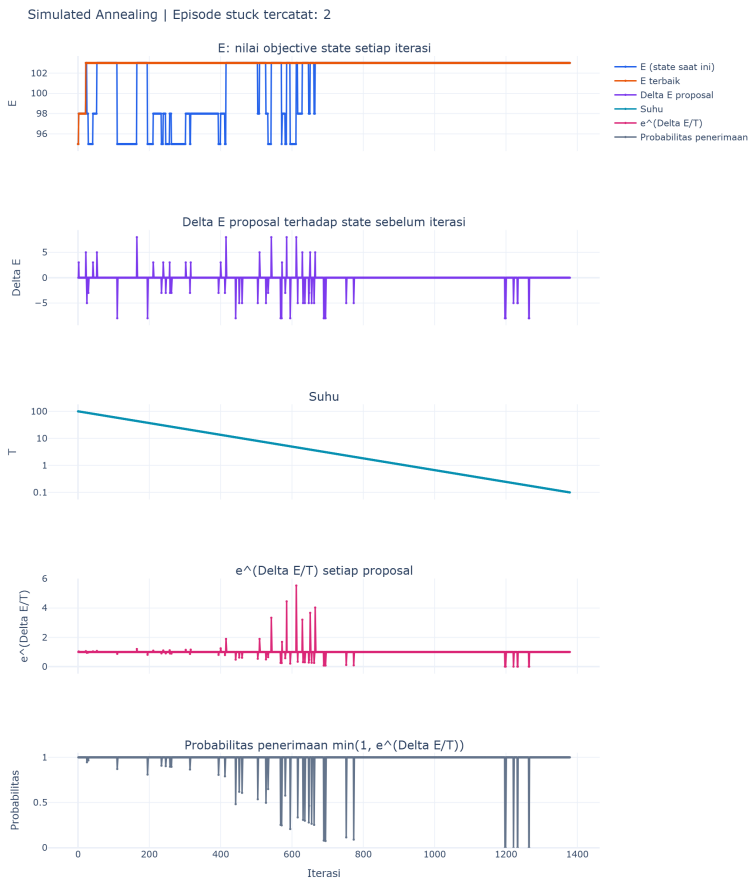
Test case tc8 — Percobaan 1

Informasi yang dicatat	**Data hasil pengujian**
State awal	
State akhir	
Nilai *objective function* akhir	103

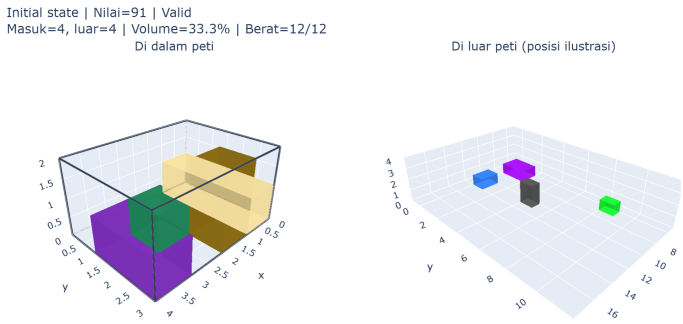
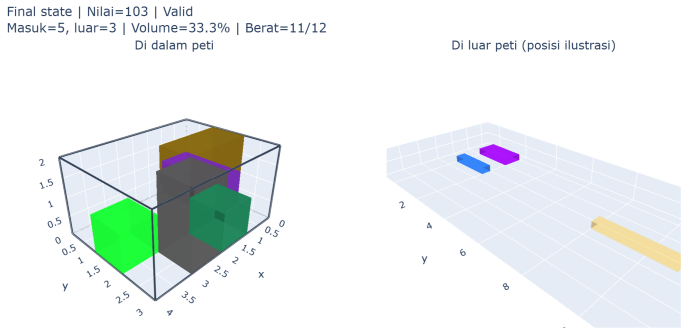
Informasi yang dicatat	**Data hasil pengujian**
Grafik *objective function* terhadap iterasi	
Grafik $e^{\Delta E/T}$ terhadap iterasi	Tercakup pada panel ke-4 grafik multi-panel di atas
Frekuensi *stuck* selama pencarian	2 episode
Durasi pencarian	0.332440 detik

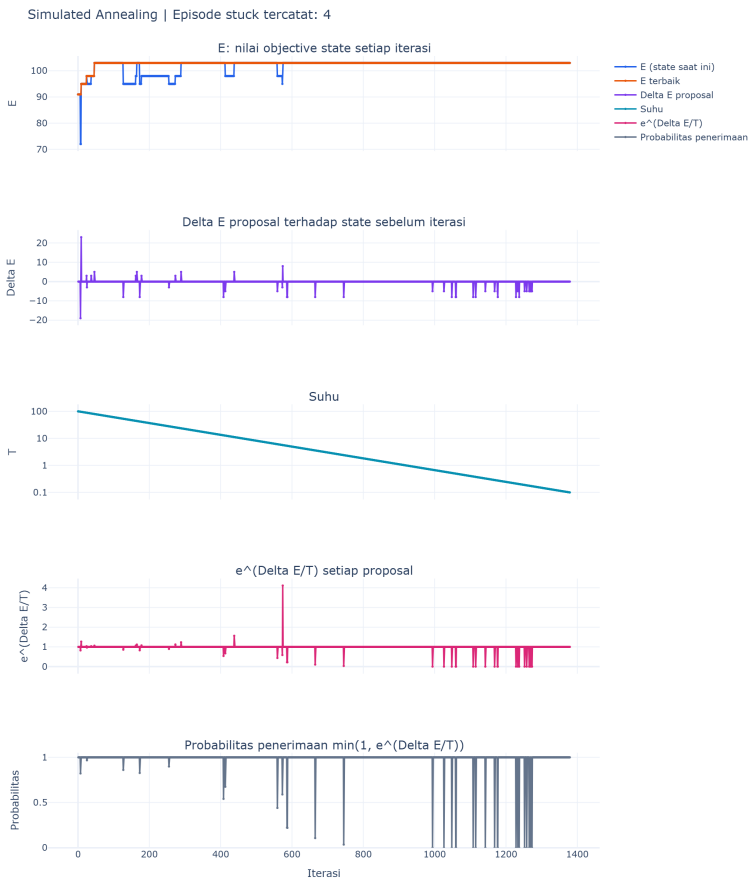
Test case tc8 — Percobaan 2

Informasi yang dicatat	**Data hasil pengujian**
State awal	
State akhir	
Nilai *objective function* akhir	103

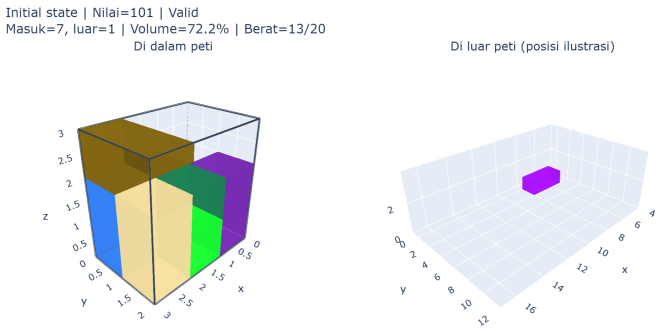
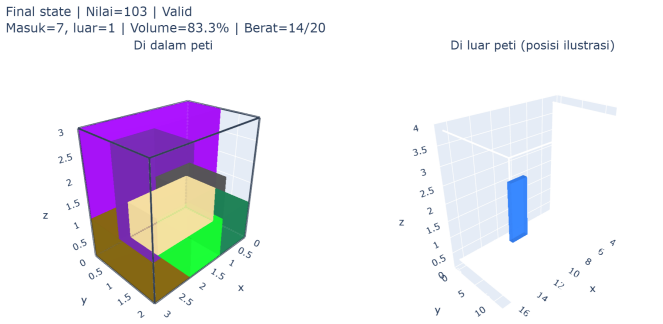
Informasi yang dicatat	Data hasil pengujian
Grafik *objective function* terhadap iterasi	
Grafik $e^{\Delta E/T}$ terhadap iterasi	Tercakup pada panel ke-4 grafik multi-panel di atas
Frekuensi *stuck* selama pencarian	2 episode
Durasi pencarian	0.338796 detik

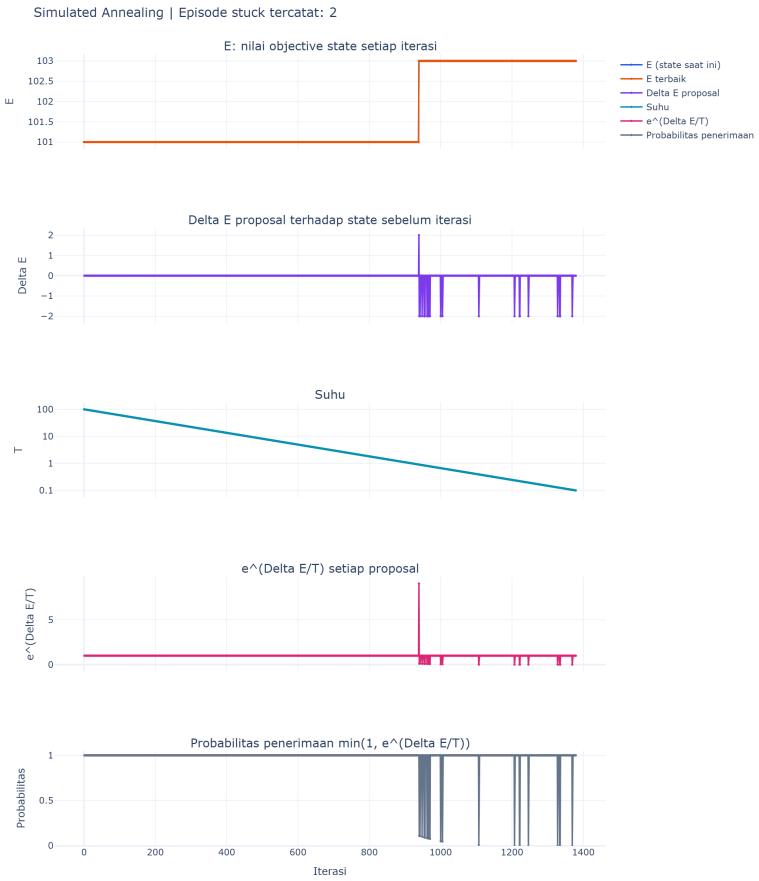
Test case tc8 — Percobaan 3

Informasi yang dicatat	**Data hasil pengujian**
State awal	
State akhir	
Nilai *objective function* akhir	103

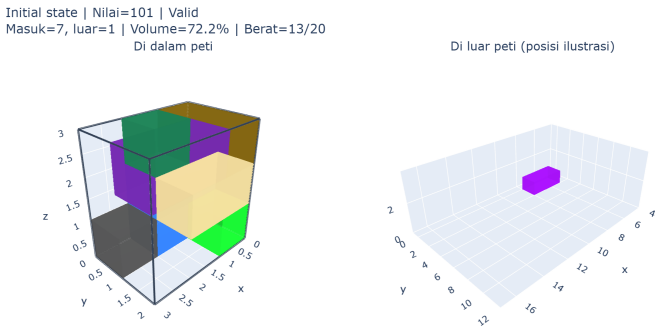
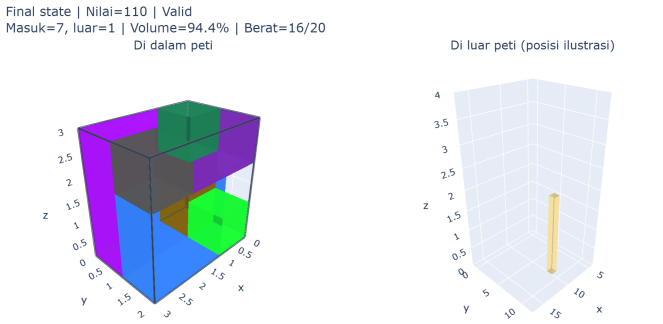
Informasi yang dicatat	**Data hasil pengujian**
Grafik *objective function* terhadap iterasi	
Grafik $e^{\Delta E/T}$ terhadap iterasi	Tercakup pada panel ke-4 grafik multi-panel di atas
Frekuensi *stuck* selama pencarian	4 episode
Durasi pencarian	0.337923 detik

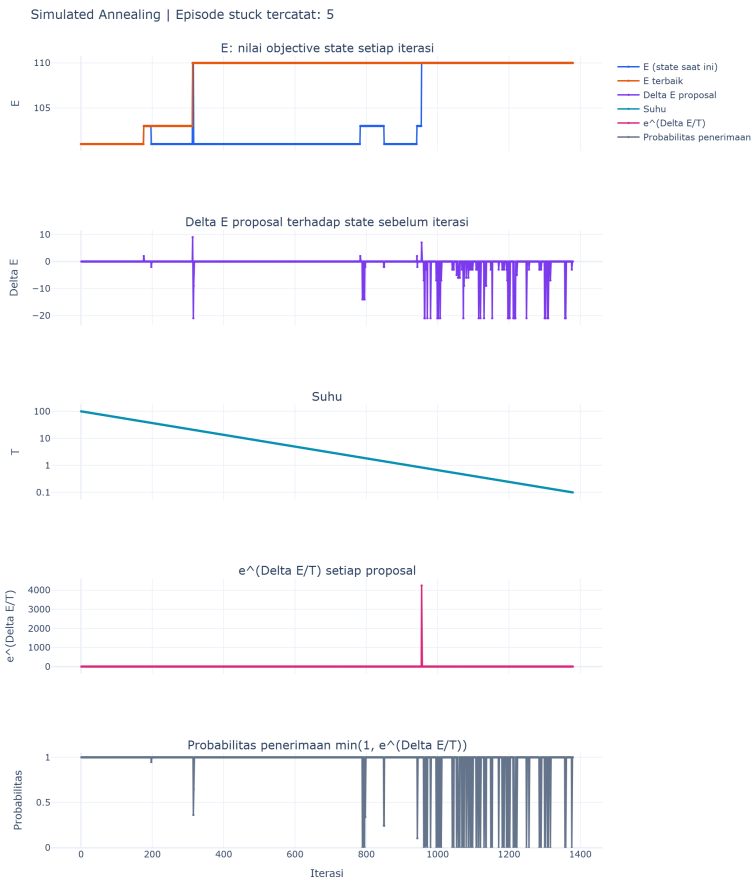
Test case tc9 — Percobaan 1

Informasi yang dicatat	**Data hasil pengujian**
State awal	
State akhir	
Nilai *objective function* akhir	103

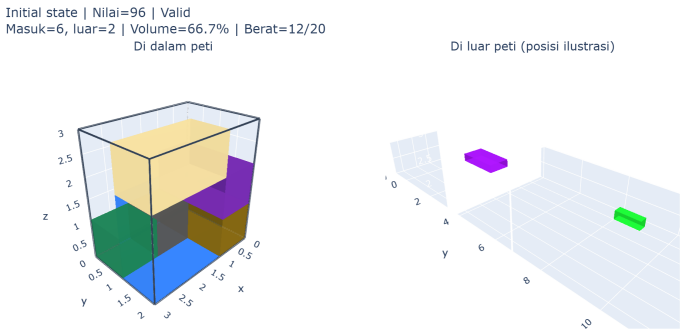
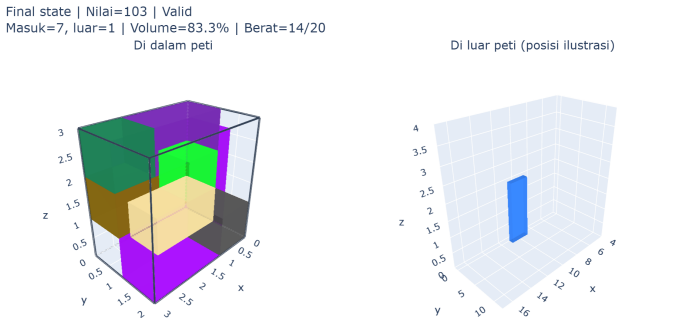
Informasi yang dicatat	**Data hasil pengujian**
Grafik *objective function* terhadap iterasi	
Grafik $e^{\Delta E/T}$ terhadap iterasi	Tercakup pada panel ke-4 grafik multi-panel di atas
Frekuensi *stuck* selama pencarian	2 episode
Durasi pencarian	1.752310 detik

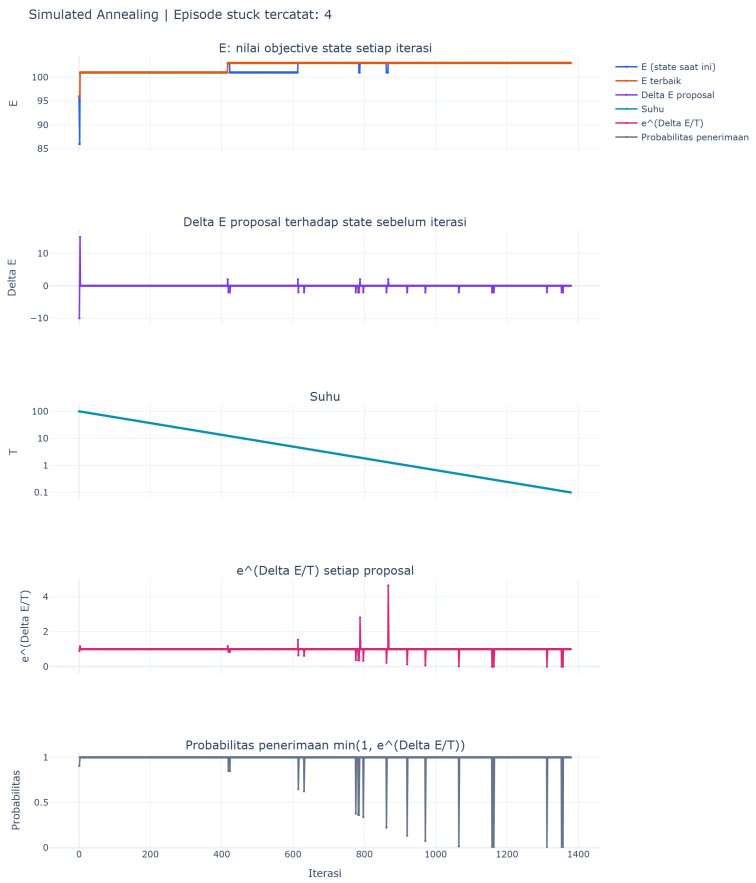
Test case tc9 — Percobaan 2

Informasi yang dicatat	**Data hasil pengujian**
State awal	
State akhir	
Nilai *objective function* akhir	110

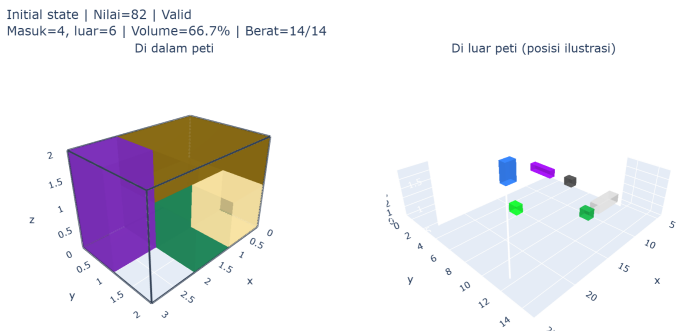
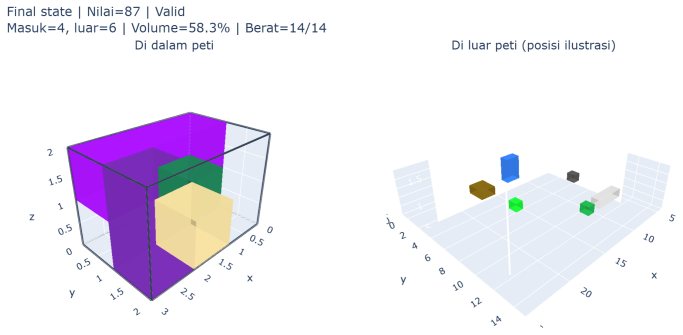
Informasi yang dicatat	**Data hasil pengujian**
Grafik *objective function* terhadap iterasi	
Grafik $e^{\Delta E/T}$ terhadap iterasi	Tercakup pada panel ke-4 grafik multi-panel di atas
Frekuensi *stuck* selama pencarian	5 episode
Durasi pencarian	1.432849 detik

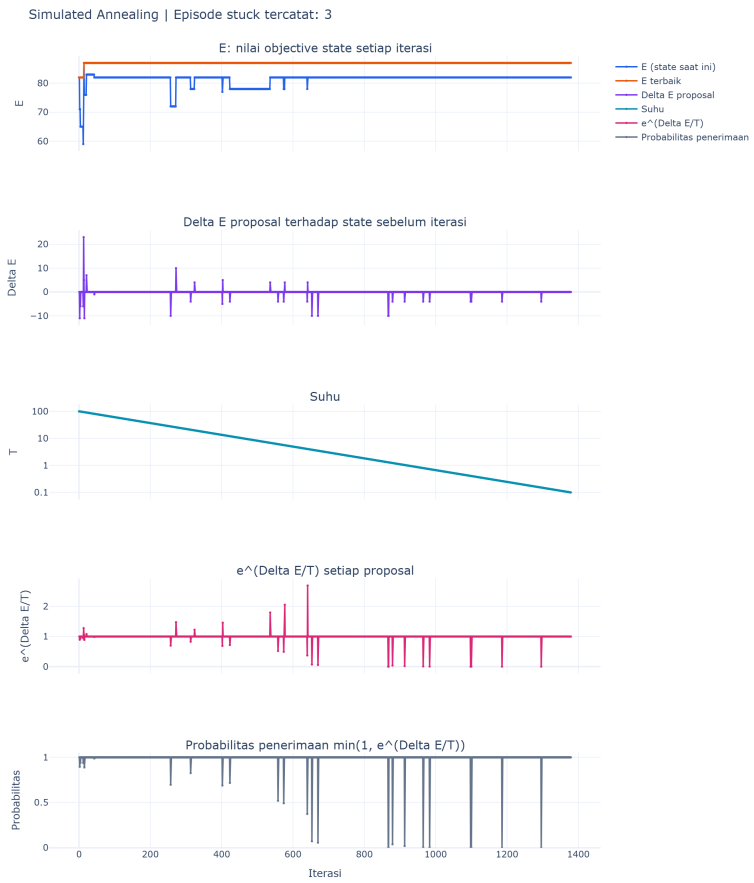
Test case tc9 — Percobaan 3

Informasi yang dicatat	**Data hasil pengujian**
State awal	
State akhir	
Nilai *objective function* akhir	103

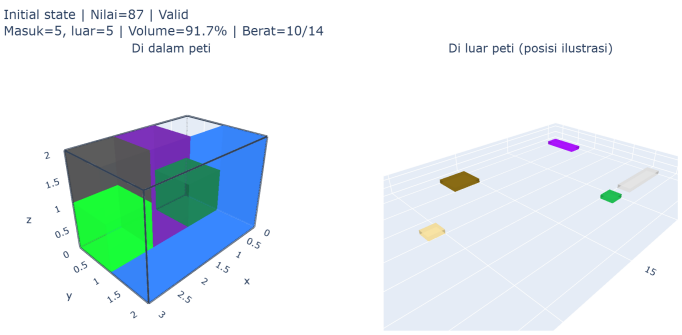
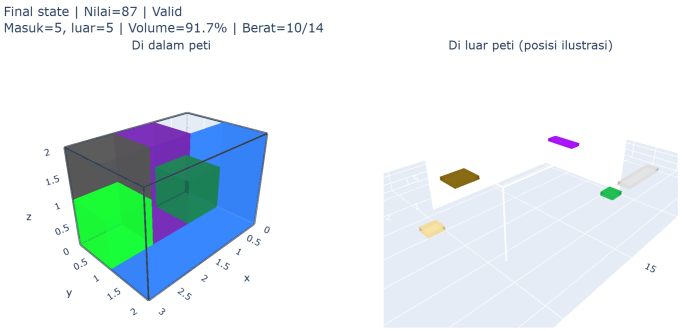
Informasi yang dicatat	**Data hasil pengujian**
Grafik *objective function* terhadap iterasi	
Grafik $e^{\Delta E/T}$ terhadap iterasi	Tercakup pada panel ke-4 grafik multi-panel di atas
Frekuensi *stuck* selama pencarian	4 episode
Durasi pencarian	1.663256 detik

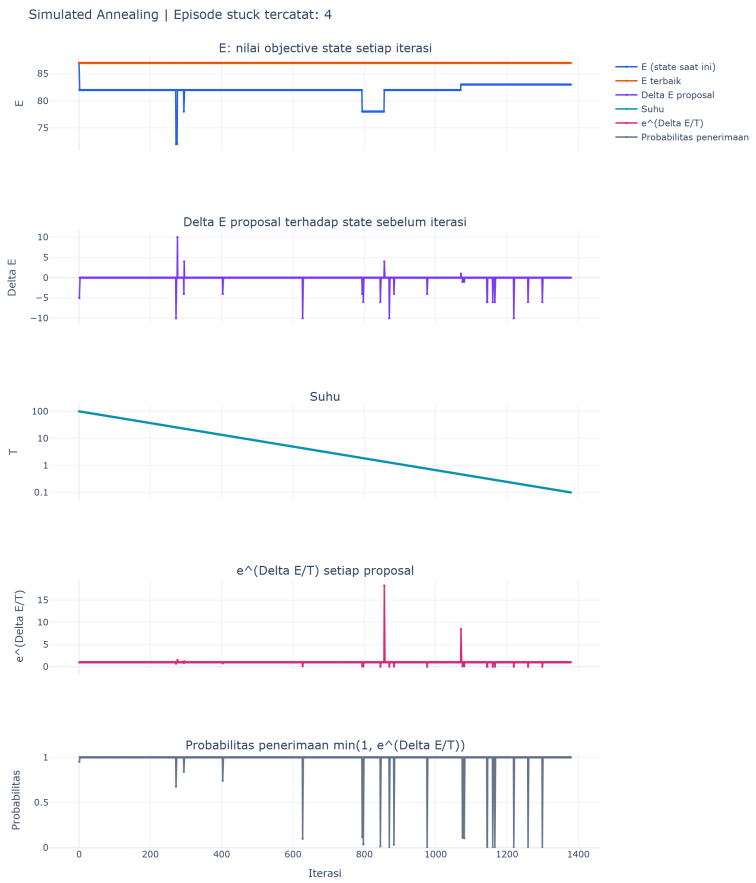
Test case tc10 — Percobaan 1

Informasi yang dicatat	**Data hasil pengujian**
State awal	
State akhir	
Nilai *objective function* akhir	87

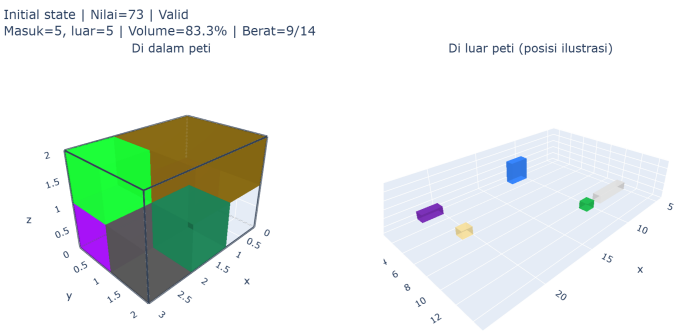
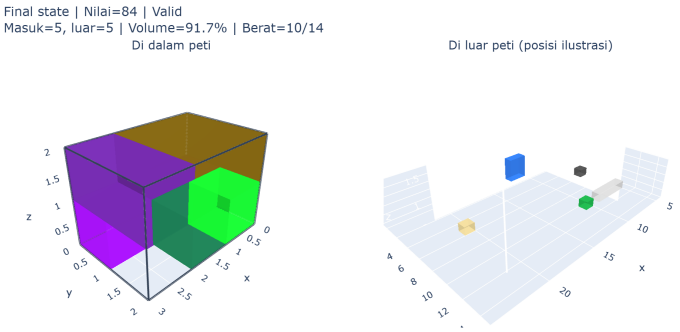
Informasi yang dicatat	**Data hasil pengujian**
Grafik *objective function* terhadap iterasi	
Grafik $e^{\Delta E/T}$ terhadap iterasi	Tercakup pada panel ke-4 grafik multi-panel di atas
Frekuensi *stuck* selama pencarian	3 episode
Durasi pencarian	0.407715 detik

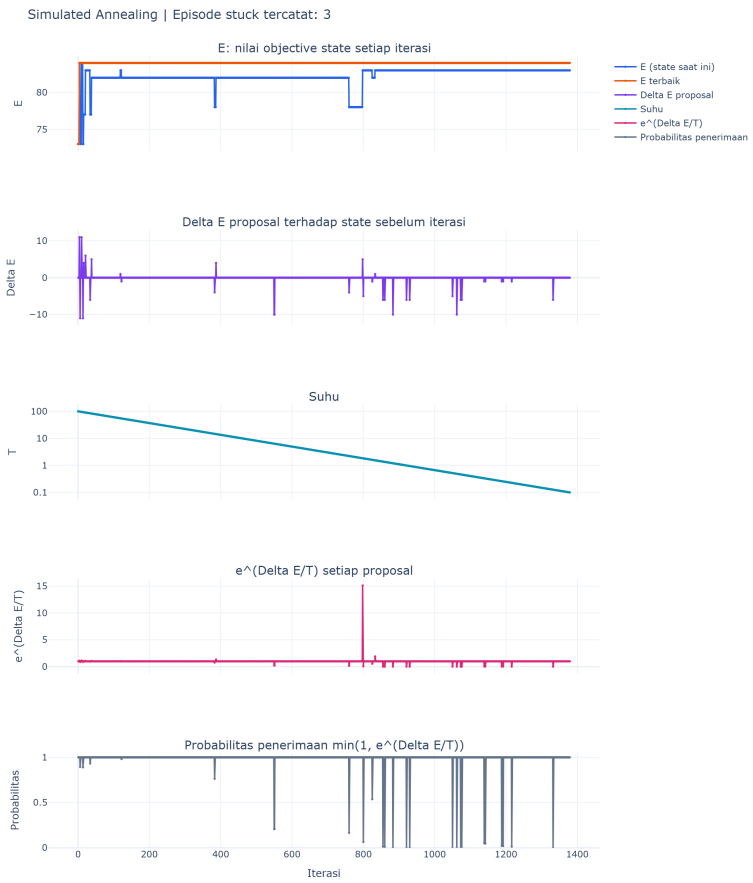
Test case tc10 — Percobaan 2

Informasi yang dicatat	**Data hasil pengujian**
State awal	
State akhir	
Nilai *objective function* akhir	87

Informasi yang dicatat	**Data hasil pengujian**
Grafik *objective function* terhadap iterasi	
Grafik $e^{\Delta E/T}$ terhadap iterasi	Tercakup pada panel ke-4 grafik multi-panel di atas
Frekuensi *stuck* selama pencarian	4 episode
Durasi pencarian	0.469470 detik

Test case tc10 — Percobaan 3

Informasi yang dicatat	**Data hasil pengujian**
State awal	
State akhir	
Nilai *objective function* akhir	84

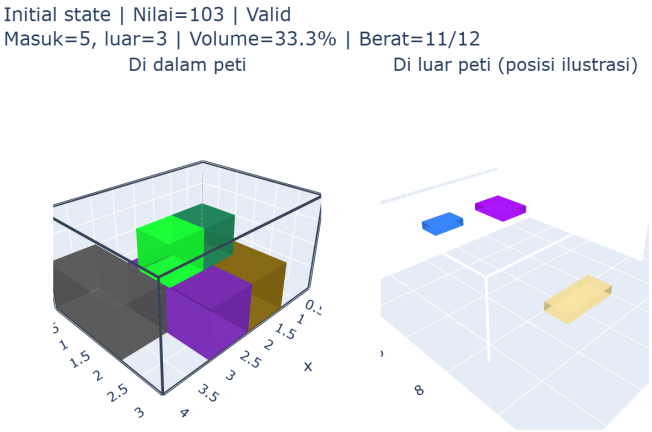
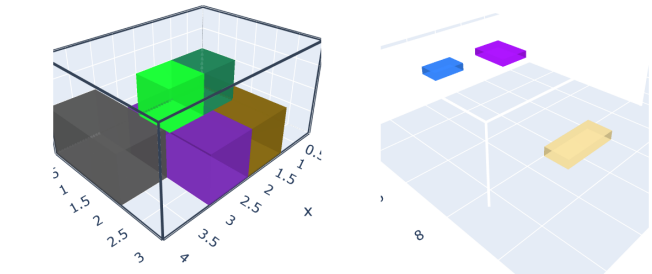
Informasi yang dicatat	**Data hasil pengujian**
Grafik *objective function* terhadap iterasi	
Grafik $e^{\Delta E/T}$ terhadap iterasi	Tercakup pada panel ke-4 grafik multi-panel di atas
Frekuensi *stuck* selama pencarian	3 episode
Durasi pencarian	0.475188 detik

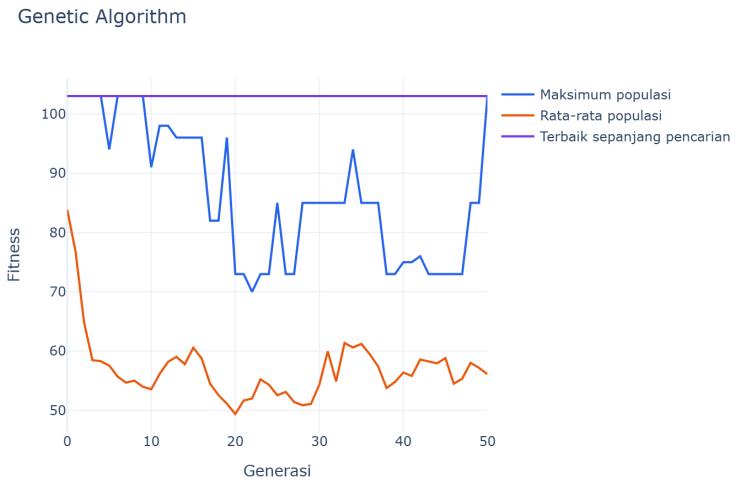
4.3 Eksperimen Algoritma *Genetic Algorithm*

Setiap konfigurasi dijalankan tiga kali. Konfigurasi populasi 50 dan iterasi 100 dapat muncul pada kedua rangkaian karena merupakan konfigurasi acuan yang sama; hasil tiga *run* yang sama dapat dirujuk pada kedua rangkaian apabila tidak dijalankan ulang.

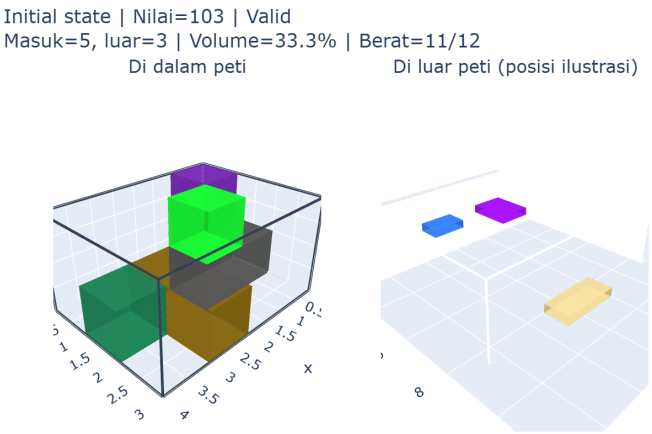
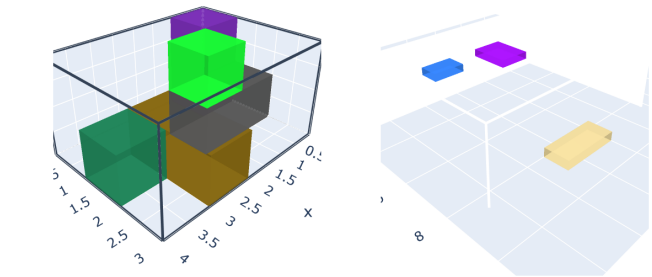
4.3.1 Populasi Tetap, Variasi Banyak Iterasi

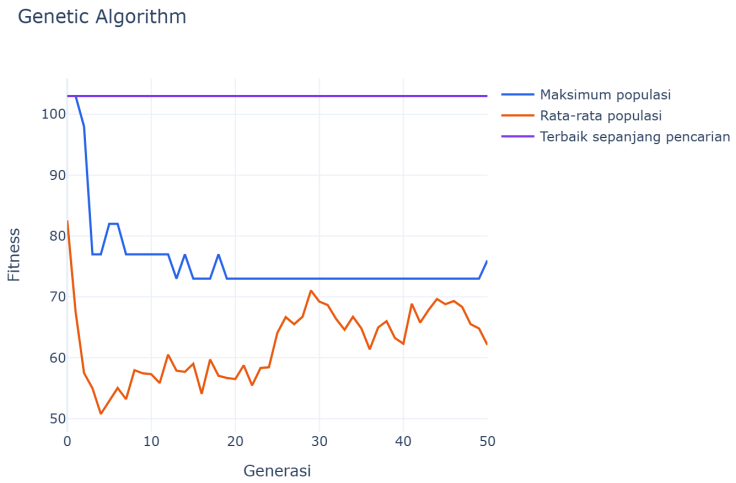
Test case tc8 — Populasi 50, iterasi 50 — Percobaan 1

Informasi yang dicatat	Data hasil pengujian
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	103

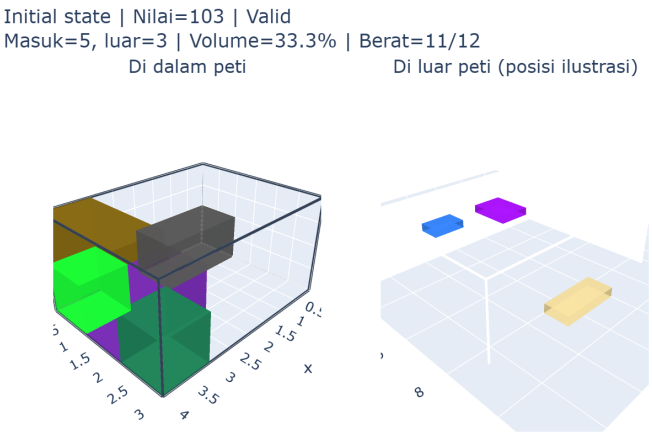
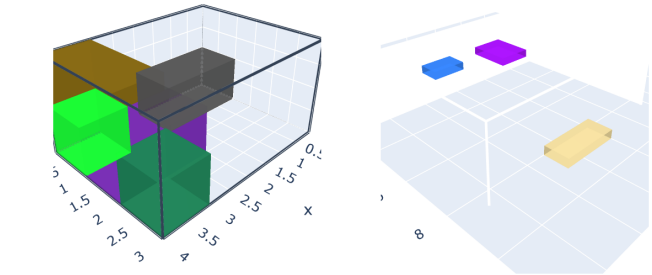
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	50
Durasi pencarian	0.134032 detik

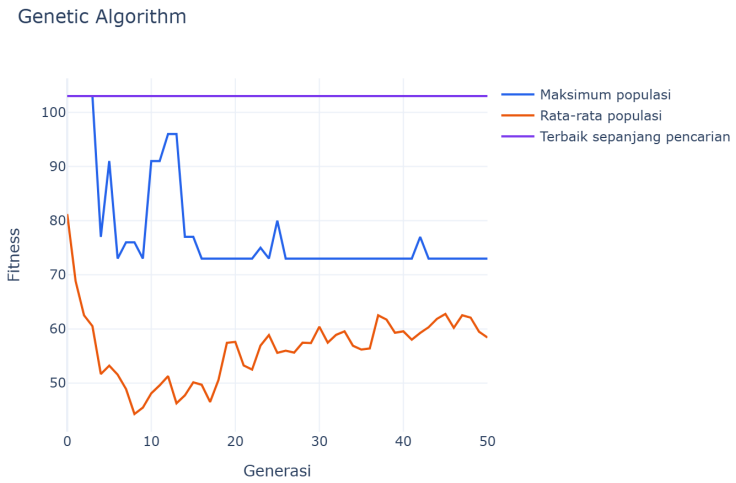
Test case tc8 — Populasi 50, iterasi 50 — Percobaan 2

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	103

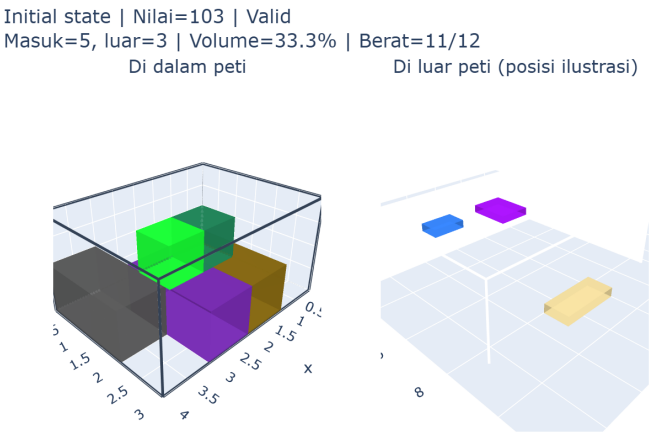
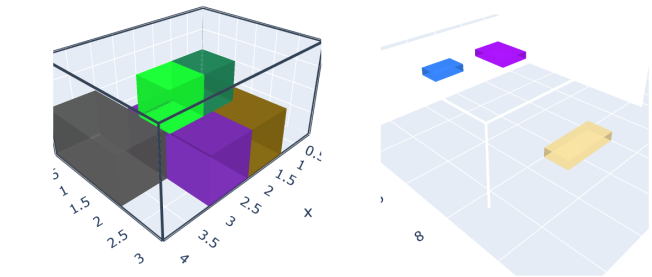
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	50
Durasi pencarian	0.127257 detik

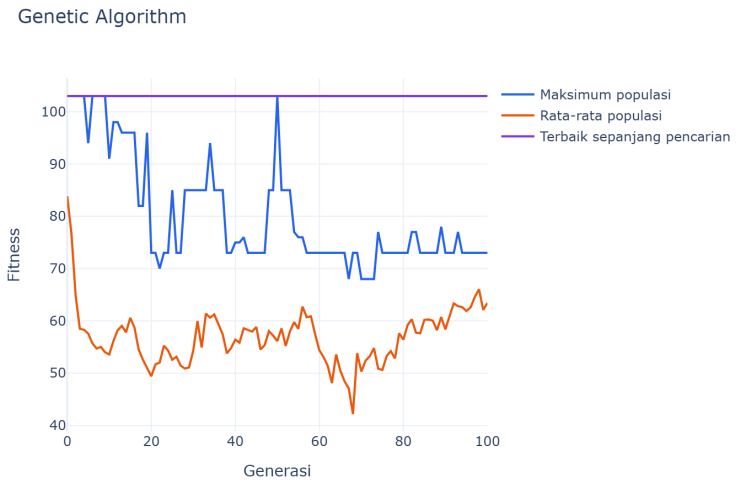
Test case tc8 — Populasi 50, iterasi 50 — Percobaan 3

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	103

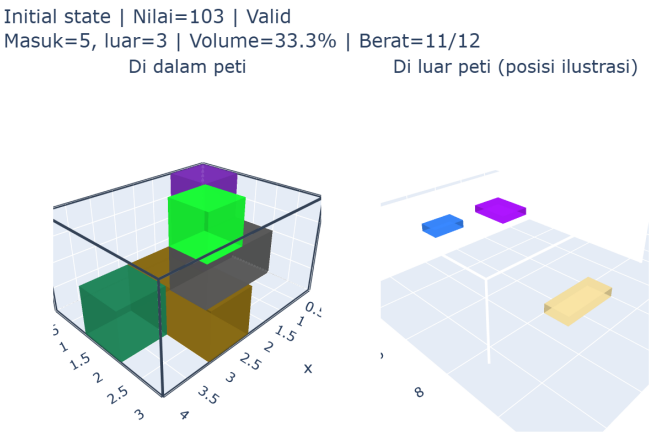
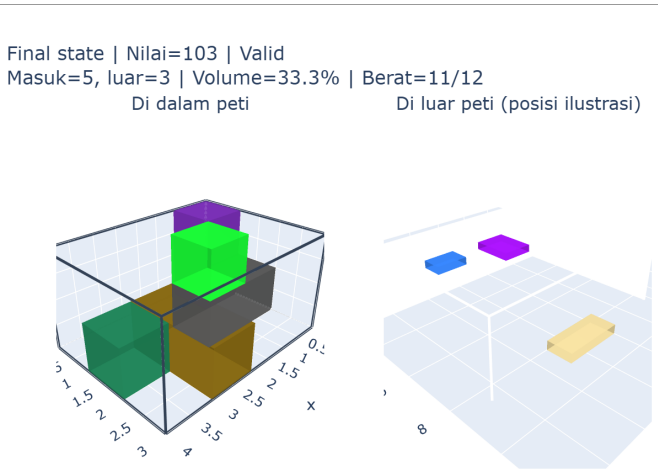
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	50
Durasi pencarian	0.131241 detik

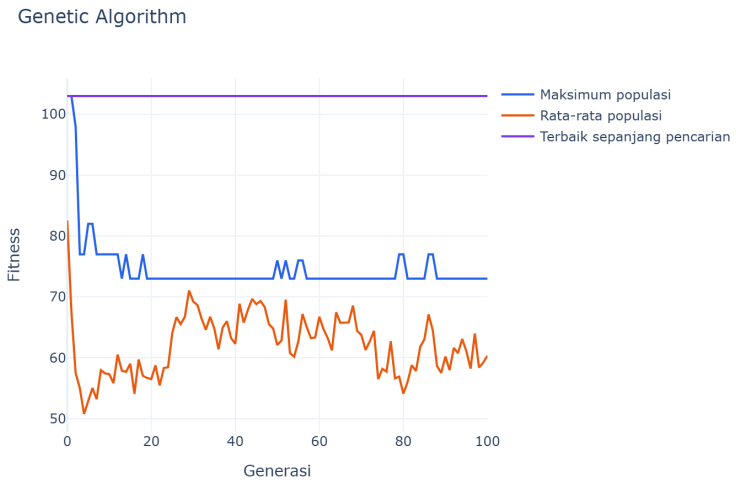
Test case tc8 — Populasi 50, iterasi 100 — Percobaan 1

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	103

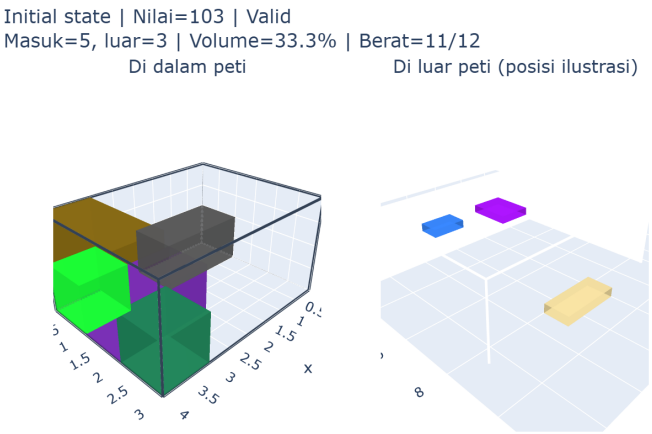
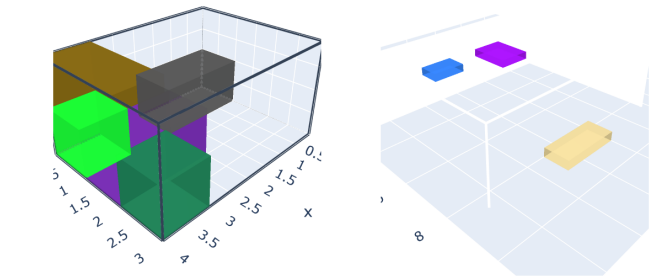
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	100
Durasi pencarian	0.254834 detik

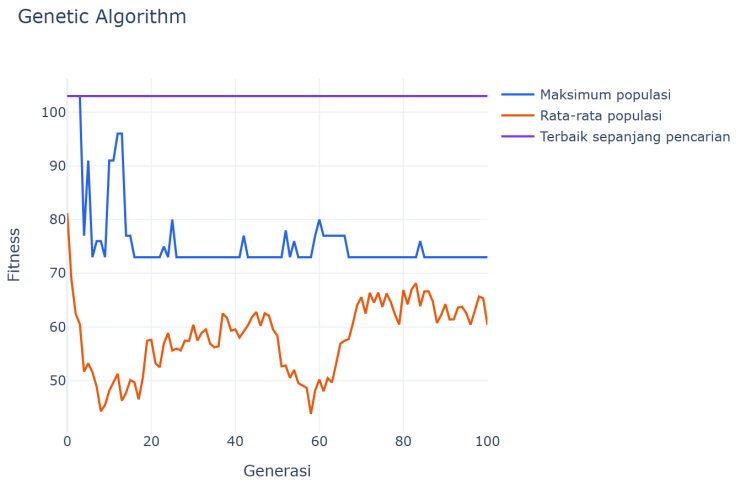
Test case tc8 — Populasi 50, iterasi 100 — Percobaan 2

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	103

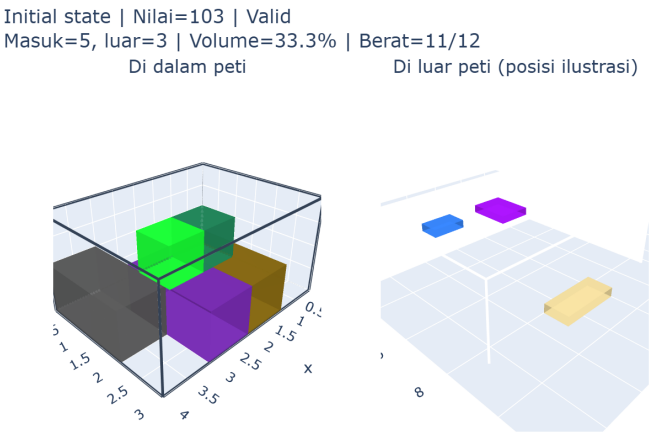
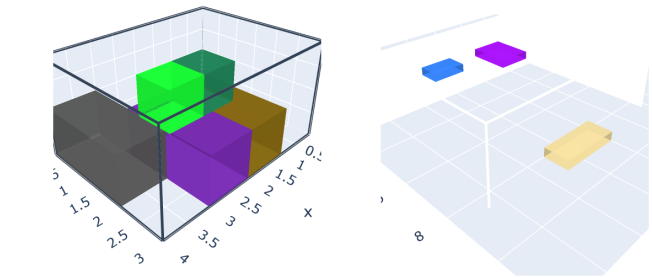
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	100
Durasi pencarian	0.26839 detik

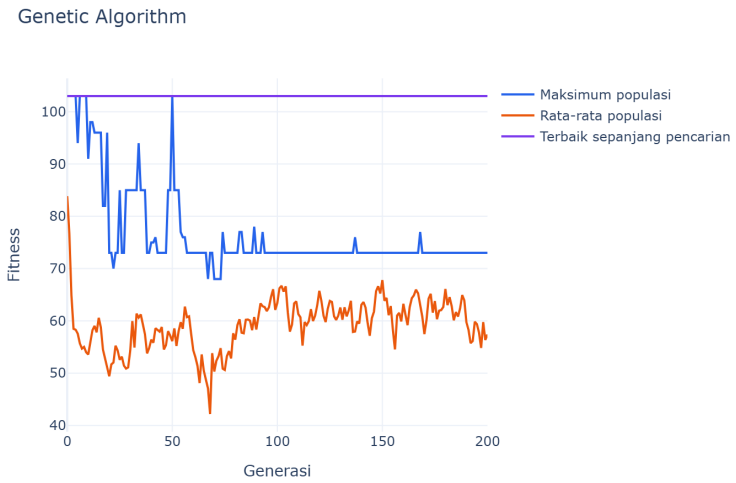
Test case tc8 — Populasi 50, iterasi 100 — Percobaan 3

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	103

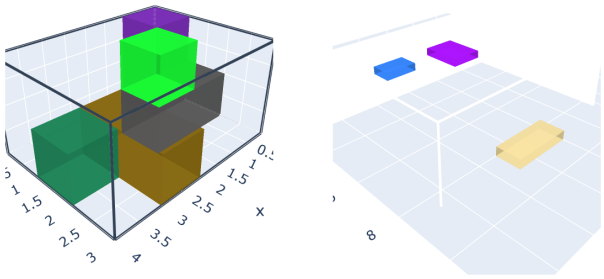
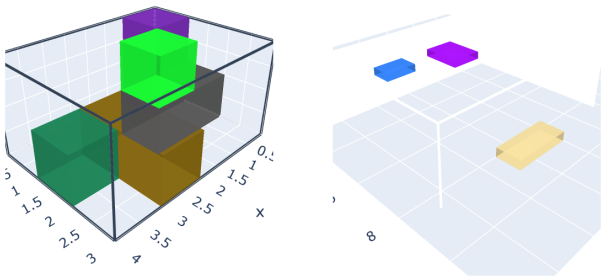
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	100
Durasi pencarian	0.259233 detik

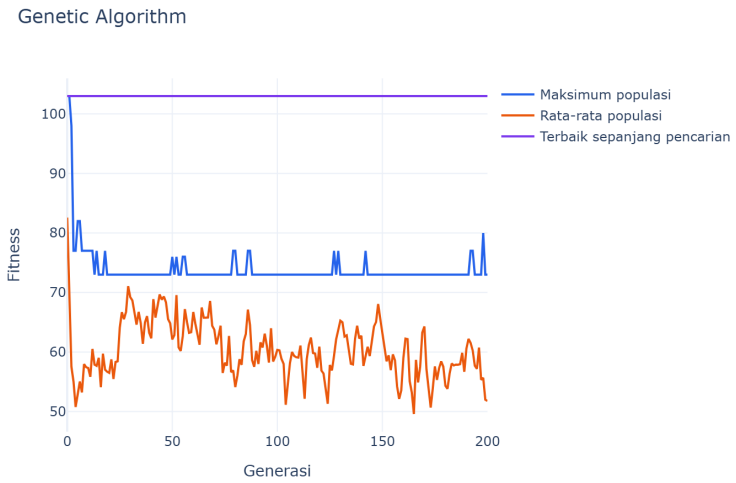
Test case tc8 — Populasi 50, iterasi 200 — Percobaan 1

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	103

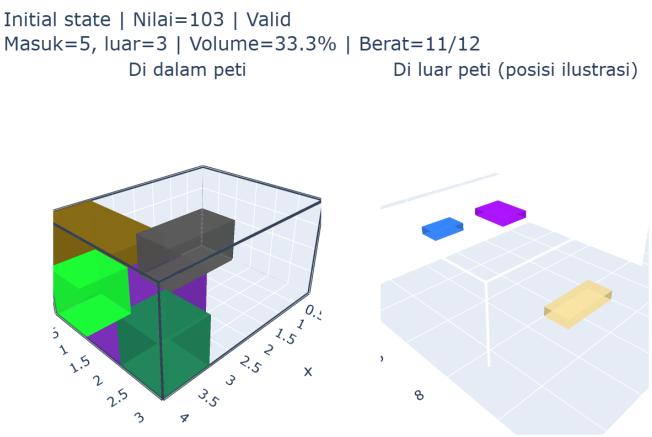
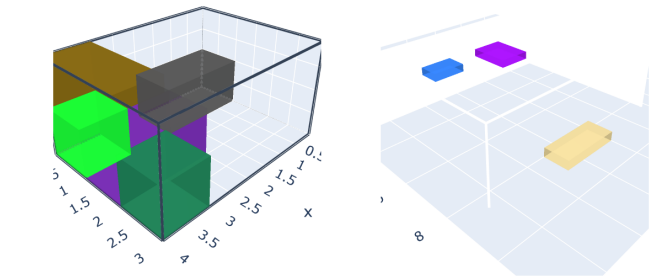
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	200
Durasi pencarian	0.542108 detik

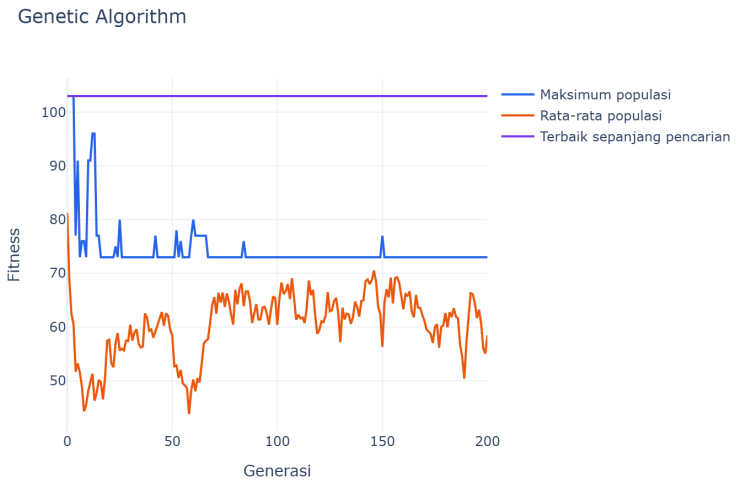
Test case tc8 — Populasi 50, iterasi 200 — Percobaan 2

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	Initial state | Nilai=103 | Valid Masuk=5, luar=3 | Volume=33.3% | Berat=11/12 Di dalam peti Di luar peti (posisi ilustrasi) 
State akhir terbaik	Final state | Nilai=103 | Valid Masuk=5, luar=3 | Volume=33.3% | Berat=11/12 Di dalam peti Di luar peti (posisi ilustrasi) 
Nilai *objective function* akhir terbaik	103

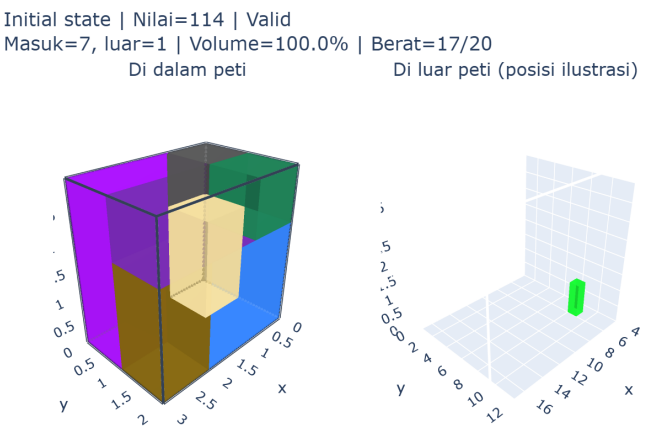
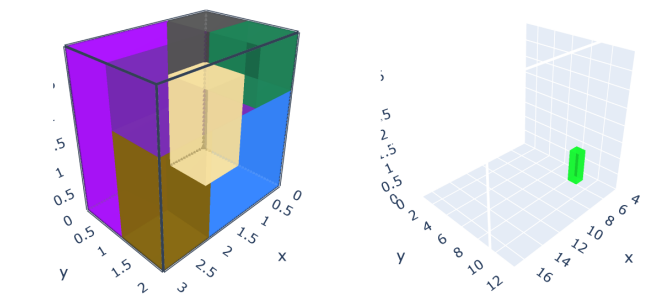
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	200
Durasi pencarian	0.511464 detik

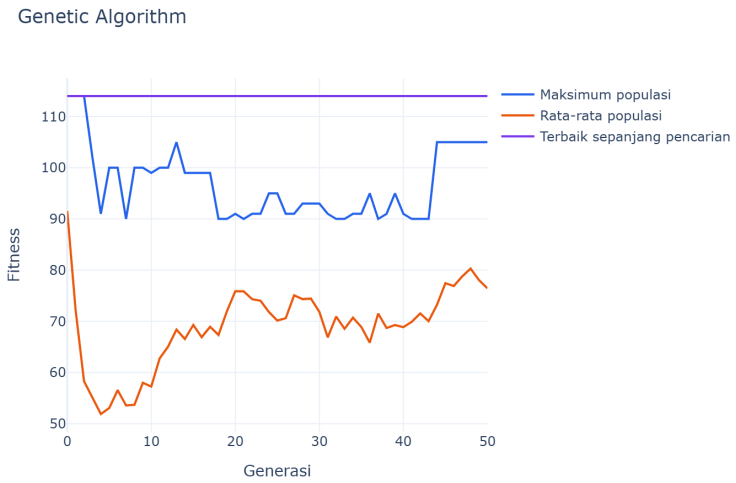
Test case tc8 — Populasi 50, iterasi 200 — Percobaan 3

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	103

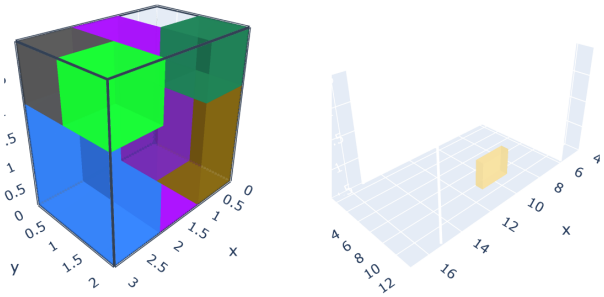
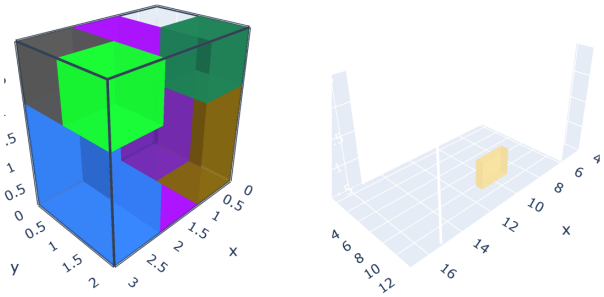
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	200
Durasi pencarian	0.560678 detik

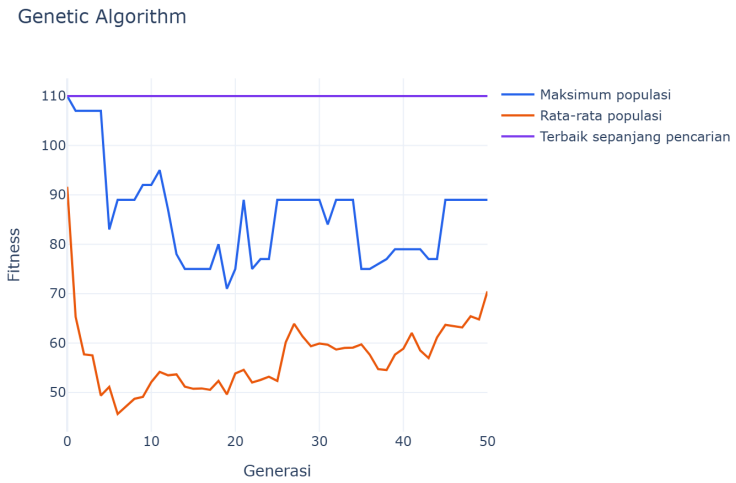
Test case tc9 — Populasi 50, iterasi 50 — Percobaan 1

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	114

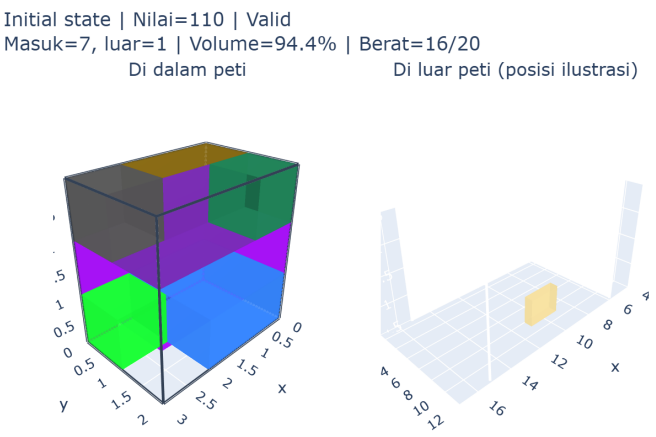
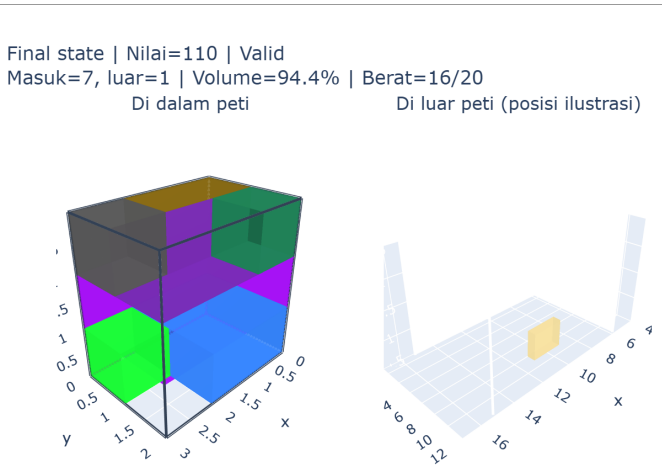
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	50
Durasi pencarian	0.240946 detik

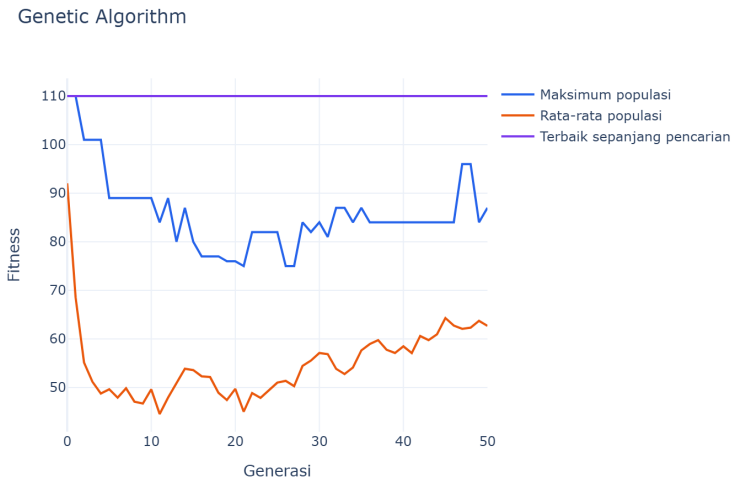
Test case tc9 — Populasi 50, iterasi 50 — Percobaan 2

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	Initial state | Nilai=110 | Valid Masuk=7, luar=1 | Volume=94.4% | Berat=16/20 Di dalam peti Di luar peti (posisi ilustrasi) 
State akhir terbaik	Final state | Nilai=110 | Valid Masuk=7, luar=1 | Volume=94.4% | Berat=16/20 Di dalam peti Di luar peti (posisi ilustrasi) 
Nilai *objective function* akhir terbaik	110

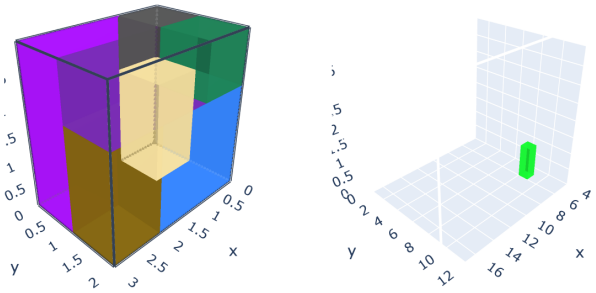
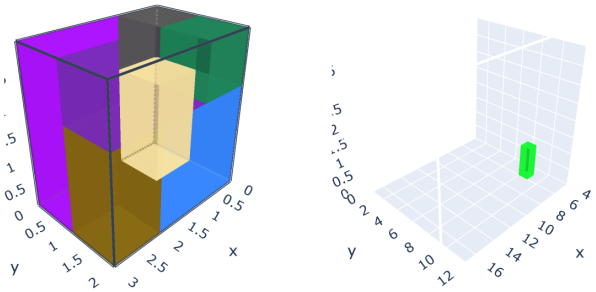
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	50
Durasi pencarian	0.202392 detik

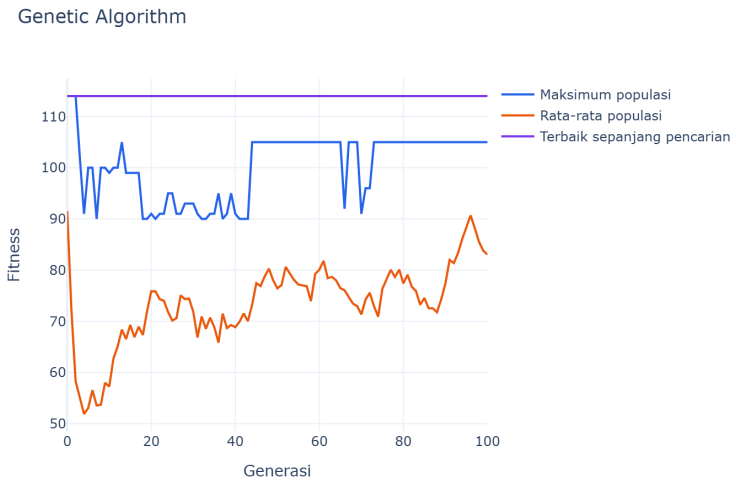
Test case tc9 — Populasi 50, iterasi 50 — Percobaan 3

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	110

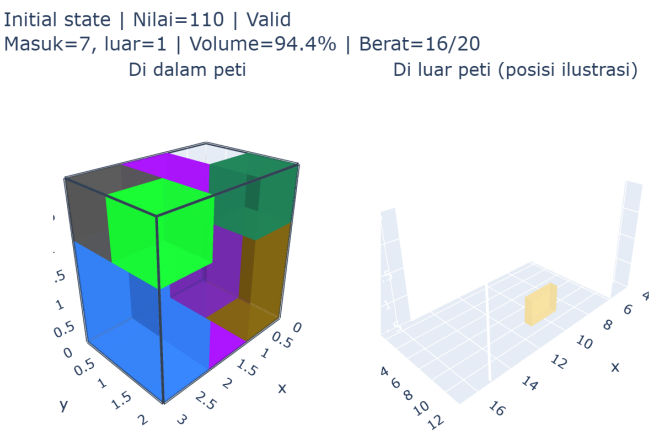
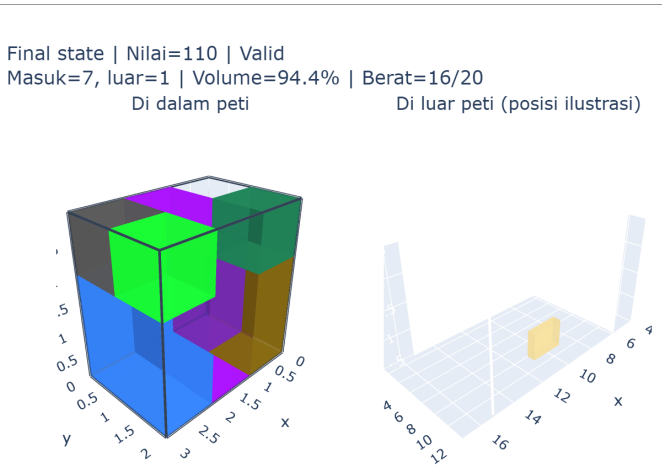
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	50
Durasi pencarian	0.267905 detik

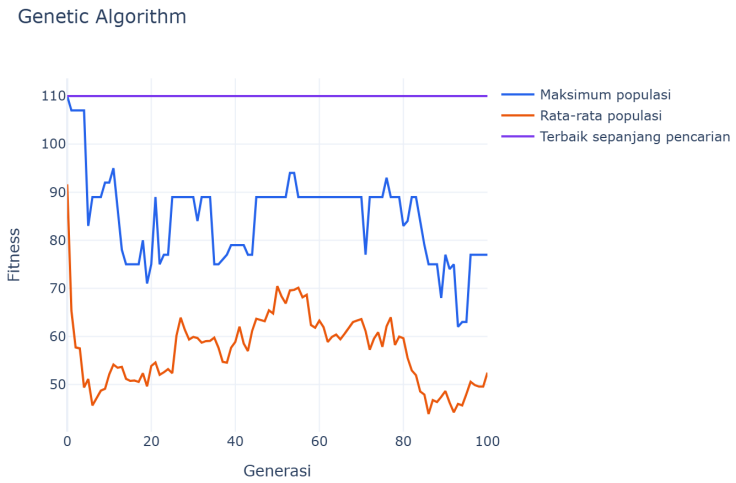
Test case tc9 — Populasi 50, iterasi 100 — Percobaan 1

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	Initial state | Nilai=114 | Valid Masuk=7, luar=1 | Volume=100.0% | Berat=17/20 Di dalam peti Di luar peti (posisi ilustrasi) 
State akhir terbaik	Final state | Nilai=114 | Valid Masuk=7, luar=1 | Volume=100.0% | Berat=17/20 Di dalam peti Di luar peti (posisi ilustrasi) 
Nilai *objective function* akhir terbaik	114

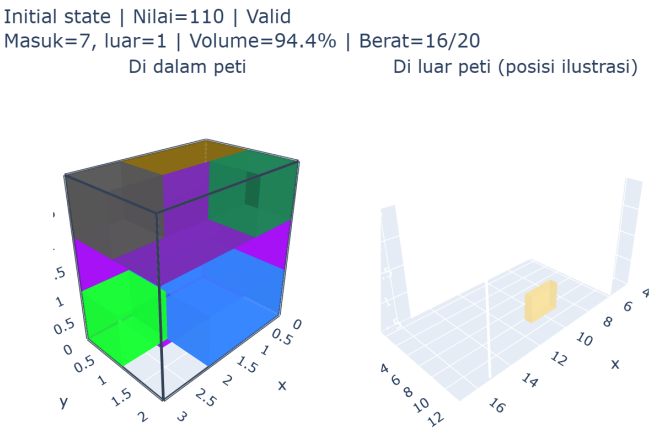
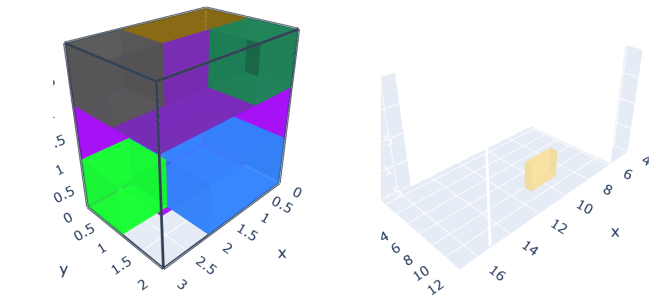
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	100
Durasi pencarian	0.482106 detik

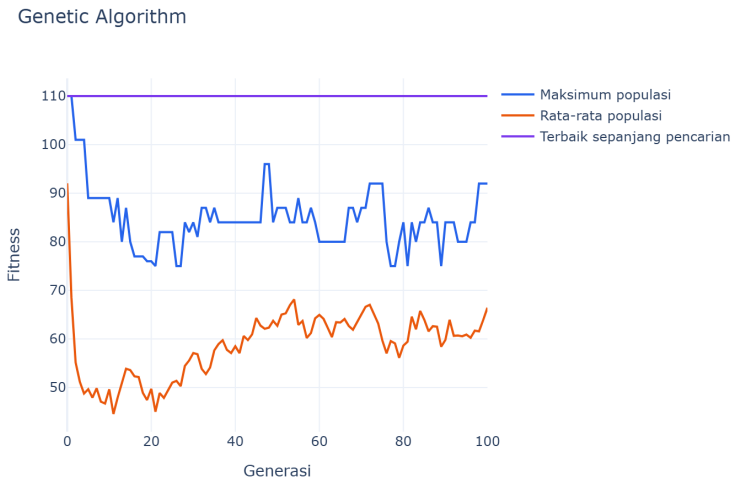
Test case tc9 — Populasi 50, iterasi 100 — Percobaan 2

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	110

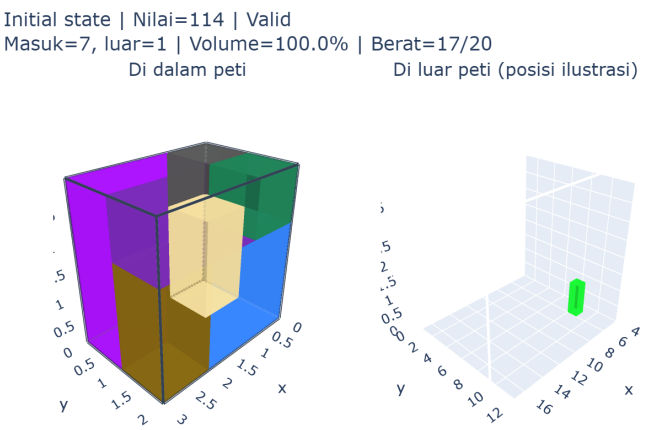
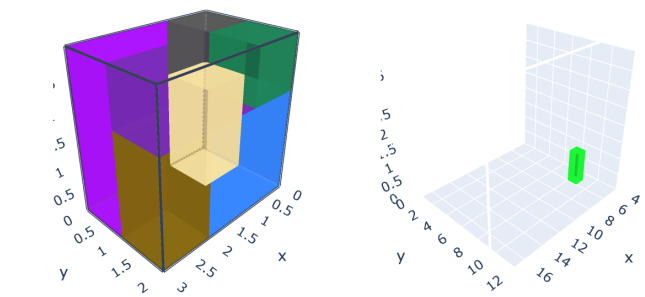
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	100
Durasi pencarian	0.387874 detik

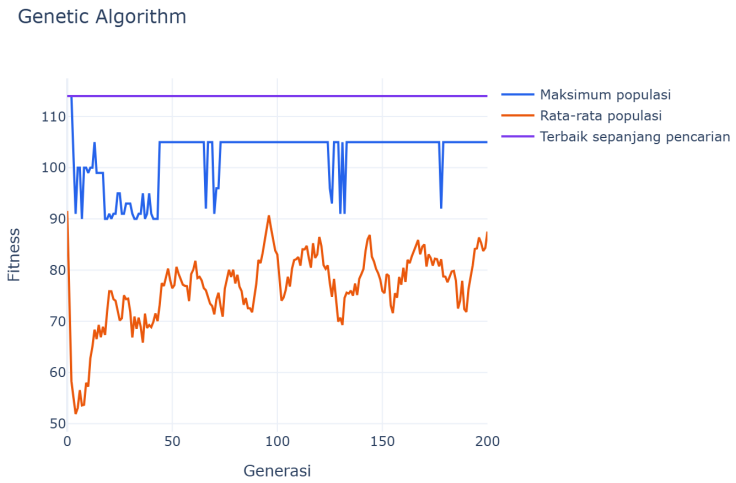
Test case tc9 — Populasi 50, iterasi 100 — Percobaan 3

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	110

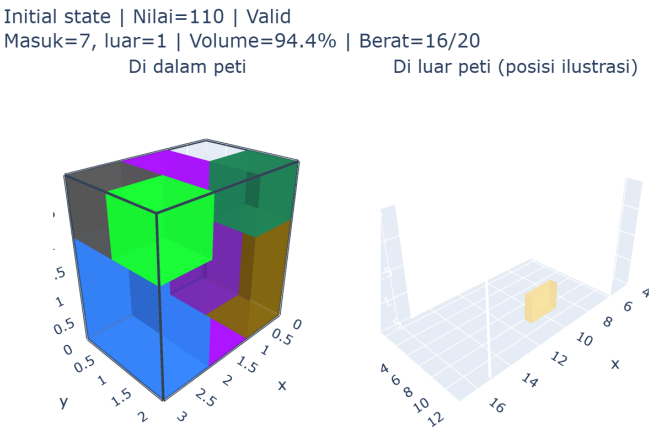
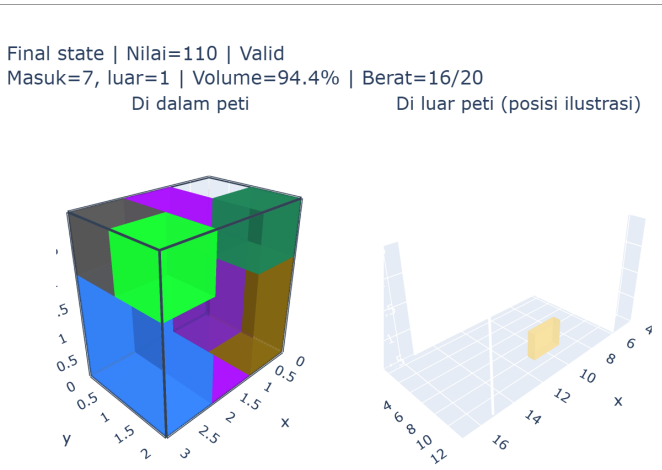
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	100
Durasi pencarian	0.372883 detik

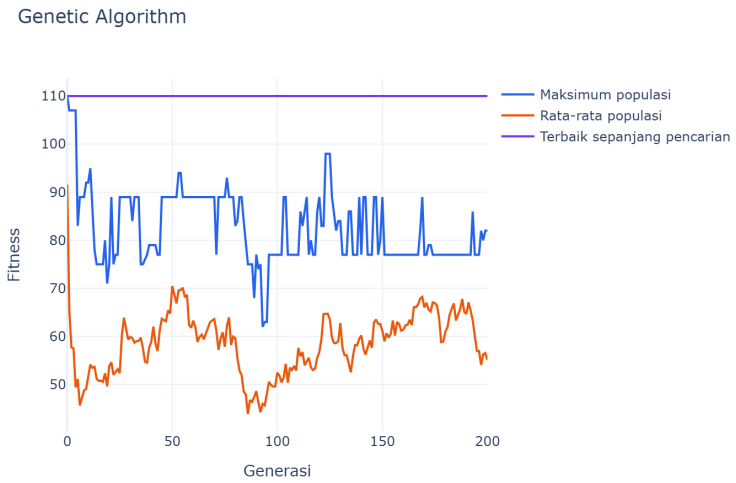
Test case tc9 — Populasi 50, iterasi 200 — Percobaan 1

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	114

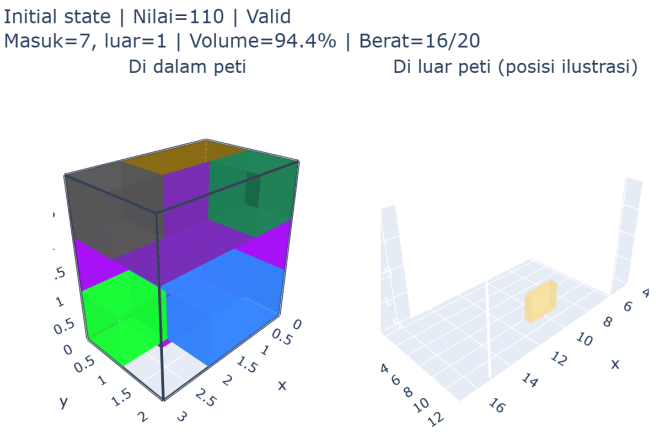
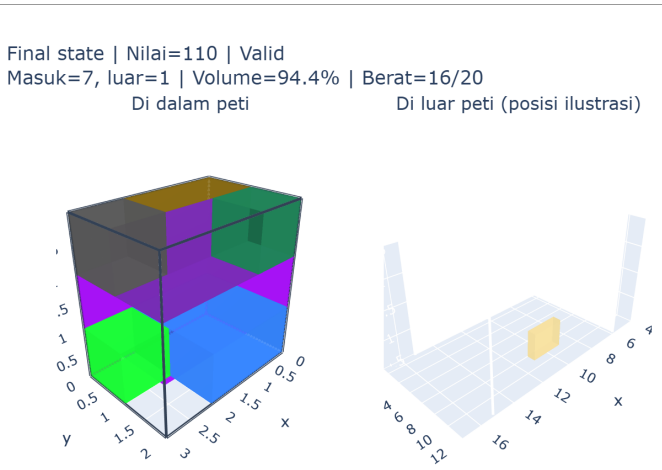
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	200
Durasi pencarian	0.931361 detik

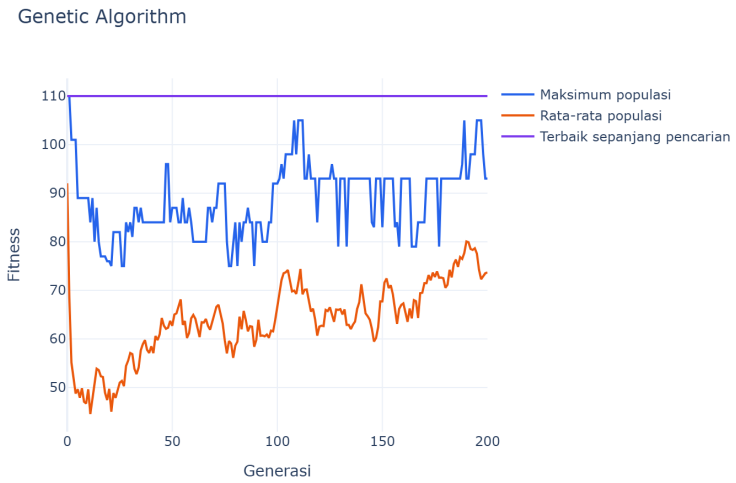
Test case tc9 — Populasi 50, iterasi 200 — Percobaan 2

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	110

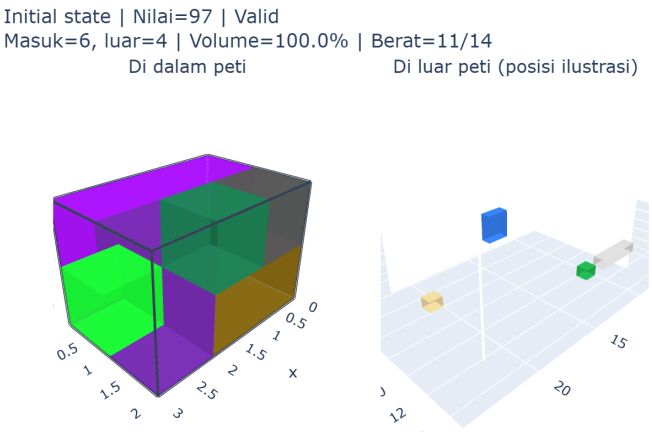
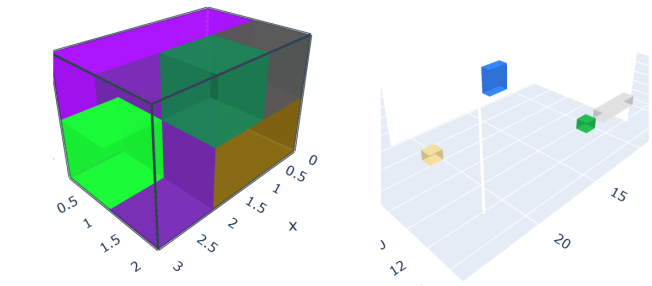
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	200
Durasi pencarian	0.677546 detik

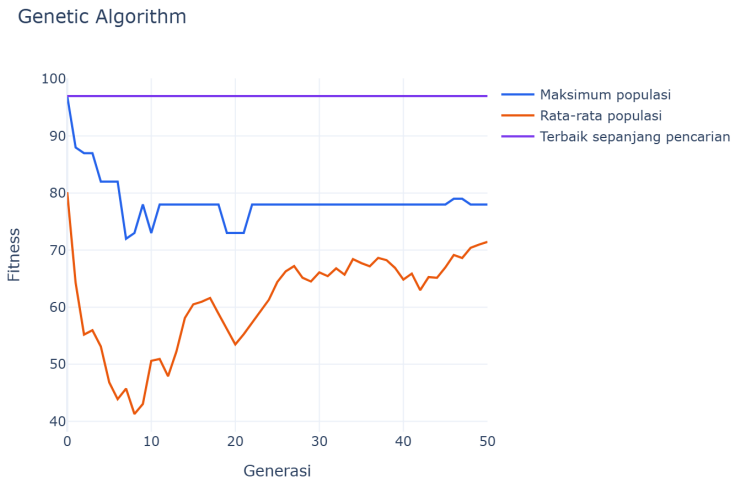
Test case tc9 — Populasi 50, iterasi 200 — Percobaan 3

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	110

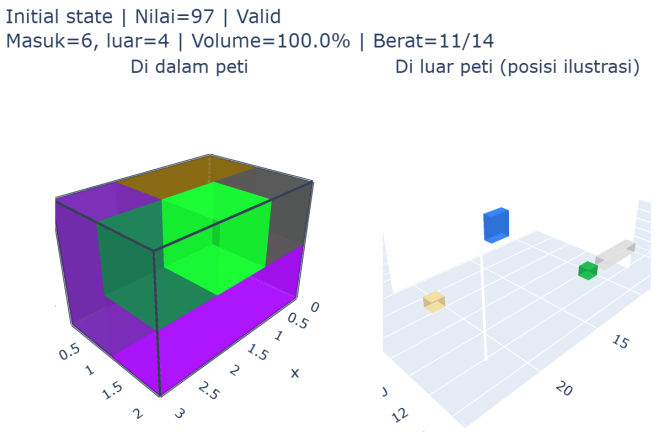
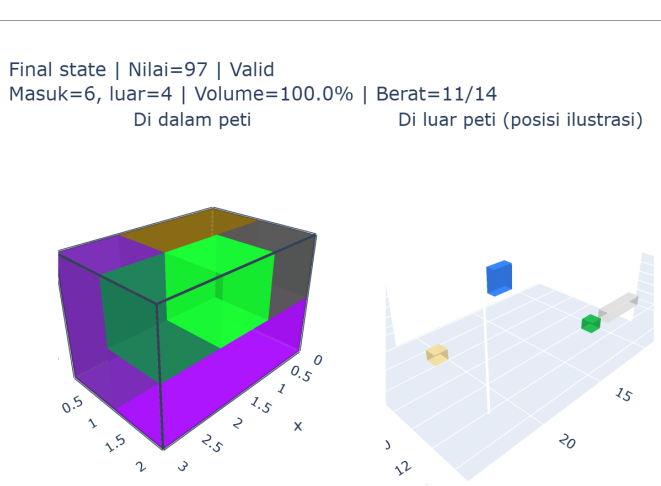
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	200
Durasi pencarian	0.765361 detik

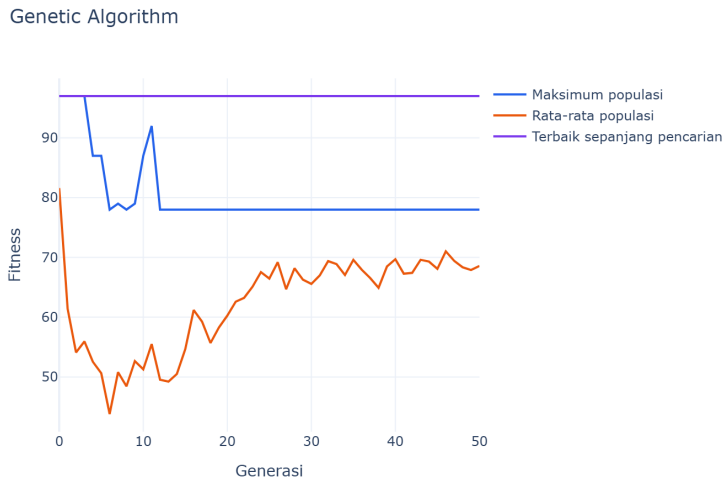
Test case tc10 — Populasi 50, iterasi 50 — Percobaan 1

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	97

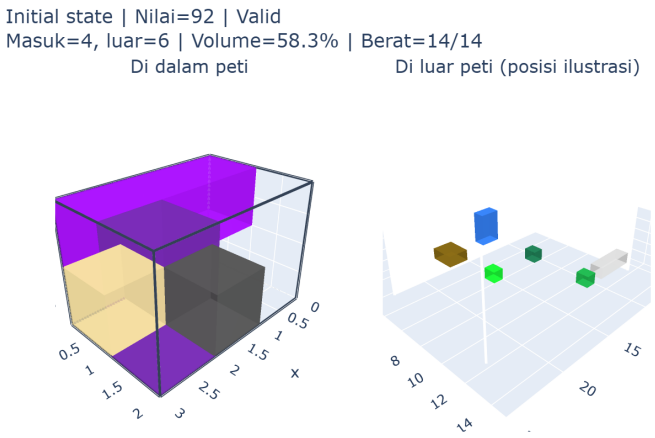
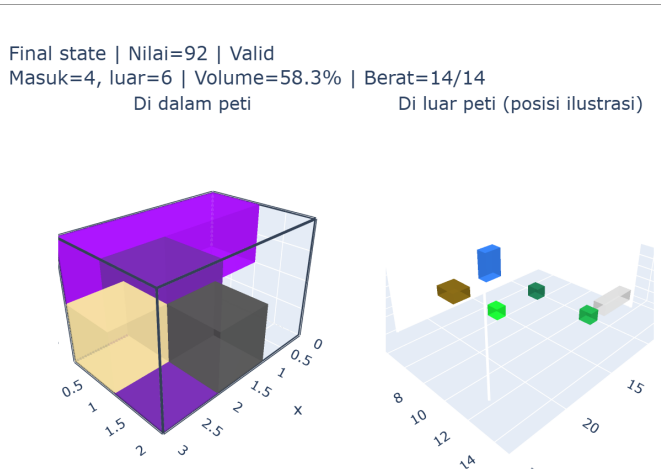
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	50
Durasi pencarian	0.192227 detik

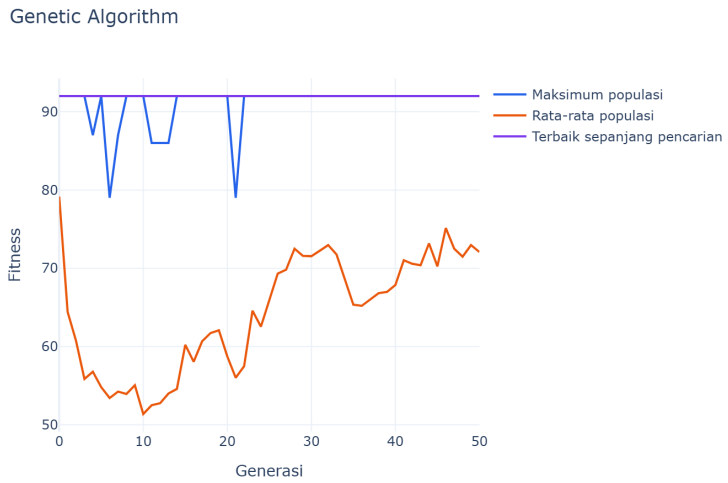
Test case tc10 — Populasi 50, iterasi 50 — Percobaan 2

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	97

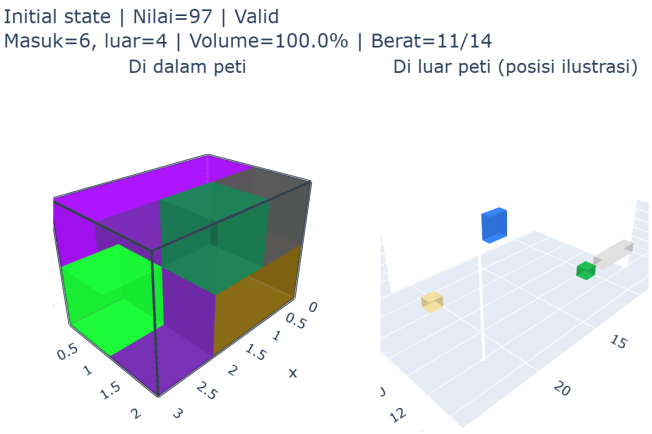
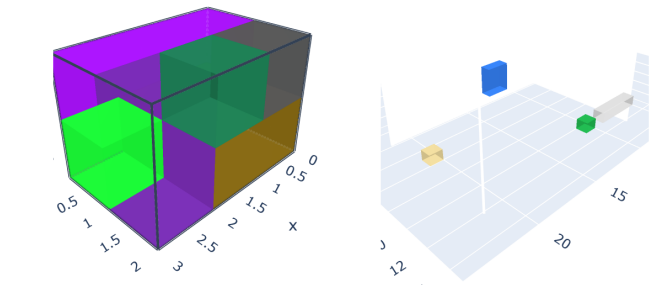
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	50
Durasi pencarian	0.191899 detik

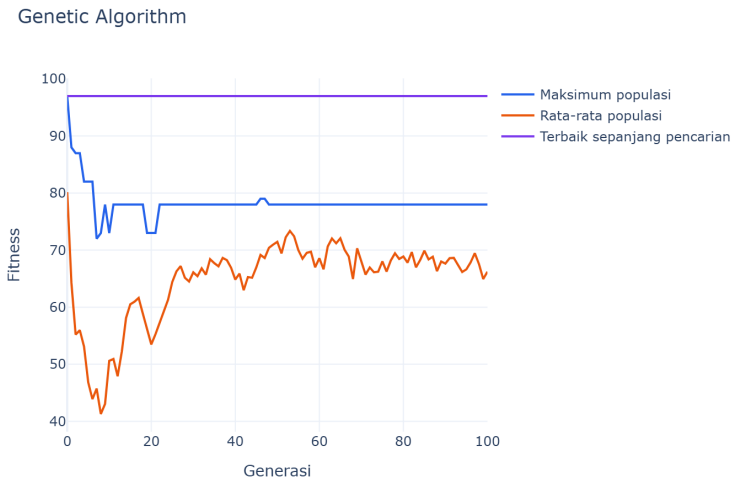
Test case tc10 — Populasi 50, iterasi 50 — Percobaan 3

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	92

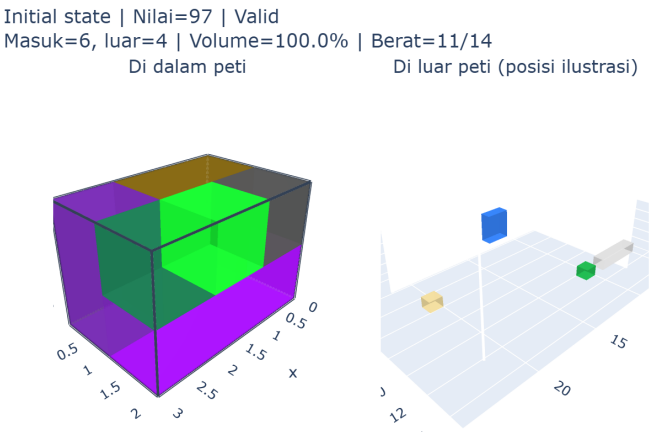
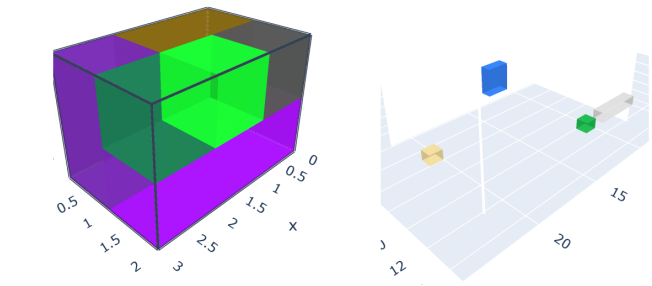
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	50
Durasi pencarian	0.186268 detik

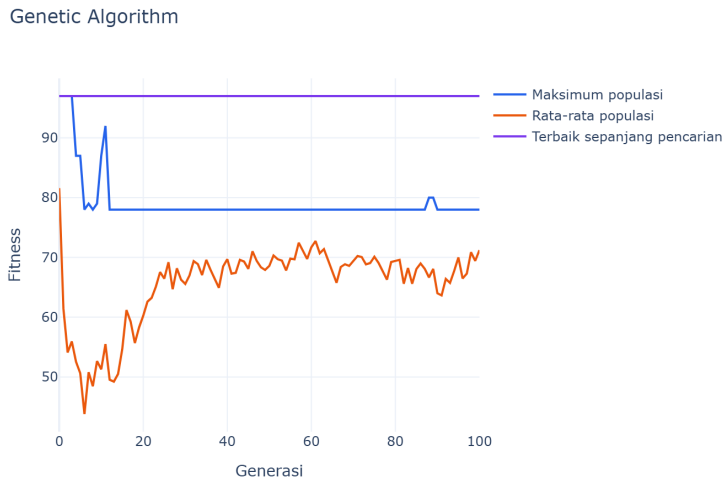
Test case tc10 — Populasi 50, iterasi 100 — Percobaan 1

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	97

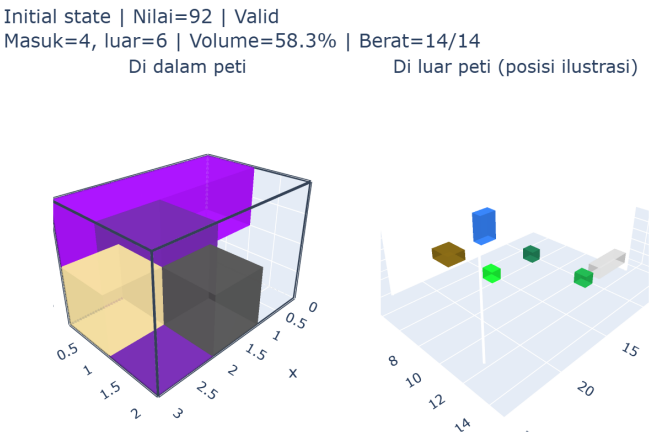
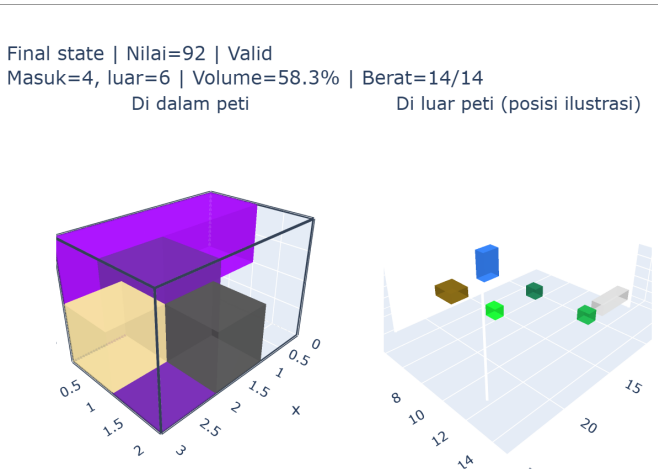
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	100
Durasi pencarian	0.40251 detik

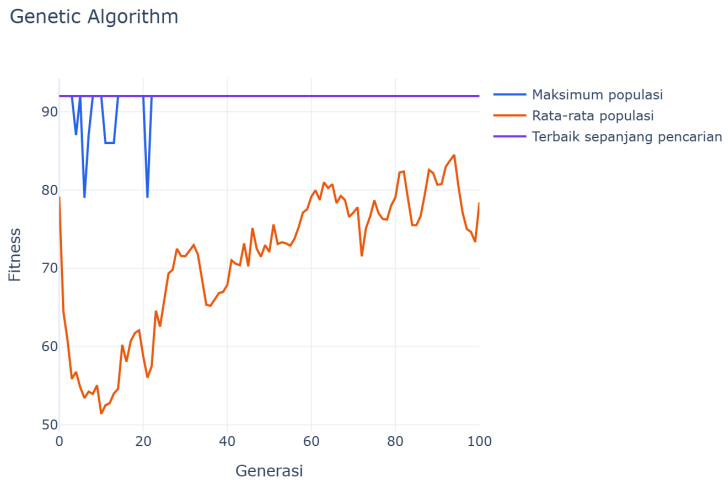
Test case tc10 — Populasi 50, iterasi 100 — Percobaan 2

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	97

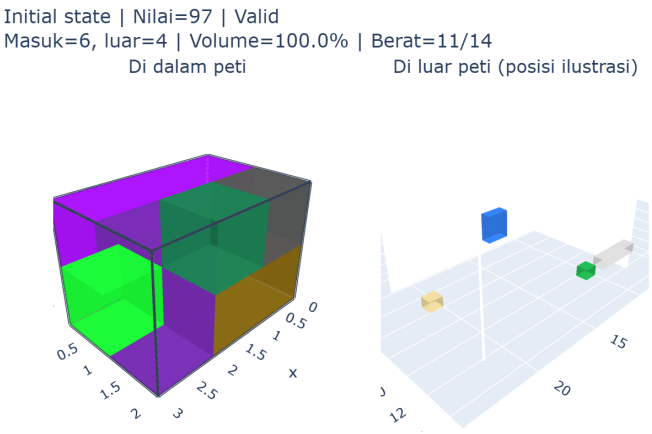
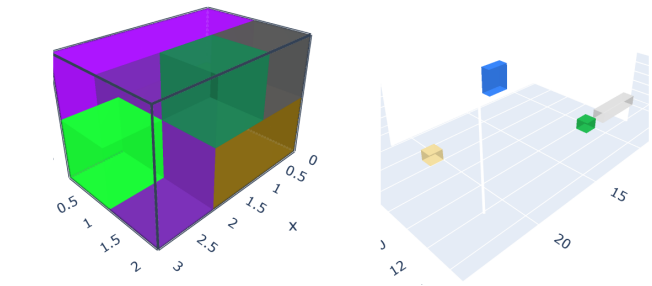
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	100
Durasi pencarian	0.384656 detik

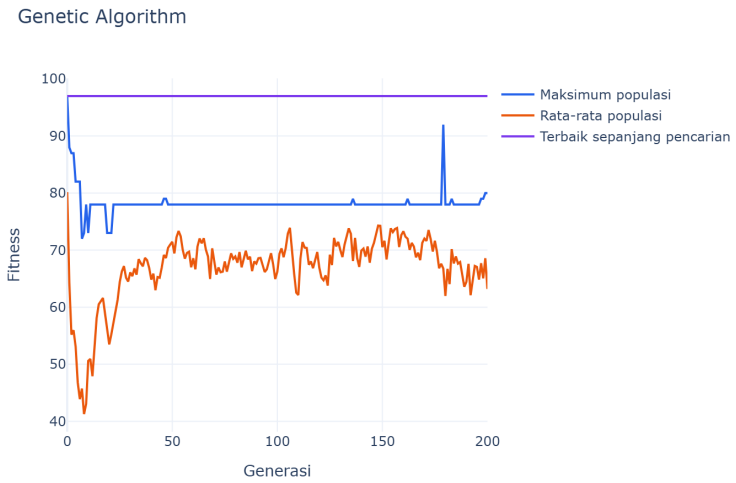
Test case tc10 — Populasi 50, iterasi 100 — Percobaan 3

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	92

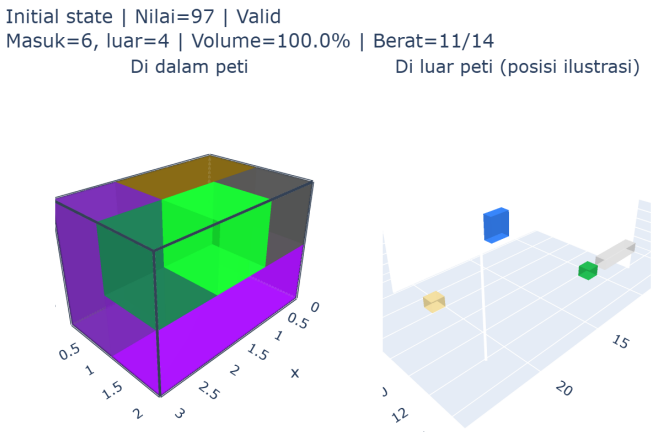
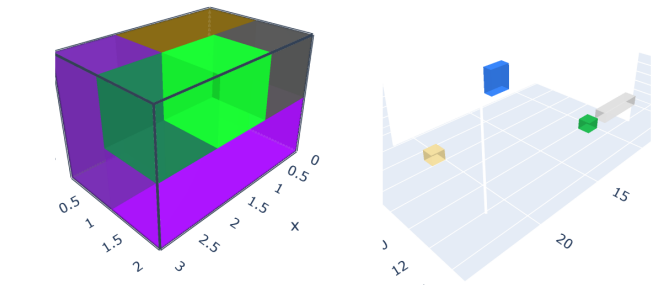
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	100
Durasi pencarian	0.38904 detik

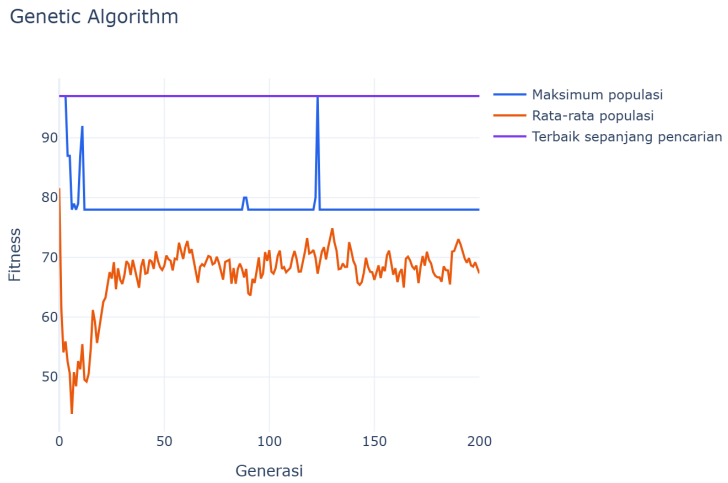
Test case tc10 — Populasi 50, iterasi 200 — Percobaan 1

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	97

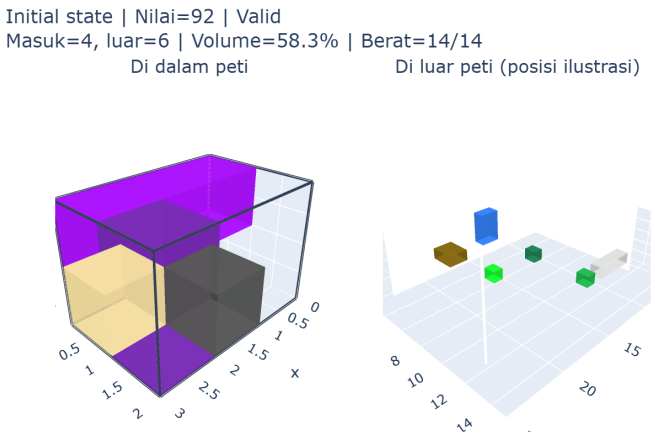
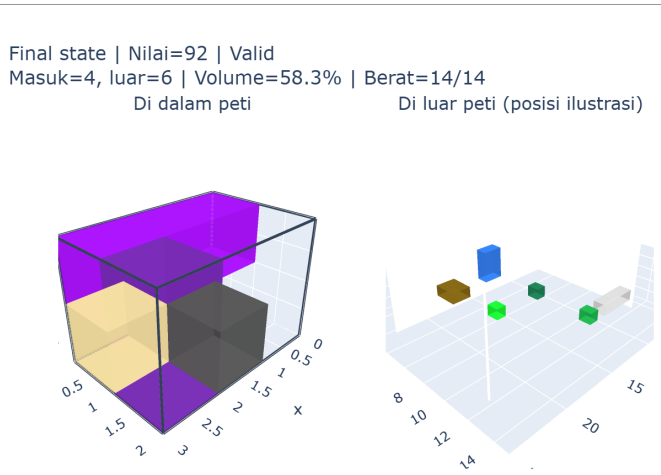
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	200
Durasi pencarian	0.790031 detik

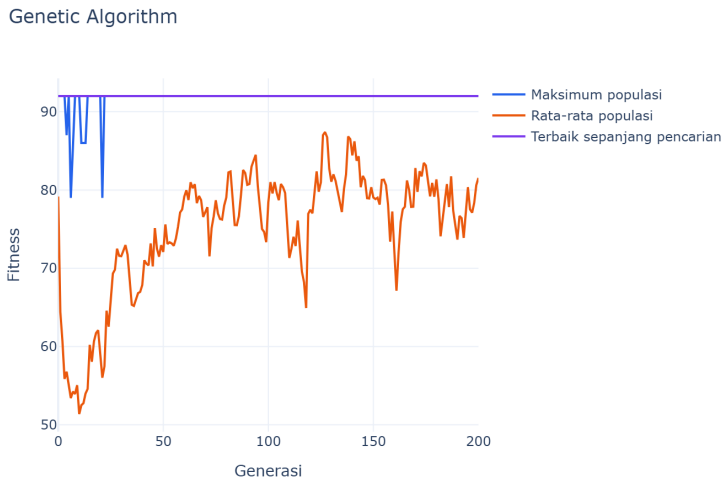
Test case tc10 — Populasi 50, iterasi 200 — Percobaan 2

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	97

Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	200
Durasi pencarian	0.809423 detik

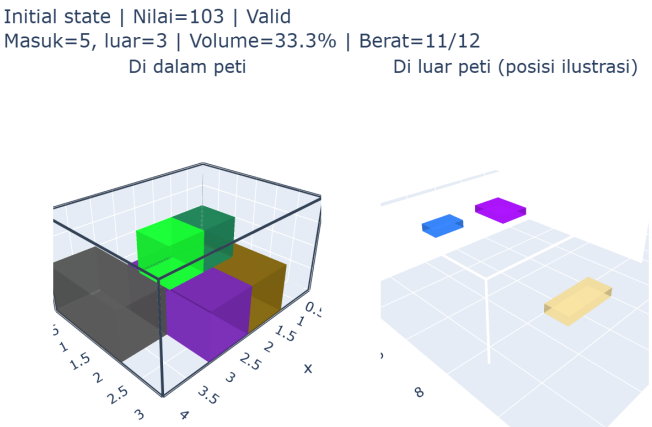
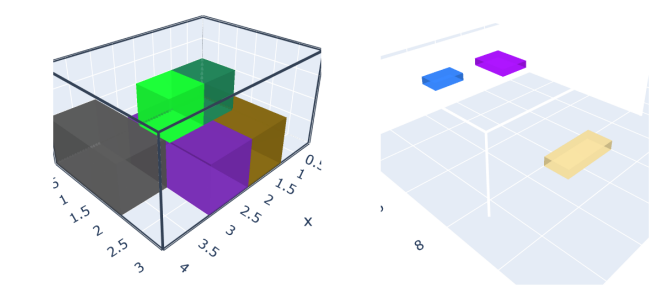
Test case tc10 — Populasi 50, iterasi 200 — Percobaan 3

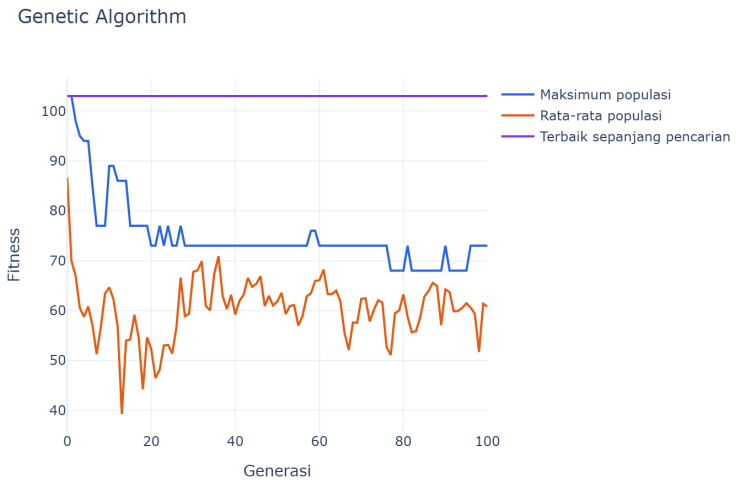
Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	92

Informasi yang dicatat	Data hasil pengujian
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	200
Durasi pencarian	0.831182 detik

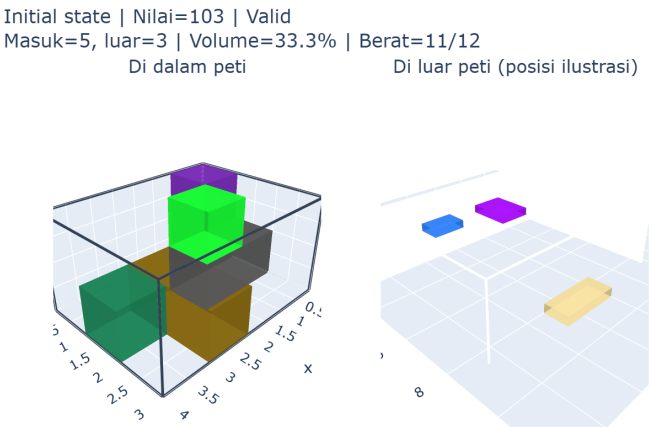
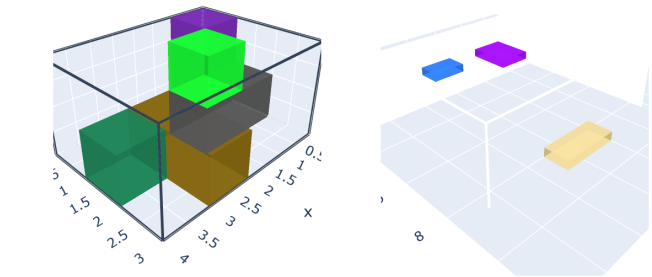
4.3.2 Iterasi Tetap, Variasi Jumlah Populasi

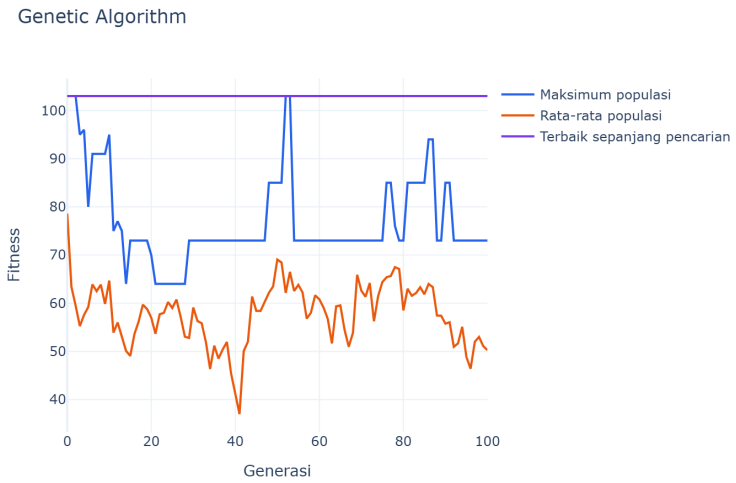
Test case tc8 — Populasi 20, iterasi 100 — Percobaan 1

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	103

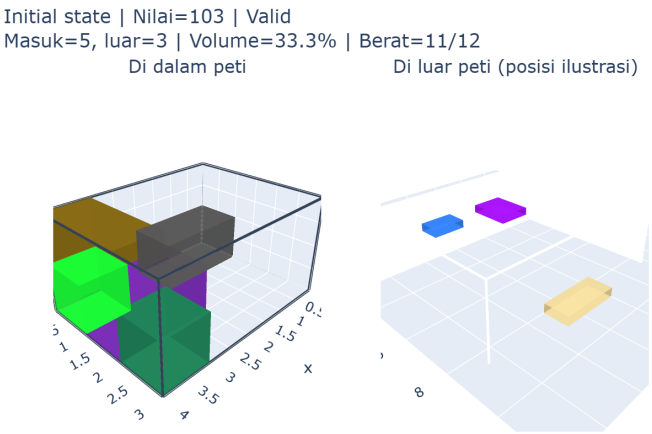
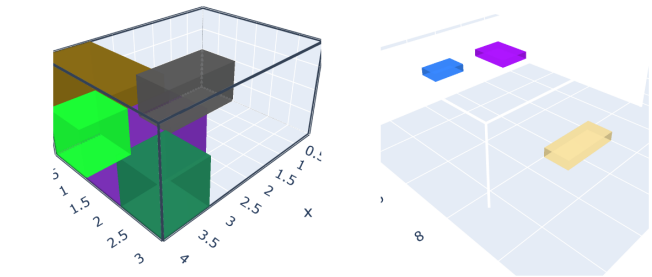
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	20
Banyak iterasi	100
Durasi pencarian	0.103802 detik

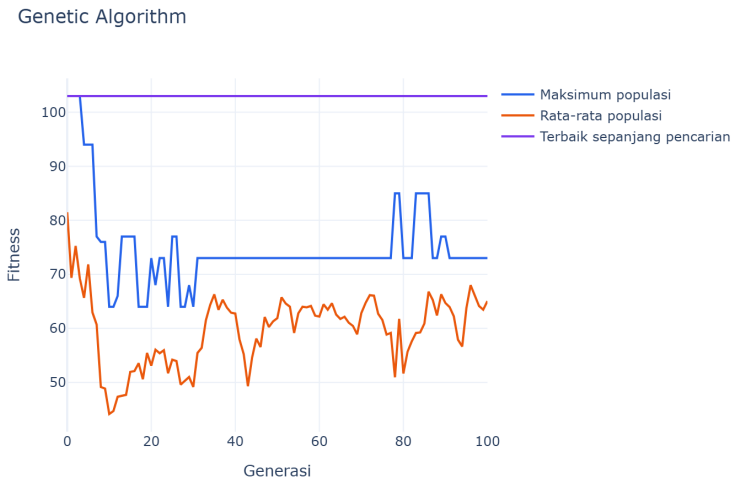
Test case tc8 — Populasi 20, iterasi 100 — Percobaan 2

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	103

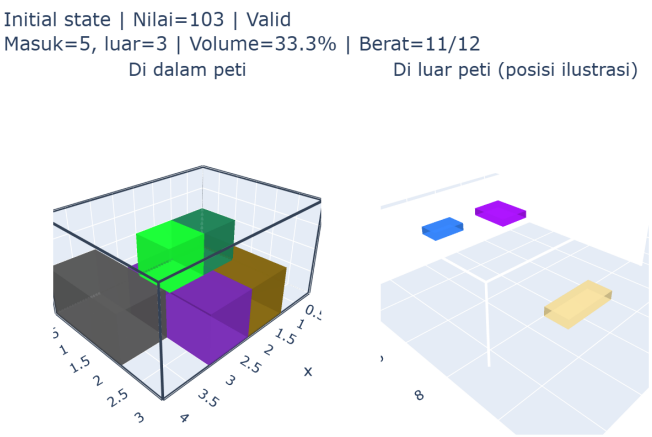
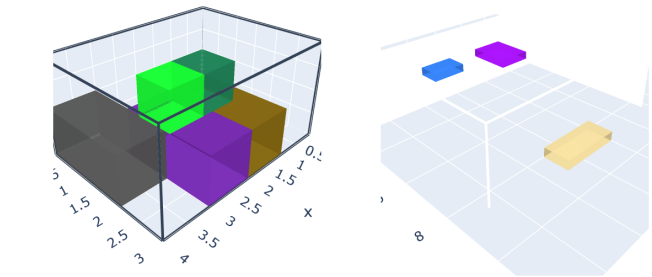
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	20
Banyak iterasi	100
Durasi pencarian	0.099828 detik

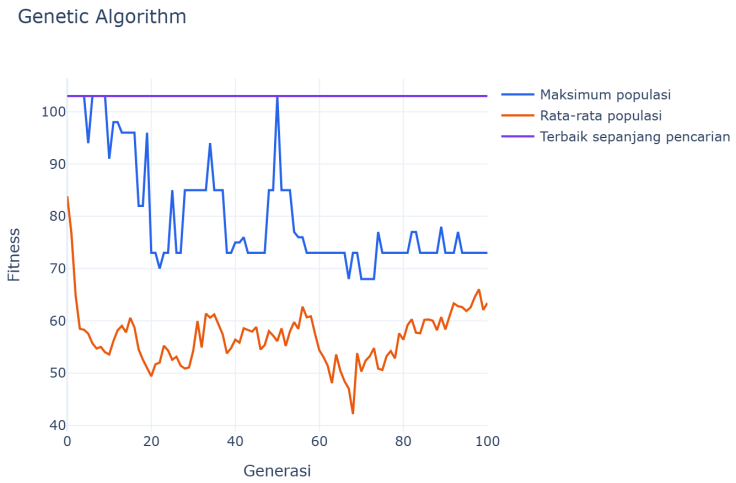
Test case tc8 — Populasi 20, iterasi 100 — Percobaan 3

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	103

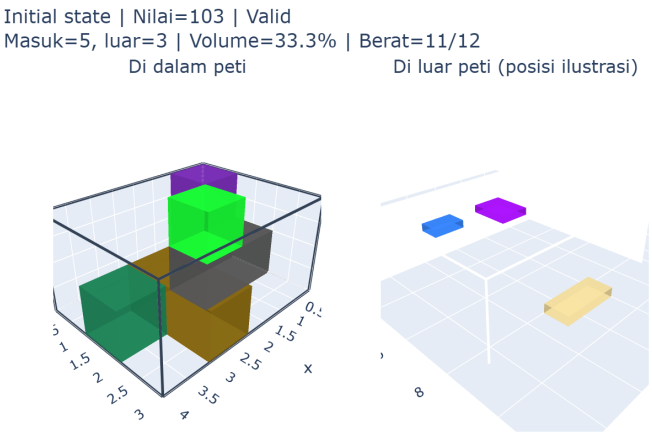
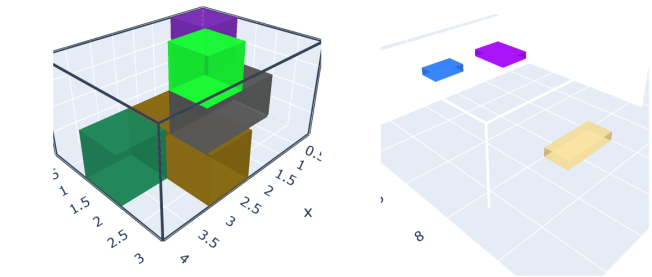
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	20
Banyak iterasi	100
Durasi pencarian	0.103273 detik

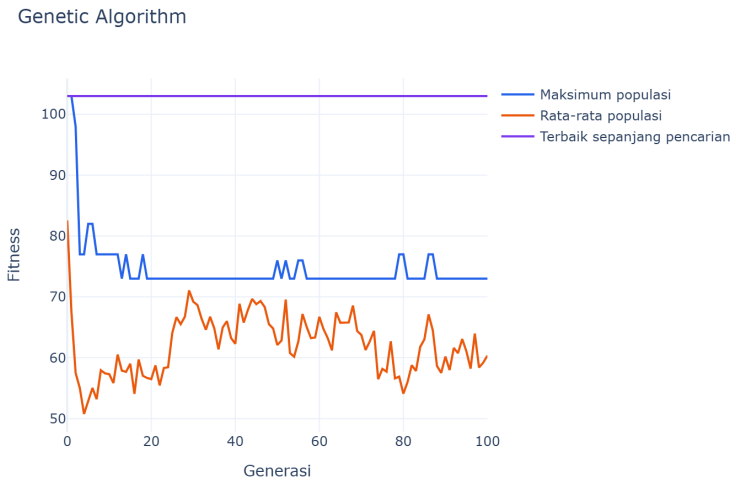
Test case tc8 — Populasi 50, iterasi 100 — Percobaan 1

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	103

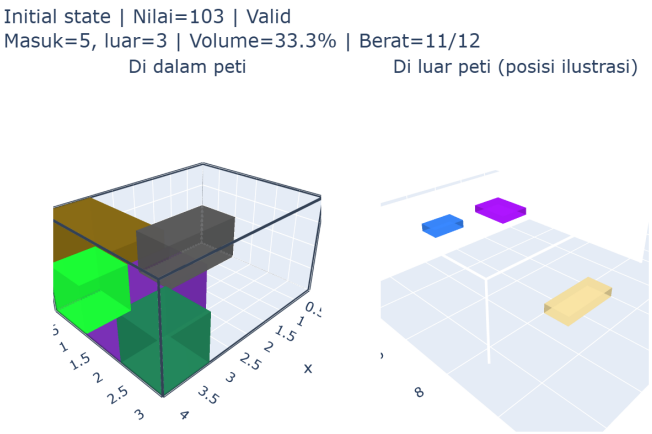
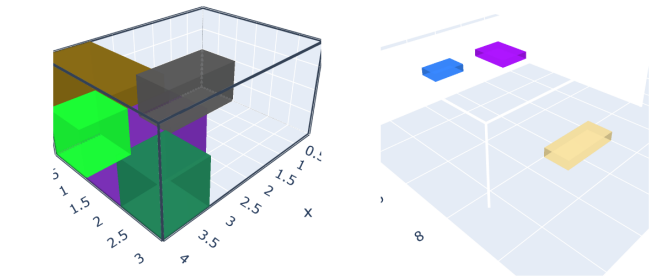
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	100
Durasi pencarian	0.254834 detik

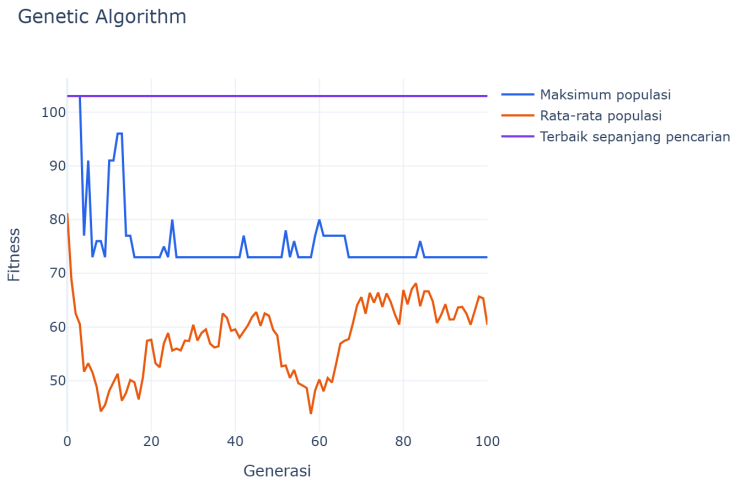
Test case tc8 — Populasi 50, iterasi 100 — Percobaan 2

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	103

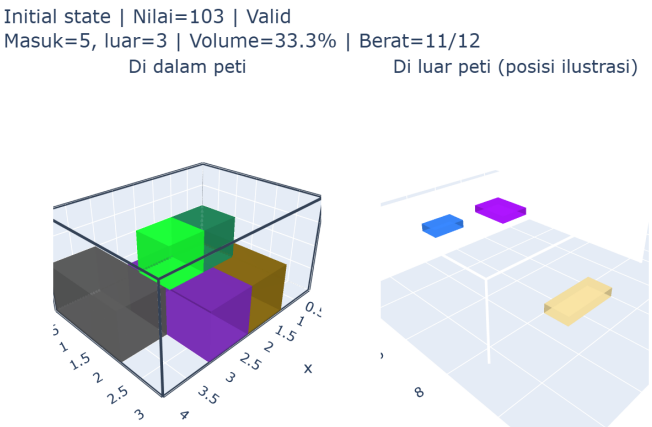
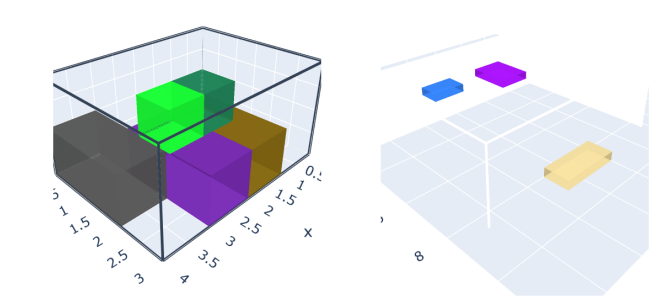
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	100
Durasi pencarian	0.26839 detik

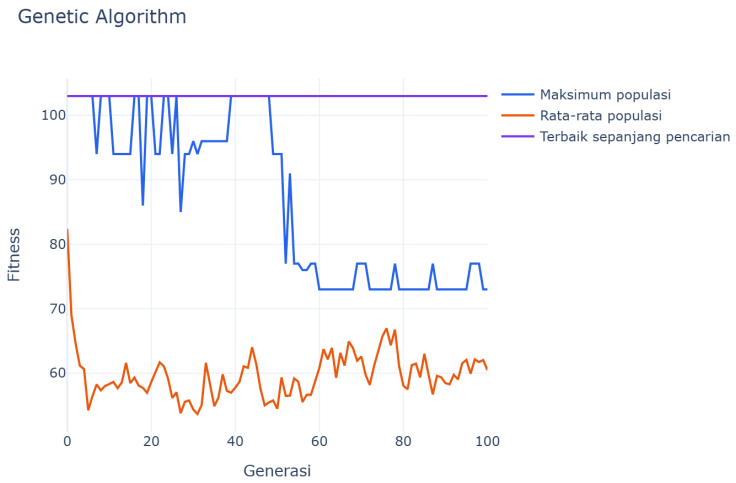
Test case tc8 — Populasi 50, iterasi 100 — Percobaan 3

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	103

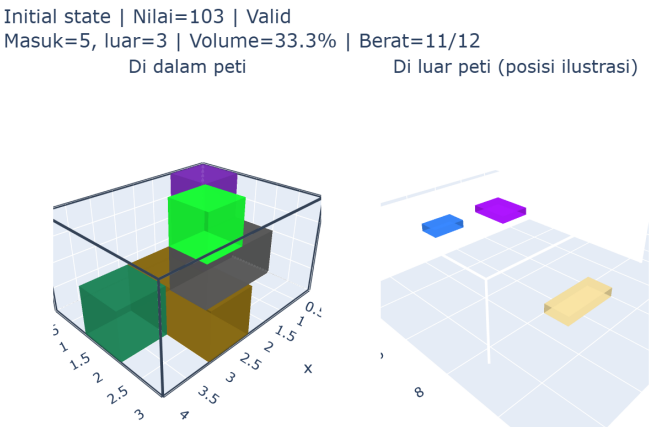
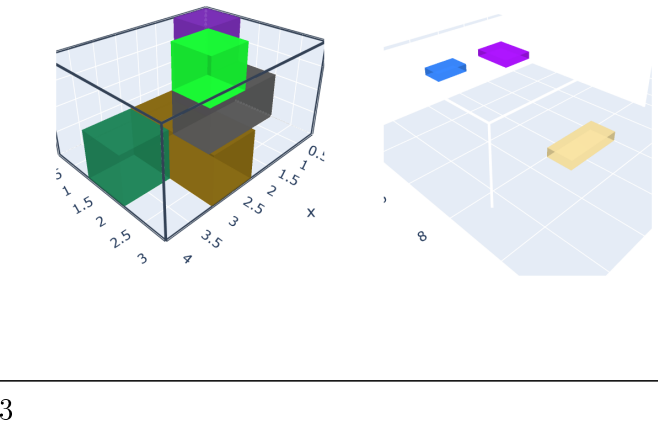
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	100
Durasi pencarian	0.259233 detik

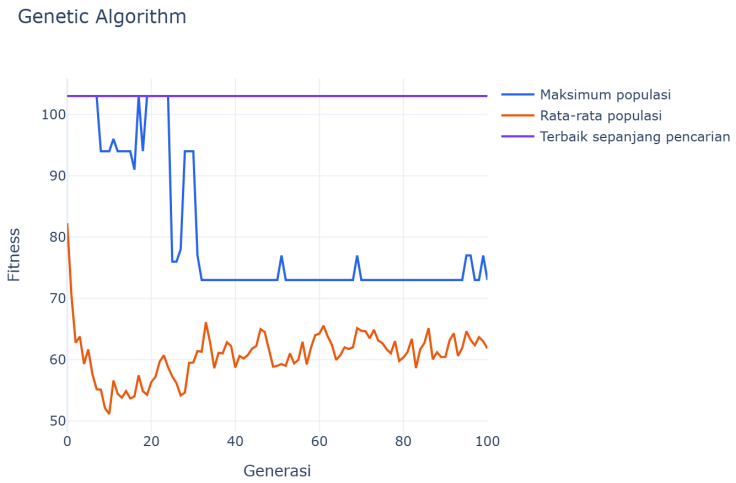
Test case tc8 — Populasi 100, iterasi 100 — Percobaan 1

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	103

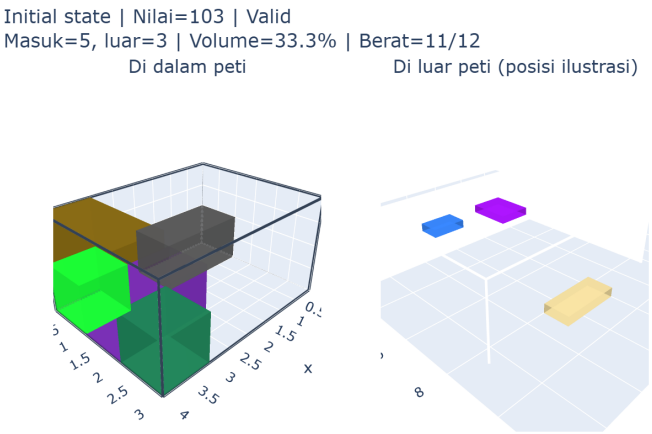
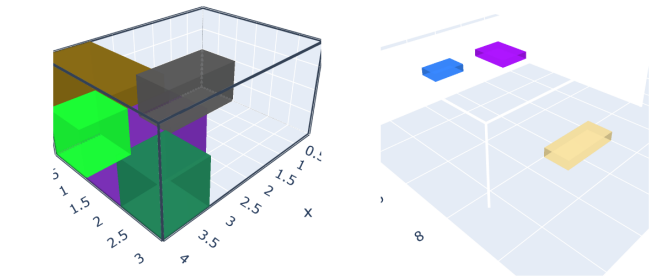
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	100
Banyak iterasi	100
Durasi pencarian	0.595717 detik

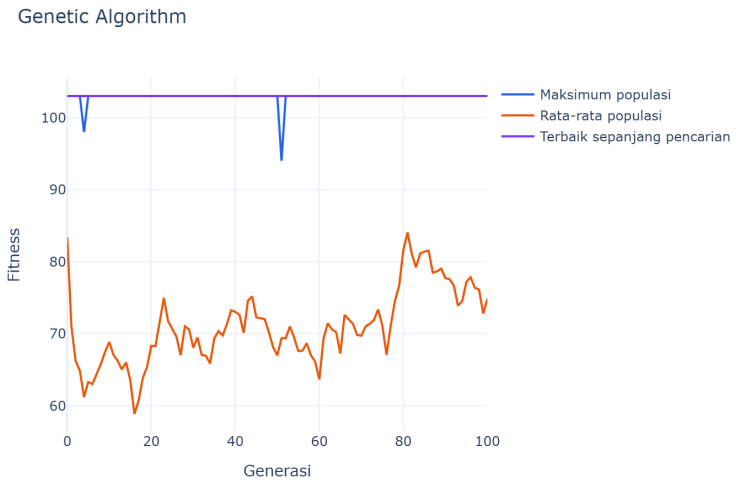
Test case tc8 — Populasi 100, iterasi 100 — Percobaan 2

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	103

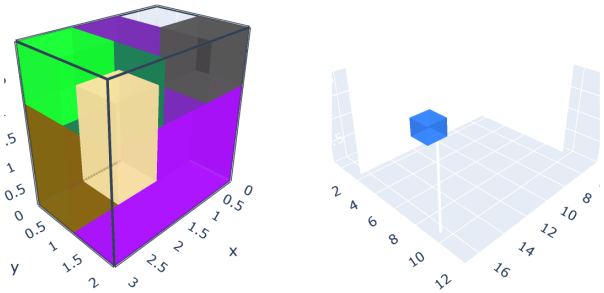
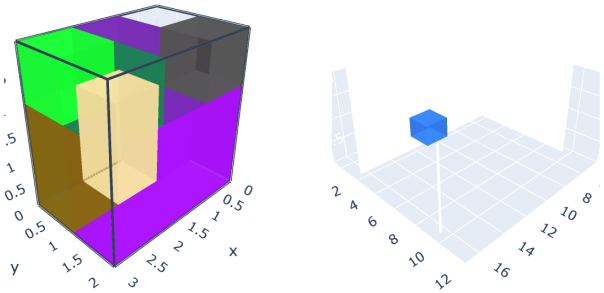
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	100
Banyak iterasi	100
Durasi pencarian	0.593048 detik

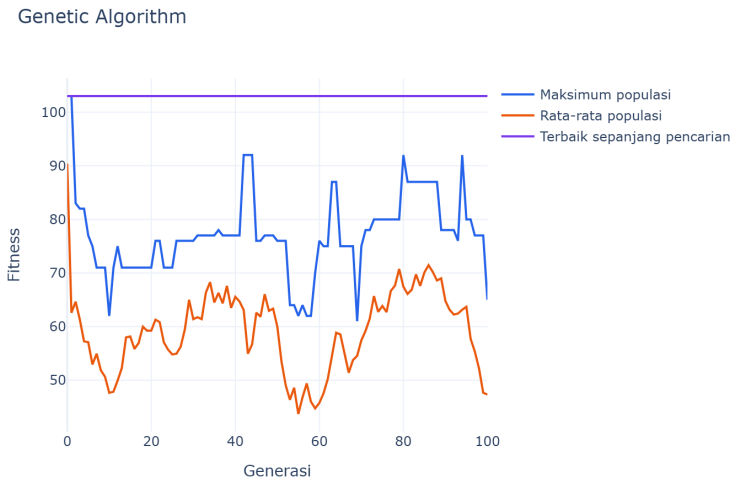
Test case tc8 — Populasi 100, iterasi 100 — Percobaan 3

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	103

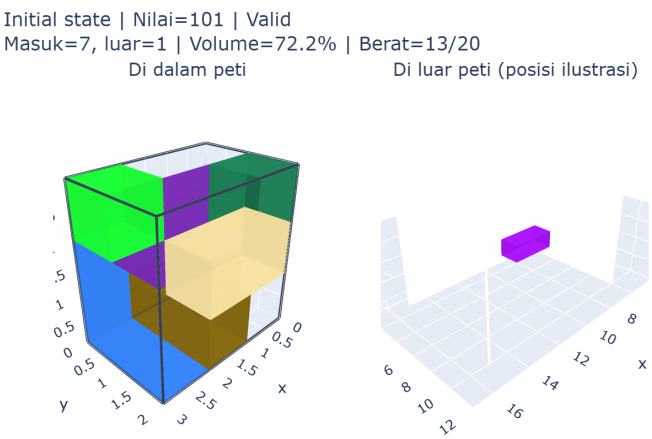
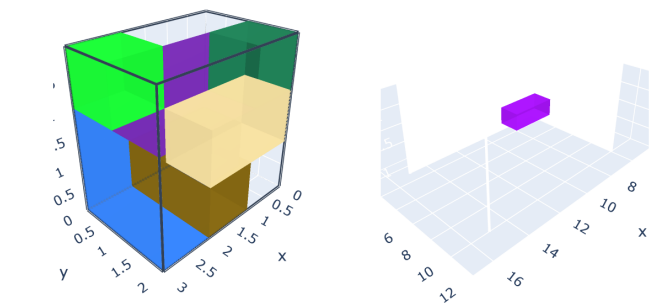
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	100
Banyak iterasi	100
Durasi pencarian	0.671984 detik

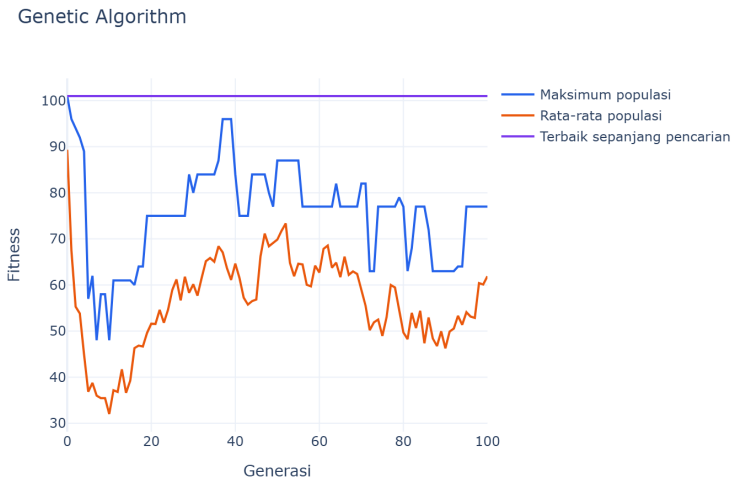
Test case tc9 — Populasi 20, iterasi 100 — Percobaan 1

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	Initial state | Nilai=103 | Valid Masuk=7, luar=1 | Volume=83.3% | Berat=14/20 Di dalam peti Di luar peti (posisi ilustrasi) 
State akhir terbaik	Final state | Nilai=103 | Valid Masuk=7, luar=1 | Volume=83.3% | Berat=14/20 Di dalam peti Di luar peti (posisi ilustrasi) 
Nilai *objective function* akhir terbaik	103

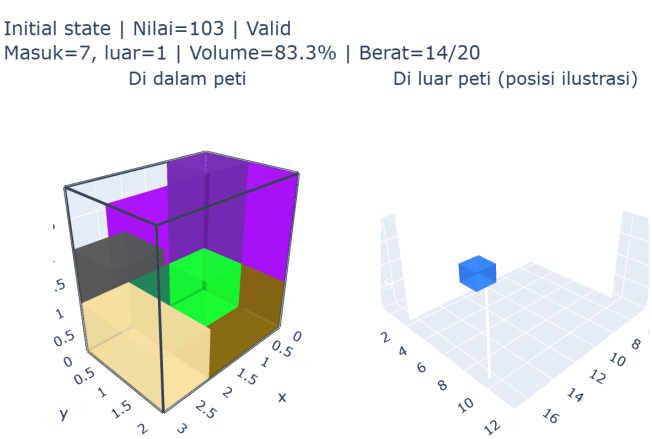
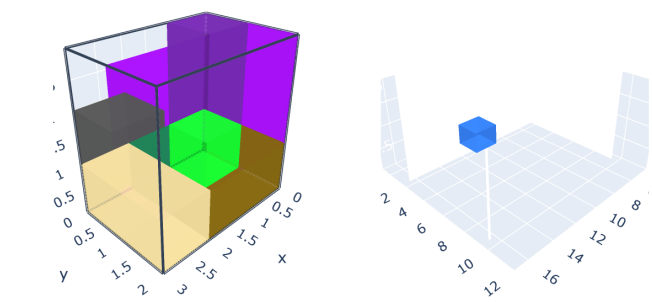
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	20
Banyak iterasi	100
Durasi pencarian	0.151308 detik

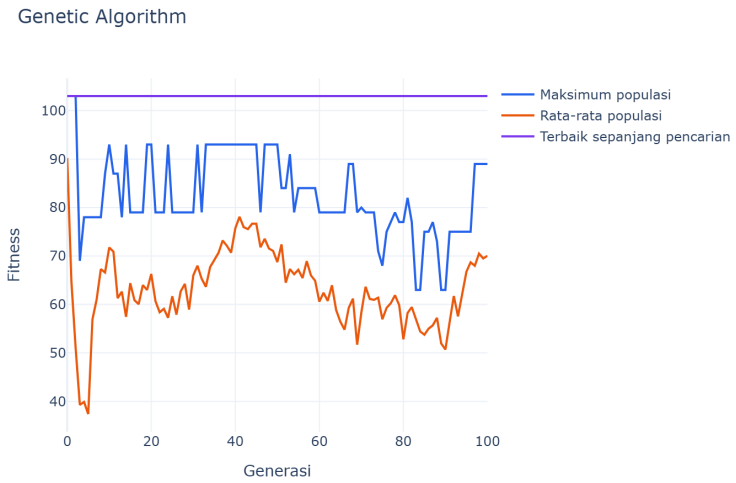
Test case tc9 — Populasi 20, iterasi 100 — Percobaan 2

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	101

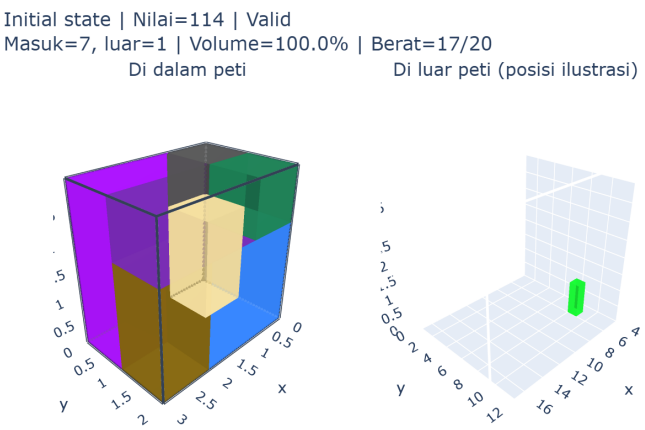
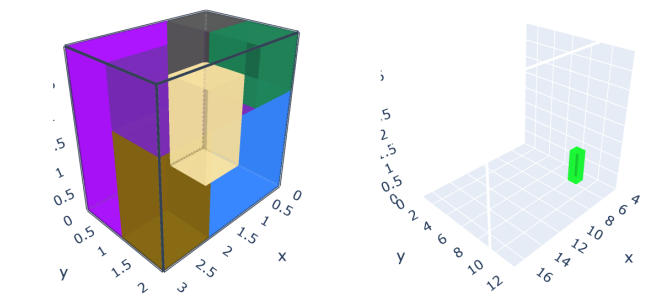
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	20
Banyak iterasi	100
Durasi pencarian	0.14194 detik

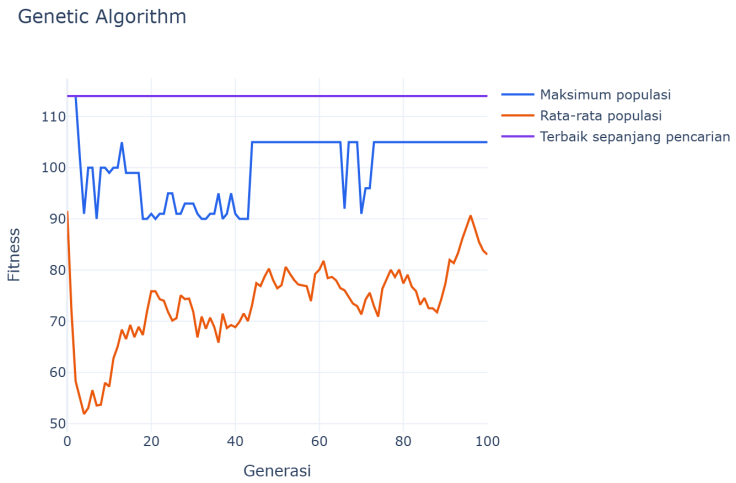
Test case tc9 — Populasi 20, iterasi 100 — Percobaan 3

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	103

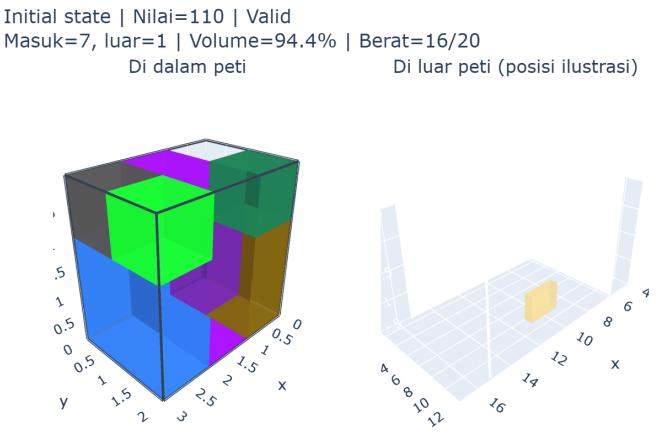
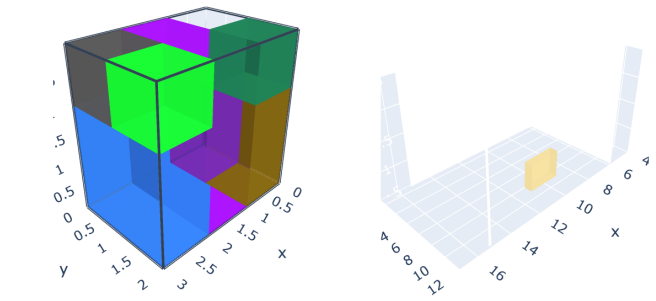
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	20
Banyak iterasi	100
Durasi pencarian	0.209694 detik

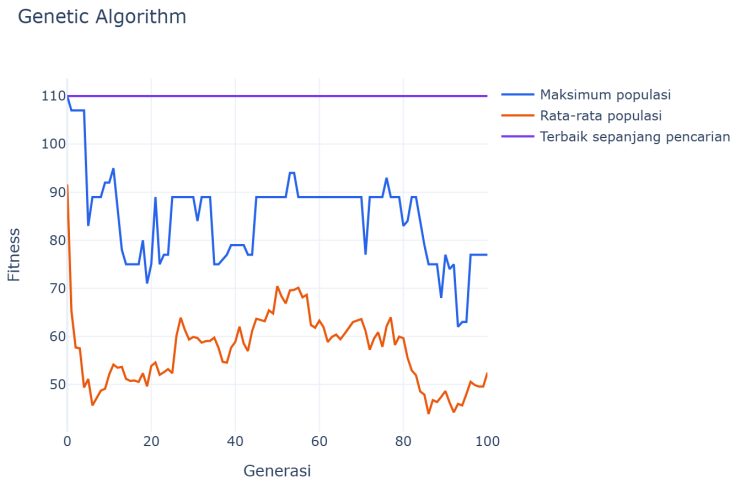
Test case tc9 — Populasi 50, iterasi 100 — Percobaan 1

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	114

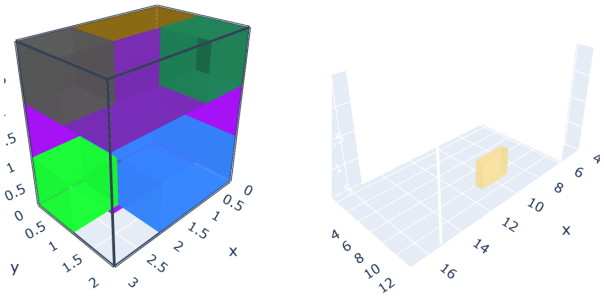
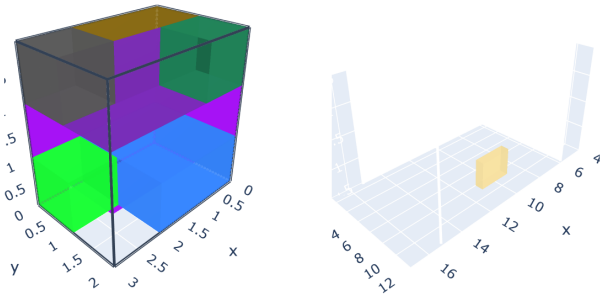
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	100
Durasi pencarian	0.482106 detik

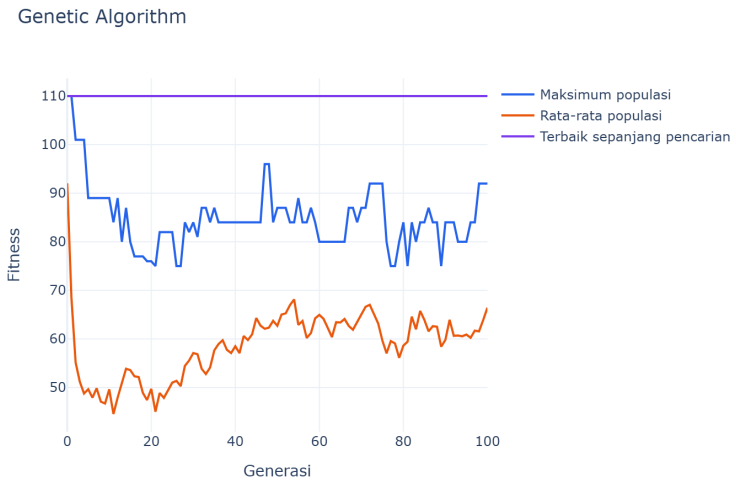
Test case tc9 — Populasi 50, iterasi 100 — Percobaan 2

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	110

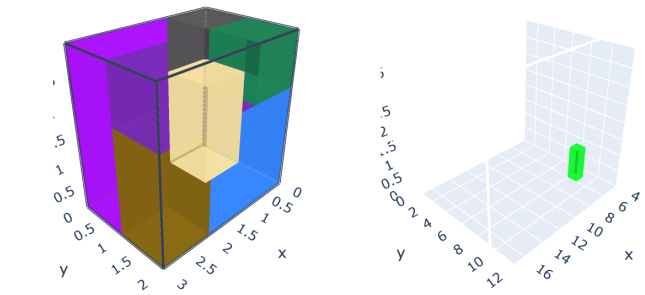
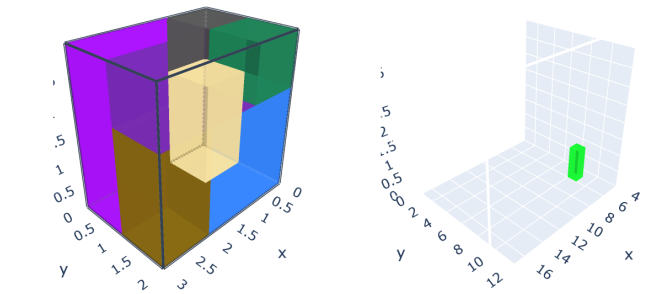
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	100
Durasi pencarian	0.387874 detik

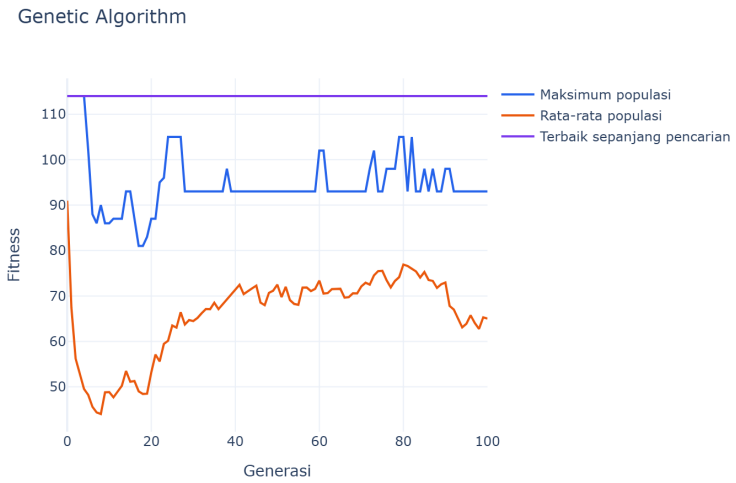
Test case tc9 — Populasi 50, iterasi 100 — Percobaan 3

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	Initial state | Nilai=110 | Valid Masuk=7, luar=1 | Volume=94.4% | Berat=16/20 Di dalam peti Di luar peti (posisi ilustrasi) 
State akhir terbaik	Final state | Nilai=110 | Valid Masuk=7, luar=1 | Volume=94.4% | Berat=16/20 Di dalam peti Di luar peti (posisi ilustrasi) 
Nilai *objective function* akhir terbaik	110

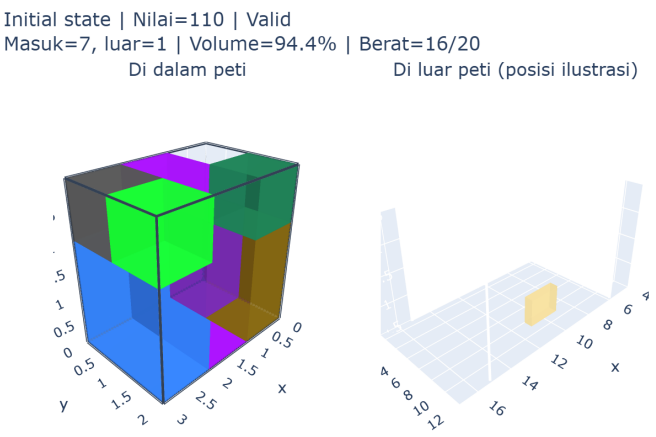
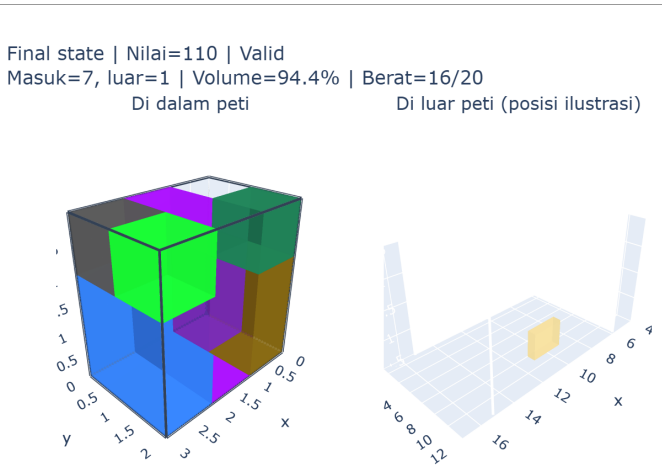
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	100
Durasi pencarian	0.372883 detik

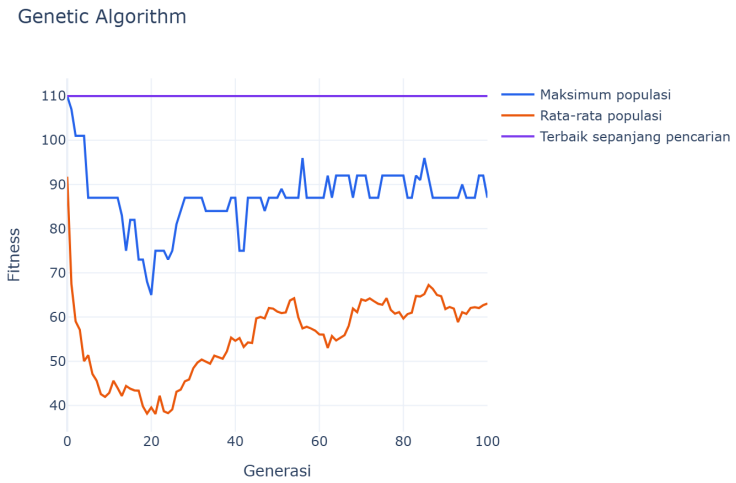
Test case tc9 — Populasi 100, iterasi 100 — Percobaan 1

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	114

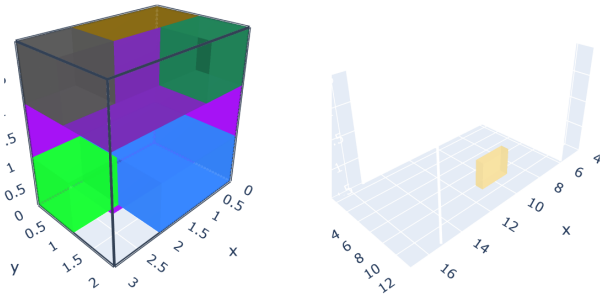
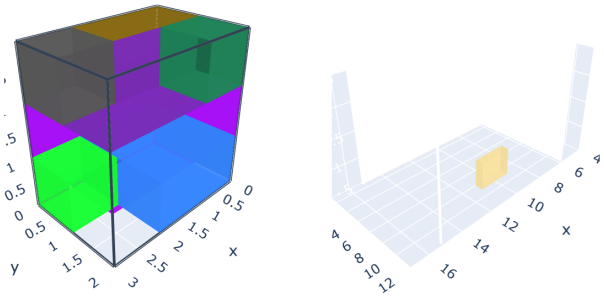
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	100
Banyak iterasi	100
Durasi pencarian	0.991931 detik

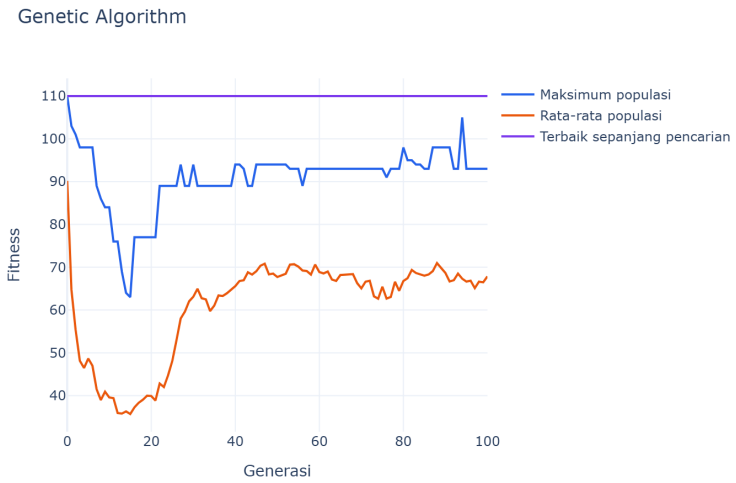
Test case tc9 — Populasi 100, iterasi 100 — Percobaan 2

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	110

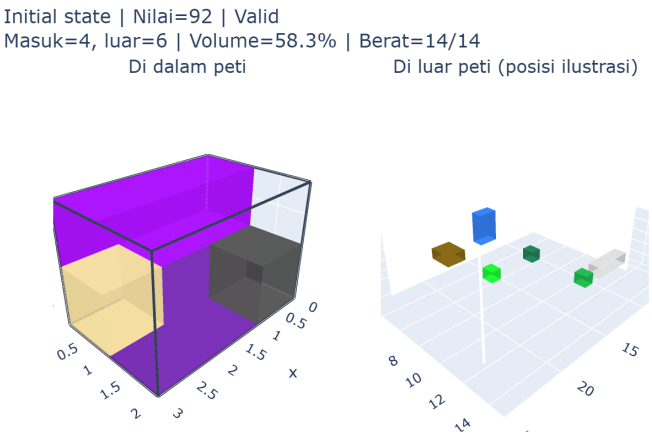
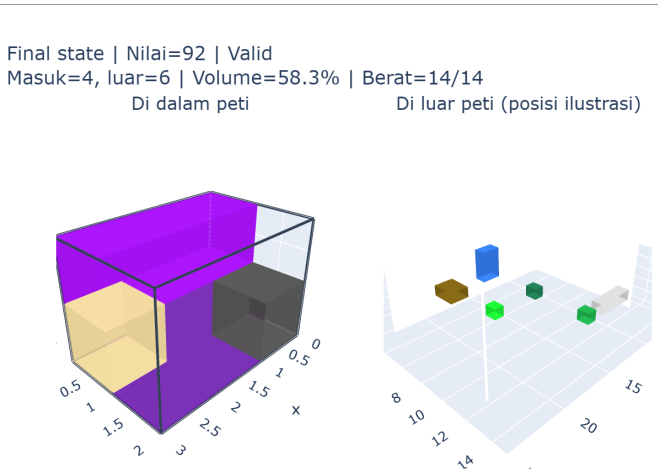
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	100
Banyak iterasi	100
Durasi pencarian	0.796501 detik

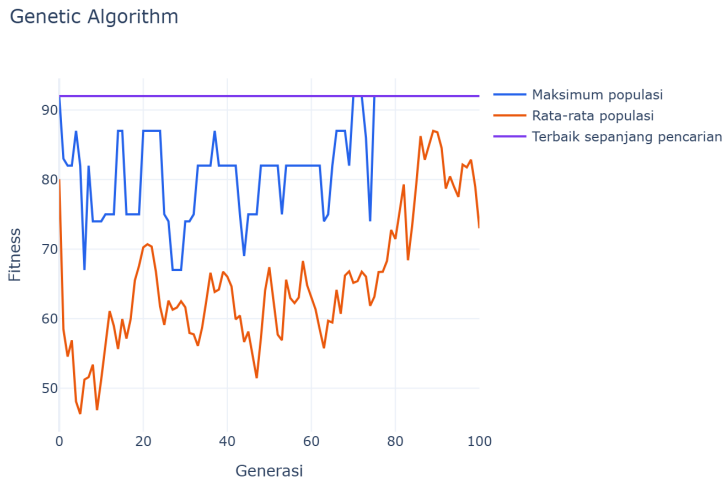
Test case tc9 — Populasi 100, iterasi 100 — Percobaan 3

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	Initial state | Nilai=110 | Valid Masuk=7, luar=1 | Volume=94.4% | Berat=16/20 Di dalam peti Di luar peti (posisi ilustrasi) 
State akhir terbaik	Final state | Nilai=110 | Valid Masuk=7, luar=1 | Volume=94.4% | Berat=16/20 Di dalam peti Di luar peti (posisi ilustrasi) 
Nilai *objective function* akhir terbaik	110

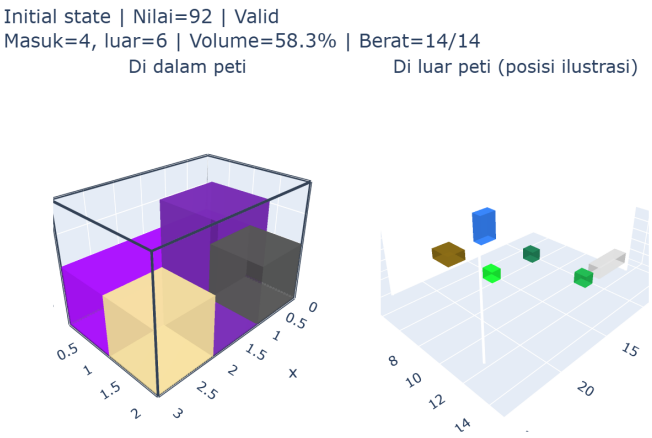
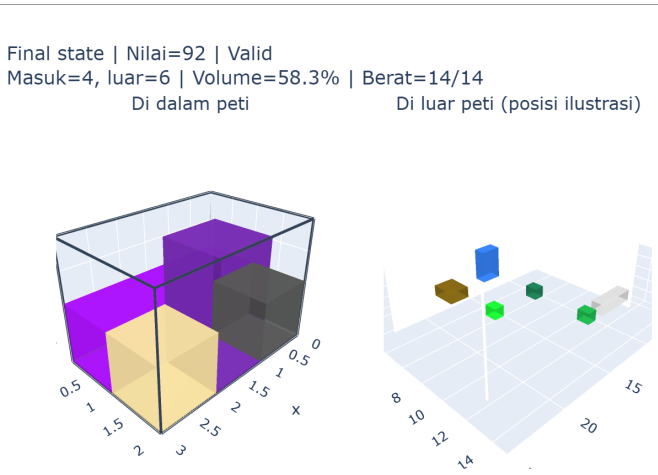
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	100
Banyak iterasi	100
Durasi pencarian	0.910508 detik

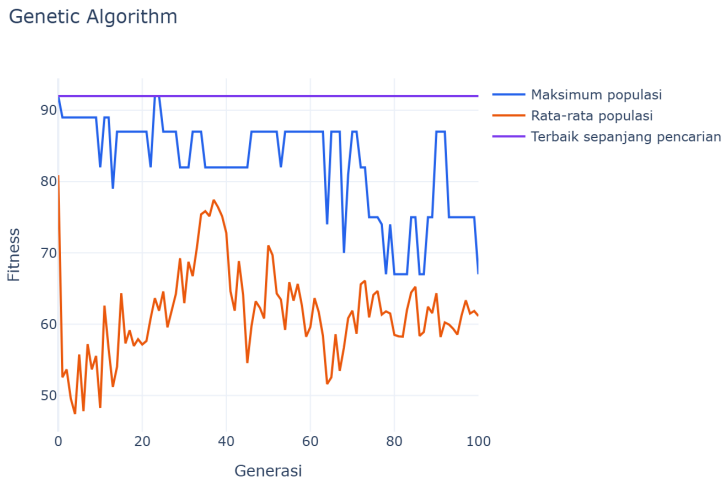
Test case tc10 — Populasi 20, iterasi 100 — Percobaan 1

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	92

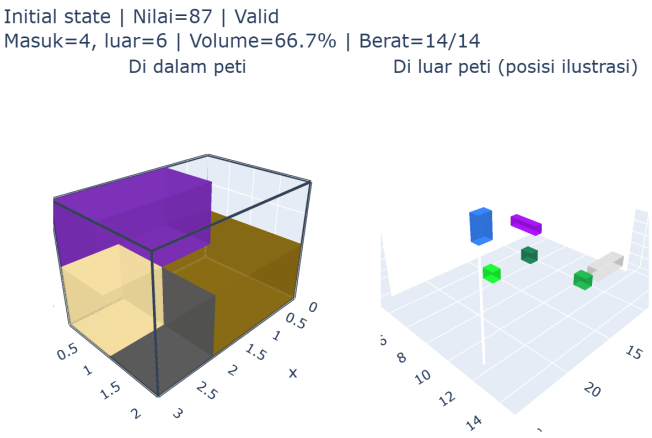
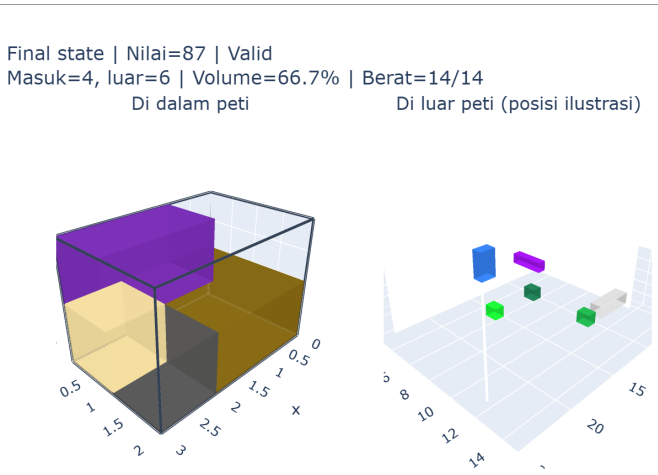
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	20
Banyak iterasi	100
Durasi pencarian	0.161459 detik

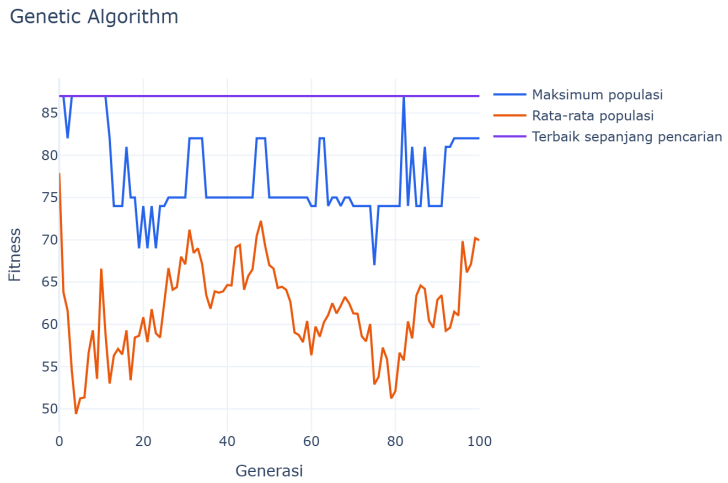
Test case tc10 — Populasi 20, iterasi 100 — Percobaan 2

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	92

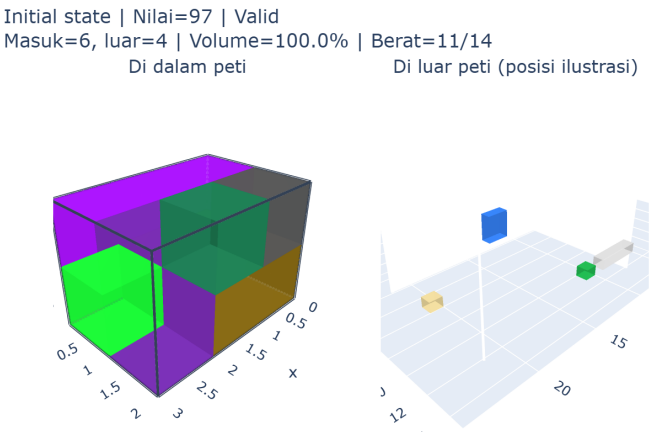
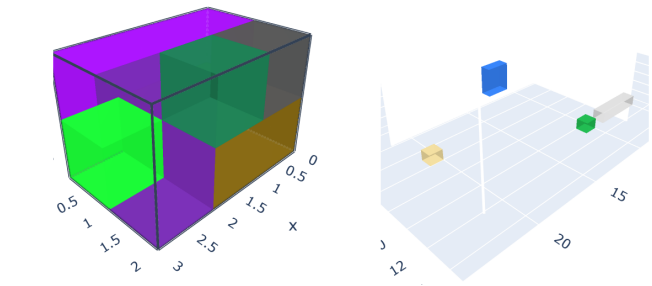
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	20
Banyak iterasi	100
Durasi pencarian	0.134301 detik

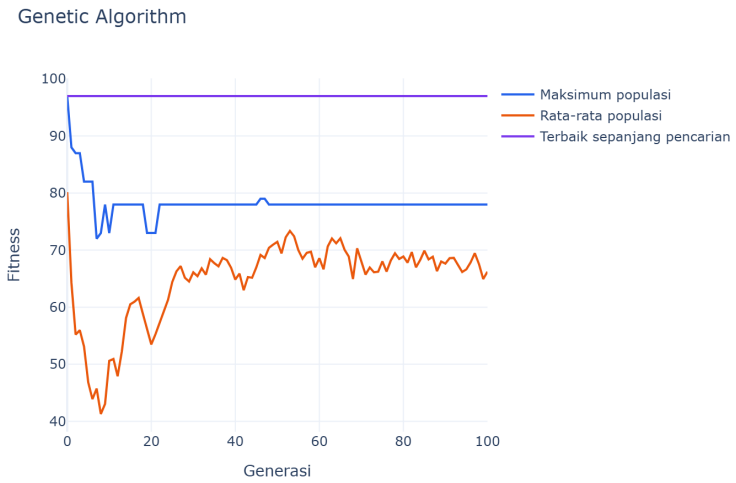
Test case tc10 — Populasi 20, iterasi 100 — Percobaan 3

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	87

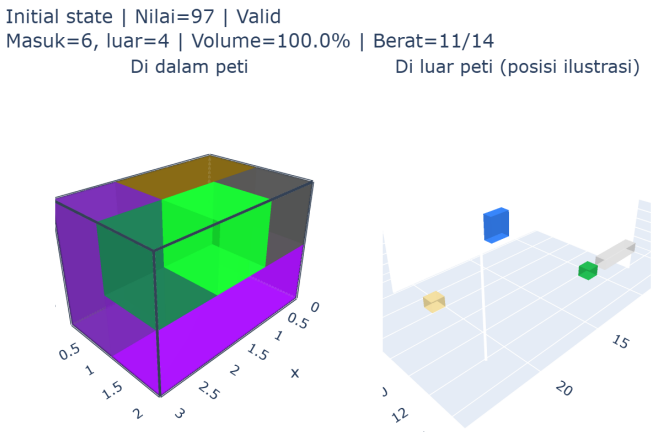
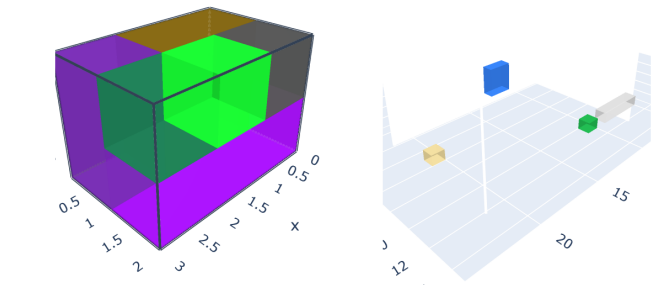
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	20
Banyak iterasi	100
Durasi pencarian	0.147517 detik

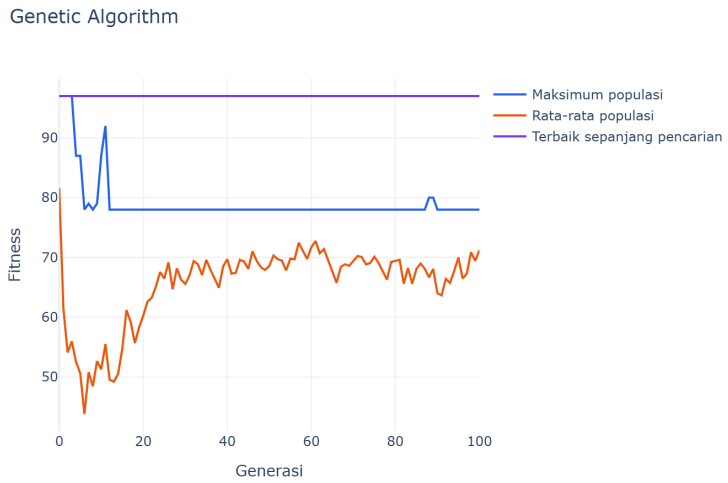
Test case tc10 — Populasi 50, iterasi 100 — Percobaan 1

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	97

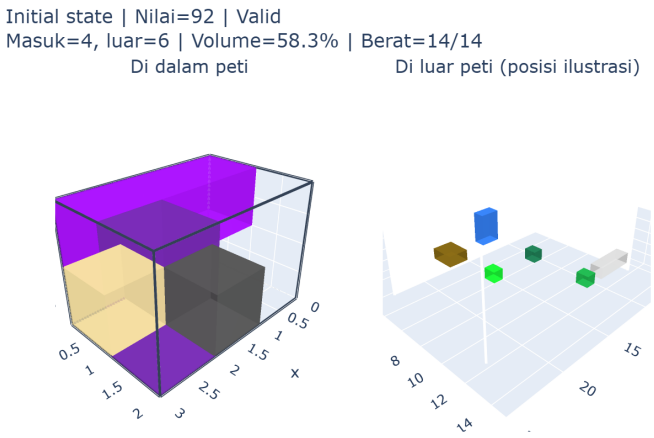
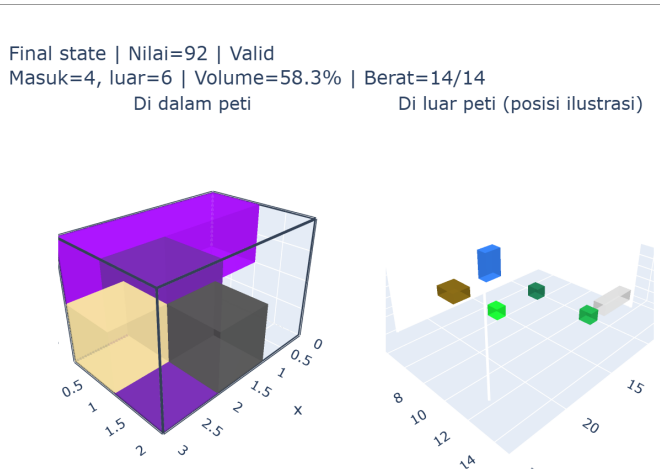
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	100
Durasi pencarian	0.40251 detik

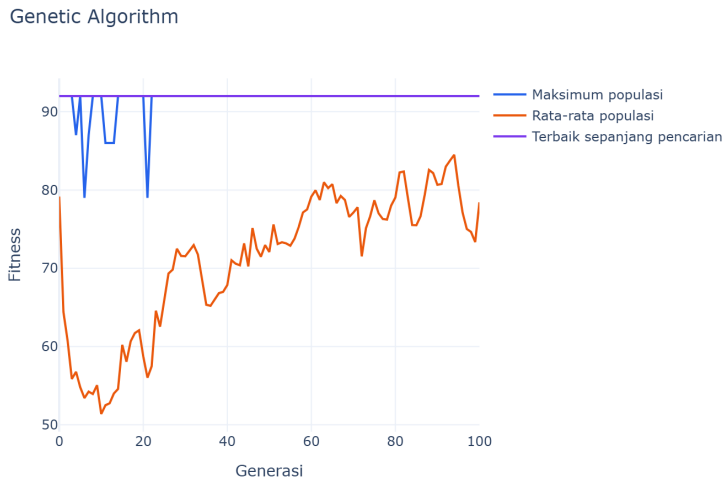
Test case tc10 — Populasi 50, iterasi 100 — Percobaan 2

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	97

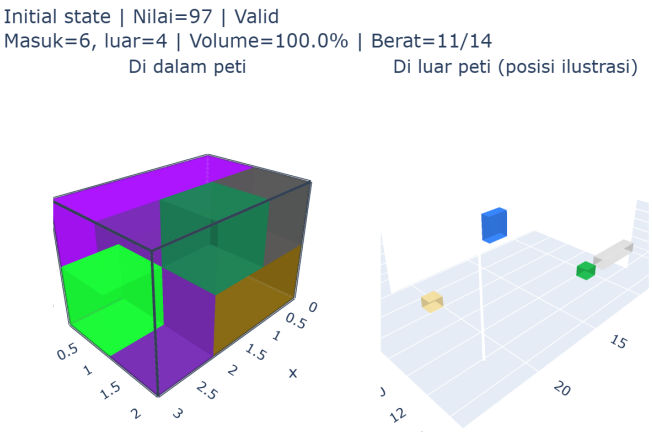
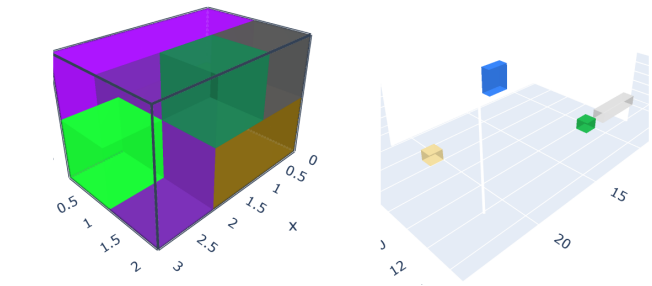
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	100
Durasi pencarian	0.384656 detik

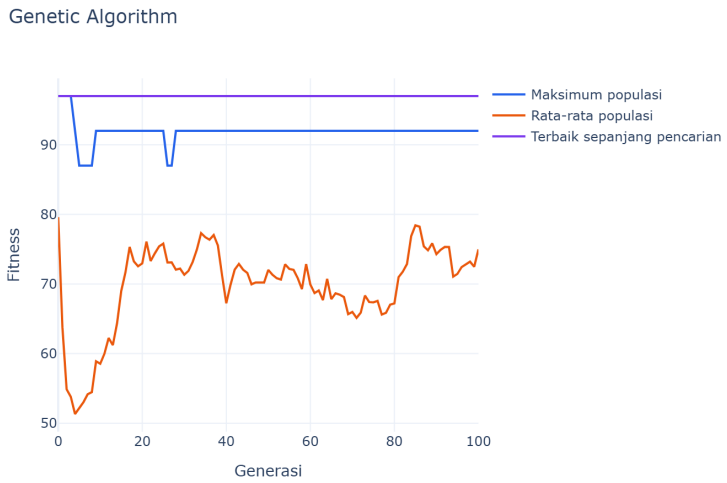
Test case tc10 — Populasi 50, iterasi 100 — Percobaan 3

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	92

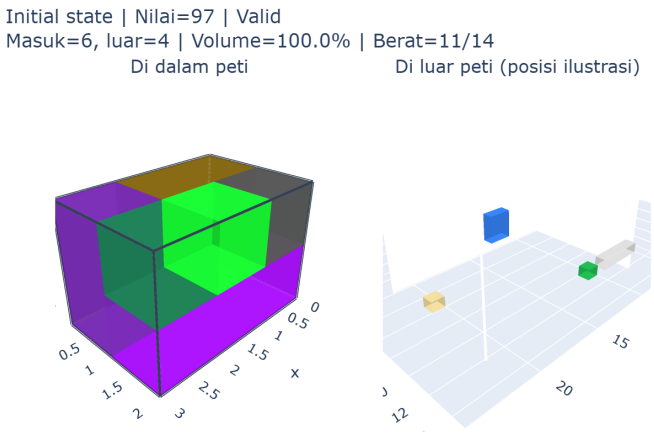
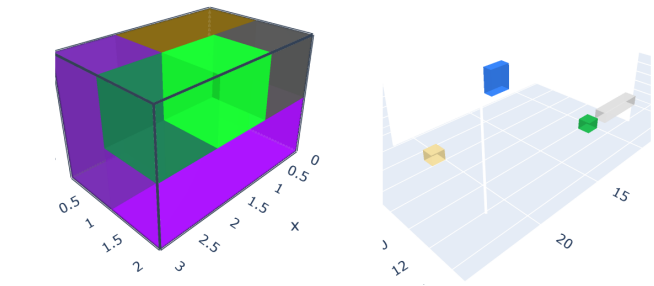
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	50
Banyak iterasi	100
Durasi pencarian	0.38904 detik

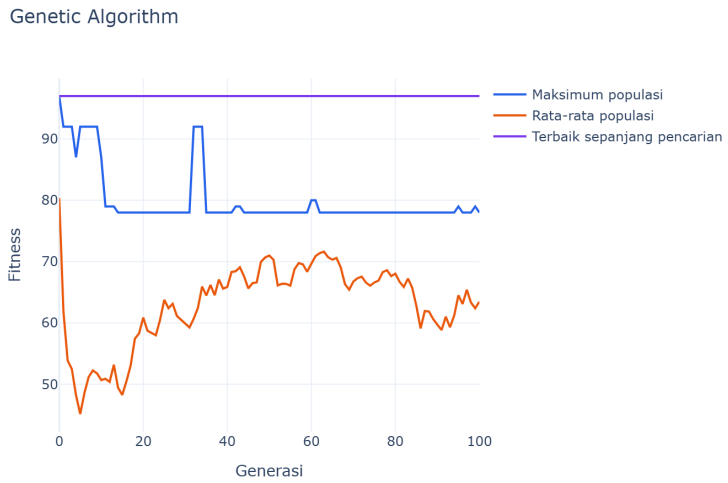
Test case tc10 — Populasi 100, iterasi 100 — Percobaan 1

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	97

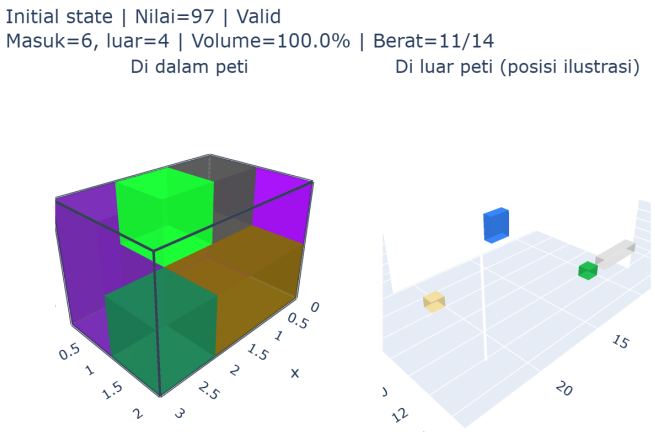
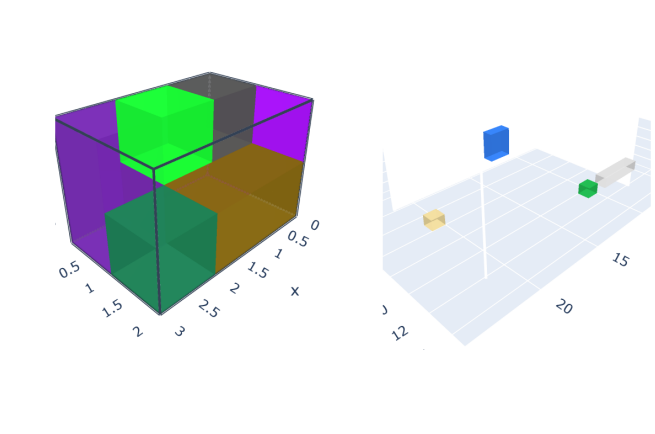
Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	100
Banyak iterasi	100
Durasi pencarian	0.923661 detik

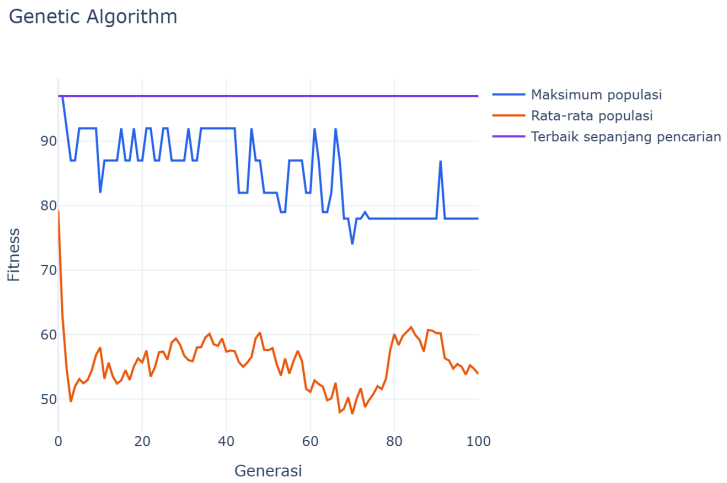
Test case tc10 — Populasi 100, iterasi 100 — Percobaan 2

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	97

Informasi yang dicatat	**Data hasil pengujian**
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	100
Banyak iterasi	100
Durasi pencarian	0.781175 detik

Test case tc10 — Populasi 100, iterasi 100 — Percobaan 3

Informasi yang dicatat	**Data hasil pengujian**
State awal (populasi awal)	
State akhir terbaik	
Nilai *objective function* akhir terbaik	97

Informasi yang dicatat	Data hasil pengujian
Grafik objektif maksimum dan rata-rata terhadap iterasi	
Jumlah populasi	100
Banyak iterasi	100
Durasi pencarian	0.745999 detik

4.4 Analisis Hasil Eksperimen

Untuk setiap konfigurasi yang diulang tiga kali, digunakan rata-rata nilai akhir $\overline{f}$, simpangan baku s, nilai minimum, nilai maksimum, dan rata-rata durasi $\overline{t}$. Nilai tersebut dihitung sebagai berikut.

$$\overline{f} = \frac{1}{3}\sum_{i=1}^{3} f_i, \qquad s = \sqrt{\frac{1}{2}\sum_{i=1}^{3}(f_i - \overline{f})^2}, \qquad \overline{t} = \frac{1}{3}\sum_{i=1}^{3} t_i.$$

4.4.1 Ringkasan Perbandingan Nilai Akhir dan Durasi

Tabel berikut merangkum statistik hasil tiga *run* untuk setiap algoritma dan konfigurasi pada ketiga test case.

Test case	Algoritma/ konfigurasi	$\overline{f}$	Min.–maks.	Standar Deviasi	Rata-rata Durasi (detik)
tc8	Steepest Ascent	97,33	96–98	1,15	0,0161
	Sideways Move	99,00	96–103	3,61	0,0890
	Random Restart	101,33	98–103	2,89	0,0817
	Stochastic HC	99,67	98–103	2,89	2,2975
	Simulated Annealing	103,00	103–103	0,00	0,3364
	GA: pop 20, it 100	103,00	103–103	0,00	0,1023
	GA: pop 50, it 50	103,00	103–103	0,00	0,1308
	GA: pop 50, it 100	103,00	103–103	0,00	0,2608
	GA: pop 50, it 200	103,00	103–103	0,00	0,5381
	GA: pop 100, it 100	103,00	103–103	0,00	0,6202
tc9	Steepest Ascent	99,33	96–101	2,89	0,0094
	Sideways Move	101,00	101–101	0,00	0,1106
	Random Restart	101,00	101–101	0,00	0,0627
	Stochastic HC	99,33	96–101	2,89	12,5682
	Simulated Annealing	105,33	103–110	4,04	1,6161
	GA: pop 20, it 100	102,33	101–103	1,15	0,1676
	GA: pop 50, it 50	111,33	110–114	2,31	0,2371
	GA: pop 50, it 100	111,33	110–114	2,31	0,4143
	GA: pop 50, it 200	111,33	110–114	2,31	0,7914
	GA: pop 100, it 100	111,33	110–114	2,31	0,8996
	Steepest Ascent	84,00	82–87	2,65	0,0083

Test case	Algoritma/ konfigurasi	$\overline{f}$	Min.–maks.	Standar Deviasi	Rata-rata Durasi (detik)
	Sideways Move	84,00	82–87	2,65	0,0657
	Random Restart	89,33	83–97	7,09	0,0616
	Stochastic HC	84,00	82–87	2,65	5,2280
	Simulated Annealing	86,00	84–87	1,73	0,4508
	GA: pop 20, it 100	90,33	87–92	2,89	0,1478
	GA: pop 50, it 50	95,33	92–97	2,89	0,1901
	GA: pop 50, it 100	95,33	92–97	2,89	0,3921
	GA: pop 50, it 200	95,33	92–97	2,89	0,8102
	GA: pop 100, it 100	97,00	97–97	0,00	0,8169

4.4.2 Kedekatan terhadap Nilai Terbaik yang Ditemukan

Nilai terbaik yang ditemukan pada `tc8` adalah 103. Nilai ini dicapai oleh *Simulated Annealing* pada ketiga *run* ($s = 0{,}00$), serta oleh seluruh konfigurasi *Genetic Algorithm* dengan simpangan baku nol. *Sideways Move* dan *Random Restart* mampu mencapai 103 pada setidaknya satu *run*, namun rata-rata keduanya sedikit di bawah 103 karena beberapa *run* terhenti di optimum lokal yang berbeda. *Steepest Ascent* paling jauh dengan rata-rata 97,33; hal ini konsisten dengan keterbatasan algoritma yang langsung berhenti saat tidak ada tetangga yang lebih baik, tanpa mekanisme keluar dari optimum lokal.

Nilai terbaik yang ditemukan pada `tc9` adalah 114, yang hanya dicapai oleh *run* pertama GA dengan populasi 50 dan 100. *Simulated Annealing* paling tinggi mencapai 110. Seluruh varian Hill-climbing terperangkap pada rentang 96–101, jauh di bawah nilai 114. Hal ini menunjukkan bahwa `tc9`, yang memiliki batasan ruang lebih ketat serta penempatan item *fragile*, menciptakan lanskap fungsi objektif yang lebih kompleks, sehingga algoritma berbasis tetangga lokal tidak cukup ekspresif untuk melewati penghalang menuju wilayah solusi yang lebih tinggi.

Pada `tc10`, nilai terbaik yang ditemukan adalah 97, dicapai oleh GA pop-100/it-100 pada ketiga *run* ($s = 0{,}00$) dan oleh dua dari tiga *run* GA pop-50 pada seluruh variasi iterasi. *Random Restart* mencapai 97 pada satu *run* ($s = 7{,}09$) namun tidak konsisten. Varian Hill-climbing lainnya dan SA hanya mencapai maksimum 87.

Perlu dicatat bahwa pada seluruh 45 *run* GA di ketiga *test case*, individu terbaik selalu ditemukan pada generasi ke-0 (best_generation = 0), sehingga nilai akhir GA selalu sama dengan nilai individu terbaik pada populasi awal. Dengan demikian, kedekatan GA terhadap nilai terbaik ditentukan oleh kualitas populasi awal, bukan oleh proses evolusinya. Penyebabnya dibahas pada Section 4.4.6.

4.4.3 Konsistensi Hasil antar-*Run*

Berdasarkan tabel ringkasan, dapat dilihat pola konsistensi antar-algoritma sebagai berikut:

- ***Simulated Annealing*** menunjukkan konsistensi tertinggi pada `tc8` ($s = 0{,}00$), namun menjadi kurang konsisten pada `tc9` ($s = 4{,}04$) karena perbedaan lintasan suhu menghasilkan eksplorasi yang berbeda-beda. Hal ini menunjukkan bahwa jadwal pendinginan yang sama dapat menghasilkan solusi yang bervariasi saat lanskap lebih kompleks.
- ***Steepest Ascent*** dan ***Stochastic HC*** menunjukkan variasi yang identik antara satu sama lain pada `tc9` dan `tc10` ($s = 2{,}89$, rentang sama). Kesamaan ini terjadi karena *state* awal dibangkitkan dari *seed* yang sama, sehingga titik awal pencarian pun serupa. Perbedaan hanya muncul dari strategi pemilihan tetangga (deterministik vs. acak).
- ***Random Restart*** memiliki simpangan baku tertinggi pada `tc10` ($s = 7{,}09$). Satu dari tiga *run* berhasil menemukan wilayah 97 melalui *restart* yang berbeda, sementara dua lainnya terhenti lebih awal pada 83 dan 88. Hal ini mencerminkan sifat probabilistik yang dimiliki mekanisme *restart* dengan jumlah terbatas.
- ***Genetic Algorithm*** dengan populasi besar (≥ 50) menghasilkan nilai yang lebih tinggi dan cenderung lebih konsisten. GA pop-100/it-100 mencapai $s = 0{,}00$ pada `tc10`. Karena nilai akhir GA selalu berasal dari populasi awal, konsistensi ini muncul karena populasi awal yang besar memuat lebih banyak *state* acak sehingga peluang memuat *state* bernilai tinggi lebih besar pada setiap *seed*. Sebaliknya, GA pop-20/it-100 menghasilkan rata-rata terendah di

antara konfigurasi GA pada `tc9` ($\overline{f} = 102{,}33$) dan `tc10` ($\overline{f} = 90{,}33$, $s = 2{,}89$).

4.4.4 Perbandingan Durasi dan Biaya Pencarian

Hierarki durasi yang konsisten dapat terlihat di seluruh test case:

1. *Steepest Ascent* paling cepat ($\overline{t} < 0{,}02$ detik),
2. *Sideways Move* dan *Random Restart* (keduanya $< 0{,}15$ detik),
3. GA dengan populasi atau iterasi kecil (pop-20/it-100 dan pop-50/it-50, 0,10–0,24 detik),
4. GA dengan populasi atau iterasi besar (pop-50/it-200 dan pop-100/it-100, 0,54–0,90 detik) serta SA (0,34–1,62 detik) yang durasinya saling bertumpang tindih bergantung *test case*, dan terakhir
5. *Stochastic HC* yang jauh lebih lambat (2,30–12,57 detik) meskipun nilai akhirnya setara atau lebih rendah dibanding SA.

Durasi *Stochastic HC* yang relatif lama dapat dijelaskan dari karakteristik pergerakannya. Karena hanya memeriksa satu tetangga acak pada setiap langkah, algoritma ini membutuhkan hingga 10.000 iterasi sebelum berhenti. Sebaliknya, *Steepest Ascent* berhenti sangat cepat (hanya dalam 1–2 iterasi) begitu tidak ada tetangga yang lebih baik di sekelilingnya. Sementara itu, durasi eksekusi *Simulated Annealing* yang berada di atas varian HC deterministik sebanding dengan efektivitasnya dalam menghasilkan solusi dengan nilai objektif yang jauh lebih optimal.

Pada GA, durasi sebanding dengan jumlah generasi dikali ukuran populasi, karena setiap generasi membentuk dan mengevaluasi population_size individu baru. GA pop-20/it-100 ($\overline{t} \approx 0{,}10$–$0{,}17$ detik) setara dengan *Sideways Move* dan menghasilkan nilai 87–103 bergantung *test case*. GA pop-100/it-100 ($\overline{t} \approx 0{,}62$–$0{,}90$ detik) merupakan konfigurasi GA termahal bersama pop-50/it-200, namun juga yang paling andal pada *test case* yang lebih sulit.

4.4.5 Perilaku Khusus Algoritma *Hill-climbing* dan *Simulated Annealing*

- ***Steepest Ascent***: grafik nilai objektif menunjukkan kenaikan tajam pada satu hingga dua iterasi pertama, lalu berhenti sama sekali. Berhentinya pencarian setelah iterasi ke-1 pada `tc9` dan `tc10` mengindikasikan bahwa *state*

awal sudah berada pada atau di dekat puncak optimum lokal yang rendah, sehingga tidak ada arah perbaikan yang dapat diambil sejak dini.

- ***Sideways Move***: parameter *maximum sideways move* = 10 memungkinkan algoritma bergerak secara lateral. Pada `tc8`, satu *run* mencapai 103 dengan memanfaatkan total 10 langkah lateral (12 iterasi), membuktikan bahwa langkah lateral membantu keluar dari dataran optimum lokal. Namun, dua *run* lainnya hanya mencapai 96–98, menunjukkan bahwa langkah lateral tidak selalu berhasil membuka jalan menuju nilai yang lebih baik.

- ***Random Restart***: dengan *maximum restart* = 3, setiap *run* menjalankan empat pencarian lokal. Pada `tc10`, satu *run* berhasil menemukan nilai 97 melalui salah satu *restart*. Namun dua *run* lainnya tidak mendapat manfaat yang sama, mencerminkan keterbatasan jumlah *restart* dan sifat probabilistik titik awal.

- ***Stochastic HC***: grafik objektif terlihat jauh lebih fluktuatif dibanding *Steepest Ascent* karena setiap langkah hanya memilih satu tetangga acak yang lebih baik. Meski demikian, batas iterasi 10.000 memberikan kesempatan lebih banyak untuk menemukan solusi yang lebih baik. Hal tersebut terbukti dari satu *run* `tc8` yang berhasil mencapai nilai 103.

- ***Simulated Annealing***: pada fase suhu tinggi, nilai $e^{\Delta E/T}$ besar sehingga probabilitas penerimaan langkah yang memperburuk solusi tinggi. Hal ini menyebabkan nilai objektif sesekali turun pada iterasi awal sebelum naik secara bertahap saat suhu mendingin. Frekuensi *stuck* yang tercatat (2–5 episode per *run*) menunjukkan bahwa algoritma memang sesekali terjebak, tetapi mekanisme penerimaan probabilistik berhasil membawanya keluar. Pada `tc9`, SA berhasil mencapai nilai 110 pada satu *run*, yang merupakan capaian tertinggi di antara seluruh varian non-GA.

4.4.6 Pengaruh Parameter pada *Genetic Algorithm*

Peran operator genetik. Pada seluruh 45 *run* GA di `tc8`, `tc9`, dan `tc10`, nilai akhir selalu sama dengan individu terbaik populasi awal (best_generation = 0). Grafik GA juga menunjukkan bahwa rata-rata *fitness* populasi justru turun setelah generasi pertama. Sebagai contoh, pada konfigurasi pop-50/it-100, rata-rata populasi turun dari 79–92 pada generasi ke-0 menjadi 52–83 pada generasi ke-100, dan nilai maksimum populasi pada generasi terakhir umumnya lebih rendah daripada

individu terbaik awal (misalnya 73 dibanding 103 pada `tc8`).

Hal ini disebabkan oleh operator *crossover* yang menjaga validitas (Section 3.7). Anak mewarisi posisi paket dari dua orang tua yang susunannya berbeda, sehingga banyak paket warisan saling bertabrakan atau kehilangan penyangga. Paket tersebut dikeluarkan dari truk tanpa dicarikan posisi pengganti, sehingga anak hampir selalu memuat lebih sedikit paket dan bernilai lebih rendah daripada orang tuanya. *Mutation* hanya mengubah satu paket dengan peluang 0,30, sehingga tidak cukup untuk memulihkan paket yang hilang. Karena tidak menggunakan *elitism*, individu terbaik juga tidak dipertahankan pada generasi berikutnya. Dengan demikian, pada implementasi ini kualitas GA terutama ditentukan oleh pengambilan sampel *state* acak pada populasi awal, sedangkan proses evolusi tidak menghasilkan individu yang lebih baik.

Variasi jumlah iterasi (populasi tetap 50). Pada ketiga *test case*, kenaikan iterasi dari 50 ke 100 dan 200 tidak mengubah nilai akhir ($\overline{f}$ tetap 103,00; 111,33; dan 95,33) maupun rentangnya. Hal ini konsisten dengan temuan di atas: karena individu terbaik selalu berasal dari generasi ke-0, penambahan generasi tidak memberikan manfaat kualitas. Sebaliknya, durasi meningkat hampir linier terhadap jumlah iterasi, misalnya pada `tc8` 0,131 $\rightarrow$ 0,261 $\rightarrow$ 0,538 detik.

Variasi jumlah populasi (iterasi tetap 100). Ukuran populasi berpengaruh nyata terhadap hasil karena menentukan banyaknya *state* acak pada populasi awal. Pada `tc9`, GA pop-20 menghasilkan $\overline{f} = 102{,}33$ ($s = 1{,}15$), sedangkan GA pop-50 dan pop-100 keduanya mencapai $\overline{f} = 111{,}33$ ($s = 2{,}31$). Pada `tc10`, GA pop-20 menghasilkan rentang 87–92 ($\overline{f} = 90{,}33$), GA pop-50 mencapai 92–97 ($\overline{f} = 95{,}33$), dan GA pop-100 secara konsisten mencapai 97 ($s = 0{,}00$). Pada `tc8`, seluruh ukuran populasi sudah cukup untuk memuat *state* bernilai 103. Durasi GA pop-100 ($\overline{t} \approx$ 0,62–0,90 detik) sekitar 5–6$\times$ lebih lama daripada pop-20 ($\overline{t} \approx$ 0,10–0,17 detik), tetapi peningkatan kualitasnya signifikan pada `tc9` dan `tc10`.

4.4.7 Kesimpulan Analisis

Dari seluruh eksperimen, karakteristik test case sangat memengaruhi kinerja relatif algoritma. Pada `tc8` yang relatif sederhana, hampir seluruh algoritma berhasil mencapai nilai terbaik 103 secara konsisten; perbedaan utama hanya terletak pada durasi. Pada `tc9` dan `tc10` yang lebih kompleks, SA dan GA unggul dalam kualitas solusi dibanding varian Hill-climbing, tetapi dengan sebab yang berbeda. SA mampu keluar dari optimum lokal melalui penerimaan probabilistik berbasis suhu. GA

unggul karena populasi awalnya memuat banyak *state* acak yang dievaluasi sekaligus, sedangkan operasi *crossover* dan mutasi pada implementasi ini tidak pernah menghasilkan individu yang lebih baik daripada populasi awal.

Dari sisi konsistensi, GA dengan populasi besar (≥ 50) dan SA pada `tc8` menunjukkan kestabilan tertinggi. Sebaliknya, *Random Restart* menjadi algoritma paling tidak konsisten karena ketergantungannya pada variasi titik awal dengan jumlah *restart* yang terbatas. Dari sisi efisiensi waktu, *Steepest Ascent* paling cepat namun paling rendah kualitasnya; *Random Restart* menawarkan keseimbangan terbaik di antara varian Hill-climbing. SA merupakan pilihan terbaik apabila kualitas solusi yang tinggi dibutuhkan dalam waktu yang masih terkendali, sedangkan GA pop-100 menghasilkan nilai tertinggi dan paling konsisten pada test case yang paling sulit dengan durasi yang masih di bawah satu detik.

Bab 5

Kesimpulan dan Saran

5.1 Kesimpulan

Pada tugas besar ini telah diimplementasikan enam algoritma *local search*, yaitu empat varian *Hill-climbing* (*steepest ascent*, *sideways move*, *random restart*, dan *stochastic*), *Simulated Annealing*, dan *Genetic Algorithm*, untuk menyelesaikan permasalahan pemuatan paket ke dalam truk yang bertujuan memaksimalkan total nilai paket yang dimuat. *State* direpresentasikan sebagai posisi dan orientasi setiap paket, sedangkan seluruh *constraint* (batas dinding, tumpang tindih, kapasitas berat, paket melayang, dan paket rapuh) diperlakukan sebagai *hard constraint*. Karena *state* awal dan seluruh *neighbor* yang dihasilkan selalu diperiksa sebelum digunakan, fungsi objektif cukup berupa jumlah nilai paket yang dimuat tanpa suku penalti. Keenam algoritma memakai komponen yang sama (representasi *state*, fungsi objektif, operator *neighbor*, dan format hasil), sehingga perbandingan antaralgoritma dapat dilakukan secara adil dan seluruh data yang diminta spesifikasi dapat dicatat dengan cara yang seragam.

Ketiga kelompok algoritma menunjukkan karakteristik yang berbeda. *Hill-climbing* hanya berpindah ke *neighbor* yang lebih baik sehingga pencarian berhenti pada *local optimum*. Varian *sideways move* dan *random restart* berupaya mengurangi masalah tersebut, masing-masing dengan melewati daerah datar dan memulai ulang dari *state* awal yang baru, tetapi tidak menjamin tercapainya *global optimum*. *Simulated Annealing* dapat menerima *neighbor* yang lebih buruk dengan probabilitas $e^{\Delta E/T}$ yang mengecil seiring turunnya suhu, sehingga mampu keluar dari *local optimum* pada tahap awal dan bersifat semakin selektif pada tahap akhir. *Genetic Algorithm* menjaga sekumpulan solusi sekaligus dan menggabungkannya melalui *crossover* dan *mutation*, dengan biaya evaluasi yang bergantung pada jumlah populasi dan jumlah generasi. Pada implementasi ini, *crossover* yang menjaga validitas mengeluarkan paket warisan yang bertabrakan atau kehilangan penyangga, sehingga anak cenderung bernilai lebih rendah daripada orang tuanya.

Pengujian dilakukan pada tiga *test case* (`tc8`, `tc9`, dan `tc10`) dengan tiga *run* untuk setiap algoritma atau konfigurasi. Dari sisi kualitas solusi, *Genetic Algorithm* dan *Simulated Annealing* menghasilkan nilai objektif akhir yang lebih tinggi daripada se-

luruh varian *Hill-climbing*. Nilai terbaik yang ditemukan adalah 103 pada `tc8`, 114 pada `tc9`, dan 97 pada `tc10`. Pada `tc8`, *Simulated Annealing* dan seluruh konfigurasi *Genetic Algorithm* mencapai 103 pada ketiga *run*, sedangkan rata-rata varian *Hill-climbing* berkisar 97,33 hingga 101,33. Pada `tc9`, *Genetic Algorithm* dengan populasi 50 atau 100 memperoleh rata-rata 111,33, *Simulated Annealing* memperoleh 105,33, dan varian *Hill-climbing* hanya 99,33 hingga 101,00. Pada `tc10`, *Genetic Algorithm* dengan populasi 100 konsisten mencapai 97, sedangkan *Simulated Annealing* memperoleh rata-rata 86,00 dan varian *Hill-climbing* 84,00 hingga 89,33. Nilai *global optimum* tidak diketahui secara pasti, sehingga nilai terbaik yang ditemukan dijadikan acuan. Sebagai gambaran, pada `tc9` total nilai seluruh paket adalah 119 (dan seluruh paket secara total tidak melebihi kapasitas berat), sehingga nilai 114 yang ditemukan mencapai sekitar 95,8% dari batas atas tersebut.

Dari sisi durasi, *Steepest Ascent* merupakan yang tercepat dengan rata-rata 0,008 hingga 0,016 detik, diikuti *Sideways Move* dan *Random Restart* (0,06 hingga 0,11 detik). *Simulated Annealing* membutuhkan 0,34 hingga 1,62 detik, sedangkan *Genetic Algorithm* membutuhkan 0,10 hingga 0,90 detik bergantung pada jumlah populasi dan iterasi. *Stochastic Hill-climbing* merupakan yang paling lambat (2,30 hingga 12,57 detik) karena pada seluruh *run* menjalankan 10.000 iterasi penuh, tetapi nilai akhirnya tidak lebih baik daripada *Simulated Annealing*. Dari sisi konsistensi, *Genetic Algorithm* dengan populasi besar paling stabil (simpangan baku 0,00 pada `tc8` dan `tc10` untuk populasi 100), sedangkan *Random Restart* paling bervariasi pada `tc10` (simpangan baku 7,09) karena hasilnya bergantung pada titik awal yang terbatas jumlahnya.

Pada *Genetic Algorithm*, jumlah populasi lebih berpengaruh daripada jumlah iterasi. Pada seluruh 45 *run*, individu terbaik selalu berasal dari populasi awal, dan rata-rata *fitness* populasi justru menurun setelah generasi pertama. Akibatnya, dengan populasi 50, menaikkan iterasi dari 50 menjadi 100 dan 200 tidak mengubah nilai akhir pada ketiga *test case* dan hanya menambah durasi. Sebaliknya, populasi 20 menghasilkan nilai yang lebih rendah (rata-rata 102,33 pada `tc9` dan 90,33 pada `tc10`) dibandingkan populasi 50 dan 100, karena populasi awal yang kecil memuat lebih sedikit *state* acak sehingga peluang memperoleh *state* bernilai tinggi lebih kecil.

Secara keseluruhan, *Genetic Algorithm* dengan populasi besar unggul dalam kualitas dan konsistensi hasil, terutama berkat populasi awal yang besar dan bukan proses evolusinya, *Simulated Annealing* memberikan keseimbangan yang baik antara kualitas dan durasi, sedangkan varian *Hill-climbing* unggul dalam kecepatan (khususnya

Steepest Ascent) tetapi mudah terjebak pada *local optimum*, terutama pada *test case* dengan *constraint* yang lebih ketat seperti `tc9`.

5.2 Saran

Terdapat beberapa hal yang dapat ditingkatkan pada algoritma pencarian. Pertama, *state* awal saat ini dibangkitkan secara acak. Pembangkitan yang lebih terarah, misalnya memasukkan paket berdasarkan rasio nilai terhadap volume atau berat terlebih dahulu, dapat memberikan titik awal yang lebih baik bagi seluruh algoritma. Kedua, operator *neighbor* saat ini hanya terdiri dari *move*, *rotate*, dan *swap*, sehingga mengeluarkan paket dari truk hanya dapat terjadi melalui *swap*. Penambahan operator khusus untuk menambah atau mengeluarkan paket dapat memperluas ruang pencarian yang terjangkau. Ketiga, pada *Hill-climbing with Sideways Move*, penghitung *sideways move* yang kumulatif dapat diganti dengan penghitung *sideways move* berurutan yang di-*reset* setiap kali terjadi perbaikan, agar pencarian tidak berhenti terlalu dini pada permasalahan dengan banyak daerah datar. Keempat, *Stochastic Hill-climbing* selalu menghabiskan seluruh 10.000 iterasi sehingga menjadi algoritma paling lambat; penambahan kriteria berhenti berdasarkan stagnasi nilai objektif dapat mengurangi durasi tanpa mengubah kualitas hasil secara berarti.

Pada *Simulated Annealing*, kualitas hasil sangat dipengaruhi oleh suhu awal, laju pendinginan, dan suhu minimum. Suhu awal sebaiknya disesuaikan dengan skala nilai paket pada setiap *test case*, karena selisih nilai objektif ΔE menentukan probabilitas penerimaan langkah yang lebih buruk. Skema pendinginan yang adaptif atau mekanisme *reheating* ketika pencarian terjebak pada *local optimum* juga dapat dicoba. Pada *Genetic Algorithm*, dapat dilakukan penyetelan parameter lebih lanjut, terutama probabilitas *crossover* dan *mutation*, serta pengujian metode seleksi lain seperti *tournament selection*. Implementasi saat ini mengganti seluruh populasi pada setiap generasi tanpa *elitism*, sehingga penerapan *elitism* untuk menyimpan individu terbaik pada setiap generasi dapat mencegah hilangnya solusi yang baik. Selain itu, karena *crossover* saat ini membuat anak kehilangan banyak paket, operator *crossover* yang lebih sesuai dengan struktur ruang, misalnya mewarisi satu bagian ruang truk secara utuh dari tiap orang tua, perlu dicoba agar proses evolusi benar-benar dapat menghasilkan individu yang lebih baik daripada populasi awal.

Dari sisi efisiensi, pembangkitan *state* awal dan *neighbor* masih mengenumerasi seluruh koordinat yang mungkin dan memvalidasi seluruh *state* dari awal setiap

kali. Pembatasan kandidat posisi pada titik-titik yang bersinggungan dengan dinding atau paket lain, serta validasi inkremental yang hanya memeriksa paket yang berubah, dapat mempercepat pencarian secara signifikan, terutama pada *test case* dengan jumlah paket besar.

Terakhir, pengujian pada tugas ini baru mencakup tiga *test case* berskala kecil (8 hingga 10 paket), sehingga perbedaan kinerja antaralgoritma pada *test case* yang lebih besar belum teruji. Penelitian selanjutnya sebaiknya mengevaluasi algoritma pada lebih banyak *test case* dengan variasi jumlah paket, ukuran truk, dan proporsi paket rapuh, serta menggunakan lebih banyak *run* dengan *seed* berbeda agar konsistensi hasil dapat dinilai lebih andal.

Referensi

Institut Teknologi Bandung. (2026). *Beyond classical search* [Slide kuliah IF3170 Inteligensi Artifisial, Semester I TA 2026/2027]. Program Studi Teknik Informatika, Institut Teknologi Bandung.

Kirkpatrick, S., Gelatt, C. D., Jr., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science, 220*(4598), 671–680. https://doi.org/10.1126/science.220.4598.671

Lampiran

Pembagian Tugas

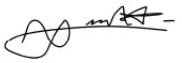
Tabel 1: Pembagian tugas anggota kelompok

No	Nama Anggota	Tugas
1	Nathan E C Marpaung	• Implementasi algoritma *Genetic Algorithm* • Implementasi visualizer • Implementasi representasi *neighbor* dan *state* awal • Penyusunan laporan
2	Kurt Mikhael Purba	• Implementasi representasi *state*, fungsi objektif, dan *constraints* • Implementasi algoritma *Steepest Ascent Hill-Climbing*, *Random Restart Hill-Climbing*, dan *Hill-Climbing with Sideways Move* • Penyusunan laporan
3	Aurelia Jennifer Gunawan	• Implementasi fungsi-fungsi pendukung • Implementasi algoritma *Simulated Annealing* • Implementasi algoritma *Stochastic Hill-Climbing* • Penyusunan laporan

Form Pernyataan Penggunaan AI

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Kurt Mikhael Purba



Nathan E C Marpaung

Aurelia Jennifer Gunawan